目　录

第一部分　名校考研真题
2015 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解
2014 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解
2013 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解
2012 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解
2011 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解
2010 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解
2009 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解
第二部分　章节题库
第 1 章　绪　论
第 2 章　线性表
第 3 章　栈和队列
第 4 章　串
第 5 章　数组和广义表
第 6 章　树和二叉树
第 7 章　图
第 8 章　动态存储管理
第 9 章　查　找
第 10 章　内部排序
第 11 章　外部排序
第 12 章　文　件
第三部分　模拟试题
严蔚敏《数据结构》（C 语言版）配套模拟试题及详解（一）
严蔚敏《数据结构》（C 语言版）配套模拟试题及详解（二）

第一部分　名校考研真题

2015 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解

一、单项选择题：1～40 小题，每小题 2 分，共 80 分。下列每题给出的四个选项中，只有一个选项符合题目要求。请在答题卡上将所选项的字母涂黑。

1．已知程序如下：

```
int S（int n）
{
return （n<＝0） ? 0 : s（n-1）+n;
}
void main（）
{
```

cout<<S（1）;
}
程序运行时使用栈来保存调用过程的信息，自栈底到栈顶保存的信息依次对应的是（　　）。
A．main（）->S（1）->S（0）
B．S（0）->S（1）->main（）
C．main（）->S（0）->S（1）
D．S（1）->S（0）->main（）
【答案】A 查看答案
【解析】函数 S（int n）是一个递归函数：①当实际参数小于等于零时则返回 0，并终止递归；②当实际参数大于零时则递归调用 S（n-1），并将 S（n-1）的结果加上 n 作为返回值。程序从 main（）函数开始，首先调用 main（）函数；在 main（）函数中调用 S（1）函数时，将 main（）函数的上下文保存到栈中，并进入函数 S（1）；由于函数 S（1）的实际参数大于零，需要调用 S（0），故将 S（1）函数的上下文保存到栈中，进入 S（0）；在 S（0）中，实际参数小于等于零，递归终止。
2．先序序列为 a，b，c，d 的不同二叉树的个数是（　　）。
A．13
B．14
C．15
D．16
【答案】B 查看答案
【解析】二叉树的先序遍历定义为：若二叉树为空，则空操作；否则，访问根节点，然后先序遍历左子树，最后先序遍历右子树。本题中，结点 a 为二叉树的根节点，左右子树的先序遍历可能存在下面四种情况：①左子树为空，bcd 为右子树；②b 为左子树，cd 为右子树；③bc 为左子树，d 为右子树；④bcd 为左子树，右子树为空。然后将左右子树继续分解，如第①种情况的右子树先序遍历（bcd）可能有：a．左子树为空，右子树为 cd；b．左子树为 c，右子树为 d；c．左子树为 cd，右子树为空。按照这种方法继续分解左右子树，直到不能再分解为止，可得第①和④种情况各包含 5 种不同情况，第②和③种情况各包含 2 种情况，因此总共有 14 种不同的二叉树。
3．下列选项给出的是从根分别到达两个叶节点路径上的权值序列，能属于同一棵哈夫曼树的是（　　）。
A．24，10，5 和 24，10，7
B．24，10，5 和 24，12，7
C．24，10，10 和 24，14，11
D．24，10，5 和 24，14，6
【答案】D 查看答案
【解析】哈夫曼树是带权路径长度最短的二叉树。由根节点出发到两个叶子节路径中，第二个被访问的两个结点的权值要么相等，要么和为根节点的权值，故 B 项错误。同理，通过第三个被访问的节点排除 A 项。C 项，由两条路径可推出三个叶子节点的权值分别是：3、10 和 11，而根据哈夫曼树的定义可知，权值为 3 的节点应该和权值为 10 的结点结合，故 C 项错误。D 项，反推出有四个叶子节点，权值分别为：5、5、6 和 8，满足哈夫曼树的条件。
4．现在有一颗无重复关键字的平衡二叉树（AVL 树），对其进行中序遍历可得到一个降序序列。下列关于该平衡二叉树的叙述中，正确的是（　　）。
A．根节点的度一定为 2
B．树中最小元素一定是叶节点

C．最后插入的元素一定是叶节点
D．树中最大元素一定是无左子树
【答案】D 查看答案
【解析】二叉树的中序遍历定义是“若二叉树为空，则空操作；否则：①中序遍历左子树；②访问根节点；③中序遍历右子树”。A 项错误，当树中仅有一个或者两个结点时，根节点的度就可能不为 2；B 项错误，树中最小元素是中序遍历时最后访问的节点，当没有右子树时，最后访问的节点是根节点；C 项错误，当最后插入的元素破坏树的平衡后，树会进行调整，使其成为中间节点；D 项正确，由中序遍历的特点可知，左子树的值大于根节点，所以最大元素一定没有左子树。
5．设有向图 G＝（V，E），顶点集 V＝{V0，V1，V2，V3}，边集 E＝{<V0, V1>，<V0, V2>，<V0, V3>，<V1, V3>}，若从顶点 V0 开始对图进行深度优先遍历，则可能得到的不同遍历序列个数是（　　）。
A．2
B．3
C．4
D．5
【答案】D 查看答案
【解析】根据题意知有向图的结构如图所示。深度优先遍历的特点是尽可能先对纵深方向进行搜索，所以可能得到的不同遍历序列分别是：①V0→V2→V1→V3；②V0→V2→V3→V1；③V0→V1→V3→V2；④V0→V3→V2→V1；⑤V0→V3→V1→V2。



6．暂缺
7．下列选项中，不能构成折半查找中关键字比较序列的是（　　）。
A．500，200，450，180
B．500，450，200，180
C．180，500，200，450
D．180，200，500，450
【答案】A 查看答案
【解析】折半查找的过程是：先确定待查找记录所在的范围，然后逐步缩小范围直到找到或找不到该记录为止。折半查找的关键字序列满足：对每一个关键字，其后面的所有关键字序列或者都小于等于该关键字或者都大于等于该关键字。A 项错误，第三次比较的关键字为 450，说明待查关键字位于 200～450 间，所以第四次比较时不会遇到关键字 180。
8．已知字符串 S 为“abaabaabacacaabaabcc”，模式串 t 为“abaabc”，采用 KMP 算法进行匹配，第一次出现“失配”（s[i]!＝t[i]）时，i＝j＝5，则下次开始匹配时，i 和 j 的值分别是（　　）。
A．i＝1，j＝0
B．i＝5，j＝0
C．i＝5，j＝2
D．i＝6，j＝2
【答案】C 查看答案

【解析】模式匹配（KMP）算法对普通的暴力匹配的改进在于：每当匹配过程中匹配失败时，主串（本题为 S）的指针（i）不需要回溯，而是利用已经得到的“部分匹配”的结果将模式串（t）向右“滑动”尽可能远的一段距离后，继续进行比较。模式串“滑动”的距离是由模式串（t）本身决定的，即 t 的子串 t[0…j-1]中前缀串和后缀串相等的最长长度。本题中第一次失配 i＝5，字串为“abaab”，其相等且最长的前后缀为“ab”，一次下一个 j＝2。

9．下列排序算法中元素的移动次数和关键字的初始排列次序无关的是（　　）。

A．直接插入排序

B．起泡排序

C．基数排序

D．快速排序

【答案】C 查看答案

【解析】C 项，基数排序是采用分配和收集实现的，不需要进行关键字的比较。ABD 三项都依赖关键字的比较，不同的初始排列次序下元素移动的次数有很大变化，最好情况元素正序，则不用移动，最坏情况元素反序，则需要移动 n（n-1）/2 次（n 为元素个数）。

10．已知小根堆为 8，15，10，21，34，16，12，删除关键字 8 之后需重建堆，在此过程中，关键字之间的比较数是（　　）。

A．1

B．2

C．3

D．4

【答案】C 查看答案

【解析】堆排序中，依次输出堆顶的最小值，然后重新调整堆，如此反复执行，便得到一个有序序列。本题中，删除堆顶元素 8 后将最后一个元素 12 置于堆顶，然后调整堆：首先与 15 比较，12 小于 15，所以不用交换；然后与 10 比较，因为 10 小于 12，所以交换 10 和 12 的位置；调整后 12 再与 16 比较，12 小于 16，调整堆过程结束。因此 12 共与 15、10、16 进行了三次比较。

11．希尔排序的组内排序采用的是（　　）。

A．直接插入排序

B．折半插入排序

C．快速排序

D．归并排序

【答案】A 查看答案

【解析】希尔排序基本思想是：先将整个待排元素序列按某个增量分割成若干个子序列，在子序列内进行直接插入排序，然后依次缩减增量再进行排序，待整个序列中的元素基本有序（增量足够小）时，再对全体元素进行一次直接插入排序。

12．计算机硬件能够直接执行的是（　　）。

Ⅰ．机器语言程序 Ⅱ.汇编语言程序 Ⅲ.硬件描述语言程序

A．仅Ⅰ

B．仅Ⅰ Ⅱ

C．仅Ⅰ Ⅲ

D．Ⅰ Ⅱ Ⅲ

【答案】A 查看答案

【解析】机器语言是计算机唯一可以直接执行的语言。汇编语言属于低级语言，但其源程必须要翻译成目标程序成为机器语言程序后才能被直接执行。硬件描述语言是电子系统硬件行为描述、结构描述、数据流描述的语言。

13．由 3 个“1”和 5 个“0”组成的 8 位二进制补码，能表示的最小整数是（　　）。

A．-126

B．-125

C．-32

D．-3

【答案】B 查看答案

【解析】能表示的最小整数一定是负数，符号位占用 1 个“1”；负数的补码和原码的转化是：原码符号位不变，数值部分按位取反，末位加“1”。因此最小的整数的补码是“10000011”，原码为“11111101”，即-125_{10}。

14．下列有关浮点数加减运算的叙述中，正确的是（　　）。

Ⅰ．对阶操作不会引起阶码上溢或下溢

Ⅱ．右规和尾数舍入都可能引起阶码上溢

Ⅲ．左规时可能引起阶码下溢

Ⅳ．尾数溢出时结果不一定溢出

A．仅Ⅱ Ⅲ

B．仅Ⅰ Ⅱ Ⅳ

C．仅Ⅰ Ⅲ Ⅳ

D．Ⅰ Ⅱ Ⅲ Ⅳ

【答案】D 查看答案

【解析】浮点数的加减运算步骤包括：①对阶，使两个操作数的小数点位置对齐，阶码小的尾数右移，可能产生溢出，但是阶码不会溢出；②尾数求和，将对阶后的尾数按定点数加（减）运算规则运算；③规格化，包括左规和右规，左规时阶码减少，可能出现阶码下溢，而右规时，阶码增加可能出现阶码上溢；④舍入，该过程可能需要右规调整，因此可能出现阶码上溢；⑤溢出判断，浮点数的溢出与否是由阶码的符号决定的，而不是由尾数溢出判断的，因此尾数溢出时结果不一定溢出。因此Ⅰ Ⅱ Ⅲ Ⅳ均正确。

15．假定主存地址为 32 位，按字节编址，主存和 Cache 之间采用直接映射方式，主存块大小为 4 个字，每字 32 位，采用回写（Write Back）方式，则能存放 4K 字数据的 Cache 的总容量的位数至少是（　　）。

A．146k

B．147K

C．148K

D．158K

【答案】B 查看答案

【解析】Cache 和主存直接映射方式的规则为：主存储器分为若干区，每个区与缓存容量相同；每个区分为若干数据块，每个块和缓存块容量相同；主存中某块只能映象到 Cache 的一个特定的块中。本题中，Cache 总共存放 4K 字数据，块大小为 4 个字，因此 cache 被分为 4K/4 ＝ 1K 个块，由 10 位表示。块内共 16 字节，所以由 4 位表示，于是标记位为 32-10-14＝18 位。所以，Cache 的每一行需要包含所存的数据 4 个字，每个字 32 位，18 位标记位和一个有效位，因此总容量为：（4*32 + 18 + 1）*1K ＝ 147K。

16．假定编译器将赋值语句“x＝x+3；”转换为指令”add xaddt，3”，其中 xaddt 是 x 对应的存储单元地址，若执行该指令的计算机采用页式虚拟存储管理方式，并配有相应的 TLB，且 Cache 使用直写（Write Through）方式，则完成该指令功能需要访问主存的次数至少是（　　）。

A．0

B．1

C．2

D．3

【答案】C 查看答案

【解析】采用页式虚拟存储管理方式时，若页表全部放在内存中，则存取一个数据最少要访问两次内存：第一次是访问页表，得到所存取的数据或指令的物理地址；第二次根据该地址存取数据或指令。在配有 TLB 的页式虚拟管理方式中，如果给出的地址在 TLB 中，则直接根据该地址取数据或指令，仅需要一次访问内存。Cache 使用直写方式时，计算完需要将数据写回到内存中，因此完成整个指令功能至少需要访问主存 2 次。

17．下列存储器中，在工作期间需要周期性刷新的是（　　）。

A．SRAM

B．SDRAM

C．ROM

D．FLASH

【答案】B 查看答案

【解析】动态随机存储器（DRAM）是利用存储元电路中栅极电容上的电荷来存储信息的，电容上的电荷一般只能维持 1～2ms，因此即使电源不掉电，信息也会自动消失。为此，每隔一定时间必须刷新。

18．某计算机使用 4 体交叉存储器，假定在存储器总线上出现的主存地址（十进制）序列为 8005，8006，8007，8008，8001，8002，8003，8004，8000，则可能发生发生缓存冲突的地址对是（　　）。

A．8004、8008

B．8002、8007

C．8001、8008

D．8000、8004

【答案】D 查看答案

【解析】交叉存储器，又称低位交叉编址，即低位地址为体号，高位地址为体内地址。本题中，主存地址对应的体号分别是：1，2，3，4，1，2，3，4，4。地址为 8004 和 8000 都是存取的四号储存器，可能导致 8004 存储还未完成而又存取 8000 地址，因此可能发生缓存冲突。

19．下列有关总线定时的叙述中，错误的是（　　）。

A．异步通信方式中，全互锁协议最慢

B．异步通信方式中，非互锁协议的可靠性最差

C．同步通信方式中，同步时钟信号可由多设备提供

D．半同步通信方式中，握手信号的采样由同步时钟控制

【答案】C 查看答案

【解析】A 项正确，异步通信方式中，全互锁协议最慢，主从模块都需要等待确认后才能撤销其信号；B 项正确，异步通信方式中，非互锁协议没有相互确认机制，因此可靠性最差；C 项错误，同步通信要遵循统一的时钟信号，不能由多设备提供；D 项正确，半同步通信方式中，握手信号的采样由同步时钟控制。

20．若磁盘转速为 7200 转/分，平均寻道时间为 8ms，每个磁道包含 1000 个扇区，则访问一个扇区的平均存取时间大约是（　　）。

A．8.1ms

B．12.2ms

C．16.3ms

D．20.5ms

【答案】B 查看答案

【解析】磁盘的平均寻址时间包括平均寻道时间和平均等待时间。平均寻道时间为 8ms，平均等待时间与磁盘转速有关，为[60s/7200]*0.5 ≈4.165ms。磁盘的存取一个扇区的时间为 60s/（7200 * 1000） ≈0.0083ms。因此总的时间为：8 + 4.165 + 0.0083 ＝ 12.1733ms。

21．在采用中断 I/O 方式控制打印输出的情况下，CPU 和打印控制接口中的 I/O 端口之间交换的信息不可能是（　　）。

A．打印字符

B．主存地址

C．设备状态

D．控制命令

【答案】B 查看答案

【解析】I/O 接口的功能包括：①选址功能；②传送命令功能；③传送数据功能；④反映 I/O 设备工作状态功能。A 项为数据，C 项为设备状态，D 项为命令。B 项，主存地址在中断方式控制下是不需要的，因此，它不可能是 CPU 和打印控制接口中的 I/O 端口之间交换的信息。

22．内部异常（内中断）可分为故障（fault）、陷阱（trap）和终止（abort）三类。下列有关内部异常的叙述中，错误的（　　）。

A．内部异常的产生与当前执行指令相关

B．内部异常的检测由 CPU 内部逻辑实现

C．内部异常的响应发生在指令执行过程中

D．内部异常处理后返回到发生异常的指令继续执行

【答案】D 查看答案

【解析】内中断分为：①由软中断指令启动的中断；②在一定条件下由 CPU 自身启动的中断。D 项错误，如突然掉电引发的内中断经处理后不会继续执行。

23．处理外部中断时，应该由操作系统保存的是（　　）。

A．程序计数器（PC）的内容

B．通用寄存器的内容

C．快表（TLB）的内容

D．Cache 中的内容

【答案】B 查看答案

【解析】外部中断处理过程首先要保护现场，使得中断处理完后能够恢复程序的状态继续执行。保护现场有两个含义：①由中断隐指令保存程序的断点（程序计数器）；②由中断服务程序保存通用寄存器和状态寄存器的内容。中断服务程序是操作系统的一部分。

24．假定下列指令已装入指令寄存器。则执行时不可能导致 CPU 从用户态变为内核态（系统态）的是（　　）。

A．DIV R0，R1；（R0）/（R1）→R0

B．INT n；产生软中断

C．NOT R0；寄存器 R0 的内容取非

D．MOV R0，addr；把地址处的内存数据放入寄存器 R0 中

【答案】C 查看答案

【解析】A 项，除法操作出现除数为零的情况时，会产生内中断，CPU 切换为内核态进行中断处理；B 项，直接产生中断，会切换到内核态；D 项，addr 出现非法地址，会出现中断，进而切换到内核态。

25．下列选项中会导致进程从执行态变为就绪态的事件是（　　）。

A．执行 P（wait）操作

B．申请内存失败

C．启动 I/0 设备

D．被高优先级进程抢占

【答案】D 查看答案

【解析】D 项，被高优先级进程抢占，进程会由执行态变为就绪态。ABC 三项，程序由于缺少资源而由执行态转为阻塞态。

26．若系统 S1 采用死锁避免方法，S2 采用死锁检测方法，下列叙述中正确的是（　　）。

Ⅰ.S1 会限制用户申请资源的顺序

Ⅱ.S1 需要进行所需资源总量信息，而 S2 不需要

Ⅲ.S1 不会给可能导致死锁的进程分配资源，S2 会

A．仅Ⅰ Ⅱ

B．仅Ⅱ Ⅲ

C．仅Ⅰ Ⅲ

D．Ⅰ Ⅱ Ⅲ

【答案】C 查看答案

【解析】死锁避免的策略是：必须知道将来的资源需求，以寻找可能的安全允许顺序，如果不存在安全序列就阻塞；死锁检测的策略是：只要允许就分配资源，它指定期检查死锁是否已经发生，如果发生就通过剥夺解除死锁。两种方式都需要所需资源的总量信息，但 S1 是用于在分配资源时判断是否会导致死锁，而 S2 是用于检测是否出现死锁。

27．系统为某进程分配了 4 个页框，该进程已访问的页号序列为 2，0，2，9，3，4，2，8，2,3,8,4,5,若进程要访问的下一页的页号为 7,依据 LRU 算法,应淘汰页的页号是(　　)。

A．2

B．3

C．4

D．8

【答案】B 查看答案

【解析】LRU 置换算法是选择最近最久未使用的页面予以淘汰。进程有 4 个页框，题中访问过程中页框的变化如下：

序列：　　2 0 2 9 3 4 2 8 2 3 8 4 5

页框：　　2 2 0 0 0 2 9 3 3 4 4 2 3

0 2 2 2 9 3 4 4 8 2 3 8

9 9 3 4 2 8 2 3 8 4

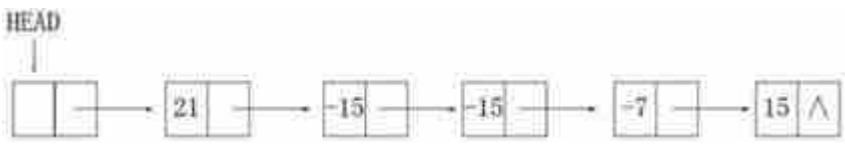
3 4 2 8 2 3 8 4 5

淘汰：　　0　9 2

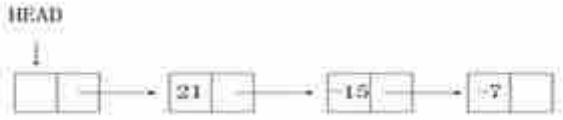
访问页号为 7 的页时，内存中存在的页的页号是：3、8、4 和 5，根据 LRU 定义应淘汰的是 3。

28．在系统内存中设置磁盘缓冲区的主要目的是（　　）。

A．减少磁盘 I/O 次数
B．减少平均寻道时间
C．提高磁盘数据可靠性
D．实现设备无关性
【答案】A 查看答案
【解析】访问磁盘的开销远远大于访问内存的开销。磁盘缓冲区便是利用主存中的存储空间，来暂存从磁盘中读出（或写入）的信息，频繁使用的一部分磁盘数据和信息，暂时存放在磁盘缓存中，可减少访问磁盘的次数。
29．在文件的索引节点中存放直接索引指针 10 个，一级二级索引指针各 1 个，磁盘块大小为 1KB。每个索引指针占 4 个字节。若某个文件的索引节点已在内存中，到把该文件的偏移量（按字节编址）为 1234 和 307400 处所在的磁盘块读入内存。需访问的磁盘块个数分别是（　　）。
A．1，2
B．1，3
C．2，3
D．2，4
【答案】B 查看答案
【解析】文件的索引结点的直接索引指针有 10 个，因此直接索引的偏移量范围是 0～2559，一级索引的偏移量范围是 2560～65791，二级索引访问的偏移量范围是 65792～45183907。偏移量 1234 可以通过直接索引得到在磁盘块的地址，因此需要一次访问，307400 需要通过二级索引查找其在磁盘的位置，需要分别访问存放二级索引的两个索引块以及对应的数据块。
30．在请求分页系统中，页面分配策略与页面置换策略不能组合使用的是（　　）。
A．可变分配，全局置换
B．可变分配，局部置换
C．固定分配，全局置换
D．固定分配，局部置换
【答案】C 查看答案
【解析】分配和置换策略有下面三个组合：①固定分配、局部置换；②可变分配、全局置换；③可变分配、局部置换。固定分配是指基于进程的类型（交互型或批处理型等），或根据程序员、程序管理员的建议，为每个进程分配一定数目的物理块，在整个运行期间都不再改变，采用该策略时，如果进程在运行中发现缺页，则只能从该进程在内存的 n 个页面中选出一个页换出，然后再调入一页，才能保证分配给该进程的内存空间不变，因此不能有固定分配，全局置换组合。
31～40．暂缺
二、综合应用题：41～47 小题，共 70 分。
41．（9 分）用单链表保存 m 个整数，节点的结构为（data，link），且|data|＜n（n 为正整数）。现要求设计一个时间复杂度尽可能高效地算法，对于链表中绝对值相等的节点，仅保留第一次出现的节点而删除其余绝对值相等的节点。
例如若给定的单链表 head 如下



删除节点后的 head 为



要求

（1）给出算法的基本思想

（2）使用 c 或 c++语言，给出单链表节点的数据类型定义。

（3）根据设计思想，采用 c 或 c++语言描述算法，关键之处给出注释。

（4）说明所涉及算法的时间复杂度和空间复杂度。

答：（1）算法思想：

定义一个大小为 n 的布尔数组 flag，初始时所有的元素都赋值为 false，用来标识遍历过程中是否出现元素绝对值为 flag 的节点。然后遍历链表，遍历过程中，每一个当前结点 data 域的绝对值所对应的 flag 位：若为真，则删除该结点；若为假（false），则将 flag 位置为真（true）。

（2）节点的数据结构定义如下：

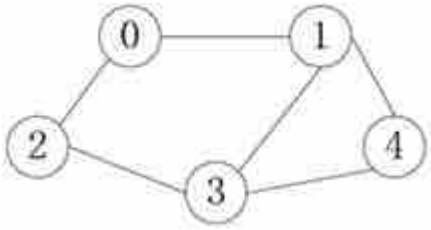
```
typedef struct Node
{
Int data;
Struct Node * next;
}Node;
```

（3）bool flag[n]； //全局数组标志节点的绝对值是否出现过

```
Node * deleteABSEnqualNode（Node * head）
{
memset（flag, false, sizeof（flag））;
Node *pre ＝ head;
Node *p ＝ head->next;
while（ p !＝ NULL ）{
if（ flag[ abs（p->data） ] ）{ //如果此绝对值已经在节点值的绝对值中出现过则删除该节点
    pre->next ＝ p->next;
    delete p;
    p ＝ pre->next;
} else {//否则，将 flag 中对应的位置置为 true，并将指针指向下一个元素
flag[ abs（p->data） ] ＝ true;
p ＝ p->next;
}
}
return head;
}
```

（4）只遍历一次链表，所以时间复杂度为 O（m）（m 为单链表中元素的个数），申请大小为 n 的数组，所以空间复杂度为 O（n）（n 为节点绝对值的最大值）。

42．（11 分）已知有 5 个顶点的图 G 如下图所示



请回答下列问题

（1）写出图 G 的邻接矩阵 A（行、列下标从 0 开始）。

（2）求 A^2，矩阵 A^2中位于 0 行 3 列元素值的含义是什么？

（3）若已知具有 n（n>=2）个顶点的邻接矩阵为 B，则 B^m（2<=m<=n）非零元素的含义是什么？

答：（1）邻接矩阵为

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

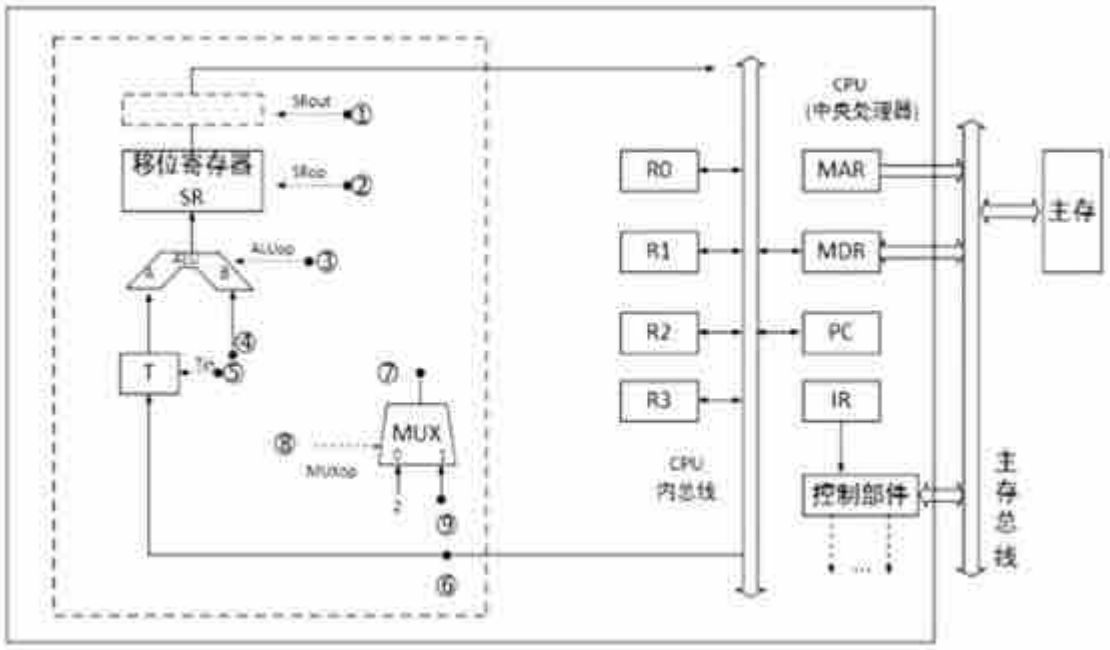
（2）A^2为：

$$\begin{bmatrix} 2 & 0 & 0 & 2 & 1 \\ 0 & 3 & 2 & 1 & 1 \\ 0 & 2 & 2 & 0 & 1 \\ 2 & 1 & 0 & 3 & 1 \\ 1 & 1 & 1 & 1 & 2 \end{bmatrix}$$

0 行 3 列的元素的含义是顶点 0 到顶点 3 间是相通的，并且路径长度为 2 的路径有 2 条。

（3）B^m中非零元素的含义是：假设此顶点位于 i 行 j 列，表示从 i 结点到 j 结点路径长度为 m 的路径的条数。。

43．（12 分）某 16 位计算机主存按字节编码。存取单位为 16 位；采用 16 位定长指令格式；CPU 采用单总线结构，主要部分如下图所示。图中 R0～R3 为通用寄存器；T 为暂存器；SR 为移位寄存器，可实现直送（mov）、左移一位（left）、右移一位（right）3 种操作，控制信号为 Srop，SR 的输出信号 Srout 控制；ALU 可实现直送 A（mova）、A 加 B（add）、A 减 B（sub）、A 与 B（and）、A 或 B（or）、非 A（not）、A 加 1（inc）7 种操作，控制信号为 ALUop。



请回答下列问题。

（1）图中哪些寄存器是程序员可见的？为何要设置暂存器 T?

（2）控制信号 ALUop 和 SRop 的位数至少各是多少？

（3）控制信号 Srout 所控制部件的名称或作用是什么？

（4）端点①～⑨中，哪些端点须连接到控制部件的输出端？

（5）为完善单总线数据通路，需要在端点①～⑨中相应的端点之间添加必要的连线。写出连线的起点和终点，以正确表示数据的流动方向。

（6）为什么二路选择器 MUX 的一个输入端是 2?

答：（1）图中程序员可见的寄存器有通用寄存器 R0～R3 和程序计数器 PC；当执行算术或逻辑操作时，由于 ALU 本身是没有内部存储功能的组合电路，因此如要执行加法运算，被相加的两个数必须在 ALU 的两个输入端同时有效，因此设置暂存器 T 用于暂存数据总线发送的数据。

【解析】程序员可见的寄存器包括：程序计数器、通用寄存器和状态寄存器。其他的 IR、MAR 和 MDR 等是 CPU 的内部工作寄存器，对程序员不可见。

（2）ALUop 和 SRop 的位数分别为 3，2。

【解析】ALU 中共有 7 种命令，用三位即可区别表示，SR 共有三种命令二位二进制即可表示。

（3）Srout 所控制的部件是状态字寄存器，用来存放 ALU 及 CPU 的指令状态。

（4）须连接到控制部件的输出端端点有①②③⑤⑧。

【解析】操作符命令，传输等都需要控制信号进行控制。

（5）⑥→⑨，⑦→④。

（6）数据宽度是 16 位，以字节编址，输入端是 2 是为了增加地址获取 ALU 的第二个操作数。

44.（10 分）题 43 中描述的计算机，其部分指令执行过程的控制信号如如题 44 图 a 所示。



题 44 图 a 部分指令控制信号

该机指令格式如题 44 图 b 所示，支持寄存器直接和寄存器间接两种寻址方式，寻址方式位分别为 0 和 1，通用寄存器 R0～R3 的编号分别为 0、1、2 和 3。



题 44 图 b 指令格式

请回答下列问题。

（1）该机的指令系统最多可定义多少条指令？

（2）假定 inc、shl 和 sub 指令的操作码分别为 01H、02H 和 03H，则以下指令对应的机器代码各是什么？

①inc R1；R1+1→R1

②shl R2，R1；（R1）<<1→R2

③sub R3，（R1），R2；（（R1））–（R2）→R3

（3）假定寄存器 X 的输入和输出控制信号分别为 Xin 和 Xout，其值为 1 表示有效，为 0 表示无效（例如，PCout＝1 表示 PC 内容送总线）；存储器控制信号为 MEMop，用于控制存储器的读（read）和写（write）操作。写出题 44 图 a 中标号①⑧处的控制信号或控制信号的取值。

（4）指令“sub R1，R3，（R2）”和“inc R1”的执行阶段至少各需要多少个时钟周期？

答：（1）128

【解析】支持两种寻址方式，使用 1 位标识，四个寄存器，使用 2 位标识，因此对于每一个操作数需要 3 位，每条指令三个操作数，一条指令总共 16 位，因此剩余 7 位，所以可以定义 2^7条指令。

（2）①0240H，②0464H，③06EEH

【解析】“inc R1；”中 R1 是直接寻址的单地址指令，代码是“0000001 001 000 000”，前面七位是操作码（01H），001 是第一个操作数地址（R1）；

“shl R2，R1；（R1）<<1→R2”中 R1 是间接寻址，R2 是直接寻址，代码是“0000010 010 101 000”，前面七位是操作码（02H），010 是第一个操作数地址（R2），101 是第二个操作数地址；
“sub R3，（R1），R2；（（R1））–（R2）→R3”中 R1 和 R2 是间接寻址，R3 是直接寻址，代码是“000011 011 101 110”，前面七位是操作码（03H），011 是第一个操作数地址（R3），101 是第二个操作数地址，110 是第三个操作数地址。
（3）①0，②mov，③mova，④left，⑤read，⑥sub，⑦mov，⑧Srout。
（4）至少各需要 8 和 1 个时钟周期。
【解析】在一个时钟周期内 CPU 仅完成一个动作。
sub R1，R3，（R2）；（R3）–（（R2））→R1 执行周期的微操作序列是：
T0：R3→MAR
T1：M（MAR）→MDR
T2：MDR→R3
T3：R2→MAR
T4：M（MAR）→MDR
T5：MDR→MAR
T6：M（MAR）→MDR
T7：R3+MDR→R1
所以至少需要 8 个时钟周期。
inc R1；R1+1→R1 执行周期的微操作序列是：
T0：R1+1→R1
所以至少需要 1 个时钟周期。
45.（15 分）有 A、B 两人通过信箱进行辩论，每人都从自己的信箱中取得对方的问题。将答案和向对方提出的新问题组成一个邮件放入对方的邮箱中，设 A 的信箱最多放 M 个邮件，B 的信箱最多放 N 个邮件。初始时 A 的信箱中有 x 个邮件（0< x < M），B 中有 y 个（0 < y < N）。辩论者每取出一个邮件，邮件数减 1.。
A、B 两人操作过程：
Code Begin
A{
While（ TRUE ）{
从 A 的信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入 B 的信箱；
}
}
B{
While（ TRUE ）{
从 B 的信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入 A 的信箱；
}
}
Code End
当信箱不为空时，辩论者才能从信箱中取邮件，否则等待。

当信箱不满时，辩论者才能将新邮件放入信箱，否则等待。
请添加必要的信号量和 P、V（或 wait signed）操作，以实现上述过程的同步，要求写出完整过程，并说明信号量的含义和初值。
答：首先定义两个互斥信号量：mutexA 和 mutexB，初始时为 1，分别用来实现对 A 的邮箱和 B 的邮箱的互斥使用；然后针对 A 的邮箱再定义两个信号量 emptyA 和 fullA，初值分别为 M – x 和 x，分别表示信箱中仍能存放信的数量和已经存放的信的数量，同理设置 emptyB 和 fullB，初值为 N – y 和 y。
初始代码：
Semaphore mutexA ＝ 1， mutexB ＝ 1；
Semaphore emptyA ＝ M – x，fullA ＝ x；
Semaphore emptyB ＝ N – y,，fullB ＝ y；
通信代码：
Code Begin
A｛
while（ TRUE ） {
P（ fullA ）;
P（ mutexA ）;
从 A 的信箱中取出一个邮件；
V（ mutexA ）;
V（ emptyA ）;
回答问题并提出一个新问题；
P（emptyB）;
P（ mutexB ）;
将新邮件放入 B 的信箱；
V（ mutexB ）;
V（ fullB ）;
}
}
B{
While（ TRUE ）{
P（ fullB ）;
P（ mutexB ）;
从 B 的信箱中取出一个邮件；
V（ mutexB ）;
V（ emptyB ）;
回答问题并提出一个新问题；
P（ emptyA ）;
P（ mutexA ）;
将新邮件放入 A 的信箱；
V（ mutexA ）;
V（ fullA ）;
}
}
Code End

46～47．暂缺

2014 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解

一、单项选择题：1～40 小题，每小题 2 分，共 80 分。下列每题给出的四个选项中，只有一个选项是符合题目要求的。

1．下列程常段的时间复杂度是（　　）

```
count＝0;
for（k＝1;k<＝n; k*2）
for（j＝1;j<＝n;j+1）
count++;
```

A．O（$\log_2 n$）

B．O（n）

C．O（$n\log_2 n$）

D．O（n^2）

【答案】C 查看答案

【解析】外部循环的退出条件是 k>n，而对于 k，每次循环都执行 k＝k*2,所以循环次数为 $\log_2 n$；内部循环的退出条件是 j>n,对于 j，每次循环都执行 j＝j+1，所以每次循环次数为 n 次。所以此程序段的时间复杂度为 O（$n\log_2 n$），即选 C。

2．假设栈初始为空，将中缀表达式 $a/b+(c*d-e*f)/g$ 转换为等价后缀表达式的过程中，当扫描到 f 时，栈中的元素依次是（　　）

A．$+(*-$

B．$+(-*$

C．$/+(*-*$

D．$/+-*$

【答案】B 查看答案

【解析】中缀表达式转后缀表达式遵循以下原则：

（1）遇到操作数，直接输出；

（2）栈为空时，遇到运算符，入栈；

（3）遇到左括号，将其入栈；

（4）遇到右括号，执行出栈操作，并将出栈的元素输出，直到弹出栈的是左括号，左括号不输出；

（5）遇到其他运算符"+""-""*""/"时，弹出所有优先级大于或等于该运算符的栈顶元素，然后将该运算符入栈；

（6）最终将栈中的元素依次出栈，输出。

所以扫描到’/’，入栈‘描到’+’,由于’+’优先级比’/’低，所以将’/’弹出，’+’入栈；扫描到’*’,优先级比’+’高，入栈；扫描到’（‘,入栈;扫描到’-‘,将栈中优先级更高的’*’弹出，‘-’ 入栈;扫描到’*’,优先级比’-‘高，入栈。所以扫描到 f 的时候，栈中元素为：$+(-*$

3．循环两列放在一维数组 A[0…M-1]中，end1 指向队头元素，end2 指向队尾元素的后一个位置。假设队列两端均可进行入队和出队操作，队列中最多能容纳 M-1 个元素。初始时为空，下列判断队空和队满的条件中，正确的是（　　）

A．队空：end1＝＝end2；　　队满：end1＝＝（end2+1）modM

B．队空：end1＝＝end2;　队满：end2＝＝（end1+1）mod（M-1）

C．队空：end2＝＝（end1+1）modM；　队满：end1＝＝（end2+1）modM

D．队空：end1＝＝（end2+1）modM;　队满：end2＝＝（end1+1）mod（M-1）

【答案】A 查看答案

【解析】在循环队列中，在少用一个元素空间的前提下，可约定入队前，测试尾指针在循环意义下加 1 后是否等于头指针，若相等，则队满。而队空的条件还是首尾指针是否相等。

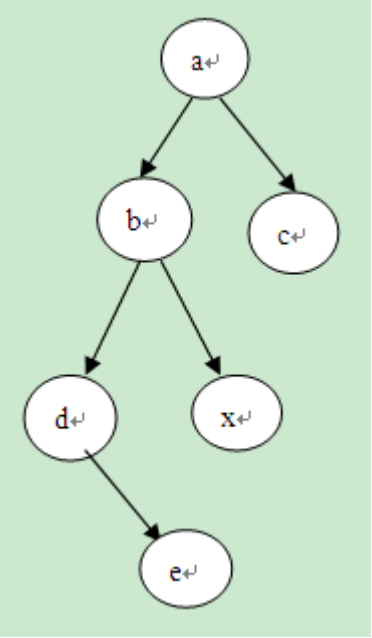
4．若对如下的二叉树进行中序线索化，则结点 x 的左、右线索指向的结点分别是（　　）

A．e,c

B．e,a

C．d,c

D．b,a



【答案】D 查看答案

【解析】此二叉树的中序遍历序列为：debxac,由于节点 x 左右孩子都为空，所有进行中序线索化时，它的左右孩子指针分别指向它的中序遍历序列的直接前驱结点 b 和直接后继结点 a，所以选 D

5．将森林 F 转换为对应的二叉树 T，F 中叶结点的个数等于（　　）

A．T 中叶结点的个数

B．T 中度为 1 的结点个数

C．T 中左孩子指针为空的结点个数

D．T 中右孩子指针为空的结点个数

【答案】C 查看答案

【解析】森林转化为对应的二叉树是‘孩子-兄弟’存储的，即左孩子指针指向当前节点的孩子节点，右孩子指针指向当前节点的兄弟节点，所以在 T 中左孩子指针为空则代表它在森林中并没有孩子即为叶结点。所以选 C

6．5 个字符有如下 4 种编码方案，不是前缀编码的是（　　）

A．01,0000,0001,001,1

B．011,000,001,010,1

C．000,001,010,011,100

D．0,100,110,1110,1100

【答案】D 查看答案

【解析】在一个字符集中，任何一个字符的编码都不是另一个字符编码的前缀。约定左分支表示字符‘0’，右分支表示字符‘1’，则可以用从根结点到叶子结点的路径上的分支字符串作为该叶子结点字符的编码。如此得到的编码必是前缀编码。D 选项中，编码 110 是编码 1100 的前缀，故不符合前缀编码的定义。

7．对如下所示的有向图进行拓扑排序，得到的拓扑序列可能是（　　）

A．3,1,2,4,5,6

B．3,1,2,4,6,5

C．3,1,4,2,5,6

D．3,1,4,2,6,5



【答案】D 查看答案

【解析】拓扑排序方法如下:

（1）从有向图中选择一个没有前驱（即入度为 0）的顶点并且输出它；

（2）从图中删去该顶点,并且删去从该顶点发出的全部有向边；

（3）重复上述两步,直到剩余的网中不再存在没有前趋的顶点为止。

对于此有向图进行拓扑排序所有序列为：3,1,4,6,2,5 和 3,1,4,2,6,5。所以选 D

8．用哈希（散列）方法处理冲突（碰撞）时可能出现堆积（聚集）现象，下列选项中，会受堆积现象直接影响的是（　　）

A．存储效率

B．数列函数

C．装填（装载）因子

D．平均查找长度

【答案】D 查看答案

【解析】哈希方法冲突会使在查找冲突的关键字时，还要根据冲突处理办法多次比较关键字，则直接影响了平均查找长度。

9．在一棵具有 15 个关键字的 4 阶 B 树中，含关键字的结点数最多是（　　）

A．5

B．6

C．10

D．15

【答案】D 查看答案

【解析】m 阶 B 树非根结点含关键字个数 ⌈m/2⌉ - 1 <= j <= m – 1。

4 阶 B 树非根结点含关键字 1～3 个，所以要使关键字结点数量最多，那么每个结点只有一个关键字，一共有 15 个关键字那么最多有 15 个含有关键字的结点

10. 用希尔排序方法对一个数据序列进行排序时，若第 1 趟排序结果为 9,1,4,13,7,8,20,23,15，则该趟排序采用的增量（间隔）可能是（　　）

A．2

B．3

C．4

D．5

【答案】B 查看答案

【解析】对于 A，增量为 2，那么 9,4,7,20,15 是一组，而它们是无序的，所以 A 错误

对于 C，增量为 4，那么 9,7,15 是一组，而它们是无序的，所以 C 错误

对于 D，增量为 5，那么 9,8 是一组，降序，1,20 是一组，而它们是升序，所以 D 也错误。

对于 B，分为 3 组：9,13,20；1,7,23；4,8,15 都是升序有序，所以 B 正确

11．下列选项中，不可能是快速排序第 2 趟排序结果的是（　　）

A．2,3,5,4,6,7,9

B．2,7,5,6,4,3,9

C．3,2,5,4,7,6,9

D．4,2,3,5,7,6,9

【答案】C 查看答案

【解析】对于快速排序，每一趟都会使一个元素位于有序时的位置，而有序序列为 2,3,4,5,6,7,9，与 C 进行对比，只有 9 位于它有序的时候的位置，显然不是第二趟快速排序的结果

12. 程序 P 在机器 M 上的执行时间是 20 秒，编译优化后，P 执行的指令数减少到原来的 70%，而 CPI 增加到原来的 1.2 倍，则 P 在 M 上的执行时间是（　　）

A．8.4 秒

B．11.7 秒

C．14 秒

D．16.8 秒

【答案】D 查看答案

【解析】20*0.7*1.2 ＝ 16.8

13. 若 x＝103,y＝-25,则下列表达式采用 8 位定点补码运算实现时，会发生溢出的是（　　）

A．x+y

B．-x+y

C．x-y

D．-x-y

【答案】C 查看答案

【解析】8 位定点补码能表示的数的范围为：-128～127

A 结果为 78，B 结果为-128，D 结果为-78 都在此范围内，只有 C 结果 128 超过了 8 位定点补码能表示的数的范围，会发生溢出

14.float 型整数据常用 IEEE754 单精度浮点格式表示，假设两个 float 型变量 x 和 y 分别在 32 为寄存器 f_1 和 f_2 中，若（f_1）＝CC900000H,（f_2）＝B0C00000H,则 x 和 y 之间的关系为：（　　）

A．x<y 且符号相同
B．x<y 且符号不同
C．x>y 且符号相同
D．x>y 且符号不同
【答案】A 查看答案
【解析】两个数对应的 IEEE754 的标准形式为：

浮点数	S	阶码	尾数
f1	1	1001 1001	001 0000 0000 0000 0000 0000
f2	1	0110 0001	100 0000 0000 0000 0000 0000

将 IEEE754 单精度形式的二进制转化为浮点数公式为 V＝（-1）^s*2^（E-Bias）*M
由于 f1，f2 的符号位都是 1，所以 f1，f2 符号相同，而阶码上 f1>f2，所以 f1>f2，所以 f1 的绝对值比 f2 大，而他们都是负数，所以 f1<f2，所以选 A
15．某容量为 256M 的存储器，由若干 4M*8 位的 DRAM 芯片构成，该 DRAM 芯片的地址引脚和数据引脚总数是：（　　）
A．19
B．22
C．30
D．36
【答案】A 查看答案
【解析】DRAM 地址线复用，4M 为 2 的 22 次方，因此除 2 为 11 根，数据线 8 根。因此地址引脚和数据引脚总数为 19 根
16．采用指令 Cache 与数据 Cache 分离的主要目的是（　　）
A．减低 Cache 的缺失损失
B．提高 Cache 的命中率
C．减低 CPU 平均访问时间
D．减少指令流水线资源冲突
【答案】D 查看答案
【解析】指令流水线不会断流，预取过来的都是指令
17．某计算机有 16 个通用寄存器，采用 32 位定长指令字操作码字段（含寻址方式位）为 8 位，Store 指令的源操作数和目的操作数分别采用寄存器直接寻址和基址寻址方式，若基址寄存器可使用任一通用寄存器，且偏移量用补码表示，则 Store 指令中偏移量的取值范围是（　　）
A．-32768～+32767
B．-32767～+32768
C．-65536～+65535
D．-65535～+65536
【答案】A 查看答案
【解析】寄存器个数 16＝2^4，偏移量有 32-8-4-4＝16 位
指令编址方式如下所示：

操作码	源地址寄存器	目的地址基址寄存器	偏移量
8	4	4	16

16 位补码取值范围为-32768～+32767，所以偏移量取值范围为-32768～+32767

18．某计算机采用微程序控制器，共有 32 条指令，公共的取指令微程序包含 2 条微程序，各指令对应的微程序平均由 4 条微指令组成，采用断定法（下址字段法）确定下条微指令的地址，则微指令中下址字段的位数至少是：（　　）

A．5

B．6

C．8

D．9

【答案】C 查看答案

【解析】32*4+2＝130，2^7＝128<130<2^8＝256，所以至少需要 8 位才能表示完 130 个地址。

19．某同步总线采用数据线和地址线复用方式。其中地址数据线有 8 根，总线时钟频率为 66MHZ，每个时钟同期传送两次数据。（上升沿和下降沿各传送一次数据）该总线的最大数据传输率是（总线带宽）：（　　）

A．132MB/S

B．264MB/S

C．528MB/S

D．1056MB/S

【答案】C 查看答案

【解析】总线带宽＝总线工作频率×（总线宽度/8），由于地址线与数据线复用，所以在两次数据传输过程中总线上数据一共传输了 8 次，那么总线带宽为 66*8＝528，所以选 C

20．一次总线事物中，主设备只需给出一个首地址，从设备就能从首地址开始的若干连续单元格读出或写入的个数，这种总线事务方式称为（　　）

A．并行传输

B．串行传输

C．突发

D．同步

【答案】C 查看答案

【解析】猝发数据传输方式：在一个总线周期内传输存储地址连续的多个数据字的总线传输方式

21．下列有关 I/O 接口的叙述中错误的是：（　　）

A．状态端口和控制端口可以合用同一寄存器

B．I/O 接口中 CPU 可访问寄存器，称为 I/O 端口

C．采用独立编址方式时，I/O 端口地址和主存地址可能相同

D．采用统一编址方式时，CPU 不能用访存指令访问 I/O 端口

【答案】D 查看答案

【解析】采用统一编码方式，存储器和 I/O 端口共用统一的地址空间，不需要专用的 I/O 指令，任何对存储器数据进行操作的指令都可用于 I/O 端口的数据操作。所以 D 错误

22．某设备中断请求的相应和处理时间为 100ns，每 400ns 发出一次中断请求，中断相应所容许的最长延迟时间为 50ns，则在该设备持续工作过程中 CPU 用于该设备的 I/O 时间占整个 CPU 时间百分比至少是（　　）

A．12.5%

B．25%

C．37.5%

D．50%

【答案】B 查看答案
【解析】每 400ns 响应一次中断并且用 100ns 进行处理，所以该设备的 I/O 时间占用 CPU 时间百分比为 100/400＝25%,中断响应容许的延迟时间对此没有影响，属于干扰条件。
23．下列调整中，不可能导致饥饿现象的是（　　）
A．时间片转移
B．静态优先及调度
C．非抢占式作业优先
D．抢占式短作业优先
【答案】A 查看答案
【解析】时间片转移方法能在一个周期内使每个进程都得到一个时间片的 CPU 使用时间，不会产生饥饿的现象，其余三个都会产生饥饿。
24．某系统有 n 台互斥使用的同类设备，3 个并发进程需要 3,4,5 台设备，可确保系统不发生死锁的设备数 n 最小为（　　）
A．9
B．10
C．11
D．12
【答案】B 查看答案
【解析】2+3+4+1 ＝ 10
25．下列指令中，不能在用户态执行的是（　　）
A．trap 指令
B．跳转指令
C．后栈指令
D．关中断指令
【答案】D 查看答案
【解析】关中断指令必须在和心态才能执行，trap 指令可以在用户态下执行，执行了就转到和心态，跳转与退栈指令都是可以在用户态下执行的指令。
26．一个进程的读磁区操作完成后，操作系统针对该进程必做的是（　　）
A．修改进程状态为就绪态
B．降低进程优先级
C．进程分配用户内存空间
D．增加进程的时间片大小
【答案】A 查看答案
【解析】进程等待的 I/O 操作完成便会从等待状态转移到就绪状态。
27．现有容量为 10GB 的磁盘分区，磁盘空间以簇（cluster）为单位进行分配，簇的大小为 4KB，若采用位图法管理该分区的空闲空间，即用一位（bit）标识一个簇是否被分配，则存放该位图所需簇的个数为（　　）
A．80
B．320
C．80K
D．320K
【答案】A 查看答案
【解析】磁盘的簇的个数为：10GB/4KB＝2.5*2^{20}个
而一个簇的位示图能管理的簇的个数为：4KB*8＝32K

所以需要簇的个数为 $2.5*2^{20}/32K$ ＝ 80 个

28．下列措施中，能加快虚实地址转换的是 1 增大快表（TLB）2 让页表常驻内存 3 增大交换区（　　）

A．仅 1

B．仅 2

C．仅 1,2

D．仅 2,3

【答案】C 查看答案

【解析】加大快表能增加快表的命中率，即减少了访问内存的次数；让页表常驻内存能够使 cpu 不用访问内存找页表，从也加快了虚实地址转换。而增大交换区只是对内存的一种扩充作用，对虚实地址转换并无影响

29．在一个文件被用户进程首次打开的过程中，操作系统需做的是（　　）

A．将文件内容读到内存中

B．将文件控制块读到内存中

C．修改文件控制块中的读写权限

D．将文件的数据缓冲区首指针返回给用户进程

【答案】B 查看答案

【解析】概念

30．在页式存储管理系统中，采用某些页面置换算法，会出现 Belady 异常现象，即进程的缺页次数会随着分配给该进程的页框个数的增加而增加。下列算法中，可能出现 Belady 异常现象的是（　　）

Ⅰ．LRU 算法　Ⅱ．FIFO 算法　Ⅲ．OPT 算法

A．仅Ⅱ

B．仅Ⅰ Ⅱ

C．仅Ⅰ Ⅲ

D．仅Ⅱ Ⅲ

【答案】A 查看答案

【解析】Belady 现象只有 FIFO 算法才会出现

31．下列关于管道（Pipe）通信的叙述中，正确的是（　　）

A．一个管道可实现双向数据传输

B．管道的容量仅受磁盘容量大小限制

C．进程对管道进行读操作和写操作都可以被阻塞

D．一个管道只能有一个读写进程或一个写进程对其操作

【答案】C 查看答案

【解析】只有写进程才能对管道写入数据，读进程对管道进行读取数据，只能半双工通信，即某一时刻只能单向传输。管道为空，则读操作被堵塞，而如果有写操作对管道进行写的话那就要堵塞了。那么 C 正确

32．下列选项中，属于多级页表优点的是（　　）

A．加快地址变换速度

B．减少缺页中断次数

C．减少页表项所占字节数

D．减少页表所占的连续内存空间

【答案】D 查看答案

【解析】多级页表避免了把所有的页表一直保存在内存中

33．在 OSI 参考模型中，直接为会话层提供服务的是（　　）
A．应用层
B．表示层
C．传输层
D．网络层
【答案】C 查看答案
【解析】OSI 参考模型中，下层直接为上层提供服务，而会话层的下层为传输层。
34．某以太网拓扑及交换机当前转发表如下图所示，主机 00-e1-d5-00-23-a1 向主机 00-e1-d5-00-23-c1 发送 1 个数据帧，主机 00-e1-d5-00-23-c1 收到该帧后，向主机 00-e1-d5-00-23-a1 发送一个确认帧，交换机对这两个帧的转发端口分别是（　　）
A．$\{3\}$ 和 $\{1\}$
B．$\{2,3\}$ 和 $\{1\}$
C．$\{2,3\}$ 和 $\{1,2\}$
D．$\{1,2,3\}$ 和 $\{1\}$

目的地址	端口
00-e1-d5-00-23-b1	2

【答案】B 查看答案
【解析】第一次交换机没有 00-e1-d5-00-23-c1 的信息，只能选择从其他端口全部发送，同时记录这个数据报源 MAC 地址的信息 00-e1-d5-00-23-a1,确认帧发送时已经有 00-e1-d5-00-23-a1 的信息了所以只用从 1 端口转发。
35．下列因素中，不会影响信道数据传输速率的是（　　）
A．信噪比
B．频率宽带
C．调制速率
D．信号传播速度
【答案】D 查看答案
【解析】信道数据传输速率与信噪比、频率宽度、调制速率都有关。
36．主机甲与主机乙之间使用后退 N 帧协议（GBN）传输数据，甲的发送窗口尺寸为 1000，数据帧长为 1000 字节，信道宽带为 100Mbps，乙每收到一个数据帧立即利用一个短帧（忽

略其传输延迟）进行确认，若甲乙之间的单向传播延迟是 50ms，则甲可以达到的最大平均数据传输速率约为（　　）

A．10 Mbps

B．20 Mbps

C．80 Mbps

D．100 Mbps

【答案】C 查看答案

【解析】1000B*1000/（50*2ms）＝10MB＝80Mbps.

37．站点 A、B、C 通过 CDMA 共享链路，A、B、C 的码片序列（chipping sequence）分别是（1,1,1,1）、（1，-1,1，-1）和（1，1,-1，-1），若 C 从链路上收到的序列是（2,0,2,0,0，-2,0，-2,0,2,0,2），则 C 收到 A 发送的数据是（　　）

A．000

B．101

C．110

D．111

【答案】B 查看答案

【解析】用 A 的码片与信息做内积运算

38．主机甲和乙已建立了 TCP 连接，甲始终以 MSS＝1KB 大小的段发送数据，并一直有数据发送；乙每收到一个数据段都会发出一个接收窗口为 10KB 的确认段。若甲在 t 时刻发生超时时拥塞窗口为 8KB，则从 t 时刻起，不再发生超时的情况下，经过 10 个 RTT 后，甲的发送窗口是（　　）

A．10KB

B．12KB

C．14KB

D．15KB

【答案】A 查看答案

【解析】发送窗口是接受窗口和拥塞窗口的最小值，这里接收窗口总是 10KB。拥塞窗口到那个时候是大于 10KB 的，取最小值。

39．下列关于 UDP 协议的叙述中，正确的是（　　）

Ⅰ 提供无连接服务

Ⅱ 提供复用/分用服务

Ⅲ 通过差错校验，保障可靠数据传输

A．仅Ⅰ

B．仅Ⅰ、Ⅱ

C．仅Ⅱ、Ⅲ

D．Ⅰ、Ⅱ、Ⅲ

【答案】B 查看答案

【解析】UDP 无连接创建，提供多路复用服务。虽然有差错检验，但是不能保证可靠数据传输，所以Ⅲ错误。

40．使用浏览器访问某大学 Web 网站主页时，不可能使用的协议是（　　）

A．PPP

B．ARP

C．UDP

D．SMTP

【答案】D 查看答案

【解析】SMTP 是简单邮件传输协议，访问主页时并不涉及邮件相关协议。

二、综合应用题：41～47 小题，共 70 分。

41.（10 分）二叉树的带权路径长度（WPL）是二叉树中所有叶结点的带权路径长度之和，给定一棵二叉树 T，采用二叉链表存储，节点结构为：

left weight right

其中叶节点的 weight 域保存该结点的非负权值。设 root 为指向 T 的根节点的指针，设计求 T 的 WPL 的算法。要求：

（1）给出算法的基本设计思想；

（2）使用 C 或 C++语言，给出二叉树结点的数据类型定义；

（3）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

答：（1）算法的基本思路是利用利用递归的思想来求解二叉树的带权路径长度，如果当前节点不是叶子节点，那么当前节点为根的树的带权路径长度便等于它的子树的带权路径长度之和，对于此函数要传入一个当前节点的树高的形参，那么递归调用孩子节点时只需要将这个形参加一即可。

（2）typedef struct BiNode

```
{
    int weight; //权值
    struct BiNode *left;     //左孩子指针
    struct BiNode *right;     //右孩子指针
}BiNode, *BiTree;
```

（3）具体算法实现如下：

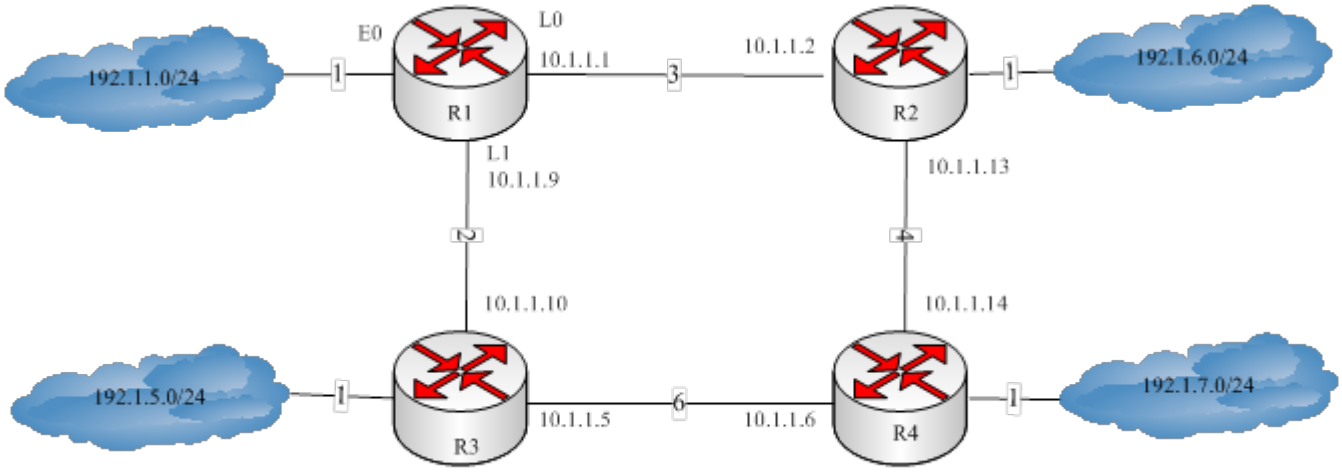
```
int CalcWPL（BiTree BT, int height）
{
    if（BT ＝＝ NULL） //如果当前节点为空节点
    return 0;
    //如果当前节点的左右孩子节点都为空，即当前节点为叶子节点，直接返回当前节点的 WPL
    if（!BT->left && !BT->right）
    return BT.weight * height;
    //如果当前节点不是叶子节点，则对当前节点的左右子树进行递归，返回左右子树 WPL 之和
    else
    return CalcWPL（BT->left, height+1） + CalcWPL（BT->right, height+1）;
}
```

42.（10 分）某网络中的路由器运行 OSPF 路由协议，题 42 表是路由器 R1 维护的主要链路状态信息（LSI），题 42 图是根据题 42 表及 R1 的接口名构造出来的网络拓扑。

题 42 表 R1 所维护的 LSI

	R1 的 LSI	R2 的 LSI	R3 的 LSI	R4 的 LSI	备注
Router ID	10.1.1.1	10.1.1.2	10.1.1.5	10.1.1.6	标识路由器的 IP 地址
Link1 ID	10.1.1.2	10.1.1.1	10.1.1.6	10.1.1.5	所连路由器的

	IP	10.1.1.1	10.1.1.2	10.1.1.5	10.1.1.6	RounterID Link1 的基本 IP 地址
	Metric	3	3	6	6	Link1 的费用
	ID	10.1.1.5	10.1.1.6	10.1.1.1	10.1.1.12	所连路由器的 RounterID
Link2	IP	10.1.1.9	10.1.1.13	10.1.1.10	10.1.1.14	Link2 基本 IP 地址
	Metic	2	4	2	4	Link2 费用
Net1	Prefix	192.1.1.0/24	192.1.6.0/24	192.1.5.0/24	192.1.7.0/24	直接网络 Net1 的网络前缀
	Metric	1	1	1	1	到达直连网络 Net1 的费用



题 42 图　R1 构造的网络拓扑

请回答下列问题。

本题中的网络可抽象为数据结构中的哪种逻辑结构？

针对题 42 表中的内容，设计合理的链式存储结构，以保存题 42 表中的链路状态信息（LSI）。要求给出链式存储结构的数据类型定义，并画出对应题 42 表的链式存储结构示意图（示意图中可仅以 ID 标识节点）。

按照迪杰斯特拉（Dijikstra）算法的策略，依次给出 R1 到达题 42 图中子网 192.1.x.x 的最短路径及费用。

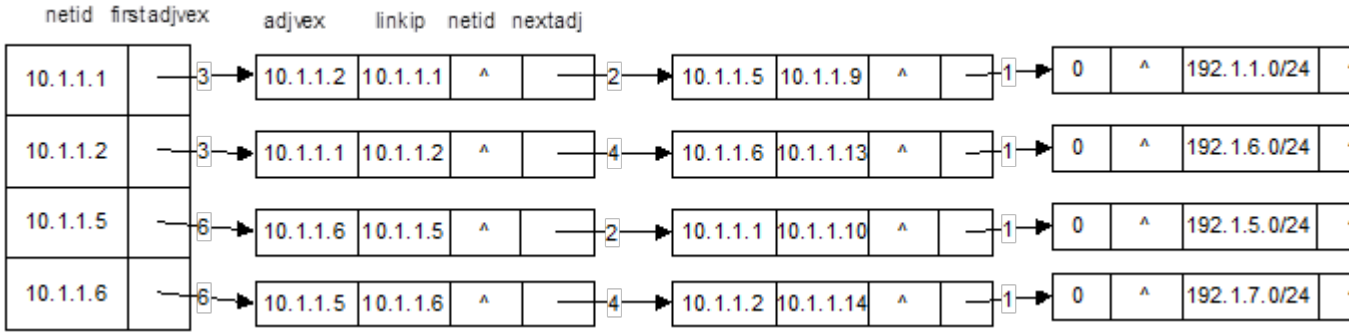
答：（1）图

（2）使用图的邻接表存储结构进行存储，数据类型定义如下：

```
typedef struct ArcNode
{
    int adjvex; //该弧指向路由器的位置，0 为没有
char *netid;     //该弧指向的网络的网络前缀,空为没有
```

```
char *linkip;     //路由的基础 IP，当 adjvex 不为 0 才有效
    struct ArcNode *nextarc;     //指向下一条弧的指针
    int metric; //连接的权值
}ArcNode;     //表结点
typedef struct RouterNode
{
char *routerid; //路由的 routerid
    ArcNode *firstarc; //第一个结点地址
}RouterNode,Router[NUM];     //头节点
```

链式存储结构示意图如下图所示：



（3）目标网络 192.1.1.0/24 记为 N1，192.1.5.0/24 记为 N2,192.1.6.0/24 记为 N3,192.1.7.0/24 记为 N4，使用 dijkstra 算法找最短路径步骤如下表所示：

步骤	S 集合	U 集合
1	选入 R1，此时 S＝{R1} 此时最短路径为 R1->R1＝0	U＝{R2,R3,R4,N1,N2,N3,N4} R1->R2＝3 R1->R3＝2 R1->N1＝1 最短
2	选入 N1，此时 S＝{R1,N1} 最短路径为 R1->R1＝0,R1->N1＝1	U＝{R2,R3,R4,N2,N3,N4} R1->R2＝3 R1->R3＝2 最短
3	选入 R3，此时 S＝{R1,N1,R3} 最短路径 R1->R1＝0,R1->N1＝1,R1->R3＝2	U＝{ R2,R4,N2,N3,N4} R1->R2＝3, R1->R3->N3＝3 最短 R1->R3->R4＝8
4	选入 N3，此时 S＝{R1,N1,R3,N2} 最短路径 R1->R1＝0,R1->N1＝1,R1->R3＝2 R1->R3->N3＝3	U＝{R2,R4,N2,N4} R1->R2＝3 最短, R1->R3->R4＝8
5	选入 R2，此时 S＝{R1,N1,R3,N2,R2} 最短路径 R1->R1＝0,R1->N1＝1,R1->R3＝2 R1->R3->N3＝3,R1->R2＝3	U＝{R4,N2,N4} R3->R4＝6, R1->R2->N2＝4 最短

6	选入 N2，此时 S＝{R1,N1,R3,N2,R2,N2} 最短路径 R1->R1＝0,R1->N1＝1,R1->R3＝2 R1->R3->N3＝3,R1->R2＝3, R1->R2->N2＝4	U＝{R4,N4} R1->R3->R4＝8,R1->R2->R4＝7 最短
7	选入 R4，此时 S＝{R1,N1,R3,N2,R2,N2,R4} 最短路径 R1->R1＝0,R1->N1＝1,R1->R3＝2 R1->R3->N3＝3,R1->R2＝3, R1->R2->N2＝4,R1->R2->R4＝7	U＝{N4} R1->R2->R4->N4＝8 最短 R1->R3->R4->N4＝9
8	选入 N4, S＝{R1,N1,R3,N2,R2,N2,R4,N4} 最短路径 R1->R1＝0,R1->N1＝1,R1->R3＝2 R1->R3->N3＝3,R1->R2＝3, R1->R2->N2＝4,R1->R2->R4＝7 R1->R3->R4->N4＝9	U 为空，查找结束

所以 R1 到达子网 192.1.1.0/24 最短路径为：R1->子网，费用为 1
R1 到达子网 192.1.6.0/24 的最短路径为：R1->R2->子网，费用为 3
R1 到达子网 192.1.5.0/24 的最短路径为：R1->R3->子网，费用为 4
R1 到达子网 192.1.7.0/24 的最短路径为 R1->R2->R4->子网，费用为 8

43. （9 分）请根据题 42 描述的网络，继续回答下列问题。

（1）假设路由表结构如下表所示，请给出题 42 图中 R1 的路由表，要求包括到达题 42 图中子网 192.1.x.x 的路由，且路由表中的路由项尽可能少。

目的网络	下一跳	接口

（2）当主机 192.1.1.130 向主机 192.1.7.211 发送一个 TTL＝64 的 IP 分组时，R1 通过哪个接口转发该 IP 分组？主机 192.1.7.211 收到的 IP 分组的 TTL 是多少？

（3）若 R1 增加一条 Metric 为 10 的链路链接 Internet，则题 42 表中 R1 的 LSI 需要增加哪些信息？

答：（1）

目的网络	下一跳	接口
192.1.1.0/24	/	E0
192.1.5.0/24	R3	L1
192.1.6.0/24	R2	L0
192.1.7.0/24	R2	L0

（2）通过接口 L0 转发该 IP 分组，由于要经过 R1，R2，R4 三个路由器，所以主机 192.1.7.211 收到的 IP 分组的 TTL 为 64-3＝61.

（3）只需要在 R1 的 LSI 中增加一项信息，prefix＝目的 internet，Metric＝10.

44．（11 分）某程序中有如下循环代码段 P：“for（i＝0;i<N;i++）sum+＝A[i];”。假设编译时变量 sum 和 i 分别分配在寄存器 R1 和 R2 中。常量 N 在寄存器 R6 中，数组 A 的首地址在寄存器 R3 中，程序段 P 起始地址为 0804 8100H，对应的汇编代码和机器代码如题 44 表所示

编号	地址	机器代码	汇编代码	注释
1	08048100H	00022080H	Loop: sll R4;R2,2	（R2）<<2→R4
2	08048104H	00083020H	Add R4;R4,R3	（R4）+（ R3）→R4
3	08048108H	8C850000H	Load R5;0（R4）	（（R4）+0）→R5
4	0804810CH	00250820H	Add R1;R1,R5	（R1）+（ R5）→R1
5	08048110H	2042000H	Add R2;R2,1	（R2）+2→R4
6	08048114H	1446FFFAH	Bne R2;R2,loop	if（R2）!＝（R6）go to loop

执行上述代码的计算机 M 采用 32 位定长指令字，其中分支指令 Bne 采用如下格式，



Op 为操作码：Rs 和 Rd 为寄存器编号：OFFSET 为偏移量，用补码表示。请回答下列问题，并说明理由。

（1）M 的存储器编址单位是什么？

（2）已知 sll 指令实现左移功能，数组 A 中每个元素占多少位？

（3）题 44 表中 bne 指令的 OFFSET 字段的值是多少？已知 bne 指令采用相对寻址方式，当前 PC 内容为 bne 指令地址，通过分析题 44 表中指令地址和 bne 指令内容，推断出 bne 指令的转移目标地址计算公式。

（4）若 M 采用如下“按序发射、按序完成”的 5 级指令流水线：IF（取指）、ID（译码及取数）、EXE（执行）、MEM（访存）、WB（写回寄存器），且硬件不采取任何转发措施，分支指令的执行均引起 3 个时钟周期阻塞，则 P 中哪些指令的执行会由于数据相关而发生流水线阻塞？哪条指令的执行会发生控制冒险？为什么指令 1 的执行不会因为与指令 5 的数据相关而发生阻塞？

答：（1）由题可知，指令为 32 为即 4 个字节，而程序执行时是以 4 为间隔逐条取指令的，故可知 M 的存储器是采用字节编址。

（2）32 位，因为 sl1 中实现左移，而（R2）<<2 R4 即左移两位就是乘以 4，所以是 4*8＝32 位

（3）由 Bne 的指令格式可知其 OFFSET 为指令的后 16 位，而 Bne 的机器码码为 1446 FFFAH，所以 Bne 的 OFFSET 为 FFFAH 即-6。

由题可知 Bne 采用相对寻址方式，故有效地址 EA＝（PC）+A＝（PC）+OFFSET，而 PC 的值为当前 Bne 指令的地址即（PC）＝08048114H，而取完 Bne 指令后，（PC）+4→PC，故（PC）＝0804 8118H。而要跳转到指令 1 的地址 08048100H，两者相差了 18H 也就是 24

个字节，而 OFFSET 是-6，故转移目标地址计算公式为（PC）+OFFSET*（24/6）＝（PC）+OFFSET*4。

（4）由指令序列可知，指令 1 需写 R4 而指令 2 需读 R4，故指令 2 会因为数据相关而发生阻塞，同理指令 3、指令 4 也会因为数据相关而发生阻塞；而指令 6 为分支指令，故其存在控制冒险。因为分支指令会有 3 个时钟周期的阻塞，故指令 1 的执行不会因为指令 5 的数据相关而发生阻塞。

45．（10 分）假设对于 44 题中的计算机 M 和程序 P 的机器代码，M 采用页式虚拟存储管理。P 开始执行时，（R1）＝ （R2）＝0. （R6）＝1000，其机器代码已调入主存但不在 Cache 中；数组 A 未调入主存，其所有数组元素在同一页，并存储在磁盘同一个扇区，请回答下列问题，并说明理由。

（1）P 执行结束时，R2 的内容是多少？

（2）M 的指令 Cache 和数据 Cache 分离，若 Cache 共有 16 行，Cache 和主存交换的块大小为 32 字节，则其数据区的容量是多少？若仅考虑程序段 P 的执行，则指令 Cache 的命中率为多少？

（3）P 在执行过程中，哪条指令的执行可能发生溢出异常？哪条指令的执行可能产生缺页异常？对于数组 A 的访问，需要读磁和 TLB 至少各多少次？

答：（1）R2 是保存变量 i 的内容，当 P 执行完即 i<N 不满足，故 i＝N＝（R6）＝1000。

（2）Cache 共有 16 行，每行大小为 32B，而本段指令大小为 6*4B＝24B，故指令占一行 Cache，所以数据 Cache 的大小为 15*32B＝160B。

由题可知 N＝1000，故循环了 1000 次，所以共执行了 1000*6＝6000 条指令，而只有第一次循环执行指令 1 时指令不在 Cache 中，故指令 Cache 命中率＝（6000-1）/6000＝99.9%。

（3）指令 4 有可能发生溢出，该指令是统计数组 A[0]～A[i]的和，而数组 A 的各元素的值未知，故该指令是可能发生溢出的。

第一次执行指令 2 时需要访问数组 A，而数组 A 未在内存中，故会发生缺页异常。由题可知数组 A 的所有元素在同一页，并存储在磁盘同一个扇区，故经过一次便将该页调入内存，所以以后访问便不会发生缺页中断，故只需读一次磁盘，999 次 TLB。

46．（9 分）文件 F 由 200 条记录组成，记录从 1 开始编号，用户打开文件后，欲将内存中的一条记录插入文件 F 中，作为其第 30 条记录，请回答下列问题，并说明理由。

（1）若文件系统为顺序分配方式，每个存储块存放一条记录，文件 F 的存储区域前后均有足够空闲的存储空间，则要完成上述操作最少要访问多少存储块？F 的文件控制区内容会有哪些改变？

（2）若文件系统为链接分配方式，每个存储块存放的一条记录和一个链接指针，则要完成上述操作最少要访问多少存储块？若每个存储块大小为 1KB，其中 4 个字节存放指针，则该系统支撑文件的最大长度是多少？

答：（1）因为要最少访问，所以选择将前 29 块前移一个存储块单元，然后将要写入的记录写入到当前的第 30 条的位置上。由于前移都要先访问原存储块将数据读出，再访问目标存储块将数据写入，所以最少需要访问 29*2+1＝59 块存储块

F 的文件区的文件长度加 1，起始块号减 1

（2）采用链接方式则需要顺序访问前 29 块存储块，然后将新纪录的存储块插入链中即可，把新的块存入磁盘要 1 次访存，然后修改第 29 块的链接地址存回磁盘又一次访存。一共就是 29+1+1＝31 次。

4 个字节的指针的地址范围为 2^{32}。

所以此系统支撑文件的最大长度为 2^{32}*（1KB-4B）＝4080GB

47.（11 分）系统中有多个生产者进程和消费者进程，共享用一个可以存 1000 个产品的缓冲区（初始为空），当缓冲区为未满时，生产者进程可以放入一件其生产的产品，否则等待；当缓冲区为未空时，消费者进程可以取走一件产品，否则等待。要求一个消费者进程从缓冲区连续取出 10 件产品后，其他消费者进程才可以取产品，请用信号量 P，V（wait，signed）操作实现进程间的互斥和同步，要求写出完整的过程；并指出所用信号量的含义和初值

答：设置 5 个信号量

empty：表示缓冲区是否为空，初值为 1000

full： 表示缓冲区是否为满，初值为 0

mutex1： 生产者之间的互斥信号，初值为 1

mutex2： 消费者之间的互斥信号，初值为 1

available: 当前消费者能否访问缓冲区，初值为 1

定义变量 in,out 分别为生产者和消费者进程所要使用的指针，指向下一个可用的缓冲区单元，MaxNum＝1000 为缓冲区的大小，count 标志当前消费者已经取的产品的数量，初值为 0

生产者进程：

```
while（true）
{
生产一个产品；
P（empty）；
P（mutex1）；
产品送入 buffer（in）；
in＝（in+1）mod MaxNum;
V（mutex1）；
V（full）；
}
```

消费者进程

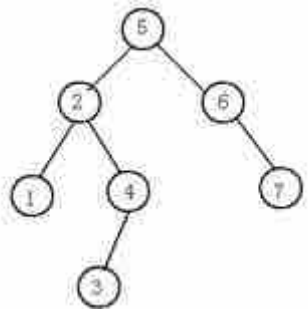
```
P（available）；
while（count !＝ 10）
{
P（full）；
P（mutex2）；
取出产品 buffer（out）；
count++
out＝（out+1）mod MaxNum;
V（mutex2）；
V（empty）；
}
count＝0;
V（available）；
```

2013 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解

一、单项选择题：1～40 小题，每小题 2 分，共 80 分。下列每题给出的四个选项中，只有一个选项符合试题要求。

1. 已知两个长度分别为 m 和 n 的升序链表，若将它们合并为一个长度为 m+n 的降序链表，则最坏情况下的时间复杂度是（　　）

A．O（n）
B．O（m*n）
C．O（min（m,n））
D．O（max（m,n））
【答案】D 查看答案
【解析】m 和 n 是两个升序链表□长度分别为 m 和 n，在合并过程中最坏的情况是两个链表中的元素依次进行比较，比较的次数是 m 和 n 中的最大值。
2．一个栈的入栈序列为 1,2,3,……,n，其出栈序列是 P_1，P_2，P_3……P_n。若，则 P_2＝3，则 P_3可能取值的个数是（　　）
A．n-3
B．n-2
C．n-1
D．无法确定
【答案】C 查看答案
【解析】除了 3 本身以外,其他的值均可以取到,因此可能取值的个数为 n-1。
3．若将关键字 1，2，3，4，5，6，7 依次插入到初始为空的平衡二叉树 T 中，则 T 中平衡因子为 0 的分支结点的个数是（　　）
A．0
B．1
C．2
D．3
【答案】D 查看答案
【解析】将图中给定的关键字序列依次插入到平衡树中，构成的平衡树如下图所示,由图可知平衡因子为 0 的分支结点为 3 个叶子结点，故答案为 D。

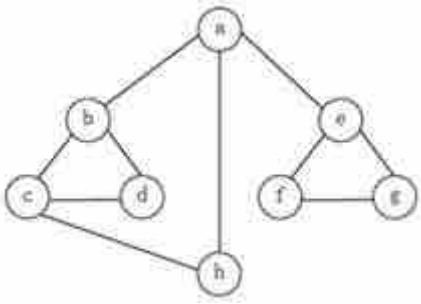


4．已知三叉树 T 中 6 个叶结点的权分别是 2，3，4，5，6，7，T 的带权（外部）路径长度最小是（　　）
A．27
B．46
C．54
D．56
【答案】B 查看答案
【解析】利用三叉树的 6 个叶子结点的权构建最小带权生成树，最小的带权路径长度为（2+3）*3+（4+5）*2+（6+7）*1＝46
5．若 X 是后序线索二叉树中的叶结点，且 X 存在左兄弟结点 Y，则 X 的右线索指向的是（　　）
A．X 的父结点

B．以 Y 为根的子树的最左下结点
C．X 的左兄弟结点 Y
D．以 Y 为根的子树的最右下结点
【答案】A 查看答案
【解析】根据后续线索二叉树的定义,X 结点为叶子结点且有左兄弟,那么这个结点为右孩子结点,利用后续遍历的方式可知 X 结点的后继是其父结点,即其右线索指向的是父结点。
6．在任意一棵非空二叉排序树 T1 中，删除某结点 v 之后形成二叉排序树 T2，再将 v 插入 T2 形成二叉排序树 T3。下列关于 T1 与 T3 的叙述中，正确的是（　　）
I．若 v 是 T1 的叶结点，则 T1 与 T3 不同
II．若 v 是 T1 的叶结点，则 T1 与 T3 相同
III．若 v 不是 T1 的叶结点，则 T1 与 T3 不同
IV．若 v 不是 T1 的叶结点，则 T1 与 T3 相同
A．仅 I、III
B．仅 I、IV
C．仅 II、III
D．仅 II、IV
【答案】C 查看答案
【解析】在一棵二叉排序树中删除一个结点后再将此结点插入到二叉排序树中，如果删除的结点是叶子结点那么在插入结点后，后来的二叉排序树与删除结点之前相同。如果删除的结点不是叶子结点，那么再插入这个结点后，后来的二叉树可能发生变化，不完全相同。
7．设图的邻接矩阵 A 如下所示,各顶点的度依次是（　　）

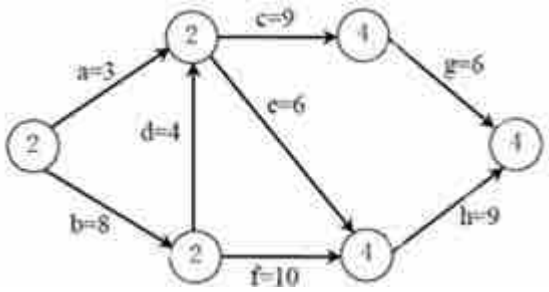
$$A=\begin{bmatrix}0&1&0&1\\0&0&1&1\\0&1&0&0\\1&0&0&0\end{bmatrix}$$

A．1，2，1，2
B．2，2，1，1
C．3，4，2，3
D．4，4，2，2
【答案】C 查看答案
【解析】当图用邻接矩阵存储时，各顶点的度是矩阵中此结点对应的横行和纵列非零元素之和。
8．若对如下无向图进行遍历，则下列选项中，不是广度优先遍历序列的是（　　）



A．h，c，a，b，d，e，g，f
B．e，a，f，g，b，h，c，d

C．d，b，c，a，h，e，f，g
D．a，b，c，d，h，e，f，g
【答案】D 查看答案
【解析】根据广度优先遍历的定义，可知选项 A、B、C 都为广度优先遍历，而选项 D 是深度优先遍历而不是广度优先遍历，故答案为 D。
9．下列 AOE 网表示一项包含 8 个活动的工程。通过同时加快若干进度可以缩短整个工程的工期。下列选项中，加快其进度就可以缩短工程工期的是（　　）



A．c 和 e
B．d 和 e
C．f 和 d
D．f 和 h
【答案】C 查看答案
【解析】根据 AOE 网的定义可知，同时缩短几条关键路径上的活动时间，可以缩短整个工期。
10．在一株高度为 2 的 5 阶 B 树中，所含关键字的个数最少是（　　）
A．5
B．7
C．8
D．14
【答案】A 查看答案
【解析】根据 B 树的定义可知，跟结点最少含有 max（2,（m-1））个关键字，高度为 2 的阶 B 树最少有（5-1）+1＝5 个关键字，其中根节点含有（5-1）个关键字，第 2 层结点含有 1 关键字。
11．对给定的关键字序列 110，119，007，911，114，120，122 进行基数排序，则第 2 趟分配收集后得到的关键字序列是（　　）
A．007，110，119，114，911，120，122
B．007，110，119，114，911，122，120
C．007，110，911，114，119，120，122
D．110，120，911，122，114，007，119
【答案】C 查看答案
【解析】基数排序的第 1 趟排序是按照个位数字来排序的,第 2 趟排序是按然十位数字的大小进行排序的,故答案是 C 选项。
12．某计算机主频为 1.2GHz,其指令分为 4 类，它们在基准程序中所占比例及 CPI 如下表所示。

指令类型	所占比例	CPI
A	50%	2
B	20%	3
C	10%	4
D	20%	5

该机的 MIPS 数是（　　）

A．100

B．200

C．400

D．600

【答案】C 查看答案

【解析】基准程序的 CPI＝2*0.5+3*0.2+4*0.1+5*0.2＝3。计算机的主频为 1.2GHz,为 1200MHz，该机器的 MIPS 为 1200/3＝400。

13．某数采用 IEEE754 单精度浮点数格式表示为 C640 0000H，则该数的值是（　　）

A．-1.5×2^{13}

B．-1.5×2^{12}

C．$-0.5x\times2^{13}$

D．-0.5×2^{12}

【答案】A 查看答案

【解析】IEEE754 单精度浮点数格式为 C640 0000H 表示为二进制格式为 1100 0110 0100 0000 0000 0000 0000 0000，转换为标准的格式为：

S	阶码	尾数
1	1000 1100	100 0000 0000 0000 0000 0000

因此，浮点数的值为$-1.5*2^{13}$。

14．某字长为 8 位的计算机中，已知整型变量 x、y 的机器数分别为$[x]_{补}$＝11110100，$[y]_{补}$＝10110000。若整型变量 z＝2*x+y/2，则 z 的机器数为（　　）

A．11000000

B．00100100

C．10101010

D．溢出

【答案】A 查看答案

【解析】将 x 左移一位,y 右移一位,两个数的补码相加的机器数为 1 1000000,故答案选择 A。

15.用海明码对长度为8位的数据进行检/纠错时，若能纠正一位错，则校验位数至少为（　　）

A．2

B．3

C．4

D．5

【答案】C 查看答案

【解析】设校验位的位数为 k,数据位的位数为 n,根据海明码编码 k 和 n 应满足下述关系。

$2^k \geq n+k+1$。n＝8,当 k＝4 时，$2^4=(16)\geq 8+4+1(=13)$，符合要求，校验位至少是 4 位，故答案为 C。

16．某计算机主存地址空间大小为 256MB，按字节编址。虚拟地空间大小为 4GB，采用页式存储管理，页面大小为 4KB，TLB（快表）采用全相联映射，有 4 个页表项，内容如下表所示。

有效位	标记	页框号	…
0	FF180H	0002H	…
1	3FFF1H	0035H	…
0	02FF3H	0351H	…
1	03FFFH	0153H	…

则对虚拟地址 03FFF180H 进行虚实地址变换的结果是（　　）

A．0153180H

B．0035180H

C．TLB 缺失

D．缺页

【答案】A 查看答案

【解析】虚拟地址为 03FF F180H，其中页号为 03FFFH，页内地址为 180H，根据题目中给出的页表项可知页标记为 03FFFH 所对应的页框号为 0153H，页框号与页内地址之和即为物理地址 015 3180H。

17．假设变址寄存器 R 的内容为 1000H，指令中的形式地址为 2000H；地址 1000H 中的内容为 2000H，地址 2000H 中的内容为 3000H，地址 3000H 中的内容为 4000H，则变址寻方式下访问到的操作数是（　　）

A．1000H

B．2000H

C．3000H

D．4000H

【答案】D 查看答案

【解析】根据变址寻址的 EA＝（IX）+A,变址寄存器的内容与形式地址的内容相加之后得到操作数的实际地址,由题可知 EA＝1000H+2000H＝3000H，根据实际地址访问内存,获取操作数 4000H。

18．某 CPU 主频为 1.03GHz，采用 4 级指令流水线，每个段的执行需要 1 个时钟周期。假定 CPU 执行了 100 条指令，在其执行过程中没有发生任何流水线阻塞，此时流水线的吞吐率为（　　）

A．0.25×109 条指令/秒

B．0.97×109 条指令/秒

C．1.0×109 条指令/秒

D．1.03×109 条指令/秒

【答案】C 查看答案

【解析】采用 4 级流水线执行 100 条指令，在执行过程中共用 4+（100-1）＝103 个时钟周期。CPU 的主频是 1.03GHz,也就是说每秒钟有 1.03G 个时钟周期。流水线的吞吐率为 1.03G*100/103＝1.0*10^9条指令/秒，故答案为 C。

19．下列选项中，用于设备和控制器（I/O 接口）之间互连的接口标准是（　　）

A．PCI

B．USB

C．AGP

D．PCI-Express
【答案】B 查看答案
【解析】设备和设备控制器之间的接口是 USB 接口,其余选项不符合，故答案为 B。
20．下列选项中，用于提高 RAID 可靠性的措施有（　　）
I．磁盘镜像　　II．条带化　　III．奇偶校验　　IV．增加 Cache 机制
A．仅 I、II
B．仅 I、III
C．仅 I、III 和 IV
D．仅 II、III 和 IV
【答案】B 查看答案
【解析】能够提高 RAID 可靠性的措施主要是对磁盘进行镜像处理和进行奇偶校验。其余选项不符合条件。
21．某磁盘的转速为 10,000 转/分，平均寻道时间是 6ms，磁盘传输速率是 20MB/s，磁盘控制器延迟为 0.2ms，读取一个 4KB 的扇区所需平均时间约为（　　）
A．9ms
B．9.4ms
C．12ms
D．12.4ms
【答案】B 查看答案
【解析】磁盘转速是 10 000 转/分钟，平均转一转的时间是 6ms，因此平均查询扇区的时间是 3ms，平均寻道时间是 6ms，读取 4KB 扇区信息的时间为 0.2ms，信息延迟的时间为 0.2ms，总时间为 3+6+0.2+0.2＝9.4 ms。
22．下列关于中断 I/O 方式和 DMA 方式比较的叙述中，错误的是（　　）
A．中断 I/O 方式请求的是方式请求的是 CPU 处理时间，DMA 方式请求的是总线使用权
B．中断响应发生在一条指令执行结束后，中断响应发生在一条指令执行结束后，DMA 响应发生在一个总线事务完成后
C．中断 I/O 方式下数据传送通过软件完成，方式下数据传送通过软件完成，DMA 方式下数据传送由硬件完成
D．中断 I/O 方式适用于所有外部设备，方式适用于所有外部设备，DMA 方式仅适用于快速外部设备
【答案】D 查看答案
【解析】中断处理方式:在 I/O 设备输入每个数据的过程中,由于无需 CPU 干预,因而可使 CPU 与 I/O 设备并行工作。仅当输完一个数据时,才需 CPU 花费极短的时间去做些中断处理。因此中断申请使用的是 CPU 处理时间,发生的时间是在一条指令执行结束之后,数据是在软件的控制下完成传送。而 DMA 方式与之不同。DMA 方式：数据传输的基本单位是数据块,即在 CPU 与 I/O 设备之间,每次传送至少一个数据块,DMA 方式每次申请的是总线的使用权,所传送的数据是从设备直接送入内存的或者相反;仅在传送一个或多个数据块的开始和结束时,才需 CPU 干预，整块数据的传送是在控制器的控制下完成的。答案 D 的说法不正确。
23．用户在删除某文件的过程中，操作系统不可能执行是（　　）
A．删除此文件所在的目录
B．删除与此文件关联的目录项
C．删除与此文件对应的控制块
D．释放与此文件关联的内存级冲区
【答案】A 查看答案

【解析】删除文件不需要删除文件所在的目录,而文件的关联目录项和文件控制块需要随着文件一同删除，同时释放文件的关联缓冲区。

24．为支持 CD-ROM 中视频文件的快速随机播放，播放性能最好的文件数据块组织方式是（　　）

A．连续结构

B．链式结构

C．直接索引结构

D．多级索引结钩

【答案】A 查看答案

【解析】为了实现快速随机播放，要保证最短的查询时间，即不能选取链表和索引结构，因此连续结构最优。

25．用户程序发出磁盘 I/O 请求后，系统的处理系统的处理流程是：用户程序→系统调用处理程序→设备骆动程序→中断处理程序。其中，计算数据所在磁盘的柱面号、磁头号、扇区号的程序是（　　）

A．用户程序

B．系统调用处理程序

C．设备驱动程序

D．中断处理程序

【答案】C 查看答案

【解析】计算磁盘号、磁头号和扇区号的工作是由设备驱动程序完成的,所以答案选 C。

26．若某文件系统索引结点（inode）中有直接地址项和间接地址项，则下列选项中，与单个文件长度无关的因素是（　　）

A．索引结点的总数

B．间接地址索引的级数

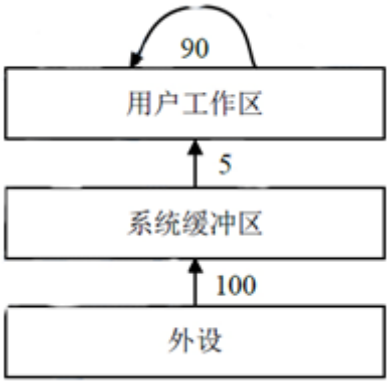
C．地址项的个数

D．文件块大小

【答案】A 查看答案

【解析】根据文件长度与索引结构的关系可知，只有选项 A 是与单个文件长度无关的。

27．设系统缓冲区和用户工作均采单，从外读入 1 个数据块到系统缓冲区的时间为 100，从系统缓冲区读入 1 个数据块到用户工作区的时间为 5，对用户工作区中的 1 个数据块行分析的时间为 90（如下图所示）。进程从外设读入并分析 2 个数据块的最短时间是（　　）



A．200

B．295

C．300

D．390

【答案】C 查看答案

【解析】数据块 1 从外设到用户工作区的总时间为 105,在这段时间中数据块 2 没有进行操作。在数据块 1 进行分析处理时,数据块 2 从外设到用户工作区的总时间为 105,这段时间是并行的。再加上数据块 2 进行处理的时间 90,总共是 300,故答案为 C。

28．下列选项中，会导致用户进程从态切换到内核的操作是（　　）

I．整数除以零　　II．sin（）函数调用　　III．read 系统调用

A．仅 I、II

B．仅 I、III

C．仅 II、III

D．I、II 和 III

【答案】B 查看答案

【解析】对于 I,系统发生异常，需要进入内核态由操作系统进行处理，而 read 系统调用函数也是在内核态执行，sin（）就是普通的用户函数，在用户态执行，故答案为 C。

29．计算机开后，操作系统最终被加载到（　　）

A．BIOS

B．ROM

C．EPROM

D．RAM

【答案】D 查看答案

【解析】系统开机后,操作系统的程序会被自动加载到内存中的系统区,这段区城是 RAM,故答案选 D。

30．若用户进程访问内存时产生缺页，则下列选项中，操作系统可能执行的是（　　）

I．处理越界错　　II．置换页　　III．分配内存

A．仅 I、II

B．仅 II、III

C．仅 I、III

D．I、II 和 III

【答案】B 查看答案

【解析】用户进程访问内存时缺页会发生缺页中断。发生缺页中断,系统地执行的操作可能是置换页面或分配内存。系统内没有越界的错误,不会进行越界出错处理。

31．某系统正在执行三个进程 P1、P2 和 P3，各进程的计算（CPUCPUCPU）时间和 I/OI/O 时间比例如下表所示。

进程	计算时间	I/O 时间
P1	90%	10%
P2	50%	50%
P3	15%	85%

为提高系统资源利用率，合理的进程优先级设置应（　　）

A．P1>P2>P3

B．P3>P2>P1

C．P2>P1＝P3

D．P1>P2＝P3

【答案】B 查看答案

【解析】为了合理地设置进程优先级，应该将进程的 CPU 利用时间和 I/O 时间做综合考虑，故答案选 B。

32．下列关于银行家算法的叙述中，正确的是（　　）

A．银行家算法可以预防死锁

B．当系统处于安全状态时，系统中一定无死锁进程

C．当系统处于不安全状态时，系统中一定会出现死锁进程

D．银行家算法破坏了死锁必要条件中的“请求和保持”条件

【答案】B 查看答案

【解析】银行家算法是避免死锁的方法。利用银行家算法，系统处于安全状态时没有死锁进程,故答案选 B。

33．在 OSI 参考摸型中，下列功能需由应用层的相邻层实现的是（　　）

A．对话管理

B．数据格式转换

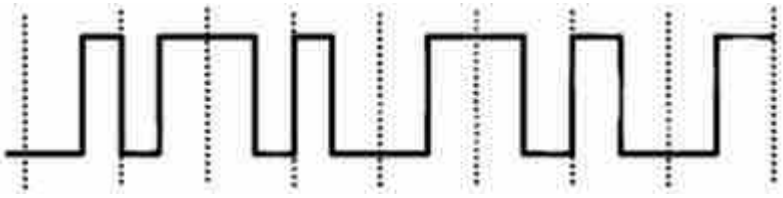
C．路由选择

D．可靠数据传输

【答案】B 查看答案

【解析】应用层的相邻层即为表示层，表示层负责管理数据的压缩、加密与解密、格式装换等，故答案为 B。

34．若下图为 10BaseT 网卡接收到的信号波形，则该比特串是（　　）



A．00110110

B．10101101

C．01010010

D．11000101

【答案】A 查看答案

【解析】以太网采用曼彻斯特编码，其将一个码元分成两个相等的间隔，前一个间隔为高电平而后一个间隔为低电平表示 1，反之则表示 0。故根据波形图，可得答案为 A。

35．主机甲通过 1 个路由器个路由器（存储转发方式）与主机乙互联，两段链路的数据传输速率均为 10Mbps,主机甲分别采用报文交换和组大小为 10kb 的分组交换向主机乙发送 1 个大小为 8Mb（1M＝106）的报文。若忽略链路传播延迟、分组头开销和拆装时间，则两种交换方式完成该报文传输所需的总时间分别为（　　）

A．800ms、1600ms

B．801ms、1600ms

C．1600ms、800ms

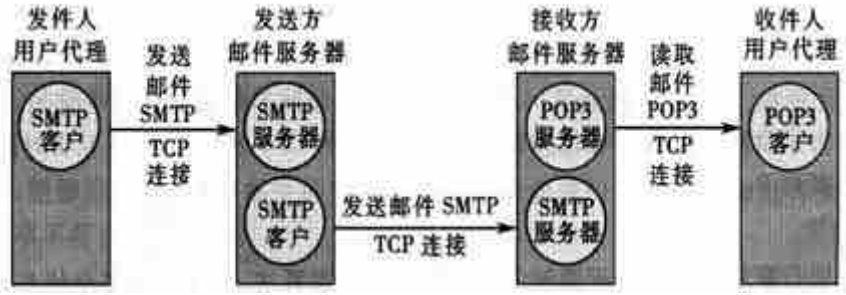
D．1600ms、801ms

【答案】D 查看答案

【解析】不进行分组时,发送一个报文的时延是 8 Mb/10 Mb/s＝800 ms,在接收端接收此报文件的时延也是 800ms□共计 1600ms。进行分组后□发送一个报文的时延是 10kb/10Mb/s＝1 ms,接收一个报文的时延也是 1 ms,但是在发送第二个报文时,第一个报文已经开始接收。共计有 800 个分组,总时间为 801 ms。

36．下列介质访问控制方法中，可能发生冲突的是（　　）
A．CDMA
B．CSMA
C．TDMAC
D．FDMA
【答案】B 查看答案
【解析】介质访向控制协议中能够发生冲突的是 CSMA 协议，答案为 B。
37.HDLC37.HDLC37.HDLC 协议对 0111110001111110 组帧后对应的比特串为（　　）
A．011111000011111010
B．01111100011111010111110
C．01111100011111010
D．01111100011111001111101
【答案】A 查看答案
【解析】HDLC 协议对比特串进行组帧时,HDLC 数据帧以位模式 0111 1110 标识每一个帧的开始和结束,因此在帧数据中凡是出现了 5 个连续的位“1”的时候,就会在输出的位流中填充一个“0”。所以答案为 A。
38．对于 100Mbps 的以太网交换机，当输出端口无排队直通（cut-throughswitching）方式转发一个以太网帧（不包括前导码）时，引入的转发延迟至少是（　　）
A．0μs
B．0.48μs
C．5.12μs
D．121.44μs
【答案】B 查看答案
【解析】直通交换方式是指以太网交换机可以在各端口间交换数据。它在输入端口检测到一个数据包时，检查该包的包头，获取包的目的地址，启动内部的动态查找表转换成相应的输出端口，在输入与输出交叉处接通，把数据包直通到相应的端口，实现交换功能。通常情况下，直通交换方式只检查数据包的包头即前 14 个字节，由于不需要考虑前导码，只需要检测目的地址的 6 B，所以最短的传输延迟是 0.48μs。
39．主机甲与乙之间已建立一个 TCP 连接，双方持续有数据传输,且无差错与丢失。若甲收到 1 个来自乙的 TCP 段，该段的序号为 1913、确认序号为 2046、有效载荷为 100 字节，则甲立即发送给乙的 TCP 段的序号和确认分别是（　　）
A．2046、2012
B．2046、2013
C．2047、2012
D．2047、2012
【答案】B 查看答案
【解析】若甲收到 1 个来自乙的 TCP 段，该段的序号 seq＝1913、确认序号 ack＝2046、有效载荷为 100 字节，则甲立即发送给乙的 TCP 段的序号 seq1＝ack＝2046 和确认序号 ack1＝seq+100＝2013,答案为 B。
40．下列关于 SMTP 协议的叙述中，正确的是（　　）
I．只支持传输 7 比特 ASCII 码内容
II．支持在邮件服务器之间发送邮件
III．支持从用户代理向邮件服务器发送邮件
IV．支持从邮件服务器向用户代理发送邮件

A．仅 I、II 和 III
B．仅 I、II 和 IV
C．仅 I、III 和 IV
D．仅 II、III 和 IV
【答案】A 查看答案
【解析】根据下图可知，SMTP 协议支持在邮件服务器之间发送邮件，也支持从用户代理向邮件服务器发送信息。SMTP 协议只支持传输 7 比特的 ASCII 码内容



二、综合应用题：41～47 小题，共 70 分。

41．已知一个整数序列 $A=(a_0,a_1\cdots\cdots a_{n-1})$，其中 $0\le a_i<n(0\le i\le n)$,若存在

$a_{p1}=a_{p2}=\cdots\cdots a_{pm}=x$ 且

$m>n/2(0\le p_k\le n,1\le k\le m)$，则称 x 为 A 的主元素。例如 $A=(0,5,5,3,5,7,5,5)$,则称 5 为主元素；又如

$A=(0,5,5,3,5,1,5,7)$ 则 A 中没有主元素。假设 A 中的 n 个元素保存在一个一维数组中，请设计一个尽可能高效的算法，找出 A 的主元素。若存在主元素，则输出该元素；否则输出-1。
要求：
（1）给出算法的基本设计思想。
（2）根据设计思想，采用 C 或 C++或 Java 语言描述算法，关键之处给出注释。
（3）说明你所设计算法的时间复杂度和空间复杂度。
解：
（1）算法的策略是从前向后扫描数组元素,标记出一个可能成为主元素的元素 Num。然后重新计数,确认 Num 是否是主元素。
算法可分为以下两步：
①选取候选的主元素：依次扫描所给数组中的每个整数，将第一个遇到的整数 Num 保存到 c 中，记录 Num 的出现次数为 1；若遇到的下一个整数仍等于 Num，则计数加 1 否则计数减 1；当计数减到 0 时，将遇到的下一个整数保存到 c 中，计数重新记为 1，开始新一轮计数，即从当前位置开始重复上述过程，直到扫描完全部数组元素。
②判断 c 中元素是否是真正的主元素，再次扫描该数组，统计 c 中元素出现的次数，若大于 n/2，则为主元素；否则，序列中不存在主元素。
（2）算法实现如下：

```
int Majority （int A[],int n）{
int i,c,count＝1;    //c 用来保存候选主元素，count 用来计数
c＝A[0];    //设置 A[0]为候选主元素
```

```
for（i＝1;i<n;i++）{ //查找候选主元素
    if（A[i]＝＝c）
count++;    //为 A 中的候选主元素计数
    eles{
if（count>0）     //处理不是候选主元素的情况
count--;
    else{ //更换候选主元素
c＝A[i];
count＝1;
    }
    }
    }
    if（count>0）
    for（i＝count＝0;i<n;i++）{    //统计候选主元素的实际出现次数
if（A[i]＝＝c）
count++;
    }
    if（count>n/2） //确认候选主元素
    return c;
    else
    return -1; //不存在主元素
}
```

（3）时间复杂度为 O（n）,空间复杂度为 O（1）。

42．（10 分）设包含 4 个数据元素的集合 S＝{“do”，“or”，“repeat”，“while”}，各元素的查找概率依次为：p1＝0.35，p2＝0.15，p3＝0.15，p4＝0.35。将 S 保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

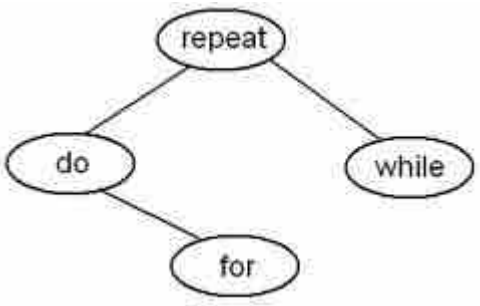
（1）若采用顺序存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

（2）若采用链式存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

解：

（1）由于个元素的查找概率不同，很自然的把查找小的位置用于存放查找概率大的元素，故要使查找长度更短，应该采用顺序存储结构，数据元素按其查找概率降序排列。这样查找成功时的平均查找长度 ASL＝0.35*1+0.35*2+0.15*3+0.15*4＝2.1。

（2）链式存储则可以采用二叉链表存储结构，构造二叉排序树，元素存储方式见下图，



采用二叉排序树的查找方法，查找成功时的平均查找长度 ASL＝
0.15*1+0.35*2+0.35*2+0.15*5＝2.0。

43．（9 分）某 32 位计算机，CPU 主频为 800MHz，Cache 命中时的 CPI 为 4，Cache 块大小为 32 字节；主存采用 8 体交叉存储方式，每个体的存储字长为 32 位、存储周期为 40ns；存储器总线宽度为 32 位，总线时钟频率为 200MHz，支持突发传送总线事务。每次读突发传送总线事务的过程包括：送首地址和命令、存储器准备数据、传送数据。每次突发传送 32 字节，传送地址或 32 位数据均需要一个总线时钟周期。请回答下列问题，要求给出理由或计算过程。

（1）CPU 和总线的时钟周期各为多少？总线的带宽（即最大数据传输率）为多少？

（2）Cache 缺失时，需要用几个读突发传送总线事务来完成一个主存块的读取？

（3）存储器总线完成一次读突发传送总线事务所需的时间是多少？

（4）若程序 BP 执行过程中，共执行了 100 条指令，平均每条指令需进行 1.2 次访存，Cache 缺失率为 5%，不考虑替换等开销，则 BP 的 CPU 执行时间是多少？

解：

（1）因为 CPU 主频为 800MHz,故 CPU 的时钟周期为：1/800MHz＝1.25ns。

总线时钟频率为 200MHz,故总线的时钟周期为：1/200MHz＝5ns。

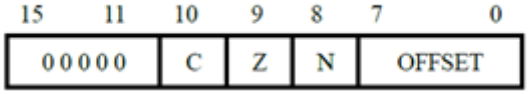
总线宽度为 32bit＝4B，故总线带宽为 4B/5ns＝800MB/s。

（2）因为 Cache 块大小为 32B，因此 Cache 缺失时需要一个读突发传送总线事务读取一个贮存块。

（3）一次读突发传送总线事务包括一次地址传送和 32B 数据传送：用 1 个总线时钟周期传输地址；每隔 40ns/8＝5ns 启动一个体工作（各进行 1 次存取），第一个体读数据花费 40ns，之后数据存取与数据传输重叠；用 8 个总线时钟周期传输数据。读突发传送总线事务时间：5ns+ 40ns +8×5ns＝85ns。

（4）BP 的 CPU 执行时间包括 Cache 命中时的指令执行时间和 Cache 缺失时带来的额外开销。即执行时间＝指令条数*CPI*时钟周期*命中率+访存次数*缺失率*缺失损失。命中时的指令执行时间：100×4×1.25ns＝500ns。指令执行过程中 Cache 缺失时的额外开销 1.2×100×5%×85ns＝510ns。BP 的 CPU 执行时间：500ns+510ns＝1010ns。

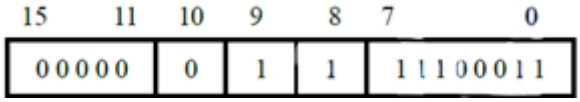
44．（14 分）某计算机采用 16 位定长指令字格式，其 CPU 中有一个标志寄存器，其中包含进位/借位标志 CF、零标志 ZF 和符号标志 NF。假定为该机设计了条件转移指令，其格式如下：



其中，00000 为操作码 OP；C、Z 和 N 分别为 CF、ZF 和 NF 的对应检测位，某测位为 1 时表示需检测对应标志，需检测的标志位中只要有一个为 1 就转移，否则就不转移，例如，若 C＝1，Z＝0，N＝1，则需检测 CF 和 NF 的值，当 CF＝1 或 NF＝1 时发生转移；OFFSET 是相对偏移量，用补码表示。转移执行时，转移目标地址为（PC）+2+2×OFFSET；顺序执行时，下条指令地址为（PC）+2。请回答下列问题。

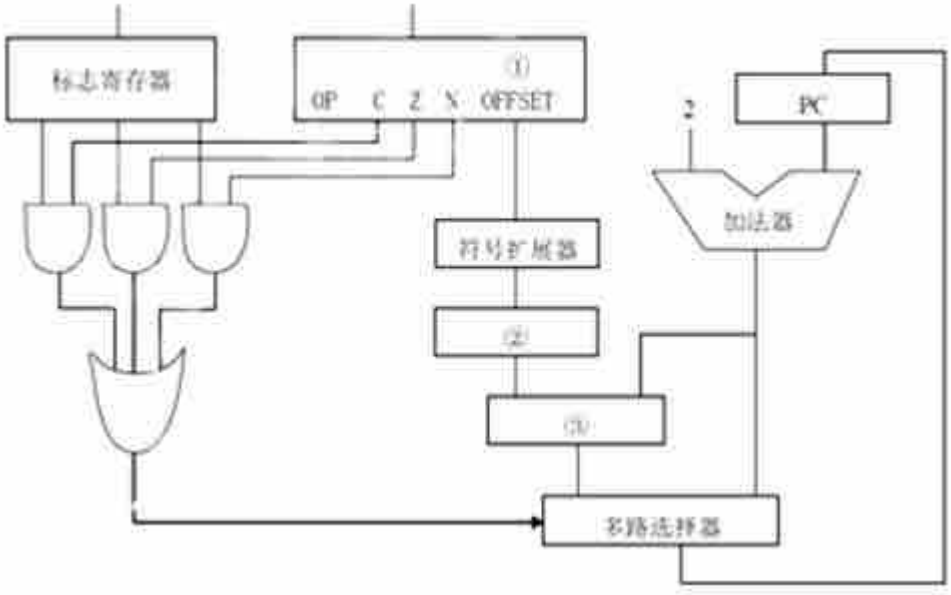
（1）该计算机存储器按字节编址，还是按字编址？该条件转移指令向后（反向）最多可跳转最多少条指令？

（2）某条件转移指令的地址为 200CH，指令内容如下图所示，若该执行时 CF＝0，ZF＝0，NF＝1，则该指令执行后 PC 的值是多少？若该指令执行时 CF＝1，ZF＝0Z，NF＝0，则该指令执行后 PC 的值又是多少？请给出计算过程。



（3）实现“无符号数比较小于等时转移”功能的指令中，C、Z 和 N 应各是什么？

（4）以下是该指令对应的数据通路示意图，要求给出中部件①～③的名称或功能说明。



解：

（1）因为指令长度为 16 位且下条指令地址为（PC）+2，故编址单位是字节。

题中给出偏移量 OFFSET 为 8 位补码，其范围为-128～127，故相对当前指令进行条件跳转，向后最多可跳转 127 条指令。

（2）指令中 C＝0，Z＝1，N＝1，故应根据 ZF 和 NF 的值来判断是否转移。当 CF＝0，ZF＝0，NF＝1 时需转移。已知指令中偏移量为 1110 0011B＝E3H，符号扩展后为 FFE3H，左移一位（乘 2）后为 FFC6H，故 PC 的值（即转移目标地址）为 200CH+2+FFC6H＝1FD4H 当 CF＝1, ZF＝0,NF＝0 时不转移。PC 的值为 200CH+2＝200EH。

（3）指令中的 C、Z 和 N 应分别设置为 C＝Z＝1，N＝0。

（4）部件①指令寄存器：用于存放当前指令；部件②移位寄存器：用于左移一位；部件③加法器：地址相加□。

45. （7 分）某博物馆最多可容纳 500 人同时参观，有一个出入口，该出入口一次仅允许个通过。参观者的活动描述如下：

cobegin

参观者进程 i：

{

…

进门；

…

参观；

…

出门；

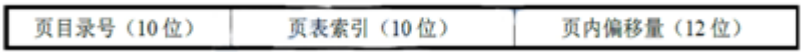
…

}

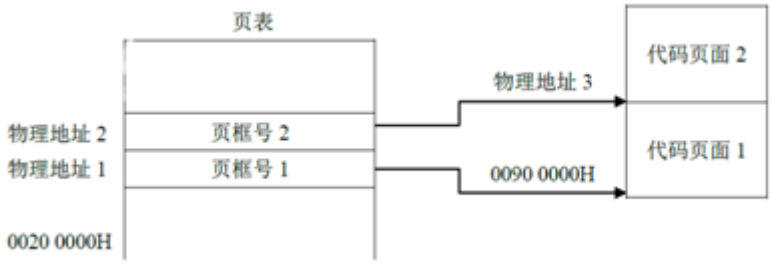
coend
请添加必要的信号量和 P、V（或 wait（　　）、signal（　　））操作，以实现上述操作过程中的互斥与同步。
要求写出完整的过程，说明信号量含义并赋初值。
解：
定义两个信号量
Semaphore empty ＝ 500;　// 博物馆可以容纳的最多人数
Semaphore mutex ＝ 1;　// 用于出入口资源的控制
cobegin
参观者进程 i:
{
……
P （ empty ）;
P （ mutex ）;
进门;
V（ mutex ）;
参观; P （ mutex ）;
出门; V（ mutex ）;
　V（ empty ）;
　……
} coend
46.（8 分）某计算机主存按字节编址，逻辑地址和物理地址都是 32 位，页表项大小为 4 字节。请回答下列问题。
（1）若使用一级页表的分存储管理方式，逻辑地址结构为：



则页的大小是多少字节？页表最大占用多少字节？
（2）若使用二级页表的分存储管理方式，逻辑地址结构为：

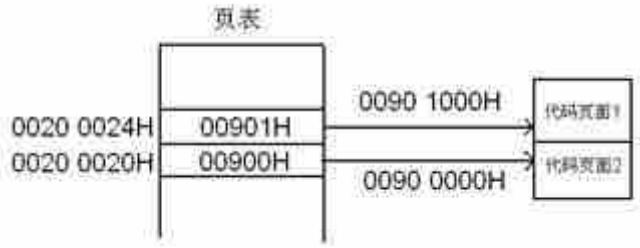


设逻辑地址为 LA，请分别给出其对应的页目录号和表索引达式。
（3）采用（1）中的分页存储管理方式，一个代码段起始逻辑地址为 00008000H，其长度为 8KB，被装载到从物理地址 00900000H 开始的连续主存空间中。页表从主存 0020 0000H 0020 0000H 开始的物理地址处连续存放，如下图所示（地址大小自下向上递增）。请计算出该代码段对应的两个页表项物理地址、这中框号以及计算出该代码段对应的两个页表项物理地址、这中框号以及计算出该代码段对应的两个页表项物理地址、这两个页表项中的框号以及代码页面 2 的起始物理地址。

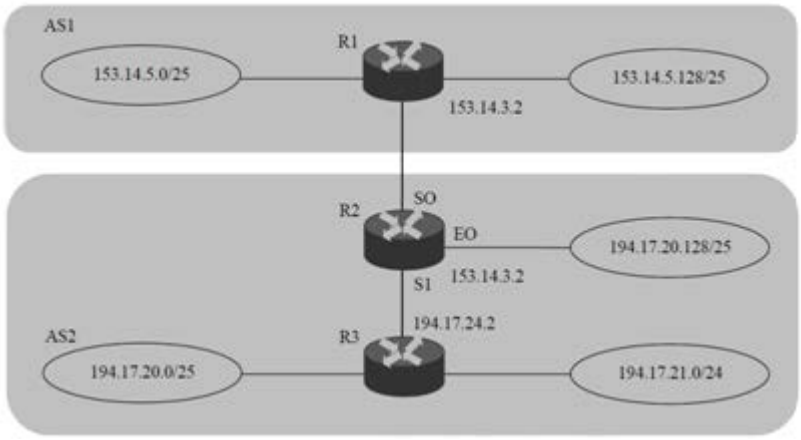


解：

（1）因为页内偏移量是 12 位□所以页大小为 4 KB
页表项数为 2 32 /4K＝2 20 □该一级页表最大为 2 20 ×4 B＝4 MB。
（2）页目录号可表示为：（（（unsigned int）（LA））>>22）&0x3FF。
页表索引可表示为：（（（unsigned int）（LA））>>12）&0x3FF。
（3）代码页面 1 的逻辑地址为 0000 8000H，表明其位于第 8 个页处，对应页表中的第 8 个页表项，所以第 8 个页表项的物理地址＝页表起始地址 + 8×页表项的字节数＝0020 0000H + 8×4 ＝ 0020 0020H，如下图所示。



47.（9 分）假设 Internet 的两个自治系统构成网络如题 47 图所示，自治系统 ASI 由路由器 R1 连接两个子网构成；自治系统 AS2 由路由器 R2、R3 互联并连接 3 个子网构成。各子网地址、R2 的接口名、R1 与 R3 的部分接口 IP 地址如题 47 图所示。请回答下列问题。



题 47 图网络拓扑结构

（1）假设路由表结构如下所示。请利用路由聚合技术，给出 R2 的路由表，要求包括到达题 47 图中所有子网的路由，且路由表中的路由项尽可能少。

目的网络	下一跳	接口

（2）若 R2 收到一个目的 IP 地址为 194.17.20.200 的 IP 分组，R2 会通过哪个接口转发该 IP 分组？
（3）R1 与 R2 之间利用哪个路由协议交换信息？该路由协议的报文被封装到哪个议的分组中进行传输？
解：
（1）在 AS1 中，子网 153.14.5.0/25 和子网 153.14.5.128/25 可以聚合为子网 153.14.5.0/24，在 AS2 中，子网 194.17.20.0/25 和子网 194.17.21.0/24 可以聚合为子网 194.17.20.0/23，但缺少 194.17.20.128/25；子网 194.17.20.128/25 单独连接到 R2 的接 口 E0。
于是可以得到 R2 的路由表如下：

目的网络	下一跳	接口
153.14.5.0/24	153.14.3.2	S0

194.17.20.0/23	194.17.24.2	S1
194.17.20.128/25		E0

（2）该 IP 分组的目的 IP 地址 194.17.20.200 与路由表中 194.17.20.0/23 和 194.17.20.128/25 两个路由表项均匹配，根据最长匹配原则，R2 将通过 E0 接口转发该 1P 分组。

（3）R1 与 R2 之间利用 BGP4（或 BGP）交换路由信息；BGP4 的报文被封装到 TCP 协议段中进行传输。

2012 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解

一、单项选择题：1～40 小题。每小题 2 分，共 80 分。下列每题给出的四个选项中，只有一个选项是最符合题目要求的。

1．求整数 n（n≥0）阶乘的算法如下，其时间复杂度是（　　）。

```
int fact( int n )
{   if ( n<=1 ) return 1;
    return n * fact( n-1 );
}
```

A．O（$\log_2 n$）

B．0（n）

C．O（$n\log_2 n$）

D．O（n^2）

【答案】B。查看答案

【解析】设 fact（n）的运行时间函数是 T（n）。

```
int fact( int n ){
    if ( n<=1 ) return 1;      //语句
    return n * fact( n-1 );    //语句
}
```

该函数中语句①的运行时间是 0（1），语句②的运行时间是 T（n-1）+O（1），其中 O（1）为乘法运算的时间。

因此，当 n≤1 时，T（n）-0（1）；当 n>1 时，T（n）＝T（n-1）+O（1）。则，T（n）＝O（1）+T（n-1）

＝2×O（1）+T（n-2）＝（n-1）×O（1）+T（1）＝n× O（1）

＝O（n）

即 fact（n）的时间复杂度为 O（n）。

2．已知操作符包括‘+’、‘-’、‘*’、‘/’、‘（’和‘）’。将中缀表达式 a+b-a*（（c+d）/ e-f）+g 转换为等价的后缀表达式 ab+acd+e / f-*-g+时，用栈来存放暂时还不能确定运算次序的操作符。若栈初始时为空，则转换过程中同时保存在栈中的操作符的最大个数是（　　）。

A．5

B．7

C．8

D．11

【答案】A。查看答案

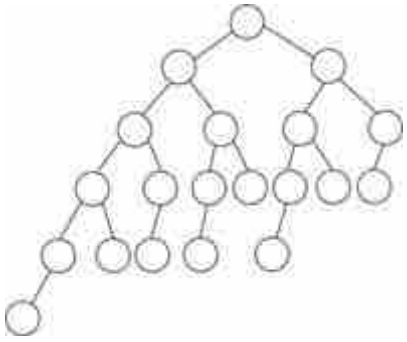
【解析】基本思想是：采用运算符栈是为了比较运算符的优先级，所有运算符必须进栈。只将大于栈顶元素优先级的运算符直接进栈，否则需要退栈栈顶运算符（先出栈的运算符先计算，同优先级的运算符在栈中的先计算）。表达式 a+b-a*（（c+d）/ e-f）+g 产生后缀表达式的过程如下表所列：

当前字符	运算符栈内容	后缀表达式	说明
a			
+	+		“+”进栈
b	+	ab	
-	-	ab+	“-”与栈顶元素“+”的优先级相同，则“+”出栈，“-”进栈
a	-	ab+a	
*	-*	ab+a	“*”的优先级大于栈顶元素“-”，则“*”进栈
（	-*（	ab+a	“（”对它之前后的运算符起隔离作用
（	-*（（	ab+a	“（”对它之前后的运算符起隔离作用
	-*（（	ab+ac	
+	-*（（+	ab+ac	“+”进栈
d	-*（（+	ab+acd	
）	-*（	ab+acd+	与其配对的左括号及其前所有运算符出栈
/	-*（/	ab+acd+	“/”进栈
e	-*（/	ab+acd+e	
-	-*（-	ab+acd+e /	“-”的优先级小于栈顶元素“/”，则“/”出栈，“-”进栈
f	-*（-	ab+acd+e / f	
）	-*	ab+acd+e / f-	与其配对的左括号及其前所有运算符出栈
+	-	ab+acd+e / f-*	“+”的优先级小于栈顶元素“*”，则“*”出栈
	+	ab+acd+e / f-*-	“+”与栈顶元素“-”的优先级相同，则“-”出栈，“+”进栈
g	+	ab+acd+e / f-*-g	
		ab+acd+e / f-*-g+	全部出栈

通过上表可以看出，显然转换过程中同时保存在栈中的操作符的最大个数是 5。

3．若一棵二叉树的前序遍历序列为 a，e，b，d，c，后序遍历序列为 b，c，d，e，a，则根结点的孩子结点（　　）。

A．只有 e
B．有 e、b
C．有 e、c
D．无法确定
【答案】A。查看答案
【解析】由题目可知，若一棵二叉树的前序遍历序列为 a，e，b，d，c，后序遍历序列为 b，c，d，e，a，其中 a 为这棵二叉树的根结点，接下来，在前序遍历的第二个结点为 e，而后序遍历的倒数第二个结点为 e，说明 a 的孩子结点只有 e。
4．若平衡二叉树的高度为 6，且所有非叶结点的平衡因子均为 1，则该平衡二叉树的结点总数为（　　）。
A．12
B．20
C．32
D．33
【答案】B。查看答案
【解析】本题题目的实际问题是，具有 6 层结点的平衡二叉树含有最少的结点数是多少。N_h 表示深度为 h 的平衡二叉树中含有的最少结点数，有 $N_0=0$，$N_1=1$，$N_2=2$……$N_h=N_{h-1}+N_{h-2}+1$
由此可得 $N_5=20$。对应的平衡二叉树如下图所示。



5．对有 2 个顶点 e 条边且使用邻接表存储的有向图进行广度优先遍历，其算法时间复杂度是（　　）。
A．0（n）
B．0（e）
C．O（n+e）
D．O（n×e）
【答案】C。查看答案
【解析】遍历图的过程实质上是对每个顶点查找其邻接点的过程。其耗费的时间则取决于所采用的存储结构。当用二维数组表示邻接矩阵图的存储结构时，查找每个顶点的邻接点所需时间为 O（n^2），其中 n 为图中顶点数。而当以邻接表作图的存储结构时，找邻接点所需时间为 0（e），其中 e 为无向图中边的数或有向图中弧的数。由此，当以邻接表作存储结构时，深度优先搜索遍历图的时间复杂度为 O（n+e）。即可得出正确答案。
6．若用邻接矩阵存储有向图，矩阵中主对角线以下的元素均为零，则关于该图拓扑序列的结论是（　　）。
A．存在，且唯一
B．存在，且不唯一不唯一
C．存在，可能不唯一

D．无法确定是否存在

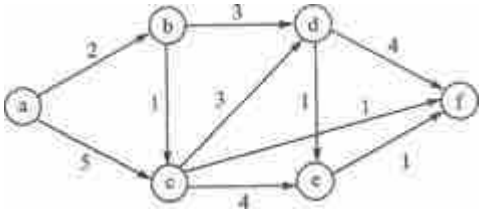
【答案】C。查看答案

【解析】图的基本应用——拓扑排序，用邻接矩阵存储有向图，矩阵中主对角线以下的元素均为零，说明该图为有向无环图，所以其拓扑序列存在，但不一定唯一，如图的邻接矩阵为



，则存在两个拓扑序列。

7．有向带权图如题 7 图所示，若采用迪杰斯特拉（Dijkstra）算法求从源点 a 到其他各顶点的最短路径，则得到的第一条最短路径的目标顶点是 b，第二条最短路径的目标顶点是 c，后续得到的其余各最短路径的目标顶点依次是（　　）。



题 7 图有向带权图

A．d, e, f

B．e，d，f

C．f，d，e

D．f，e，d

【答案】C。查看答案

【解析】本题主要考查 Dijkstra 算法的思想和解题步骤。题目执行算法过程中各步的状态如下表所示。

执行 Dijkstra 算法过程中各步的状态表，故后续目标顶点依次为 f,d,e。

顶点趟数	b	c	d	e	f	集合 S
k＝1	2（a，b）	5（a，c）				{a，b}
k＝2		（a，b，c）	（a，b，d）			{a，b，c}
k＝3			（a，b，d）	（a，b，c，e）	{a，b，c，f}	
k＝4				（a，b，c，e）	{a，b，c，f，d}	
k＝5				（a，b，d，e）	{a，b，c，f，d，e}	

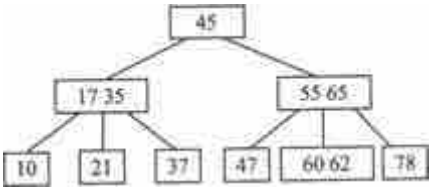
8．下列关于最小生成树的叙述中，正确的是（　　）。

Ⅰ.最小生成树的代价唯一 Ⅱ.所有权值最小的边一定会出现在所有的最小生成树中 Ⅲ.使用普里姆（Prim）算法从不同顶点开始得到的最小生成树一定相同 Ⅳ．使用普里姆算法和克鲁斯卡尔（Kruskal）算法得到的最小生成树总不相同

A．仅Ⅰ

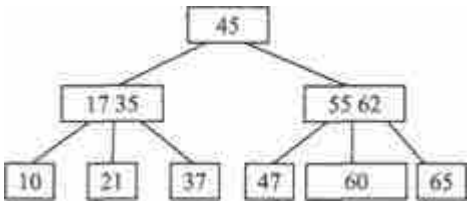
B．仅Ⅱ

C．仅Ⅰ、Ⅲ
D．仅Ⅱ、Ⅳ
【答案】A。查看答案
【解析】当图中存在相同权值的边时，其最小生成树可能是不唯一的，但最小生成树的代价一定是相同的，所以说法Ⅰ正确。从 n 个顶点的连通图中选取 n-1 条权值最小的边可能构成回路，所以说法Ⅱ错误。当某个顶点有权值相同的边，使用普里姆（Prim）算法从不同顶点开始得到的最小生成树并不一定相同，所以说法Ⅲ错误。当最小生成树不唯一时，使用普里姆算法和克鲁斯卡尔（Kruskal）算法得到的最小生成树可能相同，也可能不同，所以说法Ⅳ错误。由此可得出正确答案。
9．设有一棵 3 阶 B 树，如题 9 图所示。删除关键字 78 得到一棵新 B 树，其最右叶结点所含的关键字是（　　）。



题 9 图 3 二叉树图

A．60
B．60，62
C．62，65
D．65
【答案】D。查看答案
【解析】本题主要考查 B 树删除操作。即被删关键字所在的结点中的关键字个数等于[m/2]-1，而与该结点相邻的右兄弟（或左兄弟）结点中的关键字数目大于［m/2］-1，则需将其兄弟结点中最小（或最大）的关键字上移至双亲结点中，而将双亲结点中小于（或大于）且紧靠该上移关键字的关键字下移至被删关键字所在结点中。题目中删除关键字 78 得到一棵新 B 树如下，其最右叶结点所含的关键字是 65。



10．排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一趟排序。下列排序方法中，每一趟排序结束时都至少能够确定一个元素最终位置的方法是（　　）。
Ⅰ．简单选择排序　Ⅱ．希尔排序　Ⅲ．快速排序　Ⅳ．堆排　Ⅴ．二路归并排序
A．仅Ⅰ、Ⅲ、Ⅳ
B．仅Ⅰ、Ⅱ、Ⅲ
C．仅Ⅱ、Ⅲ、Ⅳ
D．仅Ⅲ、Ⅳ、Ⅴ
【答案】A。查看答案
【解析】其中简单选择排序、堆排序属于选择类排序，每一趟排序结束时将确定最大（或最小）关键字所在的位置。快速排序每一趟排序结束时将确定基准关键字所在的位置。希尔排序、二路归并排序每一趟排序结束时不一定能确定一个元素的最终位置。

11．对同一待排序列分别进行折半插入排序和直接插入排序，两者之间可能的不同之处是（　　）。

A．排序的总趟数

B．元素的移动次数

C．使用辅助空间的数量

D．元素之间的比较次数

【答案】D。查看答案

【解析】折半插入排序所需附加存储空间和直接插入排序相同，从时间上比较，折半插入排序仅减少了关键字间的比较次数，而记录的移动次数不变。折半插入排序的时间复杂度仍为O（n^2）,所以两者之间的不同只可能是元素之间的比较次数。

12．假定基准程序 A 在某计算机上的运行时间为 l00 秒，其中 90 秒为 CPU 时间，其余为 I/O 时间。若 CPU 速度提高 50％，I/O 速度不变，则运行基准程序 A 所耗费的时间是（　　）。

A．55 秒

B．60 秒

C．65 秒

D．70 秒

【答案】D。查看答案

【解析】CPU 速度提高 50％，即 CPU 性能提高比为 l.5，改进之后的 CPU 运行时间＝90÷1.5＝60 秒。I/O 速度不变，仍维持 l0 秒，所以运行基准程序 A 所耗费的时间为 70 秒。

13．假定编译器规定 int 和 short 类型长度分别为 32 位和 16 位，执行下列 C 语言语句：unsigned short X＝65530；unsigned int y＝X；得到 y 的机器数为（　　）。

A．00007FFAH

B．0000FFFAH

C．FFFF7FFAH

D．FFFFFFFAH

【答案】B。查看答案

【解析】X 和 y 均为无符号数，其中 X 为 16 位，y 为 32 位，将 16 位无符号数转化成 32 位无符号数，前面要补零。因为 X＝65530＝FFFAH，所以 y＝0000FFFAH。

14.float 类型（即 IEEE754 单精度浮点数格式）能表示的最大正整数是（　　）。

A．2^{126}-2^{103}

B．2^{127}-2^{104}

C．2^{127}-2^{103}

D．2^{128}-2^{104}

【答案】D。查看答案

【解析】IEEE754 单精度浮点数尾数采用隐藏位策略的原码表示，且阶码用移码表示的浮点数。规格化的短浮点数的真值为：（-1）^{S}×1.f×$2^{(E-127)}$,S 为符号位，E 的取值为 1～254，f 为 23 位；故 float 类型能表示的最大整数是 1.111^1×2（254-127）＝2^{127}×（2-2^{-23}）＝ 2^{128}-2^{104}。

15．某计算机存储器按字节编址，采用小端方式存放数据。假定编译器规定 int 和 short 型长度分别为 32 位和 16 位，并且数据按边界对齐存储。某 C 语言程序段如下：

```
struct {
        int     a;
        char    b;
        short   c;
    } record;
record.a = 273;
```

若 record 变量的首地址为 0xC008，则地址 0xC008 中内容及 record.c 的地址分别为（　　）。

A．0x00、0xC00D

B．0x00、0xCOOE

C．0x11、0xC00D

D．0x11、0xC00E

【答案】D。查看答案

【解析】32 位整数 a 需要占 4 个字节，l6 位整数 c 需要占 2 个字节，而字符数据 b 占一个字节。a＝273,转换成十六进制是 111H，采用小端方式存放数据，地址 0xC008 中的内容为 11H。由于数据按边界对齐存储，地址 0xC008～OxCOOB 中存放 a，地址 0xC00C 中存放 b，地址 0xC00D 中空闲，地址 0xC00E～0xC00F 中存放 c。

16．下列关于闪存（FlashMemory）的叙述中，错误的是（　　）。

A．信息可读可写，并且读、写速度一样快

B．存储元由 MOS 管组成，是一种半导体存储器

C．掉电后信息不丢失，是一种非易失性存储器

D．采用随机访问方式，可替代计算机外部存储器

【答案】A。查看答案

【解析】考查闪存的特性，闪存是 EEPROM 的进一步发展，可读可写，用 MOS 管的浮栅上有无电荷来存储信息，它依然是 ROM 的一种，故写速度比读速度要慢不少。闪存是一种非易失性存储器，它采用随机访问方式，现在常见的 SSD 固态硬盘就是由 flash 芯片组成的，故答案为 A。

17．假设某计算机按字编址，Cache 有 4 个行，Cache 和主存之间交换的块大小为 l 个字。若 Cache 的内容初始为空，采用 2 路组相联映射方式和 LRU 替换算法，当访问的主存地址依次为 0，4，8，2，0，6，8，6，4，8 时，命中 Cache 的次数是（　　）。

A．1

B．2

C．3

D．4

【答案】C。查看答案

【解析】Cache 有 4 个行，2 路组相联，即 Cache 被分成 2 组，每组 2 行。主存地址为 0～1、4～5、8～9 可映射到第 0 组 Cache 中，主存地址为 2～3、6～7 可映射到第 1 组 Cache 中。Cache 初始为空，采用 LRU 替换算法，当访问主存的 10 个地址依次为 0，4，8，2，0，6，8，6，4，8 时，命中 Cache 的次数共有 3 次，分别发生在第 7、8 和 10 步时。

18.某计算机的控制器采用微程序控制方式，微指令中的操作控制字段采用字段直接编码法，共有 33 个微命令，构成 5 个互斥类，分别包含 7、3、12、5 和 6 个微命令，则操作控制字段至少有（　　）。

A．5 位

B．6 位

C．15 位

D．33 位

【答案】C。查看答案

【解析】33 个微命令分成 5 个互斥类（即 5 个字段），根据每个类中微命令的多少可以分别确定字段的长度为 3、2、4、3、3 位，又因为采用直接编码方式，所以它们之和 3+2+4+3+3＝15 也就是操作控制字段的位数。

19．某同步总线的时钟频率为 l00MHz，宽度为 32 位，地址／数据线复用，每传输一个地址或数据占用一个时钟周期。若该总线支持突发（猝发）传输方式，则一次“主存写”总线事务传输 l28 位数据所需要的时间至少是（　　）。）。

A．20ns

B．40ns

C．50ns

D．80ns

【答案】C。查看答案

【解析】总线的时钟频率为 l00MHz，则时钟周期为 10ns。数据是 128 位，总线宽度是 32 位，所以需要 4 个时钟周期，而传输地址还需要一个周期，所以传输一个 128 位的数据至少需要 5 个时钟周期，所以至少需要 10ns*5＝50ns。

20．下列关于 USB 总线特性的描述中，错误的是（　　）。

A．可实现外设的即插即用和热插拔

B．可通过级联方式连接多台外设

C．是一种通信总线，可连接不同外设

D．同时可传输 2 位数据，数据传输率高

【答案】D。查看答案

【解析】USB 总线即通用串行总线，它的特点有：（1）即插即用；（2）热插拔；（3）有很强的链接能力能将所有外设链接起来，且不损失带宽；（4）有很好的可扩展性；（5）高速传输，速度可达 480Mbps。所有 A，B，C 都符合 USB 总线的特点。对于选项 D,USB 是串行总线，不能同时传输两位数据，所以答案为 D。

21．下列选项中，在 I／O 总线的数据线上传输的信息包括（　　）。

Ⅰ.I／O 接口中的命令字　Ⅱ．I／0 接口中的状态字　Ⅲ．中断类型号

A．仅Ⅰ、Ⅱ

B．仅Ⅰ、Ⅲ

C．仅Ⅱ、Ⅲ

D．Ⅰ、Ⅱ、Ⅲ

【答案】D。查看答案

【解析】在 I／O 总线的数据线上传输的信息包括 I／O 接口中的命令字、状态字以及真正的数据，而中断类型号也是通过数据线传输的。

22．响应外部中断的过程中，中断隐指令完成的操作，除保护断点外，还包括（　　）。

Ⅰ．开关中断　Ⅱ．保存通用寄存器的内容　Ⅲ．形成中断服务程序入口地址并送 PC

A．仅Ⅰ、Ⅱ

B．仅Ⅰ、Ⅲ

C．仅Ⅱ、Ⅲ

D．Ⅰ、Ⅱ、Ⅲ

【答案】B。查看答案

【解析】中断隐指令完成的操作有 3 个：①保存断点；②关中断；③引出中断服务程序（形成中断服务程序入口地址并送 PC）。而保存通用寄存器内容的操作是由软件来实现，不是由中断隐指令实现的。

23．下列选项中，不可能在用户态发生的事件是（　　）。

A．系统调用

B．外部中断

C．进程切换

D．缺页

【答案】C。查看答案

【解析】我们在学习操作系统中知道，任何一个进程在现代操作系统中为了共享和保护，设定了用户态和内核态（可以通过设置软、硬件标志位来实现），在用户态运行用户的程序，在内核运行系统的程序。所以，从选项来看，系统调用可以在任何态发生，用户可以发起系统调用，系统也可以；外部中断是不可控的，也会在任何时刻发生，缺页的发生也是不可控的，可以发生在用户代码之间；而进程切换却不会在用户态发生。我们可以考虑一下情形，进程切换是在什么时候发生的，进程切换前必定运行的是进程调度，只有进程调度选择了下一次被调度的进程，进程切换才可以进行。进程调度是 scheduler，进程切换是 dispather，这体现了现代操作系统策略与机制分离的设计思想。所以，进程切换必定不会在用户态发生（所谓发生指其起始的源头时刻），必定是在内核态（进程调度）发生的。

24．中断处理和子程序调用都需要压栈以保护现场，中断处理一定会保存而子程序调用不需要保存其内容的是（　　）。

A．程序计数器

B．程序状态字寄存器

C．通用数据寄存器

D．通用地址寄存器

【答案】B。查看答案

【解析】中断处理与子程序调用最大的区别是中断处理程序与正在运行的进程可能无关，而子程序调用与正在运行的进程有关。中断是要打断处理器的正常工作次序，并要求其去处理某一事件的一种常用手段。因此，除了要保护当前程序的地址，计数器（指针）和数据寄存器以外，还需要保存程序状态字。子程序调用是与当前进程有关，是正在运行的程序有意安排执行的，这一类调用发生的时间以及位置具有确定性，处于同一个进程内，因此不需要保存程序状态字。所以中断处理和子程序调用不同的区别是中断处理程序必定会保存程序状态字寄存器。

25．下列关于虚拟存储的叙述中，正确的是（　　）。

A．虚拟存储只能基于连续分配技术

B．虚拟存储只能基于非连续分配技术

C．虚拟存储容量只受外存容量的限制

D．虚拟存储容量只受内存容量的限制

【答案】D。查看答案

【解析】所谓虚拟存储，是指运行的进程不必全部装入内存，只需要部分装入便可以开始运行的一种技术，在运行过程中，当所需要的代码部分不在内存时，通过一种技术（例如缺页中断技术），将所需要的页面调入内存，从而继续运行。虚拟存储可以在较少的内存中运行较大的程序。但是需要有较大的外存以及相应的软、硬件机制配合才能实现。虚拟存储器可以连续分配也可以非连续分配，虚拟存储器和外存大小没有关系，所以选项中的 A,B,C 都是错误的，所以答案是 D 项。

26．操作系统的 I / O 子系统通常由四个层次组成，每一层明确定义了与邻近层次的接口。其合理的层次组织排列顺序是（　　）。

A．用户级 I / O 软件、设备无关软件、设备驱动程序、中断处理程序

B．用户级 I / O 软件、设备无关软件、中断处理程序、设备驱动程序

C．用户级 I / O 软件、设备驱动程序、设备无关软件、中断处理程序

D．用户级 I / O 软件、中断处理程序、设备无关软件、设备驱动程序

【答案】A。查看答案

【解析】对于一次设备的调用，操作系统为用户准备了系统调用的接口，当用户使用设备时，首先在用户程序中发起一次系统调用，操作系统的设备无关层软件接到该调用请求后调用处理程序进行处理，根据调用格式和形参，再转到相应的设备驱动程序去处理；大部分设备在运行时是需要时间的，所以设备驱动程序会以中断方式驱动设备，即设置好控制寄存器参数和中断向量等参数后阻塞自己；当设备准备好或所需数据到达后设备硬件发出中断，设备驱动程序唤醒，将数据按上述调用顺序逆向回传到用户程序中，或继续驱动设备执行下一条指令。因此，I/O 软件从上到下分为四个层次：用户层、与设备无关的软件层、设备驱动程序以及中断处理程序。

27．假设 5 个进程 P0、Pl、P2、P3、P4 共享三类资源 Rl、R2、R3，这些资源总数分别为 l8、6、22。T_0时刻的资源分配情况如题 27 表所示，此时存在的一个安全序列是（　　）。

题 27 表资源分配情况表

	已分配资源			资源最大需求		
进程	R1	R2	R3	R1	R2	R3
PO	3	2	3	5	5	10
P1	4	O	3	5	3	6
P2	4	O	5	4	O	11
P3	2	O	4	4	2	5
P4	3	1	4	4	2	4

A．P0，P2，P4，Pl，P3

B．Pl，P0，P3，P4，P2

C．P2，Pl，P0，P3，P4

D．P3，P4，P2，Pl，P0P0

【答案】D。查看答案

【解析】典型的死锁避免算法、银行家算法的应用。本题的题型与 2011 年的 27 题相似。银行家算法是操作系统中的一个重点知识单元，考生对此应该非常熟悉，本题并无难点。分析一下下表，可以看到，P3，P4，P2，Pl，P0 运行是可以的。

	已分配资源			尚需资源			可用资源		
进程	R1	R2	R3	R1	R2	R3	R1	R2	R3
P3	2	O	4	2	2	1	2	3	3
P4	3	1	4	l	l	O	4	3	7
P2	4	O	5	O	0	6	7	4	11
P1	4	0	3	1	3	3	11	4	16
PO	3	2	3	2	3	7	15	4	19

本题也可以排除法，T_0时刻可用资源是 R1，R2，R3 分别为 2,3,3，此时刻，P0 需要 R1，R2，R3 分别为 2,3,7，故排除 A，P1 需要 R1，R2，R3 分别为 1,3,3，P2 还需要资源 R1，R2，R3 分别为 0，0，6，故 C 排除，P3 需要 R1，R2，R3 分别为 2,2,1。所以正确答案在

B，D 之间。看 B 选项，P1 之后的可用资源 R1，R2，R3 分别变为 6, 3，6，而 P0 尚需资源 2,3,7，故 B 方案行不通。因而最终答案只有 D 项。

28．若一个用户进程通过 read 系统调用读取一个磁盘文件中的数据，则下列关于此过程的叙述中，正确的是（　　）。

Ⅰ．若该文件的数据不在内存，则该进程进入睡眠等待状态；Ⅱ．请求 read 系统调用会导致 CPU 从用户态切换到核心态；Ⅲ．read 系统调用的参数应包含文件的名称

A．仅Ⅰ、Ⅱ

B．仅Ⅰ、Ⅲ

C．仅Ⅱ、Ⅲ

D．Ⅰ、Ⅱ和Ⅲ

【答案】A 查看答案

【解析】对于Ⅰ，当所读文件的数据不再内存时，产生中断（缺页中断、缺段中断），原进程进入睡眠等待状态（阻塞状态），直到所需数据从外村调入内存后，将该进程唤醒，使其变为就绪状态。对于Ⅱ，read 系统调用 CPU 将从用户态切换到核心态，从而获取操作系统提供的服务。对于Ⅲ，在操作系统中，要读一个文件首先要 open 系统调用将该文件打开。Open 系统调用的参数需要包含文件的路径名与文件名，而 read 系统调用只需使用 open 返回的文件描述符，并不使用文件名作为参数。Read 系统调用要求用户提供三个输入参数：①文件描述符；②buf 缓冲区首址；③传送的字节数 n。read 系统调用的功能是试图从 fd 所指示的文件中读入 n 个字节的数据，并将它们送至由指针 buf 所指示的缓冲区中。

29．一个多道批处理系统中仅有 Pl 和 P2 两个作业，P2 比 Pl 晚 5ms 到达。它们的计算和 I／0 操作顺序如下：P1：计算 60ms，I／O 80ms，计算 20ms；P2：计算 120ms，I／O 40ms，计算 40ms 若不考虑调度和切换时间，则完成两个作业需要的时间最少是（　　）。

A．240ms

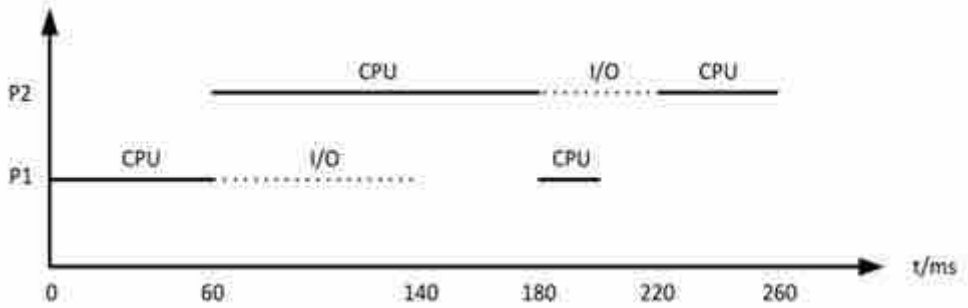
B．260ms

C．340ms

D．360ms

【答案】B。查看答案

【解析】考查处理系统的性能计算，由于 P2 比 P1 晚 5ms 到达，P1 先占用 CPU,根据 P1 和 P2 的执行过程，作业运行的甘特图如下所示，故答案为 B。



30．若某单处理器多进程系统中有多个就绪态进程，则下列关于处理机调度的叙述中，错误的是（　　）。

A．在进程结束时能进行处理机调度

B．创建新进程后能进行处理机调度

C．在进程处于临界区时不能进行处理机调度

D．在系统调用完成并返回用户态时能进行处理机调度

【答案】C。查看答案

【解析】对于 A、B、D 显然是可以进行处理机调度的，对于 C，当进程处于临界区时，只要不破坏临界资源的使用规则，是不会影响处理机调度的，比如，通常访问临界资源可能是慢速的外设（如打印机），如果在进程访问打印机时，不能处理机调度，那么系统的性能将是非常低的。几种不进行处理机调度的情况如下：①在处理机中断的过程中；②进程在操作系统内核程序临界区中；③其他需要完全屏蔽中断的原子操作过程中。

31．下列关于进程和线程的叙述中，正确的是（　　）。

A．不管系统是否支持线程，进程都是资源分配的基本单位

B．线程是资源分配的基本单位，进程是调度的基本单位

C．系统级线程和用户级线程的切换都需要内核的支持

D．同一进程中的各个线程拥有各自不同的地址空间

【答案】A。查看答案

【解析】利用排除法来确定正确答案：“线程是资源分配的基本单位，进程是调度的基本单位”这句话说反了，明显错误。“系统级线程和用户级线程的切换都需要内核的支持”也不正确，因为用户级线程的切换由用户编写的 RuntimeSystem 执行的，内核并不感知。“同一进程中的各个线程拥有各自不同的地址空间”明显错误，引入线程的目的就是为了同一进程的所有线程能共享进程的地址空间，故“不管系统是否支持线程，进程都是资源分配的基本单位”是正确的。

32．下列选项中，不能改善磁盘设备 I / O 性能的是（　　）。

A．重排 I / 0 请求次序

B．在一个磁盘上设置多个分区

C．预读和滞后写

D．优化文件物理块的分布

【答案】B。查看答案

【解析】磁盘 I / O 性能主要是指其读写速度。相对而言，磁盘的 I / O 性能是计算机性能提高的一个瓶颈。“重排 I / O 请求次序”可以优化磁臂调度的算法，减少读写时间，故正确；“预读和滞后写”是利用内存作为磁盘的缓存，使得对磁盘的访问变为对内存的访问，也可以在总体上提高其性能；“优化文件物理块的分布”减少磁臂调度和旋转调度的等待时间，也可以提高磁盘 I / O 性能，而磁盘分区仅在磁盘空间的组织上进行划分，对磁盘 I / O 性能的提升没有什么帮助，是不能改善磁盘设备 I / O 性能的，故答案为 B。

33．在 TCP / IP 体系结构中，直接为 ICMP 提供服务的协议是（　　）。

A．PPP

B．IP

C．UDP

D．TCP

【答案】B。查看答案

【解析】首先明确 ICMP 是网络层的协议，由于服务必须是下一层向上一层提供服务的，因此选项 C 项中的 UDP 和选项 D 项中的 TCP 属于传输层，在网络层上面，所以显然错误，而 PPP 协议是广域网数据链路层协议，直接为网络层，也就是 IP 层提供服务，ICMP 协议是封装在网络层，因此 PPP 不能直接为 ICMP 提供服务，ICMP 报文直接封装在 IP 分组中，故答案是 B。

34．在物理层接口特性中，用于描述完成每种功能的事件发生顺序的是（　　）。

A．机械特性

B．功能特性

C．过程特性

D．电气特性

【答案】C。查看答案

【解析】物理层的主要任务描述为确定与传输媒体接口的一些特性；机械特性：主要定义物理连接的边界点，即接插装置；电气特性：规定传输二进制位时，线路上信号的电压高低、阻抗匹配、传输速率和距离限制；功能特性：主要定义各条物理线路的功能；规程特性：主要定义各条物理线路的工作规程和时序关系。而从题干可以分析描述事件先后顺序的就是规程，也就是过程特性，答案是 C。

35．以太网的 MAC 协议提供的是（　　）。

A．无连接不可靠服务

B．无连接可靠服务

C．有连接不可靠服务

D．有连接可靠服务

【答案】A。查看答案

【解析】考查以太网 MAC 协议，考虑到局域网信道质量好，以太网采取了两项重要的措施以使通信更简洁：①采用无连接的工作方式；②不对发送的数据帧进行编号，也不要求对方发回确认。因此，以太网提供的服务是不可靠的服务，即尽最大努力交付，差错的纠正由高层完成。

36.两台主机之间的数据链路层采用后退 N 帧协议（GBN）传输数据，数据传输速率为 l6kbps，单向传播时延为 270ms，数据帧长度范围是 128～512 字节，接收方总是以与数据帧等长的帧进行确认。为使信道利用率达到最高，帧序号的比特数至少为（　　）。

A．5

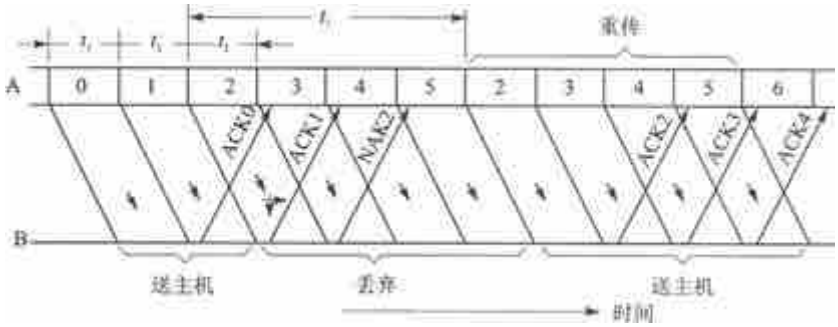
B．4

C．3

D．237

【答案】B。查看答案

【解析】GBN 的工作原理如下图所示，本题求解的是发送一个帧到接收到这个帧的确认期间最多可以发送多少数据帧，要尽可能多发送帧，应以短的数据帧计算，注意帧的单位是字节，因此首先计算出发送一帧的时间 t1＝128×8/16kbps＝64ms，故发送一帧到收到确认为止的总时间为;64+270*2+64＝668ms，这段时间总共可以发送 668/64＝10.4（帧），为了保证发送帧序号和确认帧序号在此期间不重复，因此帧序号的比特数至少为 4，答案为 B



37．下列关于 IP 路由器功能的描述中，正确的是（　　）。

Ⅰ．运行路由协议，设置路由表；Ⅱ．监测到拥塞时，合理丢弃 IP 分组；Ⅲ．对收到的 IP 分组头进行差错校验，确保传输的 IP 分组不丢失；Ⅳ．根据收到的 IP 分组的目的 IP 地址，将其转发到合适的输出线路上。

A．仅Ⅲ、Ⅳ

B．仅Ⅰ、Ⅱ、Ⅲ

C．仅Ⅰ、Ⅱ、Ⅳ
D．Ⅰ、Ⅱ、Ⅲ、Ⅳ
【答案】C。查看答案
【解析】路由器的主要功能是路由和转发，因此Ⅰ和Ⅳ是正确的，而针对Ⅱ和Ⅲ，可以从ICMP 协议的差错控制出发，注意检测到拥塞时，合理丢弃 IP 分组，并回传 ICMP 源抑制报文，Ⅱ是正确的，而Ⅲ对收到的 IP 分组头进行差错校验，确保传输的 IP 分组不丢失，差错校验是正确的，但网络层不保证 IP 分组不丢失，也就是不可靠的，因此Ⅲ的说法错误，正确的说法仅Ⅰ、Ⅱ、Ⅳ，因此答案是 C。
38.ARP 协议的功能是（　　）。
A．根据 IP 地址查询 MAC 地址
B．根据 MAC 地址查询 IP 地址
C．根据域名查询 IP 地址
D．根据 IP 地址查询域名
【答案】A。查看答案
【解析】ARP 协议是网络层协议，因此只能和传输层和数据链路层有关系，从这一点出发，域名是应用层的范畴，选项 C 和 D 是不正确的，根据 MAC 地址查询 IP 地址是 RARP 协议的功能，因此进而得出正确答案是 A。
39．某主机的 IP 地址为 180.80.77.55，子网掩码为 255.255.252.0。若该主机向其所在子网发送广播分组，则目的地址可以是（　　）。
A．180.80.76.0
B．180.80.76.255
C．180.80.77.255
D．180.80.79.255
【答案】D。查看答案
【解析】IPv4 地址中的特殊地址，直接广播地址，也就是把主机位全部设置为 1，这里 77 的二进制是 01001101，子网掩码 252 的二进制是 11111100，由此可以看到 77 的前 6 位作为子网位，后四位作为主机位，由此可以知道其广播地址是 l80.80.01001111.255，也就是 l80.80.79.255，因此答案是 D。
40．若用户 l 与用户 2 之间发送和接收电子邮件的过程如题 40 图所示，则图中①、②、③阶段分别使用的应用层协议可以是（　　）。



题 40 图电子邮件发送接收示意图

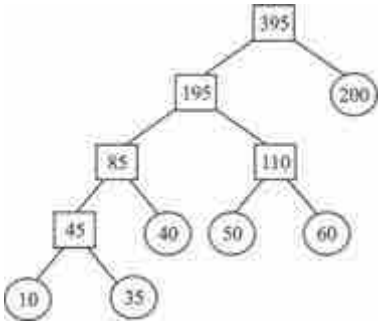
A．SMTP、SMTP、SMTP
B．POP3、SMTP、POP3
C．POP3、SMTP、SMTP
D．SMTP、SMTP、POP3
【答案】D。查看答案
【解析】题中电子邮件的工作过程如下：
①用户 l 调用用户代理来编辑要发送的邮件，用户代理用 SMTP 将邮件传送给用户 1 的发送端邮件服务器。
②发送端邮件服务器也就是用户 1 的邮件服务器将邮件放入邮件缓存队列中，等待发送。

③运行在发送端邮件服务器的 SMTP 客户进程，发现在邮件缓存中有待发送的邮件，就向运行在接收端邮件服务器也就是用户 2 的邮件服务器的 SMTP 服务器进程发起 TCP 连接建立。当 TCP 连接建立后，SMTP 客户进程开始向远程的 SMTP 服务器发送邮件。当所有的待发邮件发完了，SMTP 就关闭所建立的 TCP 连接。
④运行在接收端邮件服务器中的 SMTP 服务器进程收到邮件后，将邮件放人收信人的用户邮箱中，等待收信人在他方便时进行读取。收信人在打算收信时，调用用户代理，使用 POP 协议将自己的邮件从接收端邮件服务器的用户邮箱中取回（如果邮箱中有来信的话）。
因此题中 1，2，3 阶段分别使用的应用层协议可以是 SMTP，SMTP，POP3，因此答案是 D。
SMTP 采用“推”的通信方式，用于用户代理向邮件服务器发送邮件、以及邮件服务器之间发送邮件。POP3 采用“拉”的通信方式，用于用户从目的邮件服务器上读取邮件。

二、综合应用题：41"--47 小题。共 70 分。

41．（10 分）设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。
（1）给出完整的合并过程，并求出最坏情况下比较的总次数。
（2）根据你的合并过程，描述 n（n≥2）个不等长升序表的合并策略，并说明理由。

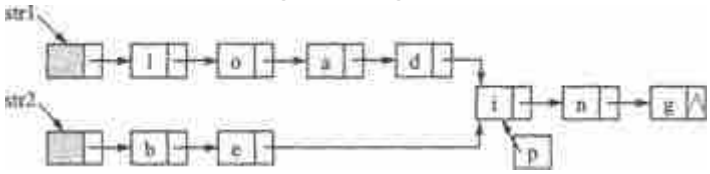
解：
（1）6 个表的合并顺序如下图所示。



对应于合并过程的哈夫曼树
根据上图中的哈夫曼树，6 个序列的合并过程为：
第 1 次合并：表 A 与表 B 合并，生成含 45 个元素的表 AB；
第 2 次合并：表 AB 与表 C 合并，生成含 85 个元素的表 ABC；
第 3 次合并：表 D 与表 E 合并，生成含 ll0 个元素的表 DE；
第 4 次合并：表 ABC 与表 DE 合并，生成含 l95 个元素的表 ABCDE；
第 5 次合并：表 ABCDE 与表 F 合并，生成含 395 个元素的最终表。
由于合并两个长度分别为 m 和 n 的有序表，最坏情况下需要比较 m+n-1 次，故最坏情况下比较的总次数计算如下：
第 1 次合并：最多比较次数＝10+35-1＝44；
第 2 次合并：最多比较次数＝45+40-l＝84；
第 3 次合并：最多比较次数＝50+60-1＝109；
第 4 次合并：最多比较次数＝85+110-1＝194；
第 5 次合并：最多比较次数＝195+200-1＝394；比较的总次数最多为：44+84+109+194+394＝825。
（2）各表的合并策略是：在对多个有序表进行两两合并时，若表长不同，则最坏情况下总的比较次数依赖于表的合并次序。可以借用哈夫曼树的构造思想，依次选择最短的两个表进行合并，可以获得最坏情况下最佳的合并效率。

解析：本题具有较强的综合性，主要考查了构造哈夫曼树的算法思想和过程、归并排序的过程等。

42．（13 分）假定采用带头结点的单链表保存单词，当两个单词有相同的后缀时，则可共享相同的后缀存储空间。例如，“loading”和“being”的存储映像如题 42 图所示。



题 42 图存储映像示意图

设 strl 和 str2 分别指向两个单词所在单链表的头结点，链表结点结构为[data，next]，请设计一个时间上尽可能高效的算法，找出由 strl 和 str2 所指的两个链表共同后缀的起始位置（如图中字符 i 所在结点的位置 p）。要求：

（1）给出算法的基本设计思想。

（2）根据设计思想，采用 C 或 C++或 JAVA 语言描述算法，关键之处给出注释。

（3）说明你所设计算法的时间复杂度。

解：

（1）算法的基本设计思想：

①分别求出 strl 和 str2 所指的两个链表的长度 m 和 n；

②将两个链表以表尾对齐：令指针 p、q 分别指向 strl 和 str2 的头结点，若 m> n，则使 p 指向链表中的第 n+1 个结点；若 m<n，则使 q 指向链表中的第 m+1 个结点，即使指针 p 和 q 所指的结点到表尾的长度相等。

③反复将指针 p 和 q 同步向后移动，并判断它们是否指向同一结点。若 p 和 q 指向同一结点，则该点即为所求的共同后缀的起始位置。

（2）用 C 语言算法描述如下：

```
LinkNode *Find_1st_Common(LinkList strl,LinkList str2){
    int len1=Length(strl),len2=Length(str2);
    LinkNode *p,*q;
    for(p=strl;len1>len2;len1--)     //使 p 指向的链表与 q 指向的链表等长
    p=p->next;
    for(q=str2;len1<len2;len2--)     //使 q 指向的链表与 p 指向的链表等长
    q=q->next;
    while(p->next!=NULL&&p->next!=q->next){//查找共同后缀起始点
    p=p->next;                       //两个指针同步向后移动
    q=q->next;
    }
    return p->next;                  //返回共同后缀的起始点
}
```

（3）参考答案的时间复杂度为：0（m+n）或 O（max（m，n））。其中 m、n 分别为两个链表的长度。

43．（11 分）假定某计算机的 CPU 主频为 80MHz，CPI 为 4，并且平均每条指令访存 1.5 次，主存与 Cache 之间交换的块大小为 168，Cache 的命中率为 99%，存储器总线宽度为 32 位。请回答下列问题。

（1）该计算机的 MIPS 数是多少?平均每秒 Cache 缺失的次数是多少?在不考虑 DMA 传送的情况下，主存带宽至少达到多少才能满足 CPU 的访存要求?

（2）假定在 Cache 缺失的情况下访问主存时，存在 0.0005％的缺页率，则 CPU 平均每秒产生多少次缺页异常?若页面大小为 4KB，每次缺页都需要访问磁盘，访问磁盘时 DMA 传送采用周期挪用方式，磁盘 I / O 接口的数据缓冲寄存器为 32 位，则磁盘 I / O 接口平均每秒发出的 DMA 请求次数至少是多少?

（3）CPU 和 DMA 控制器同时要求使用存储器总线时，哪个优先级更高?为什么?

（4）为了提高性能，主存采用 4 体交叉存储模式，工作时每 l/4 个存储周期启动一个体。若每个体的存储周期为 50ns，则该主存能提供的最大带宽是多少?

解：

（1）平均每秒 CPU 执行的指令数为：80M/4＝20M，故 MIPS 数为 20；

平均每秒 Cache 缺失的次数为：20M×1.5×（1-99％）＝300000；

当 Cache 缺失时，CPU 访问主存，主存与 Cache 之间以块为单位传送数据，此时，主存带宽为：16B×300000/s＝4．8MB/s。在不考虑 DMA 传输的情况下，主存带宽至少达到 4．8MB/s 才能满足 CPU 的访存要求。

（2）平均每秒钟“缺页”异常次数为：300000×0.0005％＝l.5 次；

因为存储器总线宽度为 32 位，所以，每传送 32 位数据，磁盘控制器发出一次 DMA 请求，故平均每秒磁盘 DMA 请求的次数至少为：1.5×4KB/4B＝1.5K＝1536。

（3）CPU 和 DMA 控制器同时要求使用存储器总线时，DMA 请求优先级更高；因为若 DMA 请求得不到及时响应，I / O 传输数据可能会丢失。

（4）4 体交叉存储模式能提供的最大带宽为：4×4B/50ns＝320MB/s。

44．（12 分）某 16 位计算机中，带符号整数用补码表示，数据 Cache 和指令 Cache 分离。题 44 表给出了指令系统中部分指令格式，其中 Rs 和 Rd 表示寄存器，mem 表示存储单元地址，（X）表示寄存器 X 或存储单元 X 的内容。

表 指令系统中部分指令格式

名称	指令的汇编格式	指令功能
加法指令	ADD Rs，Rd	（Rs）+（Rd）→Rd
算术 / 逻辑左移	SHLR d	2*（Rd）→Rd
算术右移	SHRR d	（Rd）/ 2→Rd
取数指令	LOAD RD，mem	（mem）→Rd
存数指令	STORE Rs，mem	（Rs）→mem

该计算机采用 5 段流水方式执行指令，各流水段分别是取指（IF）、译码 / 读寄存器（ID）、执行 / 计算有效地址（EX）、访问存储器（M）和结果写回寄存器（WB），流水线采用“按序发射，按序完成”方式，没有采用转发技术处理数据相关，并且同一个寄存器的读和写操作不能在同一个时钟周期内进行。请回答下列问题。

（1）若 int 型变量 x 的值为-513，存放在寄存器 Rl 中，则执行指令“SHR Rl”后，Rl 的内容是多少?（用十六进制表示）

（2）若某个时间段中，有连续的 4 条指令进入流水线，在其执行过程中没有发生任何阻塞，则执行这 4 条指令所需的时钟周期数为多少?

（3）若高级语言程序中某赋值语句为 x＝a+b，x、a 和 b 均为 int 型变量，它们的存储单元地址分别表示为[x]、[a]和[b]。该语句对应的指令序列及其在指令流水线中的执行过程如题 44 图所示。则这 4 条指令执行过程中，I_3的 ID 段和 I_4的 IF 段被阻塞的原因各是什么?

```
I1    LOAD    R1，[a]
I2    LOAD    R2，[b]
I3    ADD     R1，R2
I4    STORE   R2，[x]
```

	时间单元													
指令	1	2	3	4	5	6	7	8	9	10	11	12	13	14
I_1	IF	ID	EX	M	WB									
I_2		IF	ID	EX	M	WB								
I_3			IF				ID	EX	M	WB				
I_4							IF				ID	EX	M	WB

图 指令序列及其执行过程示意图

（4）若高级语言程序中某赋值语句为 x＝2*x+a，x 和 a 均为 unsigned int 类型变量，它们的存储单元地址分别表示为[x]、[a]，则执行这条语句至少需要多少个时钟周期?要求模仿题 44 图画出这条语句对应的指令序列及其在流水线中的执行过程示意图。

解：

（1）x 的机器码为[x]补＝1111 1101 1111B,即指令执行前（R1）＝FDFFH，右移 1wei 后为 1111 1110 1111 1111B，即指令执行后（R1）＝FEFFH。

（2）至少需要 5+（4-1）＝8 个时钟周期数。

（3） I_3的 ID 段被阻塞的原因：因为 I_3与 I_1和 I_2都存在数据相关，需等到 I_1和 I_2将结果写回寄存器后，I_3才能读寄存器内容，所以 l_3的 ID 段被阻塞。

I_4的 IF 段被阻塞的原因：因为 I_4的前一条指令 I_3在 ID 段被阻塞，所以 I_4的 IF 段被阻塞。

（4）x＝2*x+a 对应的指令序列为：

```
I1  LOADRl，  [x]
I2   LOADR2，  [a]
I3  SHLRl    R1    //或者 ADD    R1，R1
I4  ADD    R1, R2
I5     STORER2，  [x]
```

这 5 条指令在流水线中的执行过程如下图所示，执行 x＝2*x+a 语句最少需要 l7 个时钟周期。

	时间单元																
指令	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
I1	IF	ID	EX	M	WB												
I2		IF	ID	EX	M	WB											
I3			IF			ID	EX	M	WB								
I4						IF				ID	EX	M	WB				
I5										IF				ID	EX	M	WB

45.（7 分）某请求分页系统的局部页面置换策略如下：系统从 0 时刻开始扫描，每隔 5 个时间单位扫描一轮驻留集（扫描时间忽略不计），本轮没有被访问过的页框将被系统回收，并放入到空闲页框链尾，其中内容在下一次被分配之前不被清空。当发生缺页时，如果该页曾被使用过且还在空闲页框链表中，则重新放回进程的驻留集中；否则，从空闲页框链表头部取出一个页框。假设不考虑其他进程的影响和系统开销，初始时进程驻留集为空。目前系

统空闲页框链表中页框号依次为 32、15、21、41。进程 P 依次访问的<虚拟页号，访问时刻>是：<1，1>、<3，2>、<0，4>、<0，6>、<1，11>、<0，13>、<2，14>。请回答下列问题。

（1）访问<0，4>时，对应的页框号是什么?

（2）访问<1，11>时，对应的页框号是什么?说明理由。

（3）访问<2，14>时，对应的页框号是什么?说明理由。

（4）该策略是否适合于时间局部性好的程序?说明理由。

解：

（1）页框号为 21。因为起始驻留集为空，而 0 页对应的页框为空闲链表中的第三个空闲页框，其对应的页框号为 21。

（2）页框号为 32。理由：因 11>10 故发生第三轮扫描，页号为 l、3 的页框 32、15 在第二轮已处于空闲页框链表中，此刻 l 页又被重新访问，因此应被重新放回到驻留集中。其页框号为 32。

（3）页框号为 41。理由：因为第 2 页从来没有被访问过，它不在驻留集中，因此从空闲页框链表中取出链表头的页框 41，页框号为 41。

（4）适合。理由：如果程序的时间局部性越好，从空闲页框链表中重新取回的机会越大，该策略的优势越明显。

46．（8 分）某文件系统空间的最大容量为 4TB（1T＝2^{40}），以磁盘块为基本分配单位，磁盘块大小为 lKB。文件控制块（FCB）包含一个 512B 的索引表区。请回答下列问题。

（1）假设索引表区仅采用直接索引结构，索引表区存放文件占用的磁盘块号。索引表项中块号最少占多少字节?可支持的单个文件最大长度是多少字节?

（2）假设索引表区采用如下结构：第 0～7 字节采用<起始块号，块数>格式表示文件创建时预分配的连续存储空间，其中起始块号占 6B，块数占 2B；剩余 504 字节采用直接索引结构，一个索引项占 6B，则可支持的单个文件最大长度是多少字节?为了使单个文件的长度达到最大，请指出起始块号和块数分别所占字节数的合理值并说明理由。

解：

（1）文件系统存储空间共有块数 2^{42} / 2^{10}＝2^{32}。为表示 2^{32}个块号，索引表项占 32 / 8＝4B，512B 可存放 2^{7}个索引项，每个索引项对应一个磁盘块，故最大文件长度：2^{7}×2^{10}B＝2^{17}B＝128KB。

（2）块号占 6 字节，块数占 2 字节的情形下，最大文件长度：2^{16}×2^{10}+（504 / 6） ×2^{10}＝64MB+84KB＝65620KB。

合理的起始块号和块数所占字节数分别为<4,4>（或<0，8>或<1，7>或<2，6>或<3，5>）。理由：块数占 4B 或以上，就可表示 4TB 大小的文件长度，达到文件系统的空间上限。

47．（9 分）主机 H 通过快速以太网连接 Internet，IP 地址为 192.168.0.8，服务器 S 的 IP 地址为 211.68.71.80。H 与 S 使用 TCP 通信时，在 H 上捕获的其中 5 个 IP 分组如题 47-a 表所示。

题 47-a 表

编号	IP 分组的前 40 字节内容（十六进制）
1	45000030 019b4000 8006lde8 coa80008 d3444750 0bd91388 846b41c5 00000000 70024380 5db00000
2	45000030 00004000 31066e83 3d3444750 cOa80008 13880bd9 e0599fef 846b41c6

	6701216d0 37e10000
	45000028 019c4000 8006ldef cOa80008
3	d3444750 bd913888 46b41c6e 0599ff05
	01043802 b320000
	45000038 019d4000 80061dde
4	cOa80008 d3444750 0bd91388 846b4lc6
	e0599ff0 50184380 c6550000
	45000028 68114000 3106067a
5	d3444750 cOa80008
	13880bd9 e0599ff0 846b41d6
	501016d0 57d20000

请回答下列问题。

（1）题 47-a 表中的 IP 分组中，哪几个是由 H 发送的?哪几个完成了 TCP 连接建立过程?哪几个在通过快速以太网传输时进行了填充?

（2）根据题 47-a 表中的 IP 分组，分析 S 已经收到的应用层数据字节数是多少?

（3）若题 47-a 表中的某个 IP 分组在 S 发出时的前 40 字节如题 47-b 表所列，则该 IP 分组到达 H 时经过了多少个路由器?

题 47-b 表

来自 S 发出的分组	45000028 68114000 4006eCad d3444750 ca760106 1388a108 e0599ff0 846b41d6 501016dO b7d60000

注：IP 分组头和 TCP 段头结构分别如题 47-a 图、题 47-b 图所示：

版本	头部长度	服务类型（8-15）	总长度（16-31）
标识		标志	片偏移
生存时（TTL）		协议	头部校验和
源 IP 地址			
目的 IP 地址			

题 47-a 图 IP 分组头结构

源端口（0-15）			目的端口（16-31）
序号（seq）			
确认号（ack）			
数据偏移	保留 URG ACK PSH RST SYN FIN		窗口
校验和			紧急指针
选项（长度可变）			填充

题 47-b 图 TCP 段头结构

解：（1）由于题 47-a 表中 1、3、4 号分组的源 IP 地址（第 13～16 字节）均为 l92.168.0.8（coa80008H），所以 1、3、4 号分组是由 H 发送的。

题 47-a 表中 1 号分组封装的 TCP 段的 FLAG 为 02H（即 SYN＝1，ACK＝0），seq＝846b 41c5H，2 号分组封装的 TCP 段的 FLAG 为 12H（即 SY＝1，ACK＝1），seq＝e059 9fefH，ack＝846b 41c6H，3 号分组封装的 TCP 段的 FLAG 为 10H（即 ACK＝1），seq＝846b 41c6H，ack＝e059 9ff0H，所以 1、2、3 号分组完成了 TCP 连接建立过程。

由于快速以太网数据帧有效载荷的最小长度为 46 字节，表中 3、5 号分组的总长度为 40（28H）字节，小于 46 字节，其余分组总长度均大于 46 字节，所以 3、5 号分组在通过快速以太网传输时进行了填充。

（2）由 3 号分组封装的 TCP 段可知，发送应用层数据初始序号为 846b 41c6H，由 5 号分组封装的 TCP 段可知，ack 为 846b 41d6H，所以 5 号已经收到的应用层数据的字节数为 846b 41d6H-846b 41c6H＝10H＝16B。

（3）由于 S 发出的 IP 分组的标识＝6811H，所以该分组所对应的是题 47-a 表中的 5 号分组。S 发出的 IP 分组的 TTL＝40H＝64，5 号分组的 TTL＝31H＝49，64-49＝15。所以，可以推断该 IP 分组到达 H 时经过了 l5 个路由器。

2011 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解

一、单项选择题：1～40 小题。每小题 2 分。共 80 分。下列每题给出的四个选项中，只有一个选项是最符合题目要求的。

1．设 n 是描述问题规模的非负整数，下面程序片段的时间复杂度是（　　）。

```
x＝2；
while（x<n/2）
x＝2×x；
```

A．O（$\log_2 n$）

B．O（n）

C．O（$n\log_2 n$）

D．O（n_2）

【答案】A 查看答案

【解析】其中，以基本的原操作重复执行的次数作为算法的时间度量。题目中的基本运算是语句 x＝2×x，设其执行时间为 T（n），则有 $2^{T(n)}<n/2$ 即 T（n）$<\log_2(n/2)=O(\log_2 n)$。

2．元素 a，b，c，d，e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是（　　）。

A．3

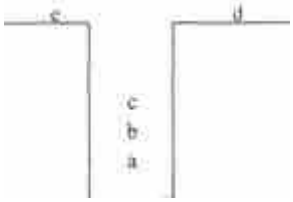
B．4

C．5

D．6

【答案】B 查看答案

【解析】d 首先出栈后的状态如下图所示。

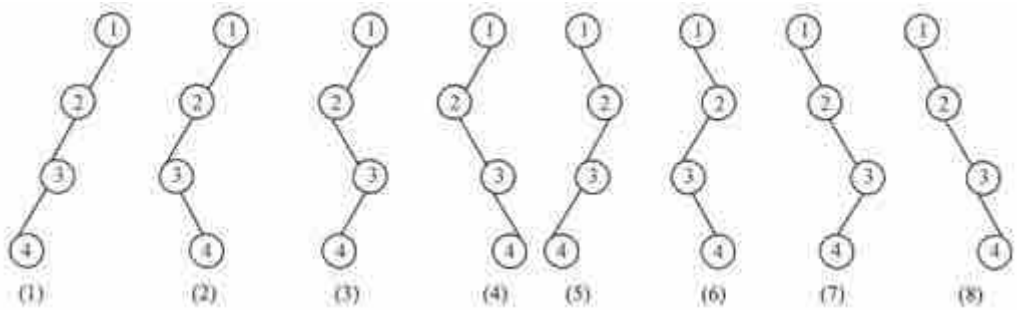


此时可有以下 4 种操作：

（1）e 进栈后出栈，出栈序列为 decba。

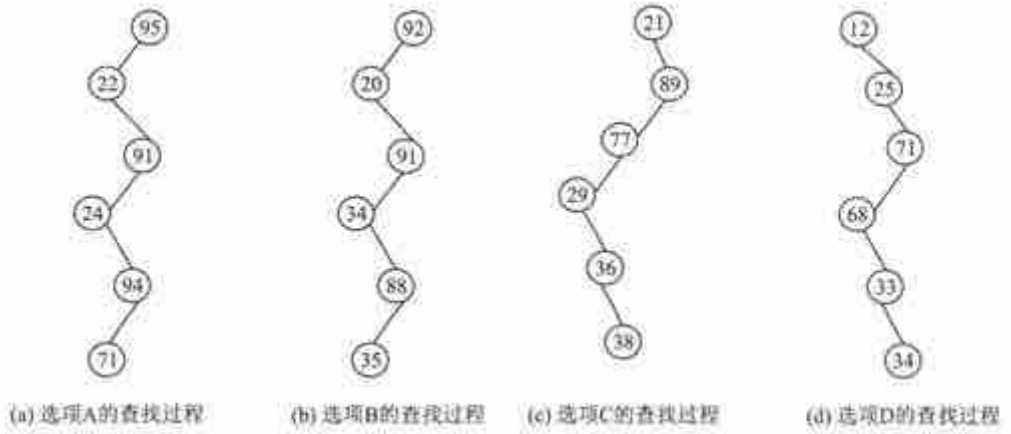
（2）c 出栈，e 进栈后出栈，出栈序列为 dceba。

（3）cb 出栈，e 进栈后出栈，出栈序列为 dcbea。
（4）cba 出栈，e 进栈后出栈，出栈序列为 dcbae。
3．已知循环队列存储在一维数组 A[0…n-1]中，且队列非空时 front 和 rear 分别指向队头元素和队尾元素。若初始时队列为空，且要求第 1 个进入队列的元素存储在 A[0]处，则初始时 front 和 rear 的值分别是（　　）。
A．0，0
B．0，n-1
C．n-1，0
D．n-1，n-1
【答案】B 查看答案
【解析】题目要求队列非空时 front 和 rear 分别指向队头元素和队尾元素，若初始时队列为空，且要求第 1 个进入队列的元素存储在 A[0]处，则此时 front 和 rear 的值都为 0。由于进队操作要执行（rear+1）% n，则初始时 front 的值为 0、rear 的值为 n-1。
4．若一棵完全二叉树有 768 个结点，则该二叉树中叶结点的个数是（　　）。
A．257
B．258
C．384
D．385
【答案】C 查看答案
【解析】由 $n=n_0+n_1+n_2$ 和 $n_0=n_2+1$ 可知，$n=2n_0-1+n_1$，即 $2n_0-1+n_1=768$，显然 $n_1=1$，$2n_0=768$，则 $n_0=384$，所以二叉树的叶结点个数是 384。还可以根据完全二叉树的另一个性质：最后一个分支结点的序号为[768/2]，故非叶子结点数为 384，而叶子结点的个数为 768-384＝384。（［x］表示不大于 x 的最大整数，比如［3.14］＝3）。
5．若一棵二叉树的前序遍历序列和后序遍历序列分别为 l，2，3，4 和 4，3，2，1，则该二叉树的中序遍历序列不会是（　　）。
A．1，2，3，4
B．2，3，4，1
C．3，2，4，1
D．4，3，2，1
【答案】C 查看答案
【解析】题目中的二叉树的先序序列和后序序列正好相反，这样的二叉树每层只有一个结点。该二叉树的形态如下图所示。



从左至右，这 8 棵二叉树的中序序列分别为：
（1）4，3，2，1，
（2）3，4，2，1
（3）2，4，3，1

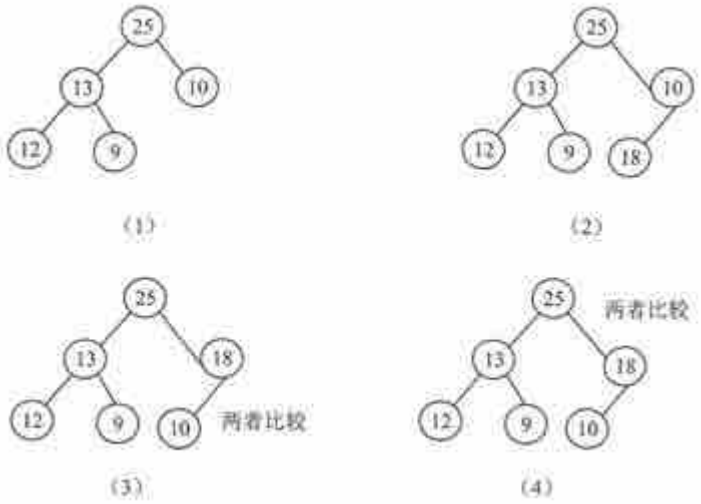
（4）2，3，4，1
（5）1，4，3，2
（6）1，3，4，2
（7）1，2，4，3
（8）1，2，3，4
显然选项 C 的中序序列不会出现。
6．已知一棵有 2011 个结点的树，其叶结点个数为 ll6，该树对应的二叉树中无右孩子的结点个数是（　　）。
A．115
B．116
C．1895
D．1896
【答案】D 查看答案
【解析】每个非终端结点转换成二叉树后都对应一个无右孩子的结点（因为一个非终端结点至少有一个孩子结点，其最右边的孩子结点转换成二叉树后一定没有右孩子），另外，树根结点转换成二叉树后也没有右孩子。题目中树的总结点数是 2011，叶结点个数是 116，则非终端结点个数是 2011-116＝1895，则该树对应的二叉树中无右孩子的结点个数是 1895+1＝1896。
7．对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的序列是（　　）。
A．95，22，91，24，94，71
B．92，20，91，34，88，35
C．21，89，77，29，36，38
D．12，25，71，68，33，34
【答案】A 查看答案
【解析】各选项对应的查找过程如下图所示，从中看到选项 B、C、D 对应的查找树都是二叉排序树，只有选项 A 对应的查找树不是一棵二叉排序树，因为在以 91 为根的左子树中出现了比 91 大的结点 94。



(a) 选项A的查找过程　(b) 选项B的查找过程　(c) 选项C的查找过程　(d) 选项D的查找过程

8．下列关于图的叙述中，正确的是（　　）。
Ⅰ．回路是简单路径
Ⅱ．存储稀疏图，用邻接矩阵比邻接表更省空间
Ⅲ．若有向图中存在拓扑序列，则该图不存在回路
A．仅Ⅱ
B．仅Ⅰ、Ⅱ

C．仅III
D．仅Ⅰ、III
【答案】C 查看答案
【解析】第一个顶点和最后一个顶点相同的路径称为回路；序列中顶点不重复出现的路径称为简单路径；回路显然不是简单路径，所以选项Ⅰ错误。稀疏图用邻接表表示比邻接矩阵节省存储空间，稠密图适合用邻接矩阵的存储表示，所以选项Ⅱ错误。利用拓扑排序算法可以判断图中是否存在回路，即在拓扑排序输出结束后所余下的顶点都有前驱，则说明了只得到了部分顶点的拓扑有序序列，图中存在回路。所以选项III正确。
9．为提高散列（Hash）表的查找效率，可以采用的正确措施是（　　）。
Ⅰ．增大装填（载）因子
Ⅱ．设计冲突（碰撞）少的散列函数
III．处理冲突（碰撞）时避免产生聚集（堆积）现象
A．仅Ⅰ
B．仅Ⅱ
C．仅Ⅰ、Ⅱ
D．仅Ⅱ、III
【答案】D 查看答案
【解析】散列表的查找效率（比较次数）取决于：散列函数、处理冲突的方法和散列表的装填因子α。α标志着散列表的装满程度，通常情况下，α越小，发生冲突的可能性越小；反之，α越大，表示已填入的记录越多，再填入记录时，发生冲突的可能性越大。因此选项Ⅰ错误，越是增大装填因子，发生冲突的可能性就越大，查找效率也越低。选项Ⅱ正确。选项III正确。采用合适的处理冲突的方法避免产生聚集现象，也将提高查找效率。例如，用拉链法解决冲突时不存在聚集现象，用线性探测法解决冲突时易引起聚集现象。
10．为实现快速排序算法，待排序序列宜采用的存储方式是（　　）。
A．顺序存储
B．散列存储
C．链式存储
D．索引存储
【答案】A 查看答案
【解析】对绝大部分内部排序而言，只适用于顺序存储结构，快速排序在排序过程中，既要从后向前查找，也要从前向后查找，因此宜采用顺序存储。
11．已知序列25，13，10，12，9是大根堆，在序列尾部插入新元素18，将其再调整为大根堆，调整过程中元素之间进行的比较次数是（　　）。
A．1
B．2
C．4
D．5
【答案】B 查看答案
【解析】对堆插入或删除一个元素，有可能不满足堆的性质，堆被破坏，需要调整为新堆。
（1）为原堆，
（2）为插入18后，
（3）比较10与18，交换后，
（4）比较25与18，不交换，即为调整后的新的大根堆。
因此调整过程中元素之间进行的比较次数为2。



12．下列选项中，描述浮点数操作速度指标的是（　　）。

A．MIPS

B．CPI

C．IPC

D．MFLOPS

【答案】D 查看答案

【解析】MFLOPS（Million Floating-point Operations per Second）表示每秒执行多少百万次浮点运算，用来描述计算机的浮点运算速度，适用于衡量处理机的性能。

MIPS（Million Instructions per Second）表示每秒执行多少百万条指令。对于一个给定的程序，MIPS 定义为

$$MIPS=\frac{指令条数}{执行时间\times 10^6}=\frac{主频}{CPI}=主频\times IPC$$

这里所说的指令一般是指加、减运算这类短指令。

CPI（Cycles per Instruction）就是每条指令执行所用的时钟周期数。由于不同指令的功能不同，造成指令执行时间不同，也即指令执行所用的时钟数不同，所以 CPI 是一个平均值。

IPC（Instructions per Cycle）每个时钟周期执行的指令数。

$$IPC=\frac{1}{CPI}$$

13.float 型数据通常用 IEEE754 单精度浮点数格式表示。若编译器将 float 型变量 x 分配在一个 32 位浮点寄存器 FRl 中，且 x＝-8.25，则 FR1 的内容是（　　）。

A．C1040000H

B．C2420000H

C．C1840000H

D．C1C20000H

【答案】A 查看答案

【解析】首先将十进制数转换为二进制数-1000.01，接着把它写成规格化形式-1.00001×2^3（按 IEEE754 标准），然后计算阶码的移码＝偏置值+阶码真值＝127+3＝130，最后短浮点数代码：数符位＝1，阶码＝10000010，尾数 00001000000000000000000，写成十六进制为 C1040000H。选项 D 是一个很容易被误选的选项，其错误在于没有考虑 IEEE754 标准中隐含最高位 1 的情况，偏置值是 128。

14．下列各类存储器中，不采用随机存取方式的是（　　）。

A．EPROM
B．CDR0M
C．DRAM
D．SRAM
【答案】B 查看答案
【解析】随机存取方式是指存储器的任何一个存储单元的内容都可以存取，而且存取时间与存储单元的物理位置无关。CDROM 是只读的光盘存储器，采用串行存取方式而不是随机存取方式。
15．某计算机存储器按字节编址，主存地址空间大小为 64MB，现用 4M×8 位的 RAM 芯片组成 32MB 的主存储器，则存储器地址寄存器 MAR 的位数至少是（　　）。
A．22 位
B．23 位
C．25 位
D．26 位
【答案】D 查看答案
【解析】虽然实际的主存储器（RAM 区）只有 32MB，但不排除还有 ROM 区，考虑到存储器扩展的需要，MAR 应保证能访问到整个主存地址空间。因为主存的地址空间大小为 64MB，所以 MAR 的位数至少需要 26 位。
16. 偏移寻址通过将某个寄存器内容与一个形式地址相加而生成有效地址。下列寻址方式中，不属于偏移寻址方式的是（　　）。
A．间接寻址
B．基址寻址
C．相对寻址
D．变址寻址
【答案】A 查看答案
【解析】在四种不同的寻址方式中，间接寻址按指令的形式地址从主存中取出操作数的有效地址，然后再按此有效地址从主存中读出操作数。其余三种寻址方式可以统称为偏移寻址。
17．某机器有一个标志寄存器，其中有进位／借位标志 CF、零标志 ZF、符号标志 SF 和溢出标志 OF，条件转移指令 bgt（无符号整数比较大于时转移）的转移条件是（　　）。
A．CF+OF＝0
B．SF+ZF＝0
C．CF+ZF＝0
D．CF+SF＝0
【答案】C 查看答案
【解析】判断无符号整数 A>B 成立，满足的条件是结果不等于 0，即零标志 ZF＝0，且不发生进位，即进位／借位标志 CF＝0。所以正确选项为 C。其余选项中用到了符号标志 SF 和溢出标志 OF，显然可以排除掉。
18．下列给出的指令系统特点中，有利于实现指令流水线的是（　　）。
Ⅰ．指令格式规整且长度一致
Ⅱ．指令和数据按边界对齐存放
Ⅲ．只有 Load／Store 指令才能对操作数进行存储访问
A．仅Ⅰ、Ⅱ
B．仅Ⅱ、Ⅲ
C．仅Ⅰ、Ⅲ

D．Ⅰ、Ⅱ、Ⅲ

【答案】D 查看答案

【解析】特点Ⅰ和Ⅲ都是 RISC 机的特征，而特点Ⅱ则有利于指令和数据的存放，所以以上三个特点都有利于实现指令流水线。

19．假定不采用 Cache 和指令预取技术，且机器处于“开中断”状态，则在下列有关指令执行的叙述中，错误的是（　　）。.

A．每个指令周期中 CPU 都至少访问内存一次

B．每个指令周期一定大于或等于一个 CPU 时钟周期

C．空操作指令的指令周期中任何寄存器的内容都不会被改变

D．当前程序在每条指令执行结束时都可能被外部中断打断

【答案】C 查看答案

【解析】本题涉及的概念比较多。首先，如果不采用 Cache 和指令预取技术，每个指令周期中至少要访问内存一次，即从内存中取指令。其次，指令有的简单有的复杂，每个指令周期总大于或等于一个 CPU 时钟周期。第三，即使是空操作指令，在指令周期中程序计数器 PC 的内容也会改变（PC 值加“1”），为取下一条指令做准备。第四，如果机器处于“开中断”状态，在每条指令执行结束时都可能被新的更高级的中断请求所打断。所以应选择选项 C。

20．在系统总线的数据线上，不可能传输的是（　　）。

A．指令

B．操作数

C．握手（应答）信号

D．中断类型号型号

【答案】C 查看答案

【解析】握手（应答）信号属于通信联络控制信号应该在通信总线上传输，不可能在数据总线上传输。而指令、操作数和中断类型码都可以在数据线上传输。

21．某计算机有五级中断 L_4～L_0，中断屏蔽字为 $M_4M_3M_2M_1M_0$，$M_i=1$（$0\le i\le 4$）表示对 L_i级中断进行屏蔽。若中断响应优先级从高到低的顺序是 $L_0\rightarrow L_1\rightarrow L_2\rightarrow L_3\rightarrow L_4$，且要求中断处理优先级从高到低的顺序为 $L_3\rightarrow L_0\rightarrow L_2\rightarrow L_1\rightarrow L_3$，则 L_1的中断处理程序中设置的中断屏蔽字是（　　）。

A．11110

B．01101

C．00011

D．01010

【答案】D 查看答案

【解析】由于 L_2的中断处理优先级下降，屏蔽字中需要 3 个 0，所以可以将选项 A、B 排除掉。L_1需要对 L_4、L_0、L_2开放，所以相应位应该为“0”，即为 01010。

22．某计算机处理器主频为 50MHz，采用定时查询方式控制设备 A 的 I / O，查询程序运行一次所用的时钟周期数至少为 500。在设备 A 工作期间，为保证数据不丢失，每秒需对其查询至少 200 次，则 CPU 用于设备 A 的 I / O 的时间占整个 CPU 时间的百分比至少是（　　）。

A．0.02％

B．0.05％

C．0.20％

D．0.50％

【答案】C 查看答案

【解析】对于设备 A，每秒中查询至少 200 次，每次查询至少 500 个时钟周期，总的时钟周期数为 100000，又因为处理器主频为 50MHz。所以 CPU 用于设备 A 的 I / O 的时间占整个 CPU 时间的百分比至少为 100000/50＝0.20％。

23．下列选项中，满足短任务优先且不会发生饥饿现象的调度算法是（　　）。

A．先来先服务

B．高响应比优先

C．时间片轮转

D．非抢占式短任务优先

【答案】B 查看答案

【解析】分析该题目可以看到，本题所提到的问题是涉及短任务调度也就是属于作业调度，因此首先排除时间片轮转算法；因为作业调度算法中没有时间片轮转的算法。其次，因为问题提到短任务，则先来先服务的算法也可以排除了，它与短任务无关。剩余高响应比优先算法和非抢占式短任务优先是哪一个?我们可以通过分析得到，非抢占式短任务优先算法不能解决饥饿问题，因为当一个系统短任务源源不断到达是，长任务必然会得不到调度，产生饥饿。而解决此方法的最好方式就是采用计算响应比的方法，并以高响应比值优先调度。这样，无论短任务或长任务，均可以得到调度，而且，较短任务会得到优先的调度。故满足短任务优先且不会发生饥饿现象的调度算法只有高响应比优先算法。

24．下列选项中，在用户态执行的是（　　）。

A．命令解释程序

B．缺页处理程序

C．进程调度程序

D．时钟中断处理程序

【答案】A 查看答案

【解析】题目是问用户态执行，可见是有关操作系统基本概念的问题。四个选项中，用户唯一能面对的是命令解释程序，缺页处理程序和时钟中断都属于中断，在核心态执行，而进城调度属于系统调用在核心态执行。只有命令解释程序属于命令接口，可以运行在用户态，接受用户的命令操作控制。

25．在支持多线程的系统中，进程 P 创建的若干个线程不能共享的是（　　）。

A．进程 P 的代码段

B．进程 P 中打开的文件

C．进程 P 的全局变量

D．进程 P 中某线程的栈指针

【答案】D 查看答案

【解析】现代操作系统中，进程是资源分配的基本单位，线程是处理机调度的基本单位。因此，进程是线程运行的容器，本题中，进程的代码段，进程打开的文件，进程的全局变量等都是进程的资源，唯有进程中某线程的栈指针是属于线程的，那么，属于进程的资源可以共享，属于线程的栈是独享的，不能共享。

26．用户程序发出磁盘 I / O 请求后，系统的正确处理流程是（　　）。

A．用户程序→系统调用处理程序→中断处理程序→设备驱动程序

B．用户程序→系统调用处理程序→设备驱动程序→中断处理程序

C．用户程序→设备驱动程序→系统调用处理程序→中断处理程序

D．用户程序→设备驱动程序→中断处理程序→系统调用处理程序

【答案】B 查看答案

【解析】对于一次设备的调用，操作系统为用户准备了系统调用的接口，当用户使用设备时，首先在用户程序中发起一次系统调用，操作系统的内核接到该调用请求后调用处理程序进行处理，根据调用格式和形参，再转到相应的设备驱动程序去处理；大部分设备在运行时是需要时间的，所以设备驱动程序会以中断方式驱动设备，即设置好控制寄存器参数和中断向量等参数后阻塞自己；当设备准备好或所需数据到达后设备硬件发出中断，设备驱动程序唤醒，将数据按上述调用顺序逆向回传到用户程序中，或继续驱动设备执行下一条指令。因此，正确的顺序应该是用户到系统调用到驱动到中断处理。中断处理处于最底层。

27．某时刻进程的资源使用情况如下表所示

	已分配资源			尚需资源			可用资源		
进程	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	2	O	O	O	O	1			
P2	1	2	O	1	3	2			
P3	O	1	1	1	3	l	0	2	1
P4	0	O	1	2	O	O			

此时的安全序列是（　　）。

A．P1，P2，P3，P4

B．P1，P3，P2，P4

C．P1，P4，P3，P2

D．不存在

【答案】D 查看答案

【解析】典型的死锁避免算法，银行家算法的应用。银行家算法是操作系统中的一个重点知识单元，考生对此应该非常熟悉，本题并无难点。分析一下下表，可以看到，经过 P1，P4 的运行以后，可用资源是 2，2,1，而 P2，P3 所需资源分别是 1,3,2 和 1,3,1。所以剩余资源已经不够 P2 或 P3 的分配，亦即找不到能够安全运行的序列，因此此时是处于不安全状态，所以不存在这样的安全序列。

	已分配资源			尚需资源			可用资源		
进程	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	2	0	O	O	0	l	O	2	1
P2	1	2	O	1	3	2	2	2	2
P3	O	1	1	1	3	1	2	2	2
P4	0	0	1	2	0	O	2	2	1

28．在缺页处理过程中，操作系统执行的操作可能是（　　）。

Ⅰ．修改页表

Ⅱ．磁盘 I/O

Ⅲ．分配页框

A．仅Ⅰ、Ⅱ

B．仅Ⅱ

C．仅Ⅲ

D．Ⅰ、Ⅱ和Ⅲ

【答案】D 查看答案

【解析】首先我们要考虑的是，为什么会发生缺页中断?当然，在一个采用虚拟存储管理技术的系统中，程序是部分装入的，还有部分是处于外存上的，因此，当需要访问那部分位于外存上的代码或数据时，系统会产生缺页中断。产生缺页中断的目的是要将位于外存上的代码或数据装入内存，据此，缺页中断接下去所做的工作就是首先要在内存中找到空闲页框并

分配给需要访问的页（若没有空闲的页面则要调用页面置换程序找到一处页面，将该页面的内容处理掉，或回写磁盘，或覆盖掉，然后将此页分配给需要访问的页），分配妥当以后，缺页中断处理程序调用设备驱动程序做磁盘 I/O，将位于外存（一般是磁盘）上的页面调入内存，调入后转身去修改页表，将页表中代表该页是否在内存的标志位（一般称为存在位或有效位、在位位）修改为“真”，将物理页框号填入相应位置，若必要还需修改其它相关表项等。完成上述任务后，缺页中断处理程序返回，继续程序的执行。从上述过程可以看出，涉及的相关处理非常多，因此，答案就显而易见了。

29．当系统发生抖动（thrashing）时，可以采取的有效措施是（　　）。

Ⅰ．撤销部分进程

Ⅱ．增加磁盘交换区的容量

Ⅲ．提高用户进程的优先级

A．仅Ⅰ

B．仅Ⅱ

C．仅Ⅲ

D．仅Ⅰ、Ⅱ

【答案】A 查看答案

【解析】“抖动”现象是指刚刚被换出的页很快又要被访问，为此，又要换出其他页，而该页又很快被访问，必须换入，如此频繁地置换页面，以致操作系统的大部分时间都花在页面置换上，引起系统性能下降甚至崩溃。引起系统抖动现象的原因是对换的信息量过大，内存容量不足，置换算法选择不当。所以解决的办法就是降低交换页面数量，加大内存容量，改变置换选择算法。但是降低交换页面数量和改变置换选择算法对于一个应用系统来讲是不可能的，只能增加内存容量。增加内存容量可以是直接添加物理内存（大型计算机都可以在不关机的情况下增加物理内存条），或者，降低进程数量，相对地增加内存。而增加交换区容量并不能解决物理内存不足的问题，提高用户进程的优先级会使系统的状态更加恶化。

30．在虚拟存储管理中，地址变换机构将逻辑地址变换为物理地址，形成该逻辑地址的阶段是（　　）。

A．编辑

B．编译

C．链接

D．装载

【答案】B 查看答案

【解析】程序的编辑阶段一般都是程序员能够识别的高级语言或低级语言的文本，不涉及到任何与计算机运行相关的事；编译是由编译程序将用户源代码编译成若干个目标模块，源地址编译成目标程序时，会形成逻辑地址；链接是由链接程序将编译后形成的一组目标模块，以及所需库函数链接，形成完整的装入模块；装入是由装入程序将装入模块装入内存。

31．某文件占 10 个磁盘块，现要把该文件磁盘块逐个读入主存缓冲区，并送用户区进行分析。假设一个缓冲区与一个磁盘块大小相同，把一个磁盘块读人缓冲区的时间为 100μs，将缓冲区的数据传送到用户区的时间是 50μs，CPU 对一块数据进行分析的时间为 50μs。在单缓冲区和双缓冲区结构下，读人并分析完该文件的时间分别是（　　）。

A．1500μs、1000μs

B．1550μs、1100μs

C．1550μs、1550μs

D．2000μs、2000Ms

【答案】B 查看答案

【解析】这是一个简单的缓冲区的问题。由于缓冲区的访问是互斥的，所以对单一缓冲区，从磁盘写入和读出到用户区的操作必须串行执行，也就是要保证互斥操作。而 CPU 对数据的分析与从用户区读数据也是需要互斥操作，但是 CPU 分析与从磁盘写入缓冲区的操作可以并行。从本题看，由于分析所用的时间小于从磁盘写入缓冲区的时间，因此，CPU 会空闲。单缓冲区的总时间＝（磁盘写入缓冲区时间+缓冲区读出时间）×10+CPU 处理最后一块数据的时间＝（100+50） ×10+50＝1550μs。当采用双缓冲区时，每块缓冲区的操作也必须满足互斥操作，但是，对两块缓冲区的操作却可以并行，所以，当第一个缓冲区写满以后，磁盘紧接着写另一个缓冲区，同时，前一个已经满了的缓冲区被读出到用户区，并立即进行 CPU 的数据分析。读出操作和数据分析必须互斥进行，故从时间上看，当数据被读出并分析后，恰好另一个缓冲区也写满了，可以立即进行读出数据到用户区并进行数据分析。两块缓冲区交替进行读写，直到数据分析完毕，因此，总时间＝（磁盘写入缓冲区时间）×10+读出最后一块数据时间+CPU 分析最后一块数据时间＝（100）×10+50+50＝1100μs。

32．有两个并发执行的进程 P1 和 P2，共享初值为 l 的变量 x。P1 对 x 加 1，P2 对 x 减 1。加 1 和减 1 操作的指令序列分别如下所示。

```
//加 1 操作                                    //减 1 操作
load    R1,x    //取 x 到寄存器 R1 中          load    R2,x
inc     R1                                   dec     R2
store   x,R1    //将 R1 的内容存入 x           store   x,R2
```

两个操作完成后，2 的值（　　）。

A．可能为-1 或 3

B．只能为 1

C．可能为 0、1 或 2

D．可能为-1、0、1 或 2

【答案】C 查看答案

【解析】这是在数据库中常有的操作。为保证数据的正确，避免产生错误，系统必须保证数据的同步。而保证数据的同步一般采取加锁的方法，让进程 P1 和 P2 互斥访问共享变量 x。当然用信号量和 P、V 操作也是可以保证互斥操作，达到数据同步的。本例中，由于没有采取保证数据同步的相应措施，则最后结果就会出现差错。例如，当正常情况下，进程 Pl 和 P2 先后对 x 操作，可以看到 x 值的变化为初始 1→2→1 的过程，若 P2，P1 先后操作，则 x 值的变化为初始 1→0→1，这是正确的。若考虑一种并发的情况，进程 P1 和 P2 先后执行了取数 load 的操作，它们得到的 x 值均为 1，运算后，P1 和 P2 的 x 值分别为 2 和 0，此时要看哪个进程后执行存数 store 的操作了，哪个进程后操作，结果就是那个进程的 x 值，所以可能的结果为 0 或 2，加上前面正确的 x 值 1，则可能的结果就有 3 种了。

33.TCP／IP 参考模型的网络层提供的是（　　）。

A．无连接不可靠的数据报服务

B．无连接可靠的数据报服务

C．有连接不可靠的虚电路服务

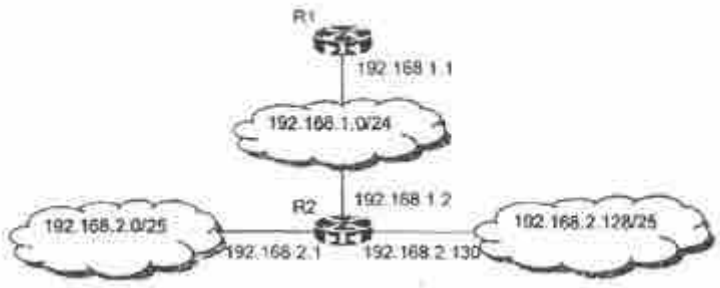
D．有连接可靠的虚电路服务

【答案】A 查看答案

【解析】TCP/IP 的网络层向上只提供简单灵活的、无链接的、尽最大努力交付的数据服务，因此答案是 A。

34. 若某通信链路的数据传输速率为 2400bps，采用 4 相位调制，则该链路的波特率是（　　）。

A．600 波特
B．1200 波特
C．4800 波特
D．9600 波特
【答案】B 查看答案
【解析】注意无噪声下的码元速率极限值 B 与信道带宽 H 的关系：B＝2×H（Baud），而奈奎斯特公式——无噪信道传输能力公式是 C＝2×H×$\log_2$N（bps），N 为一个码元所取的离散值个数。从而可以得到波特率与数据传输速率的关系，即 C＝B×$\log_2$N（bps），在本题中数据传输速率 C＝2400，N＝4，因此波特率是 1200，答案是 B。
35．数据链路层采用选择重传协议（SR）传输数据，发送方已发送了 0H3 号数据帧，现已收到 1 号帧的确认，而 0、2 号帧依次超时，则此时需要重传的帧数是（　　）。
A．1
B．2
C．3
D．4
【答案】B 查看答案
【解析】在选择重传协议中，接收方逐个地确认正确接收的分组，不管接收到的分组是否有序，只要正确接收就发送选择 ACK 分组进行确认。因此选择重传不支持累积确认，要特别注意其与 GBN 协议的区别。本题收到 l 号帧的确认，说明 1 号帧正确接收，0 和 2 号帧依次超时，因此必须重传，然而 3 号帧尚未超时，是否正确接收未知，故不用重传，因此必须重传 0 和 2 号帧，答案是 B。
36．下列选项中，对正确接收到的数据帧进行确认的 MAC 协议是（　　）。
A．CSMA
B．CDMA
C．CSMA／CD
D．CSMA／CA
【答案】D 查看答案
【解析】可采用排除法。CDMA 是码分多址复用,是物理层的内容；CSMA/CD 即带冲突检测的载波监听多路访问，接收方并不需要确认；CSMA/CD 是 CSMA 的加强版，故 CSMA 也无确定；CSMA/CD 是 802.11 中的协议，其利用 ACK 信号来避免冲突的发生，也就是说，只有当客户端收到网络上返回的 ACK 信号后才确认送出的数据已经正确到达目的地址，因此答案是 D。
37．某网络拓扑如下图所示，路由器 Rl 只有到达子网 l92.168.1.0/24 的路由。为使 R1 可以将 IP 分组正确地路由到图中所有子网，则在 R1 中需要增加一条路由（目的网络，子网掩码，下一跳）是（　　）。



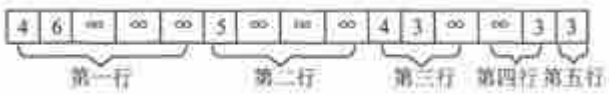
A．192.168.2.0，255.255.255.128，192.168.1.1

B．192.168.2.0，255.255.255.0， 192.168.1.1
C．192.168.2.0，255.255.255.128，192.168.1.2
D．192.168.2.0，255.255.255.0， 192.168,1,2
【答案】D 查看答案
【解析】首先从题目给出的路由表项可以确定下一跳肯定是路由器 R1 直接相连的 R2 的地址，因此是 192.168.1.2,此时可以排除 A 和 B 两个选项了。进而分析路由器 R2 所连接的网络特点，注意其连接了 2 个网络分别是 l92.168.2.0/25 和 192.168.2.128/25，但答案选项中只有一条信息，因此这里用到了超网的概念，超网是与子网类似的概念——IP 地址根据子网掩码被分为独立的网络地址和主机地址。但是，与子网把大网络分成若干小网络相反，它是把一些小网络组合成一个大网络——超网，这里 192.168.2.00000000/25 和 192.168.2.10000000/25 前 24 位是相同的，因此所构成的超网就是 l92.168.2.0/24，那么子网掩码就是 255.255.255.00000000 即 255.255.255.0，因此答案是 D。
38. 在子网 l92.168.4.0/30 中，能接收目的地址为 l92.168.4.3 的 IP 分组的最大主机数是（ ）。
A．0
B．1
C．2
D．4
【答案】C 查看答案
【解析】每个子网中忽略子网内全为 0 和全为 1 的地址剩下的就是有效主机地址，本题中由于子网的比特数是 30，因此用于主机的只有 2 位，即 O0，01，10，11，有效主机地址是 2 个，这里 192.168.4.3 显然是其广播地址，因此答案是 C。
39. 主机甲向主机乙发送一个（SYN 一 1，seq 一 11220）的 TCP 段，期望与主机乙建立 TCP 连接，若主机乙接受该连接请求，则主机乙向主机甲发送的正确的 TCP 段可能是（ ）。
A．（SYN＝0，ACK＝0，seq＝11221，ack＝11221）
B．（SYN＝1，ACK＝1，seq＝11220，ack＝11220）
C．（SYN＝1，ACK＝1，seq＝11221，ack＝11221）
D．（SYN＝0，ACK＝0，seq＝11220，ack＝11220）
【答案】C 查看答案
【解析】TCP 是面向连接的，所谓面向连接，就是当计算机双方通信时必需先建立连接，然后数据传送，最后拆除三个过程，也就是客户主动打开 TCP 传输，服务器被动打开。第一次握手：客户发送 SYN＝1，seq＝x 给服务器，即客户的 TCP 向服务器发出连接请求报文段，其首部中的同步位 SYN＝1，并选择序号 seq＝x，表明传送数据时的第一个数据字节的序号是 x。第二次握手：服务器发送 SYN＝1，ACK＝l，seq＝y，ack＝x+1 给客户，即服务器的 TCP 收到连接请求报文段后，如同意则发回确认。服务器在确认报文段中应使 SYN＝1，使 ACK＝1，其确认号 ack＝x+1，自己选择的序号 seq＝y。第三次握手：客户发送 ACK＝1，seq＝x+1，ack＝y+1 给服务器，即客户收到此报文段后向服务器给出确认，其 ACK＝1，确认号 ack＝y+1。客户的 TCP 通知上层应用进程，连接已经建立。服务器的 TCP 收到主机客户的确认后，也通知其上层应用进程：TCP 连接已经建立。因此，本题中 x＝11220，y 是主机乙自动选取的序号，可以与 x 相同，也可以不相同，从而主机乙所发出的 TCP 段应该是 SYN＝1，ACK＝1，seq＝y，ack＝x+1，即 SYN＝1，ACK＝1，seq＝y，ack＝11221，从而答案是 C。
40. 主机甲与主机乙之间已建立一个 TCP 连接，主机甲向主机乙发送了 3 个连续的 TCP 段，分别包含 300 字节、400 字节和 500 字节的有效载荷，第 3 个段的序号为 900。若主机乙仅正确接收到第 1 和第 3 个段，则主机乙发送给主机甲的确认序号是（ ）。

A．300
B．500
C．1200
D．1400
【答案】B 查看答案
【解析】本题考查 TCP 的确认机制，TCP 首部的序号字段是指本报文所发送的数据的第一个字节的序号。本题中首先根据第 3 个段的序号为 900，可以得出第 2 个段的序号为 500，第 l 个段的序号为 200，这里主机乙仅正确接收了第 1 段和第 3 段，这意味着第 2 段丢失，需要超时重传，因此主机乙发送给主机甲的确认序号，也就是此时接收端期望收到的下一个数据包中第一个字节的序号应该是第二段的第一个字节的序号，也就是 500，因此答案是 B。
二、综合应用题：41～47 小题，共 70 分。
41．（8 分）已知有 6 个顶点（顶点编号为 0--5）的有向带权图 G，其邻接矩阵 A 为上三角矩阵，按行为主序（行优先）保存在如下的一维数组中。
4 6 ∞ ∞ ∞ 5 ∞ ∞ ∞ 4 3 ∞ ∞ 3 3
要求：
（1）写出图 G 的邻接矩阵 A。
（2）画出有向带权图 G。
（3）求图 G 的关键路径，并计算该关键路径的长度。
解：（1）由题可以画出待定上三角矩阵的结构图如下（图中?为待定元素）：

$$\begin{bmatrix} 0 & ? & ? & ? & ? & ? \\ \infty & 0 & ? & ? & ? & ? \\ \infty & \infty & 0 & ? & ? & ? \\ \infty & \infty & \infty & 0 & ? & ? \\ \infty & \infty & \infty & \infty & 0 & ? \\ \infty & \infty & \infty & \infty & \infty & 0 \end{bmatrix} \begin{matrix} 5 \\ 4 \\ 3 \\ 2 \\ 1 \end{matrix}$$

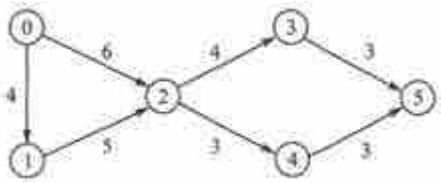
可以看出，第一行至第五行主对角线上方的元素分别为 5，4，3，2，1 个，由此可以画出压缩存储数组中的元素所属行的情况，如下图所示：



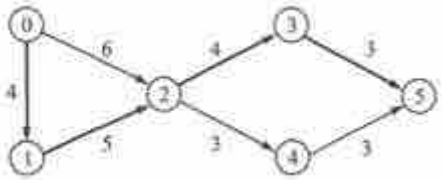
将各元素填入各行即得邻接矩阵：

$$A = \begin{bmatrix} 0 & 4 & 6 & \infty & \infty & \infty \\ \infty & 0 & 5 & \infty & \infty & \infty \\ \infty & \infty & 0 & 4 & 3 & \infty \\ \infty & \infty & \infty & 0 & \infty & 3 \\ \infty & \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & \infty & \infty & \infty & 0 \end{bmatrix}$$

（2）根据第一步所得矩阵 A 容易做出有向带权图 G，如下：



（3）下图中粗线箭头所标识的 4 个活动组成图 G 的关键路径。



由上图容易求得图的关键路径长度为：4+5+4+3＝16。

42．（15 分）一个长度为 L（L≥1）的升序序列 S，处在第「L / 2」个位置的数为 S 的中位数。例如，若序列 S1＝（11，13，15，17，19），则 Sl 的中位数是 15。两个序列的中位数是含它们所有元素的升序序列的中位数。例如，若 S2＝（2，4，6，8，20），则 Sl 和 S2 的中位数是 11。现有两个等长升序序列 A 和 B，试设计一个时间和空间两方面都尽可能高效的算法，找出两个序列 A 和 B 的中位数。要求：

（1）给出算法的基本设计思想。

（2）根据设计思想，采用 C 或 C++或 JAVA 语言描述算法，关键之处给出注释。

（3）说明你所设计算法的时间复杂度和空间复杂度。

解：（1）算法的基本设计思想：分别求两个升序序列 A 和 B 的中位数，设为 a 和 b。

①若 a＝b，则 a 或 b 即为所求的中位数。

②否则，若 a<b，中位数只能出现（a，b）范围内，舍弃 a 所在序列 A 的较小一半，同时舍弃 b 所在序列 B 的较大一半。若 a>b，中位数只能出现（b，a）范围内，舍弃 b 所在序列 B 的较小一半，同时舍弃 a 所在序列 A 的较大一半。

③在保留的两个升序序列中求出新的中位数 a 和 b，重复上述过程，直到两个序列中只含一个元素时为止，则较小者即为所求的中位数。

（2）用 C 语言算法描述如下：

```
int Search(int A[ ], int B[ ], int n){
    int s1,e1,mid1,s2,e2,mid2;
    s1 = 0;e1 = n - 1;s2 = 0;e2 = n - 1;
    while (s1! = e1||s2! = e2){
        mid1 = (s1 + e1)/2;
        mid2 = (s2 + e2)/2;
        if (A[mid1] = = B[mid2])
            return A[mid1];
        if (A[mid1]<B[mid2]){          //分别考虑奇数和偶数,保持两个子数组元素个数相等
            if ((s1 + e1) % 2 = = 0) | //若元素个数为奇数个
                s1 = mid1;             //舍弃 A 中间点以前部分且保留中间点
                e2 = mid2;             //舍弃 B 中间点以后部分且保留中间点
            }
```

```
        else {                          //若元素个数为偶数个
            s1 = mid1 + 1;              //舍弃 A 中间点及中间点以前部分
            e2 = mid2;                  //舍弃 B 中间点以后部分且保留中间点
        }
    }
    else {                              //A[mid1] > B[mid2]
        if ((s1 + e1) % 2 = = 0) {      //若元素个数为奇数个
            e1 = mid1;                  //舍弃 A 中间点以后部分且保留中间点
            s2 = mid2;                  //舍弃 B 中间点以前部分且保留中间点
        }
        else {                          //若元素个数为偶数个
            e1 = mid1;                  //舍弃 A 中间点以后部分且保留中间点
            s2 = mid2 + 1;              //舍弃 B 中间点及中间点以前的部分
        }
    }
    return(A[s1] < B[s2] ? A[s1] : B[s2]);
}
```

（3）说明算法的复杂性：算法的时间复杂度、空间复杂度分别是 O（$1og_2n$）和 O（1）。

43．（11 分）假定在一个 8 位字长的计算机中运行下列 C 程序段：

```
unsigned int  x = 134;
unsigned int  y = 246;
int  m = x;
int  n = y;
unsigned int  z1 = x - y;
unsigned int  z2 = x + y;
int  k1 = m - n;
int  k2 = m + n;
```

若编译器编译时将 8 个 8 位寄存器 Rl～R8 分别分配给变量 x、y、m、n、z1、z2、k1 和 k2。请回答下列问题。（提示：带符号整数用补码表示）

（1）执行上述程序段后，寄存器 R1、R5 和 R6 的内容分别是什么?（用十六进制表示）

（2）执行上述程序段后，变量 m 和 kl 的值分别是多少?（用十进制表示）

（3）上述程序段涉及带符号整数加／减、无符号整数加／减运算，这四种运算能否利用同一个加法器及辅助电路实现?简述理由。

（4）计算机内部如何判断带符号整数加／减运算的结果是否发生溢出?上述程序段中，哪些带符号整数运算语句的执行结果会发生溢出?

解：（1）无符号整数运算，（R1）＝x＝134＝10000110B＝86H、（R5）＝x-y＝246＝10010000B＝90H、（R6）＝x+y＝01111100B＝7CH。

（2）m 的机器数与 x 的机器数相同为 86H＝1000 0110B，解释为带符号整数（用补码表示）时,其值为-111 1010B＝-112；同理 k1＝（m-n）＝（x-y）＝90H＝1001 0000B，解释为带符号整数（用补码表示）时,其值为-111 1010B＝-112；

（3）四种运算可以利用同一个加法器及辅助电路实现，n 位加法器实现的是模 2^n无符号整数加法运算。对于无符号整数 a 和 b，a+b 可以直接用加法器实现，而 a-b＝a+[-b]$_{补}$（mod2^n）实现；对于带符号整数用补码表示，补码加减运算公式为：[a+b]$_{补}$＝[a]$_{补}$+[b]$_{补}$（mod2^n）,[a-b]$_{补}$＝[a]$_{补}$+[-b]$_{补}$（mod2^n）,所以四种运算都可在 n 位加法器中实现。

（4）判断溢出的方法有 3 种：一位符号位、进位位和双符号位。上述程序段中只有 int k2＝m+n 语句会发生溢出，因为 2 个带符号整数均为负数，它们相加之后，结果小于 8 位二进制所能表示的最小负数。

44.（12 分）某计算机存储器按字节编址，虚拟（逻辑）地址空间大小为 16MB，主存（物理）地址空间大小为 1MB，页面大小为 4KB；Cache 采用直接映射方式，共 8 行；主存与 Cache 之间交换的块大小为 32B。系统运行到某一时刻时，页表的部分内容和 Cache 的部分内容分别如题 44-a 图、题 44-b 所示，图中页框号及标记字段的内容为十六进制形式。

虚页号	有效位	页框号	…
0	1	06	…
1	1	04	…
2	1	15	…
3	1	02	…
4	0	–	…
5	1	2B	…
6	0	–	…
7	1	32	…

题 44-a 图页表的部分内容

行号	有效位	标记	…
0	1	020	…
1	0	–	…
2	1	01D	…
3	1	105	…
4	1	064	…
5	1	14D	…
6	0	–	…
7	1	27A	…

题 44-b 图 Cache 的部分内容

请回答下列问题。

（1）虚拟地址共有几位，哪几位表示虚页号?物理地址共有几位，哪几位表示页框号（物理页号）?

（2）使用物理地址访问 Cache 时，物理地址应划分哪几个字段?要求说明每个字段的位数及在物理地址中的位置。

（3）虚拟地址 001C60H 所在的页面是否在主存中?若在主存中，则该虚拟地址对应的物理地址是什么?访问该地址时是否 Cache 命中?要求说明理由。

（4）假定为该机配置一个 4 路组相联的 TLB，该 TLB 共可存放 8 个页表项，若其当前内容（十六进制）如题 44-c 图所示，则此时虚拟地址 024BACH 所在的页面是否在主存中?要求说明理由。

组号	有效位	标记	页框号	有效位	标记	页框号	有效位	标记	页框号	有效位	标记	页框号
0	0	–	–	1	001	15	0	–	–	1	012	1F
1	1	013	2D	0	–	–	1	008	7E	0	–	–

题 44-c 图 TLB 的部分内容

解：（1）由于页面大小为 4KB，页内地址需要 12 位，所以虚拟地址 24 位，其中虚页号占 12 位；物理地址 20 位，其中页框号（实页号）占 8 位。

（2）主存物理地址 20 位，从左至右应划分 3 个字段：标记字段、块号字段、块内地址字段。其中标记 12 位，块号 3 位，块内地址 5 位。

（3）虚拟地址 001C60H＝0000 0000 0001 1100 0110 0000B，该虚拟地址的虚页号为 001H，查页表可以发现，虚页号 1 对应的有效位为“1”，表明此页在主存中，页框号为 04H，对应的 20 位物理地址是 04C60H＝00000100110001100000B。

访问该地址时，Cache 不命中，因为 Cache 采用直接映射方式，对应的物理地址应该映射到 Cache 的第 3 行中，其有效位为 1，标记值 105H≠04CH（物理地址高 12 位），故访问该地址时 Cache 不命中。

（4）虚拟地址 024BACH＝000000100100101110101100B，虚页号为 024H，TLB 中存放 8 个页表项，采用 4 路组相联，即 TLB 分为 2 组，每组 4 个页表项。12 位虚页号字段中最低位作为组索引，其余 11 位为标记位。现在最低位为 0，表明选择第 0 组，11 位的标记为 012H，根据标记可以知道 TLB 命中，所在的页面在主存中。因为如果在 TLB 中查到了页表项，即 TLB 命中，说明所在页一定命中

45．（8 分）某银行提供 1 个服务窗口和 10 个供顾客等待的座位。顾客到达银行时，若有空座位，则到取号机上领取一个号，等待叫号。取号机每次仅允许一位顾客使用。当营业员空闲时，通过叫号选取一位顾客，并为其服务。顾客和营业员的活动过程描述如下：

```
cobegin
{
    process 顾客i
    {
        从取号机获得一个号码;
        等待叫号;
        获得服务;
    }
    process 营业员
    {
        while (TRUE)
        {
            叫号;
            为顾客服务;
        }
    }
}coend
```

请添加必要的信号量和 P、V（或 wait（）、signal（））操作，实现上述过程中的互斥与同步。要求写出完整的过程，说明信号量的含义并赋初值。

解：（1）互斥资源：取机号，故设一个互斥信号量 mutex。

（2）同步问题：顾客需要获得空座位等待叫号，当营业员空闲时，将选取一位顾客为其服务。空座位的有、无影响等待顾客数量，顾客的有、无决定两营业员是否能开始服务。另外，顾客获得空座位后，需要等待叫号和被服务，顾客与营业员就服务何时开始有同步关系。设信号量 teller，customer 和 mutex 初值分别为 0，0 和 1，设 waiting 为整型量，表示排队的储户数量，其初始为 0，表示顾客初始时为 0，最大不超过 10（10 把座椅），各进程的具体实现如下所示：

```
#define CHAIRS = 10                    //座椅数,也是最多排队的储户数
typedef int semaphore                  //定义信号量
semaphore teller = 0;                  //等待储户的柜员资源
semaphore customer = 0;                //排队等待服务的储户数量
semaphore mutex = 1;                   //对排队机操作的互斥量
int waiting = 0;                       //在座椅上休息等待的储户数
cobegin
{   void customer( int i )             //储户进程
    {   P(mutex);                      //先获得排队机
        if(waiting < CHAIRS )          //若还有座椅则取号
        {   waiting = waiting + 1;     //取号,占用座椅等待叫号
            V(customer);               //告知系统储户加 1
            V(mutex);                  //释放排队机
            P(teller);                 //等待柜员叫号
            serviced( );               //进入窗口被服务
        }
        else                           //若没有座椅了,则不取号
        {
            V(mutex);                  //不取号,释放排队机
        }
    }                                  //离开
    void teller( )
    {   while(TRUE)                    //并发调度无限循环
        {   P(customer);               //叫号
            P(mutex);                  //需要获得排队机的控制权
            waiting = waiting - 1;     //将等候的顾客数减 1
            V(teller);                 //提供柜员服务
            V(mutex);                  //释放排队机
            service( );                //为储户服务
        }
    }
}coend
```

46. （7 分）某文件系统为一级目录结构，文件的数据一次性写入磁盘，已写入的文件不可修改，但可多次创建新文件。请回答如下问题。

（1）在连续、链式、索引三种文件的数据块组织方式中，哪种更合适?要求说明理由。为定位文件数据块。需要在 FCB 中设计哪些相关描述字段?

（2）为快速找到文件，对于 FCB，是集中存储好，还是与对应的文件数据块连续存储好?要求说明理由。

解：根据题目所给条件，文件系统为一级目录结构，文件的数据一次性写入磁盘，已写入的文件不可修改，但是可以多次创建新文件，我们得知该文件系统是不能修改原文件的，只能将修改后的文件按新文件来存储，这与一次刻录光盘的存储方式相似。对于这样的系统，因为不需要随时添加或删除文件的内容，所以一次写入的文件的大小是固定不变的，也是可预知的，而连续存放文件的方式就有其优点。这种方式只需要知道文件的起始地址和文件的大小，便可以通过计算的方法找到文件的任何位置。文件若需要修改，则原文件作废，修改以后的文件以新文件的形式重新写入，不会产生存储碎片，高效，高利用率。所以，如下作答。

（1） 连续的数据块组织方式更合适，因为文件的数据一次性写入磁盘，已写入的文件不可修改，但是可以多次创建新文件，我们得知该文件系统是不能修改原文件的，只能将修改后的文件按新文件来存储。不需要随时添加或删除文件的内容，所以一次写入的文件的大小是

固定不变的，也是可预知的。这样，只需要知道文件的起始地址和文件的大小，便可以通过计算的方法访问文件的任意位置。

为定位文件数据块。需要在 FCB 中设计相关描述字段有：<起始块号，块数>或者<起始块号，结束块号>。

（2）将所有的 FCB 集中存放，文件数据集中存放。这样在随机查找文件名时，只需访问 FCB 对应的块，可减少磁头移动和磁盘 I/O 访问次数。

47.（9 分）某主机的 MAC 地址为 00-15-C5-Cl-5E-28，IP 地址为 10.2.128.100（私有地址）。题 47-a 图是网络拓扑，题 47-b 图是该主机进行 Web 请求的 1 个以太网数据帧前 80 个字节的十六进制及 ASCII 码内容。



题 47-a 图网络拓扑

```
0000  00 21 27 21 51 ee 00 15  c5 c1 5e 28 08 00 45 00   .!'!Q... ..^(..E.
0010  01 ef 11 3b 40 00 80 06  ba 9d 0a 02 80 64 40 aa   ...;@... .....d@.
0020  62 20 04 ff 00 50 e0 e2  00 fa 7b f9 f8 05 50 18   b ...P.. ..{...P.
0030  fa f0 1a c4 00 00 47 45  54 20 2f 72 66 63 2e 68   ......GE T /rfc.h
0040  74 6d 6c 20 48 54 54 50  2f 31 2e 31 0d 0a 41 63   tml HTTP /1.1..Ac
```

题 47-b 图以太网数据帧（前 80 字节）

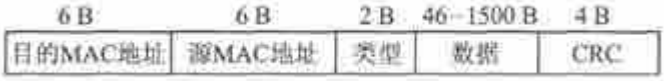
请参考图中的数据回答以下问题。

（1）Web 服务器的 IP 地址是什么?该主机的默认网关的 MAC 地址是什么?

（2）该主机在构造题 47-b 图的数据帧时，使用什么协议确定目的 MAC 地址?封装该协议请求报文的以太网帧的目的 MAC 地址是什么?

（3）假设 HTTP/1.1 协议以持续的非流水线方式工作，以此请求一响应时间为 RTT，rfc.html 页面引用了 5 个 JPEG 小图像，则从发出题 47-b 图中的 Web 请求开始到浏览器收到全部内容为止，需要多少个 RTT?

（4）该帧所封装的 IP 分组经过路由器 R 转发时，需修改 IP 分组头中的哪些字段?注：以太网数据帧结构和 IP 分组头结构分别如题 47-c 图、题 47-d 图所示。



题 47-c 图以太网帧结构



题 47-d 图 IP 分组头结构

解：（1）以太网的数据部分是 IP 数据报，只要找出相应字段所在的字节即可。根据图 47-c 可知以太网头部有 6+6+2＝14 字节，根据图 47-d 可知 IP 地址有 16 字节，从图 47-b 第一个

字节开始数 14+16＝30 字节，得目的 IP 地址为 40.aa.62.20 即 64.170.98.32。而以太网帧的前 6 字节 00-21-27-21-51-ee 是目的 MAC 地址，即为主机的默认网关 10.2.128.1 端口的 MAC 地址。

（2）该主机在构造题 47-b 图的数据帧时，使用 ARP 协议确定目的 MAC 地址。封装该协议请求报文的以太网帧的目的 MAC 地址是广播地址即 FF-FF-FF-FF-FF-FF。

（3）假设 HTTP/1.1 协议以持续的非流水线方式工作，客户机在收到前一个请求的响应后才能发出下一个请求。第一个 RTT 用于请求 Web 页面，客户机收到第一个请求的响应后，每访问一次对象就需一个 RTT。rfc.html 页面引用了 5 个 JPEG 小图像，则从发出题 47-b 图中的 Web 请求开始到浏览开始到浏览器受到全部内容为止，故共需 1+5＝6 个 RTT 后浏览器收到全部内容。

（4）私有地址要和 Internet 上的主机通信时，须由 NAT 路由器进行网络地址转换，转换为一个全球 IP 地址。IP 数据报没经过一个路由器，生存时间 TTL 值就减少 1，并重新计算首部校验和。所以需修改的信息有源 IP 地址，头部校验和，生存时间。

2010 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解

一、单项选择题：1～40 小题。每小题 2 分，共 80 分。下列每题给出的四个选项中。只有一个选项是最符合题目要求的。

1．若元素 a，b，c，d，e，f 依次进栈，允许进栈、退栈操作交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是（　　）。

A．d，c，e，b，f，a

B．c，b，d，a，e，f

C．b，c，a，e，f，d

D．a，f，e，d，c，b

【答案】D 查看答案

【解析】4 个选项所给序列的进、出栈操作序列分别为：

选项 A．Push，Push，Push，Push，Pop，Pop，Push，Pop，Pop，Push，Pop，Pop

选项 B．Push，Push，Push，Pop，Pop，Push，Pop，Pop，Push，Pop，Push，Pop

选项 C．Push，Push，Pop，Push，Pop，Pop，Push，Push，Pop，Push，Pop，Pop

选项 D．Push，Pop，Push，Push，Push，Push，Push，Pop，Pop，Pop，Pop，Pop

按照题目要求，不允许连续三次进行退栈操作，所以选项 D 所给序列为不可能得到的出栈顺序。

2．某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作，元素 a，b，c，d，e 依次入此队列后再进行出队操作，则不可能得到的出队序列是（　　）。

A．b，a，c，d，e

B．d，b，a，c，e

C．d，b，c，a，e

D．e，c，b，a，d

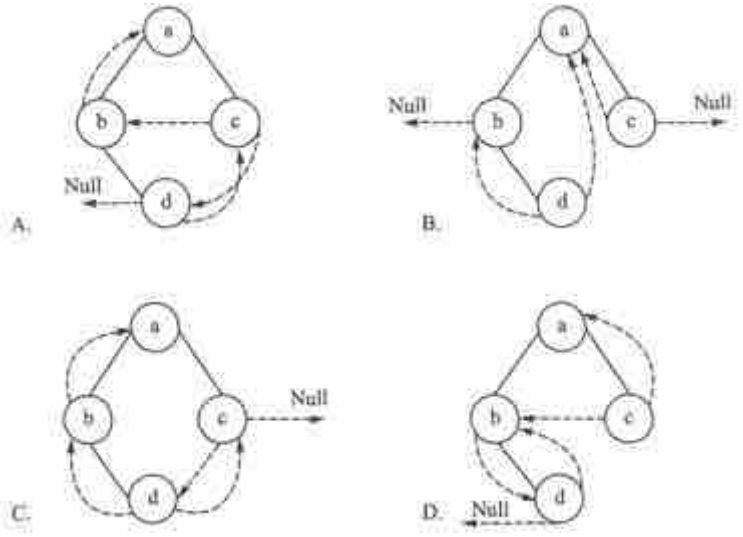
【答案】C 查看答案

【解析】根据题意，队列两端都可以输入数据元素，但是只能在一端输出数据元素，这种队列为输出受限的双端队列。本题解题方法分别判断每个选项如何入队和出队，从而得出不可能的情况。

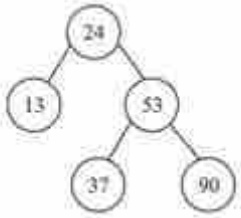
假设 L 代表从左端入队，R 代表从右端入队，出队都是从左端 L 出。四个选项所给序列的进队操作序列分别为：

选项 A．aL（或 aR），bL，cR，dR，eR

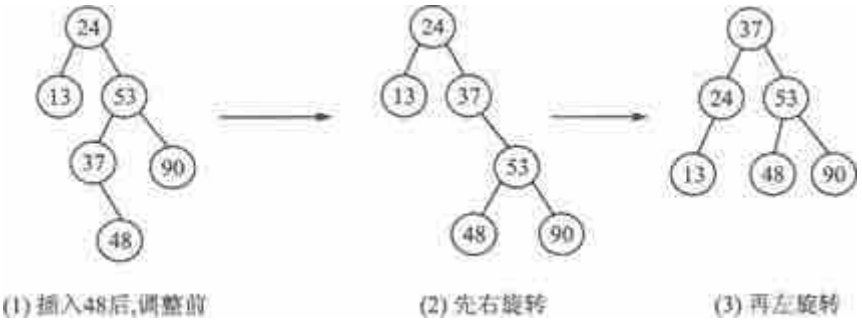
选项 B．aL（或 aR），bL，cR，dL，eR
选项 C．不可能出现
选项 D．aL（或 aR），bL，cL，dR，eL
3．下列线索二叉树中（用虚线表示线索），符合后序线索树定义的是（　　）。



【答案】D 查看答案
【解析】线索二叉树利用二叉链表的空链域来存放结点的前驱和后继信息，解题思路较简单。题中所给二叉树的后序序列为 dbca。结点 d 无前驱和左子树，左链域空，无右子树，右链域指向其后继结点 b；结点 b 无左子树，左链域指向其前驱结点 d；结点 c 无左子树，左链域指向其前驱结点 b，无右子树，右链域指向其后继结点 a。所以正确选项为 D。
4．在下图所示的平衡二叉树中，插入关键字 48 后得到一棵新平衡二叉树。在新平衡二叉树中，关键字 37 所在结点的左、右子结点中保存的关键字分别是（　　）。



A．13、48
B．24、48
C．24、53
D．24、90
【答案】C 查看答案
【解析】题目中，插入 48 以后，树根结点的平衡因子由-1 变为-2，失去平衡。这属于 RL（先右后左）型平衡旋转，需做两次（先右旋后左旋转）旋转操作。过程如下图所示：



(1) 插入48后,调整前　　(2) 先右旋转　　(3) 再左旋转

显然，在调整后的新平衡二叉树中，关键字 37 所在结点的左、右子结点中保存的关键字分别是 24，53。

5．在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度为 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的叶结点个数是（　　）。

A．41

B．82

C．113

D．122

【答案】B 查看答案

【解析】根据二叉树的性质 3 的推广公式：$N_0=l+N_2+2N_3+\ldots+(m-1)N_m$可直接在将数据带入公式，即 $N_0=l+N_2+2N_3+3N4=l+1\times1+2\times10+3\times20=82$。树 T 的叶子结点的个数是 82。如果考生不能熟练掌握二叉树的性质 3 的推广公式，得到本题的正确答案将费时费力。因此，需要熟练掌握二叉树的性质及推广。

6．对 n（n≥2）个权值均不相同的字符构成哈夫曼树。下列关于该哈夫曼树的叙述中，错误的是（　　）。

A．该树一定是一棵完全二叉树

B．树中一定没有度为 1 的结点

C．树中两个权值最小的结点一定是兄弟结点

D．树中任一非叶结点的权值一定不小于下一层任一结点的权值

【答案】A 查看答案

【解析】哈夫曼树为带权路径长度最小的二叉树，但不一定是完全二叉树，选项 A 错误；哈夫曼树中没有度为 1 的结点，选项 B 正确；构造哈夫曼树时，最先选取两个权值最小的结点作为左右子树构造一棵新的二叉树，C 正确；哈夫曼树中任一非叶结点 P 的权值为其左右子树根结点权值之和，其权值不小于其左右子树根结点的权值，在与结点 P 的左右子树根结点处于同一层的结点中，若存在权值大于结点 P 权值的结点 Q，那么结点 Q 与其兄弟结点中权值较小的一个应该与结点 P 作为左右子树构造新的二叉树，由此可知，哈夫曼树中任一非叶结点的权值一定不小于下一层任一结点的权值。

7．若无向图 G=（V，E）中含 7 个顶点，则保证图 G 在任何情况下都是连通的，则需要的边数最少是（　　）。

A．6

B．15

C．16

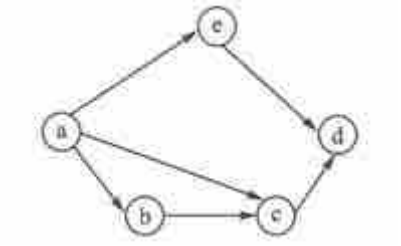
D．21

【答案】C 查看答案

【解析】要保证无向图 G 在任何情况下都是连通的，即任意变动图 G 中的边，G 始终保持连通。首先需要图 G 的任意 6 个结点构成完全连通子图 G_1，需 n（n-l）／2=6×（6—1）

／2＝15 条边，然后再添加一条边将第 7 个结点与 G_1连接起来，共需 l6 条边。本题非常容易错误地选择选项 A，主要原因是对“保证图 G 在任何情况下都是连通的”的理解，分析选项 A，在图 G 中，具有 7 个顶点 6 条边并不能保证其一定是连通图，即有 n-1 条边的图不一定是连通图。分析选项 D，图 G 有 7 个顶点 21 条边，那么图 G 一定是无向完全图，无向完全图能保证其在任何情况下都是连通的，但是这不符合题目中所需边数最少的要求。

8．对下图进行拓扑排序，可以得到不同的拓扑序列的个数是（　　）。



A．4

B．3

C．2

D．1

【答案】B 查看答案

【解析】拓扑排序的步骤为：

（1）在有向图中选一个没有前驱的顶点并且输出它；

（2）从图中删除该顶点和以它为尾的弧。重复上述两步，直至全部顶点均已输出。由于没有前驱的顶点可能不唯一，所以拓扑排序的结果也不唯一。题中所给图有三个不同的拓扑排序序列，分别为 abced，abecd，aebcd。

9．已知一个长度为 l6 的顺序表 L，其元素按关键字有序排列。若采用折半查找法查找一个 L 中不存在的元素，则关键字的比较次数最多是（　　）。

A．4

B．5

C．6

D．7

【答案】B 查看答案

【解析】折半查找法在查找不成功时和给定值进行比较的关键字个数最多为（$\log_2 n$）+1，在本题中，n＝l6，故比较次数最多为 5。

10．采用递归方式对顺序表进行快速排序。下列关于递归次数的叙述中，正确的是（　　）。

A．递归次数与初始数据的排列次序无关

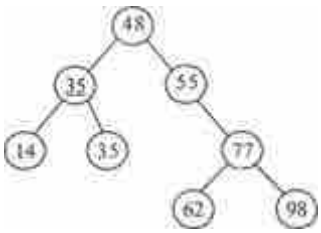
B．每次划分后，先处理较长的分区可以减少递归次数

C．每次划分后，先处理较短的分区可以减少递归次数

D．递归次数与每次划分后得到的分区的处理顺序无关

【答案】D 查看答案

【解析】快速排序是递归的，递归过程可用一棵二叉树给出，递归调用层次数与二叉树的深度一致。例如：待排序列{48，62，35，77，55，14，35，98），采用快速排序方法，其对应递归调用过程的二叉树如下图所示。



在最坏情况下，若初始序列按关键码有序或基本有序时，快速排序反而蜕化为冒泡排序。即其对应递归调用过程的二叉树是一棵单支树。因此快速排序的递归次数与初始数据的排列次序有关。但快速排序的递归次数与每次划分后得到的分区处理顺序无关，即先处理较长的分区或先处理较短的分区都不影响递归次数。

11．对一组数据（2，12，16，88，5，10）进行排序，若前三趟排序结果如下：

第一趟：2，12，16，5，10，88

第二趟：2，12，5，10，16，88

第三趟：2，5，10，12，16，88

则采用的排序方法可能是（　　）。

A．起泡排序

B．希尔排序

C．归并排序

D．基数排序

【答案】A 查看答案

【解析】题目中所给的三趟排序过程，显然是使用起泡排序方法，每趟排序时从前往后依次比较，使大值“沉底”。希尔排序的基本思想是：先对序列进行“宏观调整”，待序列中的记录“基本有序”时再进行直接插入排序。宏观调整的方法是：通过某种规则将大的待排序序列分割为若干小的待排序序列，再依次对这些小的序列直接插入排序。宏观调整可以多次，每次分割的序列数逐渐增多，而每个序列中所包含的元素数逐渐减少。归并排序的基本操作是将多个小的有序序列合并为一个大的有序序列，然后“逐趟归并”，直至整个序列为有序为止。基数排序是分配排序的一种，这类排序不是通过关键字比较，而是通过“分配”和“收集”过程来实现排序的。本题中，很容易看出大值逐渐“沉底”，显然使用的是起泡排序法。

12．下列选项中，能缩短程序执行时间的措施是（　　）。

Ⅰ．提高 CPU 时钟频率

Ⅱ．优化数据通路结构

Ⅲ．对程序进行编译优化

A．仅Ⅰ和Ⅱ

B．仅Ⅰ和Ⅲ

C．仅Ⅱ和Ⅲ

D．Ⅰ、Ⅱ和Ⅲ

【答案】D 查看答案

【解析】一般说来，CPU 时钟频率（主频）越高，CPU 的速度就越快；优化数据通路结构，可以有效提高计算机系统的吞吐量；编译优化可得到更优的指令序列。所以Ⅰ、Ⅱ、Ⅲ都是有效措施。

13．假定有 4 个整数用 8 位补码分别表示为 r_1＝FEH，r_2＝F2H，r_3＝90H，r_4＝F8H。若将运算结果存放在一个 8 位寄存器中，则下列运算会发生溢出的是（　　）。

A．$r_1 \times r_2$

B．$r_2 \times r_3$

C．$r_1 \times r_4$

D．$r_2 \times r_4$

【答案】B 查看答案

【解析】用补码表示时 8 位寄存器所能表示的整数范围为-128～+127。现在 4 个整数都是负数，r_1＝-2，r_2＝-l4，r_3＝-ll2，r_4＝-8，在 4 个选项中，只有 $r_2 \times r_3$＝1568，结果溢出，其余 3 个算式结果都未超过 127，不发生溢出。

14．假定变量 i、f 和 d 的数据类型分为 int、float 和 double（int 用补码表示，float 和 double 分别用 IEEE754 单精度和双精度浮点数格式表示），已知 i＝785，f＝1.5678e3，d＝1.5e100。若在 32 位机器中执行下列关系表达式，则结果为“真”的是（　　）。

（Ⅰ）i＝＝（int）（float）i

（Ⅱ）f＝＝（float）（int）f

（Ⅲ）f＝＝（float）（double）f

（Ⅳ）（d+f）-d＝＝f

A．仅Ⅰ和Ⅱ

B．仅Ⅰ和Ⅲ

C．仅Ⅱ和Ⅲ

D．仅Ⅲ和Ⅳ

【答案】B 查看答案

【解析】数据类型不同的数据在运算之前需要进行数据类型的转换。Ⅱ中，f 的数据类型从 float 转换为 int 时，小数点后面 4 位会丢失，故Ⅱ的结果不为真；Ⅳ中，d+f 时需要对阶，对阶后 f 的尾数有效位被舍去而变为 0，故 d+f 仍然为 d，再减去 d 后结果为 0，故Ⅳ的结果也不为真。Ⅰ和Ⅱ进行数据类型的转换的时候并没有改变其值。

15．假定用若干个 2K×4 位的芯片组成一个 8K×8 位的存储器，则地址 0B1FH 所在芯片的最小地址是（　　）。

A．0000H

B．0600H

C．0700H

D．0800H

【答案】D 查看答案

【解析】由若干芯片构成存储器，采用字和位同时扩展方法。8 片 2K×4 位的芯片分成 4 组，每组 2 个芯片，各组芯片的地址分配分别为：第 l 组，0000H～07FFH；第 2 组，0800H～0FFFH；第 3 组，l000H～17FFH；第 4 组，l800H～1FFFH。地址 0BIFH 处于第 2 组内，其芯片的最小地址为 0800H。

16．下列有关 RAM 和 ROM 的叙述中，正确的是（　　）。

Ⅰ.RAM 是易失性存储器，ROM 是非易失性存储器

Ⅱ.RAM 和 ROM 都采用随机存取方式进行信息访问

Ⅲ.RAM 和 ROM 都可用作 Cache

Ⅳ.RAM 和 ROM 都需要进行刷新

A．仅Ⅰ和Ⅱ

B．仅Ⅱ和Ⅲ

C．仅Ⅰ、Ⅱ和Ⅳ

D．仅Ⅱ、Ⅲ和Ⅳ

【答案】A 查看答案

【解析】RAM 中的内容断电后即丢失（易失性），ROM 中的内容断电后不会丢失（非易失性），同时 RAM 和 ROM 都采用随机存取方式（即 CPU 对任何一个存储单元的存取时间相同），区别在于 RAM 可读可写，ROM 只读不写。而 ROM 显然不可用作 Cache，也不需要刷新，所以Ⅲ和Ⅳ的叙述都是错误的。

17．下列命中组合情况中，一次访存过程中不可能发生的是（　　）。

A．TLB 未命中，Cache 未命中，Page 未命中

B．TLB 未命中，Cache 命中，Page 命中

C．TLB 命中，Cache 未命中，Page 命中

D．TLB 命中，Cache 命中，Page 未命中

【答案】D 查看答案

【解析】TLB（快表）和慢表（页表，Page）构成二级存储系统，若 TLB 命中，则 Page 必命中。因此不可能发生的是 D 选项。

18．下列寄存器中，汇编语言程序员可见的是（　　）。

A．存储器地址寄存器（MAR）

B．程序计数器（PC）

C．存储器数据寄存器（MDR）

D．指令寄存器（IR）

【答案】B 查看答案

【解析】CPU 有 5 个专用寄存器，它们是程序计数器（PC）、指令寄存器（IR）、存储器地址寄存器（MAR）、存储器数据寄存器（MBR）和状态标志寄存器（PSWR），这些寄存器中有些是 CPU 的内部工作寄存器，对汇编语言程序员来说是透明的，在汇编语言程序设计中不会出现。但汇编语言程序员可以通过制定待执行指令的地址来设置 PC 的值,所以程序计数器（PC）对于汇编语言程序员可见的。

19．下列选项中，不会引起指令流水线阻塞的是（　　）。

A．数据旁路（转发）

B．数据相关

C．条件转移

D．资源冲突

【答案】A 查看答案

【解析】由于采用流水线方式，相邻或相近的两条指令可能会因为存在某种关联，后一条指令不能按照原指定的时钟周期运行，从而使流水线断流。有三种相关可能引起指令流水线阻塞：

①结构相关，又称资源相关；

②数据相关；

③控制相关，又称指令相关，主要由转移指令引起。

20．下列选项中的英文缩写均为总线标准的是（　　）。

A．PCI、CRT、USB、EISA

B．ISA、CPI、VESA、EISA

C．ISA、SCSl、RAM、MIPS

D．ISA、EISA、PCI、PCI-Express

【答案】D 查看答案

【解析】选项 A 中的 CRT 和 USB、选项 B 中的 CPI、选项 C 中的 RAM 和 MIPS 均不是总线标准的英文缩写，只有选项 D 中的英文缩写均为总线标准。

21．单级中断系统中，中断服务程序内的执行顺序是（　　）。

Ⅰ保护现场；Ⅱ开中断；Ⅲ关中断；Ⅳ保存断点；Ⅴ中断事件处理；Ⅵ恢复现场；Ⅶ中断返回

A．Ⅰ→Ⅴ→Ⅵ→Ⅱ→Ⅷ

B．Ⅲ→Ⅰ→Ⅴ→Ⅶ

C．Ⅲ→Ⅳ→Ⅴ→Ⅵ→Ⅶ

D．Ⅳ→Ⅰ→Ⅴ→Ⅵ→Ⅶ

【答案】A 查看答案

【解析】程序中断有单级中断和多级中断之分，单级中断在 CPU 执行中断服务程序的过程中不能被打断，即不允许中断嵌套。保存断点与关中断的任务是由硬件（中断隐指令）完成的，所以在单级中断系统中，中断服务程序内应完成的任务有：①保存现场；②中断事件处理；③恢复现场；④开中断；⑤中断返回。

22．假定一台计算机的显示存储器用 DRAM 芯片实现，若要求显示分辨率为 1600×1200，颜色深度为 24 位，帧频为 85Hz，显存总带宽的 50％用来刷新屏幕，则需要的显存总带宽至少约为（　　）。

A．245Mbps

B．979Mbps

C．1958Mbps

D．7834Mbps

【答案】D 查看答案

【解析】显存的容量＝分辨率×色深，带宽＝分辨率×色深×帧频，考虑到 50％的时间用来刷新屏幕，故显存总带宽应加倍。所以需要的显存总带宽至少约为：1600×1200×24×85×2＝7834Mbps。

23．下列选项中，操作系统提供的给应用程序的接口是（　　）。

A．系统调用

B．中断

C．库函数

D．原语

【答案】A 查看答案

【解析】操作系统提供给用户应用程序的接口只有两种：命令输入和系统调用。其中，命令输入又有不同的形式，例如常规的命令行、图形化人机交互接口（GUI）、自然命令用户接口（NUI）等，而系统调用中除了常规的一些传统的系统调用（例如 read（））以外，还有经过扩展的复杂调用（例如多种 API），以及包含在 Lib 库中的各种封装好的过程调用（最终都是通过系统调用陷入到操作系统中去的）等。

24．下列选项中，导致创建新进程的操作是（　　）。

Ⅰ．用户登录成功

Ⅱ．设备分配

Ⅲ．启动程序执行

A．仅Ⅰ和Ⅱ

B．仅Ⅱ和Ⅲ

C．仅Ⅰ和Ⅲ

D．Ⅰ、Ⅱ和Ⅲ

【答案】C 查看答案

【解析】进程创建是需要填写 PCB 表的，其中唯一不需要的是Ⅱ。考察一个进程创建的过程是这样的：当进程被创建，可以是用户创建，例如双击相关图标；也可以由父进程创建，

例如 lock（）时，操作系统首先到 PCB 表区搜索空闲的表格，若无则直接拒绝创建进程，若有则填写 PCB 表创建进程。通常填写 PCB 表的过程有一段时间（主要涉及资源分配需要协调），许多操作系统为此设立了一个中间状态称为“初始化”，也有的操作系统不设这个中间状态。此时操作系统填写进程 ID 号、处理机参数、进程参数（状态、特权、优先级）、分配内存（若是虚拟存储就分配虚拟地址）、映射文件等，一切就绪，将控制权交给系统进行下一步调度。设备分配可能引起进程状态的改变，但不会创建新进程，用户登录成功和启动程序执行都会创建新的进程，所以本题答案为 C。

25．设与某资源相关联的信号量初值为 3，当前为 1，若 M 表示该资源的可用个数，N 表示等待该资源的进程数，则 M，N 分别是（　　）。

A．0、1

B．1、0

C．1、2

D．2、0

【答案】B 查看答案

【解析】信号量初值是 3 表示资源数有 3 个，当前为 1 表示已经用掉 2 个，剩余可用的资源数就只有 1 个了，由于资源有剩余，可见没有其他进程等待使用该资源，故进程数为 0。

26．下列选项中，降低进程优先级的合理时机是（　　）。

A．进程的时间片用完

B．进程刚完成 I/O，进入就绪队列

C．进程长期处于就绪队列

D．进程从就绪状态转为运行态

【答案】A 查看答案

【解析】进程时间片用完可以降低其优先级，完成 I/O 的进程应该提升其优先级，处于就绪队列等待调度的进程一般不会改变其优先级。进行这样的操作主要是为了改善交互式系统的响应时间，并均衡各个作业的公平性。采用时间片轮转技术主要为改善交互式用户的感受，使其觉得是独享计算机（时间片轮转可以有效地防止计算繁忙型的进程独占计算机），时间片用完后降低其优先级是为了改善新进程的响应时间（新进程优先级较高，老进程降低优先级可以保证新进程具有优先权），对于刚进入就绪队列的新进程，往往在创建时已经根据其特点和要求确定好优先级，不会随意改变。而对于从阻塞状态唤醒的进程，由于阻塞带来了较长时间的等待，一般会根据阻塞队列的不同适当地提高优先级，以改善用户响应时间。

27．进程 P0 和 Pl 的共享变量定义及若进程 P0 和 Pl 访问临界资源的类 C 伪代码实现如下：

```
boolean flag[2];
int turn = 0;
flag[0] = FALSE;  flag[1] = FALSE;
```

```
void P0( )      //进程 P0
{while(TRUE){
    flag[0] = TRUE; turn = 1;
    while(flag[1]&&(turn = = 1));
    临界区;
    flag[0] = FALSE;
    }
}
```

```
void P1( )      //进程 P1
{while(TRUE){
    flag[1] = TRUE; turn = 0;
    while(flag[0]&&(turn = = 0));
    临界区;
    flag[1] = FALSE;
    }
}
```

则并发执行进程 P0 和 Pl 时产生的情况是（　　）。

A．不能保证进程互斥进入临界区，会出现“饥饿”现象

B．不能保证进程互斥进入临界区，不会出现“饥饿”现象

C．能保证进程互斥进入临界区，会出现“饥饿”现象

D．能保证进程互斥进入临界区，不会出现“饥饿”现象

【答案】D 查看答案

【解析】这是皮特森算法（Peterson’SAlgorithm）的实现，保证进入临界区的进程合理安全。该算法为了防止两个进程为进入临界区而无限期等待，设置变量 turn，表示不允许进入临界区的编号，每个进程在先设置自己标志后再设置 turn 标志，不允许另一个进程进入，这时，再同时检测另一个进程状态标志和不允许进入标志，这样可以保证当两个进程同时要求进入临界区时只允许一个进程进入临界区。保存的是较晚的一次赋值，则较晚的进程等待，较早的进程进入。先到先人，后到等待，从而完成临界区访问的要求。

28．某基于动态分区存储管理的计算机，其主存容量为 55MB（初始为空闲），采用最佳适配（BestFit）算法，分配和释放的顺序为：分配 15MB、分配 30MB、释放 15MB、分配 8MB、分配 6MB，此时主存中最大空闲分，区的大小是（　　）。

A．7MB

B．9MB

C．10MB

D．15MB

【答案】B 查看答案

【解析】对于简单分区内存分配，需要将进程的所有代码和数据装入内存。故 55MB 先分配 15MB 余 40MB，再分配 30MB 后余 l0MB，释放 15MB 后出现一个 15MB 和一个 10MB 的空闲空间，分配 8MB 时按最佳适配（BestFit）算法应该使用 l0MB 的空闲块，余 2MB 的碎片，分配 6MB 时占用 15MB 的空间余 9MB 的碎片（空闲空间），因此最大空闲区为 9MB。

29．某计算机采用二级页表的分页存储管理方式，按字节编址，页大小为 2^{10}字节，页表项大小为 2 字节，逻辑地址结构为：

页目录号　　页号　　页内偏移量

逻辑地址空间大小为 2^{16}页，则表示整个逻辑地址空间的页目录表中包含表项的个数至少是（　　）。

A．64

B．128

C．256

D．5l2

【答案】B 查看答案

【解析】地址空间分为逻辑地址空间和物理地址空间。页的大小为 2^{10}字节，页表项大小为 2B,采用二级页表，一页可存放 $2^{10}/2=2^9$个页表项，本题中逻辑地址空间大小为 2^{16}字节，故最少需要 $2^{16}/2^9=2^7$128 个页面来保存页表项，故本题答案为 B。

30．设文件索引节点中有 7 个地址项，其中 4 个地址项为直接地址索引，2 个地址项是一级间接地址索引，1 个地址项是二级间接地址索引，每个地址项大小为 4 字节，若磁盘索引块和磁盘数据块的大小均为 256 字节，则可表示的单个文件最大长度是（　　）。

A．33KB

B．519KB

C．1057KB

D．16513KB

【答案】C 查看答案

【解析】4 个地址项为直接地址索引，其指向的数据块大小 4×256B＝1KB，一级间接地址索引可以索引 256/4＝64 个直接地址索引，故 2 个一级间接地址索引指向的数据块大小为 2×64×256B＝32KB，二级间接地址索引为 256/4×256/4＝4096 个直接地址索引，故 1 个二级间接地址索引指向的数据块大小为 4096×256B＝1024KB，共计 1KB+32KB+1024KB＝1057KB。

31．设置当前工作目录的主要目的是（　　）。

A．节省外存空间

B．节省内存空间

C．加快文件的检索速度

D．加快文件的读 / 写速度

【答案】C 查看答案

【解析】工作目录只是指出了当前操作的默认目录，使得在每次访问的时候不需要由根目录一层一层地解析，在文件路径比较长时，可以节省许多解析的时间，从而加快了文件的检索速度。

32．本地用户通过键盘登录系统时，首先获得的键盘输入信息的程序是（　　）。

A．命令解释程序

B．中断处理程序

C．系统调用服务程序

D．用户登录程序

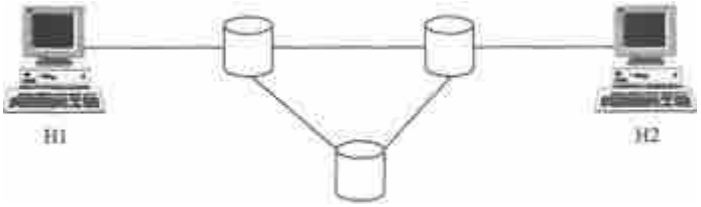
【答案】B 查看答案

【解析】外部设备在与计算机连接时有多种方式，中断技术就是一种常用方式。其工作原理是：利用处理机中断信号线，外部设备在需要服务的时候将该线设置为有效，计算机若同意接受中断则会停止当前进程的运行，转而服务发出中断的物理设备（注意与陷阱，即软中断有区别），那么对不同外部设备进行服务的程序代码是不同的，如何找到这些代码呢?这就要借助中断向量，中断向量一般是由硬件根据中断的类型（不同外设不同）计算所得，或计算机系统在开机配置时所配置的。处理机取得中断向量，其实就是一个物理地址，该地址下存放的是为此中断服务的代码的起始地址。所以，当键盘按下的时候，键盘控制器获得该操作动作，先将键盘扫描码读入键盘缓冲区，再向处理机发出键盘中断，适当的时候（一条指令的末尾或一条原语结束）处理机会响应中断，调用指定服务程序将键盘缓冲区中的键盘扫描码输入到登录进程中去。如此，最先响应键盘的必然是中断处理程序。本题中，像命令解释器（例如 cmd 窗口）、系统调用服务和用户登录程序都在中断处理程序后面。

33．下列选项中，不属于网络体系结构中所描述的内容是（　　）。

A．网络的层次

B．每一层使用的协议
C．协议的内部实现细节
D．每一层必须完成的功能
【答案】C 查看答案
【解析】体系结构仅规定协议的功能和消息格式，但对具体的实现细节由具体设备厂商来确定，对于网络的层次，以及每一个层次的协议及其功能都是网络体系结构所要描述的内容，因此答案为选项 C。
34．在下图所示的采用“存储一转发”方式的分组交换网络中，所有链路的数据传输速率为 100Mbps，分组大小为 1000B，其中分组头大小 20B，若主机 Hl 向主机 H2 发送一个大小为 980000B 的文件，则在不考虑分组拆装时间和传播延迟的情况下，从 H1 发送开始到 H2 接收完为止，需要的时间至少是（　　）。



A．80ms
B．80.08ms
C．80.16ms
D．80.24ms
【答案】C 查看答案
【解析】由题设可知，分组携带的数据长度为 980B，文件长度为 980000B，需拆分为 1000 个分组，加上头部后，每个分组大小为 1000B，总共需要传送的数据量大小为 lMB。由于所有链路的数据传输速度相同，因此文件传输经过最短路径时所需时间最少，最短路径经过分组交换机。当 t＝1M×8/100Mbps＝80ms 时，HI 发送完最后一个比特；到达目的地，最后一个分组，需经过两个分组交换机的转发，每次转发的时间为 t_0＝lK×8/100Mbps＝0.08ms，所以，在不考虑分组拆装时间和传播延时的情况下，当 t＝80ms+2t_0＝80.16ms 时，H2 接受完文件，即所需的时间至少为 80.16ms。
35．某自治系统内采用 RIP 协议，若该自治系统内的路由器 R1 收到其邻居路由器 R2 的距离矢量，距离矢量中包含信息“<net1，16>”，则能得出的结论是（　　）。
A．R2 可以经过 R1 到达 net1，跳数为 17
B．R2 可以到达 net1，跳数为 16
C．R1 可以经过 R2 到达 net1，跳数为 17
D．R1 不能经过 R2 到达 net1
【答案】D 查看答案
【解析】RIP 允许一条路径最多只能包含 l5 个路由器，因此距离等于 16 时相当于不可达，因此 RIP 协议里规定 16 为路由不可达，答案为 D。
36．若路由器 R 因为拥塞丢弃 IP 分组，则此时 R 可向发出该 IP 分组的源主机发送的 ICMP 报文件类型是（　　）。
A．路由重定向
B．目的不可达
C．源抑制
D．超时
【答案】C 查看答案

【解析】当路由器或主机由于拥塞而丢弃数据报时，就向源点发送源点抑制报文，使源点知道把数据报的发送速率放慢，正确选项为 C。

37. 某网络的 IP 地址空间为 192.168.5.0/24，采用定长子网划分，子网掩码为 255.255.255.248，则该网络的最大子网个数、每个子网内的最大可分配地址个数分别是（　　）。

A．32，8

B．32，6

C．8，32

D．8，30

【答案】B 查看答案

【解析】子网号为 5 位，在 CIDR 中可以表示 2^5＝32 个子网，主机号为 3 位，除去全 0 和全 1 的情况可以表示 6 个主机地址，答案为 B。

38．下列网络设备中，能够抑制广播风暴的是（　　）。

Ⅰ．中继器

Ⅱ．集线器

Ⅲ．网桥

Ⅳ．路由器

A．仅Ⅰ和Ⅱ

B．仅Ⅲ

C．仅Ⅲ和Ⅳ

D．仅Ⅳ

【答案】D 查看答案

【解析】中继器和集线器工作在物理层，不能抑制网络风暴。为了解决冲突域的问题，提高共享介质的利用率，通常利用网桥和交换机来分隔互联网的各个网段中的通信量，以建立多个分离的冲突域。但是，当网桥和交换机接收到一个未知转发信息的数据帧时，为了保证该帧能被目的结点正确接收，将该帧从所有的端口广播出去。于是可以看出，网桥和交换机的冲突域等于端口的个数，广播域为 1。因此网桥不能抑制网络风暴。

39．主机甲和主机乙之间已建立了一个 TCP 连接，TCP 最大段长度为 1000 字节，若主机甲的当前拥塞窗口为 4000 字节，在主机甲向主机乙连续发送两个最大段后，成功收到主机乙发送的对第一个段的确认段，确认段中通告的接收窗口大小为 2000 字节，则此时主机甲还可以向主机乙发送的最大字节数是（　　）。

A．1000

B．2000

C．3000

D．4000

【答案】A 查看答案

【解析】发送方的发送窗口的上限值应该取接收方窗口和拥塞窗口这两个值中较小的一个，于是此时发送方的发送窗口为 min{4000，2000）＝2000 字节，由于发送方还没有收到第二个最大段的确认，所以此时主机甲还可以向主机乙发送的最大字节数为 2000-1000＝1000 字节，正确选项为 A。

40．如果本地域名服务无缓存，当采用递归方法解析另一网络某主机域名时，用户主机、本地域名服务器发送的域名请求消息数分别为（　　）。

A．1 条，1 条

B．1 条，多条

C．多条，1 条

D．多条，多条

【答案】A 查看答案

【解析】所谓递归查询方式就是：如果主机所询问的本地域名服务器不知道被查询域名的 IP 地址，那么本地域名服务器就以 DNS 客户的身份向其他服务器继续发出查询请求报文，而不是让该主机自行下一步的查询。所以主机只需向本地域名服务器发送一条域名请求，采用递归查询方法，本地域名服务器也只需向上一级的根域名服务器发送一条域名请求，然后依次递归。正确选项为 A。

二、综合应用题：41—47 小题，共 70 分。

41．（10 分）将关键字序列（7，8，30，11，18，9，14）散列存储到散列表中，散列表的存储空间是一个下标从 0 开始的一维数组。散列函数是：H（key）＝（key×3）MOD7，处理冲突采用线性探测再散列法，要求装填（载）因子为 0.7。

（1）请画出所构造的散列表。

（2）分别计算等概率情况下查找成功和查找不成功的平均查找长度。

解：（1）要求装填因子为 0.7，数组的长度应该为 7/0.7＝10，数组下标为 0～9。各关键字的散列函数值如下表：

key	7	8	30	11	18	9	14
H（key）	O	3	6	5	5	6	O

采用线性探测法再散列法处理冲突，所构造的散列表为：

地址	0	1	2	3	4	5	6	7	8	9
关键字	7	14		8		11	30	18	9	

（2）查找成功时，在等概率情况下，查找表中每个元素的概率是相等的，因此是根据表中元素个数来计算平均查找长度，各关键字的比较次数如下表所示：

关键字	7	8	30	11	18	9	14
次数	l	1	1	1	3	3	2

故查找成功的平均查找长度为（1+1+1+1+3+3+2）/7＝12/7。

在不成功的情况下，由于任意关键字 key，H（key）的值只能是 0～6 之间，H（key）为 0 需要比较 3 次，H（key）为 1 需要比较 2 次，H（key）为 2 需要比较 l 次，H（key）为 3 需要比较 2 次，H（key）为 4 需要比较 l 次，H（key）为 5 需要比较 5 次，H（key）为 6 需要比较 4 次，共 7 种情况，如下表所示：

H（key）	0	1	2	3	4	5	6
次数	3	2	1	2	l	5	4

所以，在等概率下，查找失败的平均查找长度为：（3+2+1+2+1+5+4）/7＝18/7。

42．（13 分）设将 n（n>1）个整数存放到一维数组 R 中。试设计一个在时间和空间两方面都尽可能高效的算法，将 R 中存有的序列循环左移 P（0<P<n）个位置，即将 R 中的数据由（X_0，X_1，…，X_{n-1}）变换为（x_p，x_{p+1}，…，x_{n-1}，x_0，x_1，…，x_{p-1}）。要求：

（1）给出算法的基本设计思想。

（2）根据设计思想，采用 C 或 C++或 JAVA 语言描述算法，关键之处给出注释。

（3）说明你所设计算法的时间复杂度和空间复杂度。

解：（1）算法的基本设计思想：先将 n 个数据由 $x_0,x_1,\ldots,x_p,\ldots,x_{n-1}$ 原地逆置，得到 $x_{n-1},\ldots,x_p,x_{p-1}$，…，$x_0$ 然后再将数组 R 中的前，n-P 个数和后 P 个数分别原地逆置，最终得到结果 $x_p,x_{p+1},\ldots,x_{n-1},x_0,x_1,\ldots,x_{p-1}$。

（2）用 C 语言算法描述如下：

```
void reverse (int R[],int left,int right){
    int k = left,j = right, temp;          //k 等于左边界 left,j 等于右边界 right
    while (k<j){                           //交换 R[k]与 R[j]
        temp = R[k];
        R[k] = R[j];
        R[j] = temp;
        k++;                               //k 右移一个位置
        j--;                               //j 左移一个位置
    }
}
void leftshift (int R[], int n, int p){
    if (p>0 && p<n){
        reverse (R,0,n-1);                 //将全部数据逆置
        reverse (R,0,n-p-1);               //将前 n-p 个元素逆置
        reverse (R,n-p,n-1);               //将后 p 个元素逆置
    }
}
```

（3）说明算法的复杂性：上述算法中 3 个 Reverse 函数的的时间复杂度分别为 O（p/2）、O（（p-2）/2）为 0（n/2），故算法的时间复杂度为 O（n）,算法的空间复杂度为 0（1）。

43.（11 分）某计算机字长 16 位，主存地址空间大小为 128KB，按字编址，采用单字长指令格式，指令各字段定义如下：



转移指令采用相对寻址方式，相对偏移量用补码表示，寻址方式定义如下：

Ms / Md	寻址方式	助记符	含义
000B	寄存器直接	Rn	操作数一（Rn）
001B	寄存器间接	（Rn）	操作数一（（Rn））
010B	寄存器间接、自增	（Rn）+	操作数一（（Rn）），（Rn）+1 一 Rn
011B	相对	D（Rn）	转移目标地址一（PC）+（Rn）

注：（X）表示存储器地址 X 或寄存器 X 的内容。请回答下列问题：

（1）该指令系统最多可有多少条指令?该计算机最多有多少个通用寄存器?存储器地址寄存器（MAR）和存储器数据寄存器（MDR）至少各需要多少位?

（2）转移指令的目标地址范围是多少?

（3）若操作码 0010B 表示加法操作（助记符为 add），寄存器 R4 和 R5 的编号分别为 100B 和 101B，R4 的内容为 l234H，R5 的内容为 5678H，地址 1234H 的内容为 5678H，地址 5678H 中的内容为 1234H，则汇编语句“add（R4），（R5）+”（逗号前为源操作数，逗号后为目的操作数）对应的机器码是什么（十六进制表示）?该指令执行后，哪些寄存器和存储单元的内容会改变?改变后的内容是什么?

解：（1）指令操作码占 4 位，则指令系统最多可有 $2^4=16$ 条不同的指令；指令操作上占 6 位，寻址方式占 3 位，于是寄存器编号占 3 位，该计算机最多可以有 $2^3=8$ 个通用寄存器；

主存容量为 128KB,计算机字长为 16 位，故主存有 l28KB/2B＝2^{16}个存储单元，故 MDR 和 MAR 至少各需 l6 位。

（2）由于寄存器字长为 l6 位，所以转移指令的目标地址范围为 0000H～FFFFH。。

（3）汇编语句 add（R4），（R5）+对应的机器码为 O010001100010101B＝2315H，该指令执行后，寄存器 R5 和地址为 5678H 的存储单元的内容会改变，改变后的内容分别为：

（ACC）＝（（R4））+（（R5））＝5678H+1234H＝68ACH

（R5）＝（R5）+1＝5678H+1＝5679H

（1234H）＝（ACC）＝68ACH

44.（12 分）某计算机的主存地址空间大小为 256MB，按字节编址，指令 Cache 和数据 Cache 分离，均有 8 个 Cache 行，每个 Cache 行大小为 64B，数据 Cache 采用直接映射方式。现有两个功能相同的程序 A 和 B，其伪代码如下所示：程序 A：程序 B：

```
int a[256][256];
……
int sum_array1( )
{
    int i,j,sum = 0;
    for(i = 0;i<256;i + + )
      for(j = 0;j<256;j + + )
        sum + = a[i][j];
    return sum;
}
```

```
int a[256][256];
……
int sum_array2( )
{
    int i,j,sum = 0;
    for(j = 0;j<256;j + + )
      for(i = 0;i<256;i + + )
        sum + = a[i][j];
    return sum;
}
```

假定 int 类型数据用 32 位补码表示，程序编译时 i，j，sum 均分配在寄存器中，数组 a 按行优先方式存放，首地址 320（十进制数）。请回答下列问题，要求说明理由或给出计算过程。

（1）若不考虑用于 Cache 一致性维护和替换算法的控制位，则数据 Cache 的总容量为多少?

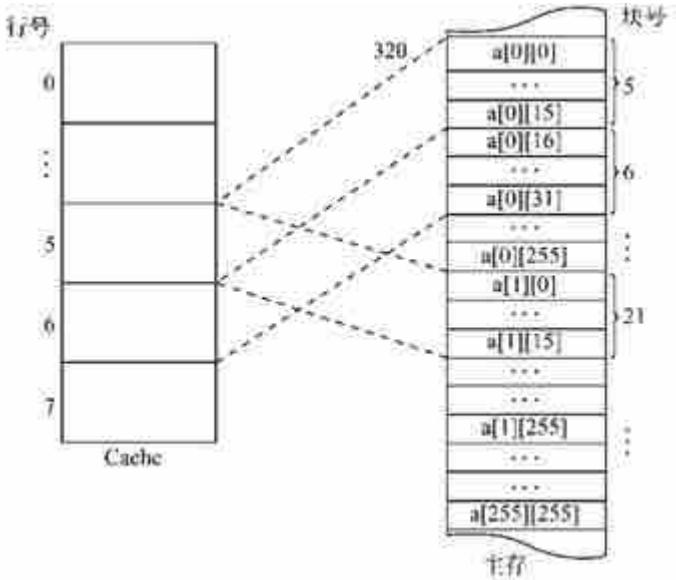
（2）数组数据 a[0][31]和 a[1][1]各自所在的主存块对应的 Cache 行号分别是多少（Cache 行号从 0 开始）?

（3）程序 A 和 B 的数据访问命中率各是多少?哪个程序的执行时间更短?

解：

（1）每个 Cache 行对应一个标记项，标记项包括有效位、脏位、替换控制位以及标记位。由主存空间大小为 256M 可知地址总长度为 28 位，其中块内地址为 $\log_2 64$＝6 位，Cache 块号为 $\log_2 8$＝3 位，不考虑一致性维护和替换算法的控制位，则 Tag 的位数为 28-6-3＝19 位，还需一位有效位,数据 Cache 共有 8 行，故 Cache 的总容量为 8*（64+20/8）B＝532B

（2）数组 a 在主存的存放位置及其与 Cache 之间的映射关系如下图所示：



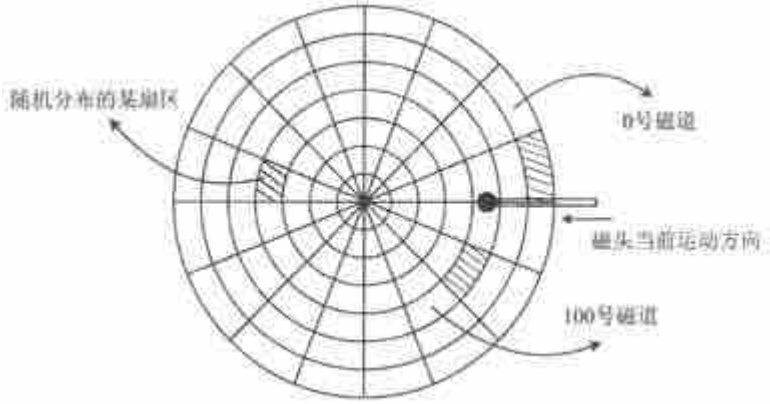
数组按行优先方式存放，首地址为 320，数组元素占 4 个字节。a[0][31]所在的主存块对应的 Cache 行号为（320+31*4）/64＝6；a[1][1]所在的主存块对应的 Cache 行号为（320+ 256*4+1*4）/64%8＝5。

（3）数组 a 的大小为 256*256*4B＝2^{18}B，占用 2^{18}/64＝212 个主存块，按行优先存放，程序 A 逐行访问数组 a,共需访问的次数为 2^{16}次，每个字块的第一个数未面中，因此未面中次数为 2^{12}次，程序 A 的数据访问命中率为（2^{16}-2^{12}）/2^{16}*100%＝93.75%；Cache 总容量为 64B*8＝512B，数组 a 一行的大小为 1KB 正好是 Cache 容量的 2 倍，可知不同行的同一列数组元素使用的是同一个 Cache 单元，而程序 B 逐列访问数组 a 的数据时,都会将之前的字块置换出，也即每次访问都不会面中，故程序 B 的数据访问命中率是 0，因此程序 A 的执行过程更短。

45．（7 分）假设计算机系统采用 CSCAN（循环扫描）磁盘调度策略。使用 2KB 的内存空间记录 16384 个磁盘块的空闲状态。

（1）请说明在上述条件下如何进行磁盘空闲状态的管理。

（2）设某单面磁盘旋转速度为每分钟 6000 转，每个磁道有 100 个扇区，相邻磁道间的平均移动时间为 1ms。若在某时刻，磁头位于 100 号磁道处，并沿着磁道号增大的方向移动（如下图所示），磁道号请求队列为 50，90，30，120，对请求队列中的每个磁道需要读取 1 个随机分布的扇区，则读完这 4 个扇区总共需要多少时间?要求给出计算过程。



（3）如果将磁盘替换为随机访问的 Flash 半导体存储器（如 U 盘、SSD 等），是否有比 CSCAN 更高效的磁盘调度策略?若有，给出磁盘调度策略的名称并说明理由；若无，说明理由。

解：

（1）采用位示图法管理磁盘空闲块，每一位表示一个磁盘块的状态，共需要 16384/32＝512 个字节即 2KB 的空间,正好可放在系统提供的内存中。

（2）采用 CSCAN 调度算法，访问磁道的顺序和移动的磁道数如下表所示：

被访问的下一个磁道号	移动距离（磁道数）
120	20
30	90
50	20
90	40

故移动的磁道数为 20+90+20+40＝170，所花的时间为 170ms。由于转速为 6000r/min，则平均旋转延迟为 5ms,总的旋转延迟时间为 20ms,读取一个扇区的平均时间为 0.1ms,故读取 4 个扇区所花的时间为 0.4ms。综上所述，读取磁道上所有扇区所花的总时间为 170+5*4+0.4＝190.4ms。

（3）采用先来先服务（FCFS）调度策略更高效，因为 Flash 半导体存储器的物理结构不需要考虑寻道时间和旋转延迟，可直接按 I/O 请求的先后顺序服务。

46．（8 分）设某计算机的逻辑地址空间和物理地址空间均为 64KB，按字节编址。若某进程最多需要 6 页（Page）数据存储空间，页的大小为 1KB，操作系统采用固定分配局部置换策略为此进程分配 4 个页框（PageFrame）。在时刻 260 前的该进程访问情况如下表所示（访问位即使用位）。

页号	页框号	装入时刻	访问位
O	7	130	1
1	4	230	1
2	2	200	1
3	9	160	1

当该进程执行到时刻 260 时，要访问的逻辑地址为 17CAH 的数据，请回答下列问题：

（1）该逻辑地址对应的页号是多少?

（2）若采用先进先出（FIFO）置换算法，该逻辑地址对应的物理地址是多少?要求给出计算过程。

（3）若采用时钟（CLOCK）置换算法，该逻辑地址对应的物理地址是多少?要求给出计算过程。（设搜索下一页的指针沿顺时针方向移动，且当前指向 2 号页框，如图所示）



解：

（1）由题可知计算机的逻辑地址空间和物理地址空间均为 64KB＝216B，按字节编址，并且页的大小为 1K＝210,故逻辑地址和物理地址的地址格式均为:

页号/页框号（6bit）　　页内偏移量（10bit）

地址 17CA＝0001 0111 1100 1010B,故可知其逻辑页号为 000101B＝5。

（2）根据 FIFO 算法，需要替换出最早装入的页，故需置换 0 号页，将 5 号页装入 7 号页框中，所以物理地址为 0001 1111 1100 1010B＝1FCAH。

（3）根据 CLOCK 算法，如果当前指针所指页框的使用位为 0，则替换该页，否则将使用位清零，并将指针指向下一个页框，继续查找。由题可知，将从 2 号页框开始，前 4 次查找页框号的顺序为 2、4、7、9,并将对应页框的使用位清零。在第 5 次查找中，指针指向 2 号页框，因 2 号页框的使用位已经为 0，故将 2 号页框的页将 5 号装入 2 号页框，并将其对应使用位设置为 1，所以对应的物理地址为 0000 1011 1100 1010B＝0BCAH。

47．（9 分）某局域网采用 CSMA／CD 协议实现介质访问控制，数据传输率为 10Mbps，主机甲和主机乙之间的距离为 2km，信号传播速度是 200000km/s。请回答下列问题，要求说明理由或写出计算过程。

（1）若主机甲和主机乙发送数据时发生冲突，则从开始发送数据时刻起，到两台主机均检测到冲突时刻止，最短需经过多长时间?最长需经过多长时间?（假设主机甲和主机乙发送数据过程中，其他主机不发送数据）

（2）若网络不存在任何冲突与差错，主机甲总是以标准的最长以太网数据帧（1518 字节）向主机乙发送数据，主机乙每成功收到一个数据帧后立即向主机甲发送一个 64 字节的确认帧，主机甲收到确认帧后方可发送下一个数据帧，此时主机甲的有效数据传输速率是多少?（不考虑以太网帧的前导码）

解：（1）当甲乙两台主机同时向对方发送数据时，两台主机均检测到冲突的时间最短：T_{min}＝（1km/200000km/s）×2＝10μs；当一台主机发送的数据就要到达另一台主机时，另一台主机才发送数据，两台主机均检测到冲突的时间最长：T_{min}＝（2km/200000km/s）×2＝20μs

（2）有效数据传输速率＝发送的有效数据／发送有效数据所用的总时间。发送的有效数据＝l500B＝1500×8bit＝12000bit；发送 l518B 的发送时间＝l518×8/10Mbps＝1214.4μs；数据帧的传播时间＝2km/200000km/s＝10μs；确认帧的发送时间＝64×8/10Mbps＝51.2μs ；确认帧的传播时间＝2km/200000km/s＝10μs ； 发送 l518B 所用的总时间为 1214.4μs+10μs+10μs+51.2μs＝1285.6μs 主机甲的有效数据传输率为 l2000bit／1285.6μs＝9.33Mbps。

2009 年全国硕士研究生入学统一考试计算机科学与技术学科联考计算机学科专业基础综合真题及详解

一、单项选择题：1～40 小题，每小题 2 分，共 80 分。下列每题给出的四个选项中。只有一个选项是最符合题目要求的。

1．为解决计算机主机与打印机之间速度不匹配问题，通常设置一个打印数据缓冲区，主机将要输出的数据依次写入该缓冲区，而打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应该是（　　）。

A．栈

B．队列

C．树

D．图

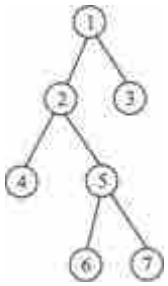
【答案】B 查看答案

【解析】这类问题一般都先分析题目中的数据具有什么操作特性或是结构特性比如“先进后出”、“先进先出”等再判断其逻辑结构。栈和队列是操作受限的线性表，栈具有先进后出的特性而队列具有先进先出的特性。由于本题中先进入打印数据缓冲区的文件先被打印，因此打印数据缓冲区具有先进先出性，则它的逻辑结构应该是队列。

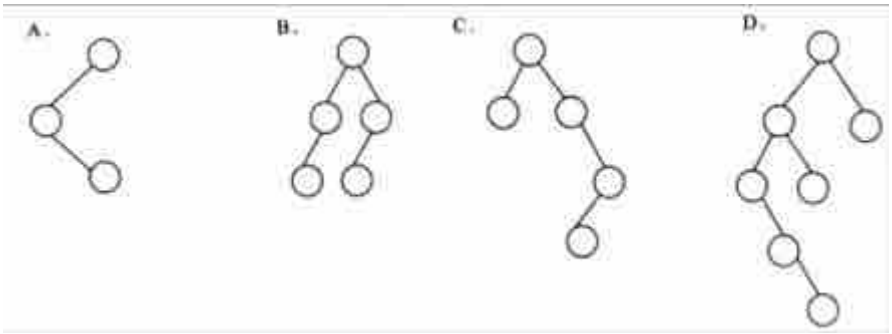
2．设栈 S 和队列 Q 的初始状态均为空，元素 a，b，c，d，e，f，g 依次进入栈 S。若每个元素出栈后立即进入队列 Q，且 7 个元素出队的顺序是 b，d，c，f，e，a，g，则栈 S 的容量至少是（　　）。

A．1

B．2
C．3
D．4
【答案】C 查看答案
【解析】由于栈具有先进后出的特性，队列具有先进先出的特性，出队顺序即为入队顺序。在本题中，每个元素出栈 S 后立即进入队列 Q，出栈顺序即为入队顺序，所以本题中队列的作用形同虚设，根据题意出队顺序即为出栈顺序。根据出栈顺序可以分析各个元素进出栈的过程：第一个出栈元素为 b，表明栈内还有元素 a，b 出栈前的深度为 2；第二个出栈元素为 d，栈内元素为 a 和 c，d 出栈前的深度为 3；c 出栈后，剩余元素为 a，c 出栈前的深度为 2；f 出栈后，剩余元素为 a 和 e，f 出栈前的深度为 3；e 出栈后，剩余元素为 a，e 出栈前的深度为 2；a 出栈后，无剩余元素，a 出栈前的深度为 1；g 出栈后，无剩余元素，g 出栈前的深度为 1。所以栈容量至少是 3。
3．给定二叉树如下图所示。设 N 代表二叉树的根，L 代表根结点的左子树，R 代表根结点的右子树。若遍历后的结点序列为 3，1，7，5，6，2，4，则其遍历方式是（　　）。
A．LRN
B．NRL
C．RLN
D．RNL



【答案】D 查看答案
【解析】对“二叉树”而言，一般有三条搜索路径：
①先上后下的按层次遍历；
②先左（子树）后右（子树）的遍历；
③先右（子树）后左（子树）的遍历。
其中第 1 种搜索路径方式就是常见的层次遍历，第 2 种搜索路径方式包括常见的先序遍历 NLR、中序遍历 LNR、后序遍历 LRN，第 3 种搜索路径方式则是不常使用的 NRL、RNL、RLN。本题考查的是第 3 种搜索路径方式的一种情况。根据遍历的序列以及树的结构图，可以分析出该遍历的顺序是先右子树再跟结点最后左子树，故答案为 D。
4．下列二叉排序树中，满足平衡二叉树定义的是（　　）。



【答案】B 查看答案

【解析】平衡二叉树是指左右子树高度差（平衡因子）的绝对值不超过 1 的二叉树。A 项中根结点的平衡因子是 2；B 项中每个结点的平衡因子的绝对值均不超过 1；C 项中根结点的平衡因子是-2；D 项中根结点的平衡因子是 3。

5．已知一棵完全二叉树的第 6 层（设根为第 1 层）有 8 个叶结点，则该完全二叉树的结点个数最多是（　　）。

A．39

B．52

C．111

D．119

【答案】C 查看答案

【解析】完全二叉树的一个特点是：叶子结点只能出现在最下层和次下层。题目中没有说明完全二叉树的高度，首先由完全二叉树的特点确定题目中树的高度。根据题意，一棵完全二叉树的第 6 层（设根为第 1 层）有 8 个叶结点，可知此二叉树的高度是 6 或 7。题目中求二叉树的结点数最多的情况，因此此完全二叉树的高度为 7。由于高度为 7 的完全二叉树的前 6 层是一棵满二叉树，根据二叉树的性质 2 可知，高度为 6 的满二叉树的结点数是 $2^6-1=63$。又根据二叉树的性质 1 可知，题目中二叉树的第 6 层结点数是 $2^5=32$ 个结点，已知有 8 个叶子结点，那么其余 32-8＝24 个结点均为分支结点，这些结点在第 7 层上最多有 48 个子结点（即叶子结点）。所以此二叉树的结点数最多可达 $2^6-1+(2^5-8)\times2=111$。

6．将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 的父结点的父结点，则在原来的森林中，u 和 v 可能具有的关系是（　　）。

Ⅰ．父子关系

Ⅱ．兄弟关系

Ⅲ.u 的父结点与 v 的父结点是兄弟关系

A．只有Ⅰ

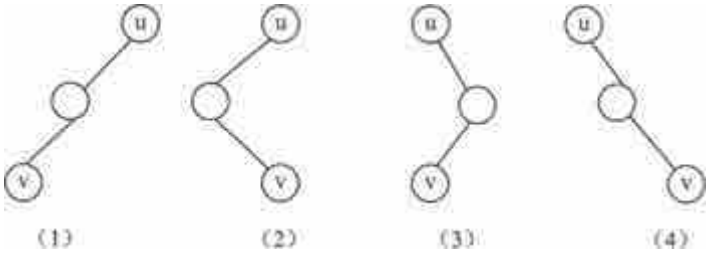
B．Ⅰ和Ⅱ

C．Ⅰ和Ⅲ

D．Ⅰ、Ⅱ和Ⅲ

【答案】B 查看答案

【解析】首先，在二叉树中，若结点 u 是结点 v 的父结点的父结点，那么 u 和 v 的关系有如下 4 种情况：



接下来，根据森林与二叉树的转换规则，将这 4 种情况还原成森林中结点的关系。其中：

情况（1），在原来的森林中 u 是 v 的父结点的父结点；

情况（2），在森林中 u 是 v 的父结点；

情况（3），在森林中 u 是 v 的父结点的兄弟；

情况（4），在森林中 u 与 v 是兄弟关系。

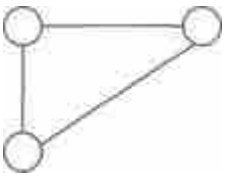
由此可知，题目中的Ⅰ、Ⅱ是正确的。

7．下列关于无向连通图特性的叙述中，正确的是（　　）。

Ⅰ．所有的顶点的度之和为偶数

Ⅱ．边数大于顶点个数减 1
Ⅲ．至少有一个顶点的度为 1
A．只有Ⅰ
B．只有Ⅱ
C．Ⅰ和Ⅱ
D．Ⅰ和Ⅲ
【答案】A 查看答案

【解析】在图中，顶点的度 TD（V_i）之和与边的数目满足关系式：$\sum_{i=1}^{n} TD(V_i)$ ＝2e（n 为图的总结点数，e 为总边数），因此，Ⅰ项正确。对于Ⅱ、Ⅲ项中的特性不是一般无向连通图的特性，可以轻松地举出反例。“至少有一个顶点的度为 1”的反例如下图（1）所示，“边数大于顶点个数减 1”的反例如下图（2）所示。

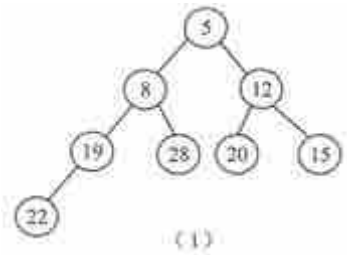


（1）

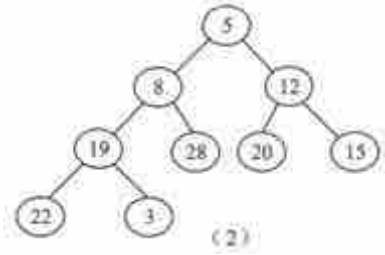


（2）

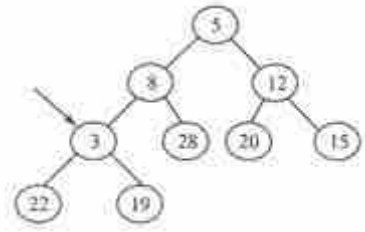
8．下列叙述中，不符合 m 阶 B 树定义要求的是（　　）。
A．根结点最多有 m 棵子树
B．所有叶结点都在同一层上
C．各结点内关键字均升序或降序排列
D．叶结点之间通过指针链接
【答案】D 查看答案
【解析】B 树就是指 B-树。根据 B-树的定义，m 阶 B-树中每个结点最多有 m 个分支，因此，根结点最多有 m 棵子树，A 项正确；B-树中所有叶结点都在最底层，位于同一层，B 项正确；结点内各关键字互不相等且有序排列，C 项正确。但是，所有叶子结点之间通过指针链接，是 B+树的定义，而 B-树中没有。因此，D 项是错误的。
9．已知关键字序列 5，8，12，19，28，20，15，22 是小根堆（最小堆），插入关键字 3，调整后的小根堆是（　　）。
A．3，5，12，8，28，20，15，22，19
B．3，5，12，19，20，15，22，8，28
C．3，8，12，5，20，15，22，28，19
D．3，12，5，8，28，20，15，22，19
【答案】A 查看答案
【解析】在堆中插入或删除一个元素后，将不再满足堆的性质。为了使其成为新堆，在输出堆顶元素后，需要调整剩余元素。具体过程如图（1）～（5）所示，（1）为原堆，（2）为插入 3 后，（3）、（4）为调整过程，（5）为调整后的小根堆。



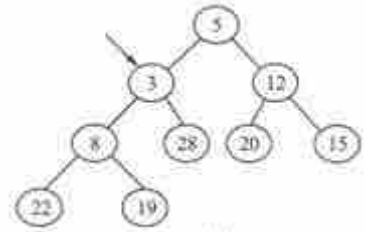
（1）



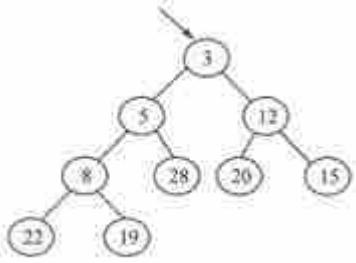
（2）



（3）



（4）



（5）

10．若数据元素序列 11，12，13，7，8，9，23，4，5 是采用下列排序方法之一得到的第二趟排序后的结果，则该排序算法只能是（　　）。

A．起泡排序

B．插入排序

C．选择排序

D．二路归并排序

【答案】B 查看答案

【解析】经过两趟排序后，A 项起泡排序的结果是两个最小或最大的元素放到了序列的最终位置；B 项插入排序的结果是前三个数有序即可；C 项选择排序结果是两个最小的元素在最前面按顺序排好；D 项二路归并排序的结果是长度为 4 的子序列有序，即前 4 个数排好序，接下来的 4 个数排好序。显然题目中的元素序列只能是插入排序第二趟排序后的结果，因此，B 项正确。

11．冯·诺依曼计算机中指令和数据均以二进制形式存放在存储器中，CPU 区分它们的依据是（　　）。

A．指令操作码的译码结果

B．指令和数据的寻址方式

C．指令周期的不同阶段

D．指令和数据所在的存储单元

【答案】C 查看答案

【解析】在冯·诺依曼结构计算机中指令和数据均以二进制形式存放在同一个存储器中，CPU 可以根据指令周期的不同阶段来区分是指令还是数据，通常在取指阶段取出的是指令，其他阶段（分析取数阶段、执行阶段）取出的是数据。所以，CPU 区分指令和数据的依据是指令周期的不同阶段。

12．一个 C 语言程序在一台 32 位机器上运行。程序中定义了 3 个变量 x、Y 和 z，其中 x 和 z 为 int 型，Y 为 short 型。当 x＝127，Y＝-9 时，执行赋值语句 z＝x+Y 后，x、Y 和 z 的值分别是（　　）。

A．x＝0000007FH，Y＝FFF9H，z＝00000076H

B．x＝0000007FH，Y＝FFF9H，z＝FFFF0076H

C．x＝0000007FH，Y＝FFF7H，z＝FFFF0076H

D．x＝0000007FH，Y＝FFF7H，z＝00000076H

【答案】D 查看答案

【解析】当两个不同长度的数据，要想通过算术运算得到正确的结果，必须将短字长数据转换成长字长数据，这被称为“符号扩展”。例如，x 和 z 为 int 型，数据长 32 位，Y 为 short 型，数据长 16 位，因此首先应将 y 转换成 32 位的数据，然后再进行加法运算。

运算采用补码的形式，而 x 的补码是 0000007FH，Y 的补码是 FFFFFFF7H，所以 x+Y＝00000076H 。

13．浮点数加、减运算一般包括对阶、尾数运算、规格化、舍入和判溢出等步骤。设浮点数的阶码和尾数均采用补码表示，且位数分别为 5 位和 7 位（均含 2 位符号位）。若有两个数 X＝2^7×29／32，Y＝2^5×5／8，则用浮点加法计算 X+Y 的最终结果是（　　）。

A．001111100010

B．001110100010

C．010000010001

D．发生溢出

【答案】D 查看答案

【解析】浮点数加、减运算一般包括对阶、尾数运算、规格化、舍入和判溢出等步骤，难点在对阶、规格化、判溢出这三步。X 和 Y 的阶码不同，所以应该先对阶，对阶原则为：小阶向大阶看齐。因此将 Y 对阶后得到：Y＝2^7×5／32，然后将尾数相加，得到尾数之和为：34／32。因为这是两个同号数相加，尾数大于 1，则需要右规，阶码加 1。由于阶码的位数为 5 位，且含两位符号位，即阶码的表示范围在-8～+7 之间。而阶码本身等于 7，再加 1 就等于 8。因此，最终结果发生溢出。

14．某计算机的 Cache 共有 16 块，采用 2 路组相联映射方式（即每组 2 块）。每个主存块大小为 32 字节，按字节编址。主存 129 号单元所在主存块应装入到的 Cache 组号是（　　）。

A．0

B．2

C．4

D．6

【答案】C 查看答案

【解析】首先根据主存地址计算所在的主存块号，然后根据组相联映射的映射关系 K＝I mod Q（K 代表 Cache 的组号，I 代表主存的块号，Q 代表 Cache 的组数）来计算 Cache 的组号。由于每个主存块大小为 32 字节，按字节编址，那么主存 129 号单元所在的主存块号是 4，Cache 共有 16 块，采用 2 路组相联映射方式（即每组 2 块），故 Cache 有 8 组，按照上面的公式可以计算得到 Cache 的组号＝4 mod 8＝4。

15．某计算机主存容量为 64 KB，其中 ROM 区为 4 KB，其余为 RAM 区，按字节编址。现要用 2 K×8 位的 ROM 芯片和 4 K×4 位的 RAM 芯片来设计该存储器，则需要上述规格的 ROM 芯片数和 RAM 芯片数分别是（　　）。

A．1、15

B．2、15

C．1、30

D．2、30

【答案】D 查看答案

【解析】主存储器包括 RAM 和 ROM 两部分，由于 ROM 区为 4KB，则 RAM 区为 60KB。存储容量的扩展方法有字扩展、位扩展、字和位同时扩展三种。选用 2K×8 位的 ROM 芯片，只需采用 2 片芯片进行字扩展便可得到 4KB 的 ROM 区；选用 4K×4 位的 RAM 芯片，需采用（60）/4*2 片芯片进行字和位同时扩展便可得 60KB 的 RAM 区。

16．某机器字长 16 位，主存按字节编址，转移指令采用相对寻址，由两个字节组成，第 1 字节为操作码字段，第 2 字节为相对位移量字段。假定取指令时，每取一个字节 PC 自动加 1。若某转移指令所在主存地址为 2000H，相对位移量字段的内容为 06H，则该转移指令成功转移后的目标地址是（　　）。

A．2006H

B．2007H

C．2008H

D．2009H

【答案】C 查看答案

【解析】相对寻址方式的有效地址 EA＝（PC）+D，其中 PC 为程序计数器，D 为相对偏移量。主存按字节编址，取指令时，每取一个字节 PC 值自动加 1。由于转移指令由两个字节组成，取出这条转移指令之后的 PC 值自动加 2，为 2002H，故转移的目标地址为 2002H+06H＝2008H。

17．下列关于 RISC 的叙述中，错误的是（　　）。

A．RISC 普遍采用微程序控制器

B．RISC 大多数指令在一个时钟周期内完成

C．RISC 的内部通用寄存器数量相对 CISC 多

D．RISC 的指令数、寻址方式和指令格式种类相对 CISC 少

【答案】A 查看答案

【解析】B 项、C 项、D 项都是 RISC 的特点之一，所以它们都是正确的，只有 A 项是 CISC 的特点，因为 RISC 的速度快，所以普遍采用硬布线控制器，而非微程序控制器。

18．某计算机的指令流水线由 4 个功能段组成，指令流经各功能段的时间（忽略各功能段之间的缓存时间）分别为 90ns、80ns、70ns 和 60ns，则该计算机的 CPU 时钟周期至少是（　　）。

A．90ns

B．80ns

C．70ns

D．60ns

【答案】A 查看答案

【解析】对于各功能段执行时间不同的指令流水线，计算机的 CPU 时钟周期应当以最长的功能段执行时间为准。

19．相对于微程序控制器，硬布线控制器的特点是（　　）。

A．指令执行速度慢，指令功能的修改和扩展容易

B．指令执行速度慢，指令功能的修改和扩展难

C．指令执行速度快，指令功能的修改和扩展容易

D．指令执行速度快，指令功能的修改和扩展难

【答案】D 查看答案

【解析】在同样的半导体工艺条件下，硬布线（组合逻辑）控制器的速度比微程序控制器的速度快。这是因为硬布线控制器的速度主要取决于逻辑电路的延迟，而微程序控制器增加了一级控制存储器，执行的每条微指令都要从控制存储器中读取，影响了速度。由于硬布线控制器一旦设计完成就很难改变，所以指令功能的修改和扩展难。因此，硬布线控制器的特点是指令执行速度快，指令功能的修改和扩展难。

20．假设某系统总线在一个总线周期中并行传输 4 字节信息，一个总线周期占用 2 个时钟周期，总线时钟频率为 10MHz，则总线带宽是（　　）。

A．10MB/s

B．20MB/s

C．40MB/s

D．80MB/s

【答案】B 查看答案

【解析】因为一个总线周期占用 2 个时钟周期，完成一个 32 位数据的传送。总线时钟频率为 10MHz，时钟周期为 0.1μs，总线周期占用 2 个时钟周期，为 0.2μs。一个总线周期中并行传输 4 字节信息，则总线带宽是 4B÷0.2μs ＝20MB/s。

21．假设某计算机的存储系统由 Cache 和主存组成。某程序执行过程中访存 1000 次，其中访问 Cache 缺失（未命中）50 次，则 Cache 的命中率是（　　）。

A．5％

B．9.5％

C．50％

D．95％

【答案】D 查看答案

【解析】Cache 的命中率 H＝N_1/（N_1+N_2）,其中 N_1为访问 Cache 的次数，N_2为访存主存的次数，程序总访存次数为 N_1+N_2，程序访存次数减去失效次数就是访问 Cache 的次数 N_1。所以根据公式可得：H＝（1000-50）/1000＝95%。

22．下列选项中，能引起外部中断的事件是（　　）。

A．键盘输入

B．除数为 0
C．浮点运算下溢
D．访存缺页
【答案】A 查看答案
【解析】所谓外部中断是指由外部事件引起的中断，在这 4 个选项中，只有键盘输入是真正由外部事件引起的中断。
23．单处理机系统中，可并行的是（　　）。
Ⅰ．进程与进程
Ⅱ．处理机与设备
Ⅲ．处理机与通道
Ⅳ．设备与设备
A．Ⅰ、Ⅱ和Ⅲ
B．Ⅰ、Ⅱ和Ⅳ
C．Ⅰ、Ⅲ和Ⅳ
D．Ⅱ、Ⅲ和Ⅳ
【答案】D 查看答案
【解析】注意区分并发和并行。在单处理机系统中，进程只能并发。微观上同一时刻占用处理机的进程只有一个，因此，进程之间不是并行的。通道是独立于 CPU 控制的输入／输出的设备，处理机与通道两者是可以并行。显然，设备和设备之间也是可以并行的。
24．下列进程调度算法中，综合考虑进程等待时间和执行时间的是（　　）。
A．时间片轮转调度算法
B．短进程优先调度算法
C．先来先服务调度算法
D．高响应比优先调度算法
【答案】D 查看答案
【解析】时间片轮转法和先来先服务算法都是公平的方法，并未考虑进程等待时间和执行时间，而短进程优先考虑的是进程执行时间。最高响应比优先调度算法是最先执行响应比最高的进程（响应比＝1+等待时间／估计运行时间）。该算法综合了先来先服务（FCFS）和短作业优先（SJF）算法，FCFS 只考虑每个作业的等待时间，而未考虑执行时间的长短。SJF 只考虑执行时间的长短，而未考虑等待时间的长短，HRRN 算法则同时考虑执行时间和等待时间。
25．某计算机系统中有 8 台打印机，由 K 个进程竞争使用，每个进程最多需要 3 台打印机。该系统可能会发生死锁的 K 最小值是（　　）。
A．2
B．3
C．4
D．5
【答案】C 查看答案
【解析】死锁的抽屉原理一般描述是：将 5 个苹果放进 4 个抽屉，那么，必然有 1 个抽屉中至少有 2 个苹果。计算机系统的资源分配充分体现了这一原理。考察进程运行的特点，只要有一个进程能够运行，则运行结束后必然会归还资源，其余的进程也就会得到满足从而可以执行（这里考虑的资源主要是可重用的资源，不可重用的资源会消失，就不可用上述方法分析）。所以最少需要 4 个进程竞争使用，每个进程占用 2 台打印机，此时会产生死锁。
26．分区分配内存管理方式的主要保护措施是（　　）。

A．界地址保护
B．程序代码保护
C．数据保护
D．栈保护
【答案】A 查看答案
【解析】对于连续分配算法，无论固定分区或动态分区方法，程序都必须全部调入内存，不同的进程放于不同的内存块中，相互之间不可越界，因此需要进行界地址保护。通常的界地址保护方法采用软硬件结合的方法。考生要注意本题与虚拟存储方法的区别。
27. 一个分段存储管理系统中，地址长度为 32 位，其中段号占 8 位，则最大段长是（　　）。
A．2^8字节
B．2^{16}字节
C．2^{24}字节
D．2^{32}字节
【答案】C 查看答案
【解析】段内位移的最大值就是最大段长。段号长度占了 8 位，剩下 32-8＝24 位是段内位移空间，因此最大段长为 2^{24}B。
28．下列文件物理结构中，适合随机访问且易于文件扩展的是（　　）。
A．连续结构
B．索引结构
C．链式结构且磁盘块定长
D．链式结构且磁盘块变长
【答案】B 查看答案
【解析】连续结构的优点是结构简单，缺点是不易于文件扩展，不易随机访问。链式结构的优点是文件易于扩展，缺点是不易随机访问。索引结构的优点是具有链式结构的优点并克服了它的缺点，可随机存取，易于文件扩展。
29. 假设磁头当前位于第 105 道，正在向磁道序号增加的方向移动。现有一个磁道访问请求，序列为 35，45，12，68，110，180，170，195，采用 SCAN 调度（电梯调度）算法得到的磁道访问序列是（　　）。
A．110，170，180，195，68，45，35，12
B．110，68，45，35，12，170，180，195
C．110，170，180，195，12，35，45，68
D．12，31，45，68，110，170，180，195
【答案】A 查看答案
【解析】SCAN 算法类似电梯工作原理，即朝一个固定方向前进，经过的磁道有访问请求则马上服务，直至到达一端顶点，再掉头往回移动以服务经过的磁道，并这样在两端之间往返。因此，当磁头从 105 道向序号增加的方向移动时，便会服务所有大于 105 的磁道号（从小到大的顺序）；往回返时又会按照从大到小的顺序进行服务。注意与循环扫描算法的区别，所以 SCAN 算法的访问序列是：110，170，180，195，68，45，35，12。
30．文件系统中，文件访问控制信息存储的合理位置是（　　）。
A．文件控制块
B．文件分配表
C．用户口令表
D．系统注册表
【答案】A 查看答案

【解析】文件控制块是文件存在的标志，文件的相关信息（基本信息、存取控制信息以及使用信息）都存储在文件控制块中，系统对文件的管理全是依靠文件控制块里的信息。

31．设文件 F1 的当前引用计数值为 1，先建立 F1 的符号链接（软链接）文件 F2，再建立 F1 的硬链接文件 F3，然后删除 F1。此时，F2 和 F3 的引用计数值分别是（　　）。

A．0、1

B．1、1

C．1、2

D．2、1

【答案】B 查看答案

【解析】为了使文件实现共享，通常在使用该形式文件系统的文件索引节点中设置一个链接计数字段，用来表示链接到本文件的用户目录项的数目（引用计数值），这是共享的一种方法。当新文件建立时，一般默认引用计数值为 1。硬链接可以看作是已存在文件的另一个名字，新文件和被链接文件指向同一个节点，引用计数值加 1。当删除被链接文件时，只是把引用计数值减 1，直到引用计数值为 0 时，才能真正删除文件。软链接又叫符号链接，在新文件中只包含了被链接文件的路径名，新文件和被链接文件指向不同的节点。建立软链接文件时，文件的引用计数值不会增加。在这种方式下，当被链接文件删除时，新文件仍然是存在的，只不过是不能通过新文件的路径访问被链接文件而已。因此，在本题中，当建立 F2 时，F1 和 F2 的引用计数值都为 1。当再建立 F3 时，F1 和 F3 的引用计数值就都变成了 2。当后来删除 F1 时，F3 的引用计数值为 2-1＝1。F2 的引用计数值仍然保持不变，所以 F2 和 F3 的引用计数值分别是：1,1。

32．程序员利用系统调用打开 I / O 设备时，通常使用的设备标识是（　　）。

A．逻辑设备名

B．物理设备名

C．主设备号

D．从设备号

【答案】A 查看答案

【解析】设备管理具有设备独立性的特点，操作系统以系统调用方式提供给应用程序使用逻辑设备名来请求使用某类设备时，调用中使用的是逻辑设备名，例如 LPT1 或 COM1 等。而操作系统内部管理设备使用的是设备编号。

33．在 OSI 参考模型中，自下而上第一个提供端到端服务的层次是（　　）。

A．数据链路层

B．传输层

C．会话层

D．应用层

【答案】B 查看答案

【解析】题目中指明了这一层能够实现端到端传输，也就是端系统到端系统的传输，数据链路层主要负责传输路径上相邻结点间的数据交付，这些结点包括了交换机和路由器等数据通信设备，这些设备不能被称为端系统，因此数据链路层不满足题意。题目中指明了这一层能够实现传输，会话层只是在两个应用进程之间建立会话而已，应用层只是提供应用进程之间通信的规范，都不涉及传输。所以本题答案应该是 B 项。在 OSI 模型中网络层提供的是主机到主机的通信服务。

34．在无噪声情况下，若某通信链路的带宽为 3kHz，采用 4 个相位，每个相位具有 4 种振幅的 QAM 调制技术，则该通信链路的最大数据传输速率是（　　）。

A．12 kbps

B．24 kbps
C．48 kbps
D．96 kbps
【答案】B 查看答案
【解析】首先要根据信道有无噪声来确定是否采用奈奎斯特定理。解题难点在于离散数值的确定，先确定调制技术的码元数，此处为 4 个相位乘以 4 种振幅，共 16 种，即该通信链路的最大数据传输速率＝2×3×log 2（4×4）＝6×4＝24kbps。
35．数据链路层采用后退 N 帧（GBN）协议，发送方已经发送了编号为 0～7 的帧。当计时器超时，若发送方只收到 0、2、3 号帧的确认，则发送方需要重发的帧数是（　　）。
A．2
B．3
C．4
D．5
【答案】C 查看答案
【解析】后退 N 帧协议，即 GO-BACK—N 策略的基本原理是，当接收方检测出失序的信息帧后，要求发送方重发最后一个正确接收的信息帧之后的所有未被确认的帧；或者当发送方发送了 N 个帧后，若发现该 N 帧的前一个帧在计时器超时后仍未返回其确认信息，则该帧被判为出错或丢失，此时发送方就不得不重新发送出错帧及其后的 N 帧。本题收到 3 号帧的确认，说明 0，1，2，3 号帧已经收到，丢失的是 4，5，6，7 号帧，共 4 帧。因此答案为 C 项。
36．以太网交换机进行转发决策时使用的 PDU 地址是（　　）。
A．目的物理地址
B．目的 IP 地址
C．源物理地址
D．源 IP 地址
【答案】A 查看答案
【解析】交换机会监测发送到每个端口的数据帧，通过数据帧中的有关信息（源结点的 MAC 地址、目的结点的 MAC 地址），就会得到与每个端口所连接结点的 MAC 地址，并在交换机的内部建立一个“端口-MAC 地址”映射表。建立映射表后，当某个端口接收到数据帧后，交换机会读取出该帧中的目的结点的 MAC 地址，并通过“端口-MAC 地址”的对应关系，迅速将数据帧转发到相应的端口，注意这里的交换机工作在数据链路层，因此关于 IP 地址的选项是不对的，因此答案为 A。
37．在一个采用 CSMA／CD 协议的网络中，传输介质是一根完整的电缆，传输速率为 1 Gbps，电缆中的信号传播速度是 200000km/s。若最小数据帧长度减少 800bit，则最远的两个站点之间的距离至少需要（　　）。
A．增加 160m
B．增加 80m
C．减少 160m
D．减少 80m
【答案】D 查看答案
【解析】以太网采用 CSMA／CD 访问协议，在发送的同时要进行冲突检测，这就要求在能检测出冲突的最大时间内数据包不能够发送完毕，否则冲突检测不能有效地工作。所以，当发送的数据包太短时必须进行填充。最小帧长度＝碰撞窗口大小×报文发送速率，本题最小

数据帧长度减少 800b，那么碰撞的窗口也要减少，因此距离也要减少，从而（$800\times2\times10^8$）/（1×10^9）＝160m，由于时间延时存在两倍的关系，因此减少的距离为 80m。

38．主机甲和主机乙间已建立一个 TCP 连接，主机甲向主机乙发送了两个连续的 TCP 段，分别包含 300 字节和 500 字节的有效载荷，第一个段的序列号为 200，主机乙正确接收到两个段后，发送给主机甲的确认序列号是（　　）。

A．500

B．700

C．800

D．1000

【答案】D 查看答案

【解析】TCP 使用滑动窗口流控协议，窗口大小的单位是字节，本题中分别包含 300 字节和 500 字节的有效载荷，第一个段的序列号为 200，那么确认序列号为 200+300+500＝1000。

39．一个 TCP 连接总是以 1KB 的最大段发送 TCP 段，发送方有足够多的数据要发送。当拥塞窗口为 16KB 时发生了超时，如果接下来的 4 个 RTT（往返时间）时间内的 TCP 段的传输都是成功的，那么当第 4 个 RTT 时间内发送的所有 TCP 段都得到肯定应答时，拥塞窗口大小是（　　）。

A．7KB

B．8KB

C．9KB

D．16KB

【答案】C 查看答案

【解析】回顾 TCP 流量控制和拥塞控制（慢启动）的知识点，从第一个 MSS 开始，每次发送成功，拥塞窗口值翻倍，四次以后，应该为 16，但是由于拥塞阈值变为 16 / 2＝8，故三次成功后为 8，以后为线性增长，故为 8+1＝9，答案为 C。

40.FTP 客户和服务器间传递 FTP 命令时，使用的连接是（　　）。

A．建立在 TCP 之上的控制连接

B．建立在 TCP 之上的数据连接

C．建立在 UDP 之上的控制连接

D．建立在 UDP 之上的数据连接

【答案】A 查看答案

【解析】对于 FTP，为了保证可靠性，选择 TCP。FTP 应用需要建立两条 TCP 连接：一条为控制连接，另一条为数据连接。FTP 服务器打开 21 号端口，被动的等待客户的连接建立请求。客户则以主动方式与服务器建立控制连接，客户通过控制连接将命令传给服务器，而服务器则通过控制连接将应答传给客户，命令和响应都是以 NVTASCII 形式表示的。

二、综合应用题：41～47 小题。共 70 分。

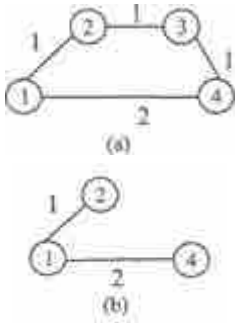
41．（10 分）带权图（权值非负，表示边连接的两顶点间的距离）的最短路径问题是找出从初始顶点到目标顶点之间的一条最短路径，假设从初始顶点到目标顶点之间存在路径，现有一种解决该问题的方法：

①该最短路径初始时仅包含初始顶点，令当前顶点为初始顶点；

②选择离 υ 最近且尚未在最短路径中的顶点 ν，加入到最短路径中，修改当前顶点 υ＝ν；

③重复步骤②，直到 υ 是目标顶点时为止。请问上述方法能否求得最短路径?若该方法可行，请证明之；否则请举例说明。

解：题目中方法不一定能（或不能）求得最短路径。举例说明：



图（a）中，假设初始顶点 1 到目标顶点 4 之间有一条边，权值 x＝2。显然图（a）中这顶点 1 和顶点 4 之间的最短路径长度为 2。若按照题目中给定的方法找到的路径为初始顶点 1 经过中间结点 2、3 到目标顶点 4，即初始顶点 1—2—3 一目标顶点 4，所经过的边的权值分别为 y_1＝1、y_2＝1、y_3＝1。显然，$y_1+y_2+y_3$＝3 大于 2。因此按照题目中给定的方法所求得的路径并不是这两个顶点之间的最短路径。

图（b）中，假设初始顶点为 1、目标顶点为 4，欲求从顶点 1 到顶点 4 之间的最短路径。显然，按照题目中给定的方法无法求出顶点 1 到顶点 4 的路径，而事实上顶点 1 到顶点 4 的最短路径为 1 到 4。

42．（15 分）已知一个带有表头结点的单链表，结点结构为 data | link，假设该链表只给出了头指针 1ist。在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第 k 个位置上的结点（k 为正整数）。若查找成功，算法输出该结点的 data 域的值，并返回 1；否则，只返回 0。要求：

（1）描述算法的基本设计思想；

（2）描述算法的详细实现步骤；

（3）根据设计思想和实现步骤，采用程序设计语言描述算法（使用 C 或 C++或 JAVA 语言实现），关键之处请给出简要注释。

解：（1）算法的基本设计思想定义两个指针变量 p 和 q，初始时均指向头结点的下一个结点。p 指针沿链表移动；当 p 指针移动到第 k 个结点时，q 指针开始与 p 指针同步移动；当 p 指针移动到链表最后一个结点时，因为 p 和 q 相隔 k，故 q 指针所指元素为倒数第 k 个结点。以上过程对链表仅进行一遍扫描。

（2）算法的详细实现步骤

①count＝0，p 和 q 指向链表表头结点的下一个结点；

②若 p 为空，转⑤；，

③若 count 等于 k，则 q 指向下一个结点；否则，count＝count+1；

④p 指向下一个结点，转步骤②；

⑤若 count 等于 k，则查找成功，输出该结点的 data 域的值，返回 1；否则，查找失败，返回 0；

⑥算法结束。

（3）算法实现

```
typedef struct LNode{
int          data;                    //数据域
struct LNode * link;                  //指针域
} * LinkList;
int SearchN1(LinkList List, int k) {
   LinkList p,q;
   int count = 0;                     //计数器赋初值
   p = q = List -> link;              //p和q指向链表表头结点的下一个结点
   while (p! = NULL){
      if (count<k) count + + ;        //计数器+1
      else q = q -> link;             //q移到下一个结点
      p = p -> link;                  //p移到下一个结点
   }
   if (count<k) return 0;             //如果链表的长度小于k,查找失败
   else {
      printf(" %d",p -> data);        //查找成功
```

43.（8分）某计算机的CPU主频为500MHz，CPI为5（即执行每条指令平均需要5个时钟周期）。假定某外设的数据传输率为0.5 MB/s，采用中断方式与主机进行数据传送，以32位为传输单位，对应的中断服务程序包含18条指令，中断服务的其他开销相当于2条指令的执行时间。请回答下列问题，要求给出计算过程。

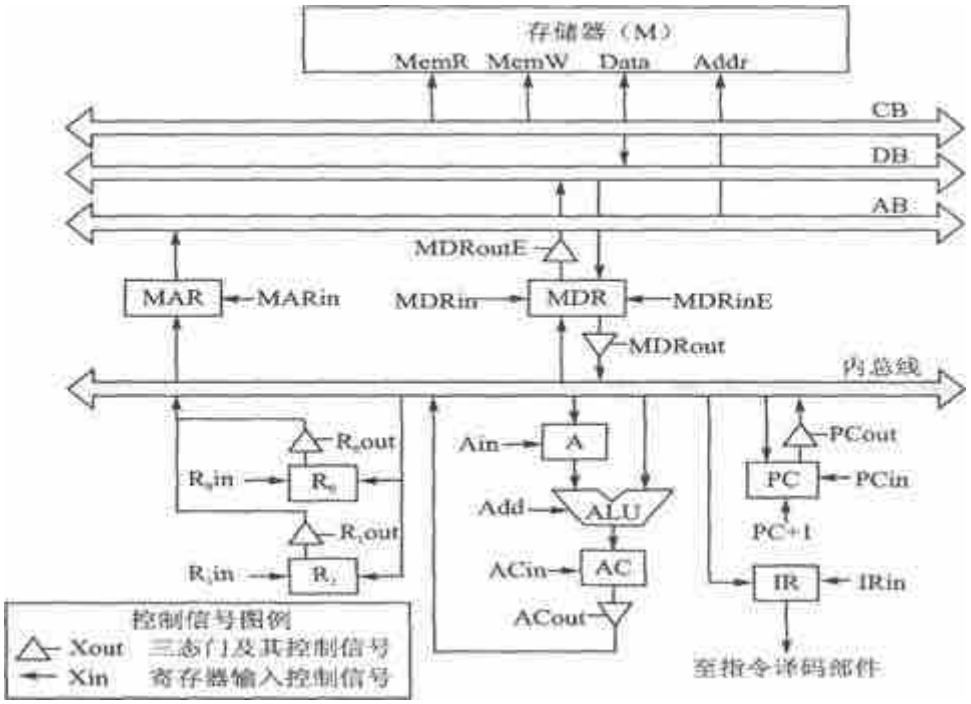
（1）在中断方式下，CPU用于该外设I / O的时间占整个CPU时间的百分比是多少?

（2）当该外设的数据传输率达到5MB/s时，改用DMA方式传送数据。假定每次DMA传送块大小为5000B，且DMA预处理和后处理的总开销为500个时钟周期，则CPU用于该外设I / O时间占整个CPU时间的百分比是多少?（假设DMA与CPU之间没有访存冲突）

解：（1）已知主频为500MHz，则时钟周期＝1÷500MHz ＝2ns，因为CPI＝5，所以每条指令平均5×2＝10ns。又已知每中断一次传送32位（4个字节），数据传输率为0.5MB/s，所以传送时间＝4÷O.5MB/s＝8μs。CPU用于该外设I / O共需20条指令（中断服务程序包括18条指令+其他开销折合2条指令），花费时间＝20×10＝200ns。CPU用于该外设I / O的时间占整个CPU时间的百分比＝200/8000×100％＝0.025×100％＝2.5％。

（2）改用DMA方式传送数据，数据传输率为5MB/s，传送5000B的时间＝5000B÷5MB/s＝1ms。预处理和后处理的总开销时间＝500×2ns＝1μs。CPU用于该外设I / O时间占整个CPU时间的百分比＝预处理和后处理的总开销时间÷传送数据的时间＝1/1000×100％＝0.001×100％＝0.1％。

44.（13分）某计算机字长16位，采用16位定长指令字结构，部分数据通路结构如下图所示，图中所有控制信号为1时表示有效，为0时表示无效，例如控制信号MDRinE为1表示允许数据从DB打入MDR，MDRin为1表示允许数据从内总线打入MDR。假设MAR的输出一直处于使能状态。加法指令“ADD（R1），R0”的功能为（R0）+（（R1））一（R1），即将R0中的数据与R1的内容所指主存单元的数据相加，并将结果送入R1的内容所指主存单元中保存。



下表给出了上述指令取指和译码阶段每个节拍（时钟周期）的功能和有效控制信号，请按表中描述方式用表格列出指令执行阶段每个节拍的功能和有效控制信号。

时钟	功能	有效控制信号
C_1	MAR←（PC）	PCout，MARin
C_2	MDR←M（MAR）PC←（PC）+1	MemR,MDRinEPC+1
C_3	IR←（MDR）	MDROut,IRin
C_4	指令译码	无

解：执行阶段每个节拍（时钟周期）的功能和有效控制信号见下表。

时钟	功能	有效控制信号
C_5	MAR←（R_1）	R_1out，MARin
C_6	MDR←M（MAR）	MemR，MDRinE
C_7	A←（MDR）	MDRout，Ain
C_8	AC←（Ro）+（A）	Roout，Add，ACin
C_9	MDR←（AC）	ACout，MDRin
c_{10}	M←（MDR）	MDRoutE，MemW

45.（7 分）三个进程 P1、P2，P3 互斥使用一个包含 N（N>0）个单元的缓冲区。P1 每次用 produce（）生成一个正整数并用 put（）送入缓冲区某一空单元中；P2 每次用 getodd（）从该缓冲区中取出一个奇数并用 countodd（）统计奇数个数；P3 每次用 geteven（）从该缓冲区中取出一个偶数并用 counteven（）统计偶数个数。请用信号量机制实现这三个进程的同步与互斥活动，并说明所定义信号量的含义。要求用伪代码描述。

解：定义信号量 S1 控制 P1 与 P2 之间的同步；S2 控制 P1 与 P3 之间的同步；empty 控制生产者与消费者之间的同步；mutex 控制进程间互斥使用缓冲区，程序如下：

```
semaphore mutex = 1,odd = 0,even = 0,empty = N;
//缓冲区可用,没有放置奇数和偶数,全空,odd + even + empty = N
main()
  cobegin{                              //并发进程
    process P1                          //生产者进程
    while(true)                         //等待调度
    {  number = produce();              //生产者生产数
       P(empty);                        //有无空间
       P(mutex);                        //能否进入缓冲区
       put();                           //放置数字
       V(mutex);                        //释放缓冲区
       If  number % 2 = = 0             //是否偶数
         V(even);                       //偶数信号量加 1
       else
         V(odd);}                       //否则奇数信号量加 1

  process P2                            //消费者进程 1
    while(true)
    {  P(odd);                          //有无奇数
       P(mutex);                        //能否进入缓冲区
       getodd();                        //取奇数
       V(mutex);                        //释放缓冲区
       V(empty);                        //空间加 1
       countodd();}                     //计算奇数个数

process P3
  while(true)
  {  P(even);                           //有无偶数
     P(mutex);                          //能否进入缓冲区
     geteven();                         //取偶数
     V(mutex);                          //释放缓冲区
     V(empty);                          //空间加 1
     counteven();}                      //计算偶数个数
  }coend                                //并发结束
```

46. （8 分）请求分页管理系统中，假设某进程的页表内容如下表所示：

页号	页框（Page Frame）号	有效位（存在位）
O	101H	1
1		0
2	254H	1

页面大小为 4KB，一次内存的访问时间是 100ns，一次快表（TLB）的访问时间是 10ns，处理一次缺页的平均时间为 108ns（已含更新 TLB 和页表的时间），进程的驻留集大小固定为 2，采用最近最少使用置换算法（LRU）和局部淘汰策略。假设①TLB 初始为空；②地址转

换时先访问 TLB，若 TLB 未命中，再访问页表（忽略访问页表之后的 TLB 更新时间）；③有效位为 0 表示页面不在内存，产生缺页中断，缺页中断处理后，返回到产生缺页中断的指令处重新执行。设有虚地址访问序列 2362H、1565H、25A5H，请问：

（1）依次访问上述三个虚地址，各需多少时间?给出计算过程。

（2）基于上述访问序列，虚地址 1565H 的物理地址是多少?请说明理由。

解：（1）210ns；10^8ns；110ns。

页面大小为 4KB，因此，虚地址的低 12 位是页内偏移，其余高位是页号。

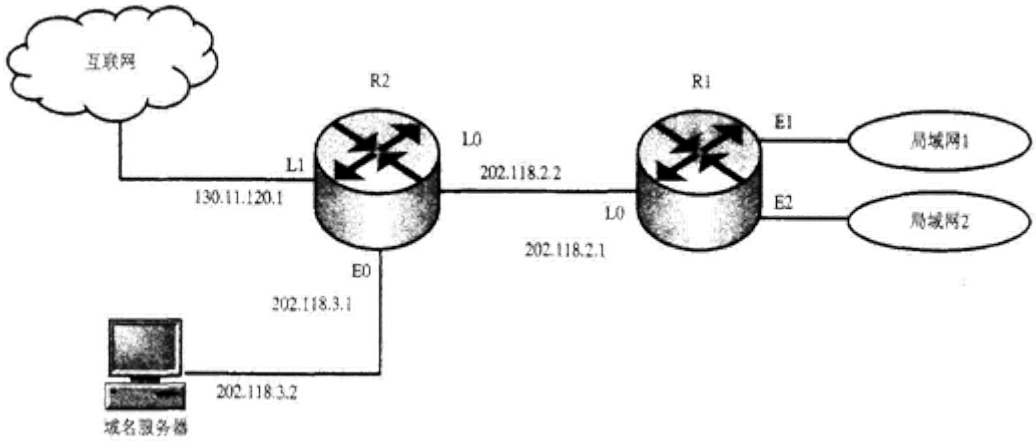
访问虚地址 2362H，虚页号为 2，页内偏移 362H。查找 TLB，TLB 初始为空，未命中，耗时 10ns;访问页表,2 号页面所在页框号为 254H,耗时 100ns;计算得到的物理地址 254362H，访问内存，耗时 100ns。因此，总共用时 10+100+100＝210ns。

访问虚地址 1565H，虚页号为 1，页内偏移 565H。查找 TLB，未命中，耗时 10ns；访问页表，有效位是 0，未命中，耗时 100ns；产生缺页中断，进行缺页中断处理，耗时 108ns；采用 LRU 置换算法，虚页 1 装入页帧号 101H，缺页中断处理完后，再次访问页表，命中，耗时 100ns；计算得到物理地址 101565H，再次访问内存，耗时 100ns。因此，总共用时 $10+100+10^8+100\approx10^8$ns。

访问虚地址 25A5H，虚页号为 2，页内偏移 5A5H。查找 TLB，命中，耗时 10ns；虚页 2 对应的页帧为 254H，因此计算得物理地址为 2545A5H，访问内存，耗时 100ns。因此，总共用时 10+100＝110ns。

（2）当访问虚地址 1565H 时，产生缺页中断，合法驻留集为 2，必须从页表中淘汰一个页面，根据题目的置换算法，应淘汰 0 号页面，因此 1565H 的对应的页框号为 101H，故可知虚地址 1565H 的物理地址为 101565H。

47.（9 分）某公司网络拓扑图如下图所示，路由器 R1 通过接口 E1、E2 分别连接局域网 1、局域网 2，通过接口 10 连接路由器 R2，并通过路由器 R2 连接域名服务器与互联网。R1 的 1O 接口的 IP 地址是 202.118.2.1；R2 的 10 接口的 IP 地址是 202.118.2.2，11 接口的 IP 地址是 130.11.120.1，E0 接口的 IP 地址是 202.118.3.1；域名服务器的 IP 地址是 202.118.3.2。



R1 和 R2 的路由表结构为：

目的网络 IP 地址	子网掩码	下一跳 IP 地址	接口

（1）将 IP 地址空间 202.118.1.0/24 划分为两个子网，分配给局域网 1、局域网 2，每个局域网分配的地址数不少于 120 个，请给出子网划分结果。说明理由或给出必要的计算过程。

（2）请给出 R1 的路由表，使其明确包括到局域网 1 的路由、局域网 2 的路由、域名服务器的主机路由和互联网的路由。

（3）请采用路由聚合技术，给出 R2 到局域网 1 和局域网 2 的路由。

解：（1）把 IP 地址空间 202.118.1.0/24 划分为 2 个等长的字网。两个局域网的 IP 地址数都不能少于 120 个，所以划分子网后，IP 中主机所占的位数不能少于 7 位，要划分为两个子网又子少需要 1 位。可以将 IP 地址空间 202．118．1.0／24 的最后一个字节，划分为 1 位和 7 位两个部分，高一位表示子网号，低 7 位表示主机号，故划分结果为:

①子网 1：子网地址为 202.118.1.0，子网掩码为 255.255.255.128（或者子网 1：202.118.1.0/25）。

②子网 2：子网地址为 202.118.1.128，子网掩码为 255.255.255.128（或者子网 1：202.118.1.128/25）。地址分配方案：子网 1 分配给局域网 1，子网 2 分配给局域网 2；或子网 1 分配给局域网 2，子网 2 分配给局域网 1。每个子网可分配的地址数为 2^7-2＝126 大于 120 满足题目要求。

（2）·填写到局域网 1 和局域网 2 的路由。两个局域网的网络地址和掩码在问题（1）已经求出来了,分别为 202．118．1．0／25 和 202．118．1．128／25。则 R1 路由表应该填入的网络地址为 202．118．1．0 和 202．118．1．128，掩码为 255．255．255．128 和 255.255.255.0。由于两个局域网是分别直接连接到路由器 R1 的 E1 口、E2 口上的，因此，下一跳地址填写直接路由（Direct）。接口填写 E1、E2。

·填写到域名服务器的路由。由于图 6 给出了域名服务器的 IP 地址为 202．118．3．2，而该地址为主机地址，因此掩码为 255．255．255．255。同时，路由器 R1 要到 DNS 服务器，就需要通过路由器 R2 的接口 L0 才能到达，因此下一跳地址填写 L0 的 IP 地址（202．118．2．2）。

·填写互联网路由。本题的实质是编写默认路由，默认路由叫做“0／0”路由，因为路由的 IP 地址是 0．0．0．0，而子网掩码也是 0．0．0．0。同时路由器 R1 连接的网络需要通过路由器 R2 的 L0 口才能到达互联网络，因此下一跳地址填写 L0 的 IP 为 202．118．2．2。

综合以上叙述，所填写的路由表如下表所示：

参考答案一

（若子网 1 分配给局域网 1。子网 2 分配给局域网 2）

目的网络 IP 地址	子网掩码	下一跳 IP 地址	接口
202.118.1.0	255.255.255.128	Direct	E1
202.118.1.128	255.255.255.128	Direct	E2
202.118.3.2	255.255.255.255	202.118.2.2	10
0.0.0.0	0.0.0.0	202.118.2.2	1O

参考答案二

（若子网 1 分配给局域网 2，子网 2 分配给局域网 1）

目的网络 IP 地址	子网掩码	下一跳 IP 地址	接口
202.118.1.0	255.255.255.128	Direct	E2
202.118.1.128	255.255.255.128	Direct	E1
202.118.3.2	255.255.255.255	202.118.2.2	10
0.0.0.0	0.0.0.0	202.118.2.2	10

（3） 填写 R2 到局域网 1 和局域网 2 的路由表 2。局域网 1 和局域网 2 的地址可以聚合为 202.118.1.0／24，而 R2 去往局域网 1 和局域网 2 都是同一条路径。因此，路由表里面只需要填写到 202.118.1.0／24 网络的路由即可，如下表所示：

R2 路由表

目的网络 IP 地址	子网掩码	下一跳 IP 地址	接口

202.118.1.0　255.255.255.0　202.118.2.1　10

第二部分　章节题库

第 1 章　绪　论

一、选择题

1．算法的计算量的大小称为计算的（　　）。

A．效率　B．复杂性　C．现实性　D．难度

【答案】B 查看答案

【解析】算法复杂度通常分为时间复杂度和空间复杂度，算法的计算量的大小可以用时间复杂度衡量，即可以称为计算的复杂度。

2．计算机算法指的是解决问题的步骤序列，它必须具备（　　）三个特性。

A．可执行性、可移植性、可扩充性　B．可执行性、确定性、有穷性

C．确定性、有穷性、稳定性　D．易读性、稳定性、安全性

【答案】B 查看答案

【解析】计算机算法是以一步接一步的方式来详细描述计算机如何将输入转化为所要求的输出的过程，或者说，算法是对计算机上执行的计算过程的具体描述，也就是解决问题的步骤序列。一个算法通常需要具备五大特性：有穷性；确定性；可执行性；输入一个算法有零个或多个输入；输出一个算法有零个或者多个输出。

3．以下说法错误的是（　　）。

（1）算法原地工作的含义是指不需要任何额外的辅助空间

（2）在相同的规模 n 下，复杂度 O（n）的算法在时间上总是优于复杂度 O（2^n）的算法

（3）所谓时间复杂度是指最坏情况下，估算算法执行时间的一个上界

（4）同一个算法，实现语言的级别越高，执行效率就越低

A．（1）　B．（1），（2）　C．（1），（4）　D．（3）

【答案】A 查看答案

【解析】算法原地工作的含义不是指不需要任何额外的辅助,而是算法所需要的辅助空间不随着问题的规模而变化，是一个确定的值。

4．以下与数据的存储结构无关的术语是（　　）。

A．循环队列　B．链表　C．哈希表　D．栈

【答案】D 查看答案

【解析】循环队列体现线性表是以顺序存储。用散列法存储的线性表称散列表。链表说明线性表是以链式结构存储的。栈不能体现出是顺序还是链式存储结构。

5．在下面的程序段中，对 x 的赋值语句的时间复杂度为（　　）

FOR i：=1 TO n DO

FOR j：=1 TO n DO

x：=x+1：

A．O（2n）　B．O（n）　C．O（n^2）　D．O（$\log 2^n$）

【答案】C 查看答案

【解析】两个循环嵌套，那么语句 x：=x+1：则被执行了 n^2次。

6．程序段

FOR i：=n-1 DOWNT0 1 DO

FOR j：=l TO i DO

IF A[j]>A[j+1]

THEN A[j]与 A[j+1]对换；

其中 n 为正整数，则最后一行的语句最坏情况下的时间复杂度是（　　）。

A．D（n）　B．O（nlogn）　C．O（n^3）　D．O（n^2）

【答案】D 查看答案

【解析】这个是冒泡排序，最坏的情况下需要进行 1+2+...+n-1 次交换，即时间复杂度是 O（n^2）。

7．以下数据结构中，（　　）是非线性数据结构。

A．树　B．字符串　C．队　D．栈

【答案】A 查看答案

【解析】非线性结构是指存在一对多或者多对一的关系。常见的非线性结构有树结构和图结构。

8．连续存储设计时，存储单元的地址（　　）。

A．一定连续　B．一定不连续　C．不一定连续　D．部分连续，部分不连续

【答案】A 查看答案

【解析】连续存储是指数据的物理存储相连，即存储单元的地址是连续的。

9．可以用（　　）定义一个完整的数据结构。

A．数据元素　B．数据对象　C．数据关系　D 抽象数据类型

【答案】D 查看答案

【解析】抽象数据类型可以定义一个完整的数据结构。包括数据元素，数据元素之间的关系，以及可以进行的操作。

10．下列不是设计一个“好”的算法应考虑达到的目标是（　　）。

A．可行的　B．健壮的　C．无二义性的　D．可读性好的

【答案】A 查看答案

【解析】设计一个“好”的算法应考虑以下目标：正确性；可读性；健壮性；效率和低存储量需求。可行性是算法的五个基本特征之一，不是一个好的算法该达到的目标。

二、判断题

1．数据元素是数据的最小单位。（　　）

【答案】×查看答案

【解析】数据项是数据的不可分割的最小单位，而数据元素是数据的基本单位。

2．数据的逻辑结构是指数据的各数据项之间的逻辑关系。（　　）

【答案】×查看答案

【解析】数据的逻辑结构是指数据元素之间的逻辑关系。

3．算法的优劣与算法描述语言无关，但与所用计算机有关。（　　）

【答案】×查看答案

【解析】算法的优劣和它的时间复杂度和空间复杂度有关，与算法描述语言和所用计算机都无关。

4．健壮的算法不会因非法的输入数据而出现莫名其妙的状态。（　　）

【答案】√查看答案

【解析】算法的健壮性是指当输入数据非法时，算法能作适当的处理并作出反应，而不应死机或输出异常结果。

5．程序一定是算法。（　　）

【答案】×查看答案

【解析】一个程序不一定满足有穷性。而算法是对问题的解，用程序设计语言来实现来描述，这时算法就是一个程序。

6．数据结构的抽象操作的定义与具体实现有关。（　　）

【答案】×查看答案

【解析】数据结构的抽象操作定义取决于客观存在的一组逻辑特性，与其在计算机内具体表示和实现无关。

7．顺序存储方式的优点是存储密度大，且插入、删除运算效率高。（　　）

【答案】×查看答案

【解析】前者正确，后者错误。顺序存储方式在插入、删除元素时需要挪动大量的元素，执行效率较低。

8．抽象数据类型与计算机内部表示和实现无关。（　　）

【答案】√查看答案

【解析】抽象数据类型只表示数据的逻辑结构，与计算机内部表示和实现无关。

三、填空题

1．对于给定的元素，可以构造出的逻辑结构有（1），（2），（3），（4）四种。

【答案】集合；线性结构；树形结构；图状结构(网状结构)查看答案

2．抽象数据类型的定义仅取决于它的一组（1），而与（2）无关，即不论其内部结构如何变化，只要它的（3）不变，都不影响其外部使用。

【答案】逻辑特性；在计算机内部如何表示和实现；数学特性查看答案

3．数据结构中评价算法的两个重要指标是_____。

【答案】算法的时间复杂度和空间复杂度查看答案

4．数据结构是研讨数据的（1）和（2）以及它们之间的相互关系，并对与这种结构定义相应的（3），设计出相应的（4）。

【答案】逻辑结构；物理结构；操作（运算）；算法查看答案

5．一个算法具有 5 个特性：（l）、（2）、（3）、有零个或多个输入、有一个或多个输出。

【答案】有穷性；确定性；可行性查看答案

6．已知如下程序段：

```
FOR i：=n DOWNTO 1 DO          {语句 1}
BEGIN
x：=x+1；       {语句 2}
FOR j：=n DOWNTO i DO     {语句 3}
Y：=y+1；       {语句 4}
END；
```

语句 1 执行的时间复杂度为（1）；语句 2 执行的时间复杂度为（2）；语句 3 执行的时间复杂度为（3）；语句 4 执行的时间复杂度为（4）。

【答案】（1）n+1 查看答案

（2）n

（3）n(n+3)/2

（4）n(n+1)/2

【解析】语 s 句 1 执行到不符合条件情况下，执行了 n+1 次。当语句 1 不符合条件了是不会执行语句 2 的，所以语句 2 被执行了 n 次。语句 3 每次都要执行到不符合条件，故为 2+3+4……+(n+1)加起来就是 n(n+3)/2。语句 3 不符合条件了是不会执行语句 4 的。所以语句 4 被执行了 1+2+3……+n 即 n(n+1)/2。

7．在下面的程序段中，对 X 的赋值语句的时间复杂度为_____（表示为 n 的函数）。

```
FOR     i：=l TO n DO
FOR     j：=l TO i DO
FOR     k：=1 TO j DO
x：=x+delta；
```

【答案】1+（1+2）+（1+2+3）+…+（1+2+…+n）=n（n+1）（n+2）/ 6，即 O（n^3）查看答案

【解析】当 i=1 时，赋值语句就被执行了一次。当 i=2 时，赋值语句被执行了 1+2 次。当 i=3 时，赋值语句被执行了 1+2+3 次。……可以推出赋值语句总共被执行了 1+（1+2）+（1+2+3）+…+（1+2+…+n）=n（n+1）（n+2）/ 6 次。

8．设 m、n 均为自然数，m 可表示为一些不超过 n 的自然数之和，f（m，n）为这种表示方式的数目。例 f（5，3）=5，有 5 种表示方式：3+2，3+1+1，2+2+1，2+1+1+1，1+1+1+1+1。

①以下是该函数的程序段，请将未完成的部分填入，使之完整。

```
int f（m，n）
int m，n;
{if（m==1）
return （1）;
if（n==1）{
return （2）; }
if（m<n）
{return f（m，m）; }
if（m==n）
{return l+（3） ; }
return f（m，n-1）+f（m-n，（4））;
}
```

②执行程序，f（6，4）=_____。

【答案】①1；1；f（m，n-1）；n　② 9 查看答案

9．设有两个算法在同一机器上运行，其执行时闻分别为 $100n^2$和 2^n，要使前者快于后者，n 至少为_____。

【答案】15 查看答案

【解析】当 n≤14 时，$1OOn^2>2^n$，而，n≥15 时，$1OOn^2<2^n$

四、应用题

1．数据类型和抽象数据类型是如何定义的?二者有何相同和不同之处?抽象数据类型的主要特点是什么?使用抽象数据类型的主要好处是什么?

答：（1）数据类型的定义

数据类型是程序设计语言中的一个概念，它是一个值的集合和操作的集合。如 c 语言中的整型、实型、字符型等。整型值的范围（对具体机器都应有整数范围），其操作有加、减、乘、除、求余等。实际上数据类型是厂家提供给用户的已实现了的数据结构。

（2）抽象数据类型的定义

“抽象数据类型（ADT）”指一个数学模型及定义在该模型上的一组操作。“抽象”的意义在于数据类型的数学抽象特性。抽象数据类型的定义仅取决于它的逻辑特性，而与其在计算机内部如何表示和实现无关。无论其内部结构如何变化，只要它的数学特性不变就不影响它的外部使用。

（3）两者的不同

抽象数据类型和数据类型实质上是一个概念。此外，抽象数据类型的范围更广，它已不再局限于机器已定义和实现的数据类型，还包括用户在设计软件系统时自行定义的数据类型。

（4）抽象数据类型的主要特点

抽象数据类型的出现使程序设计不再是“艺术”，而是向“科学”迈进了一步。

（5）使用抽象数据类型的好处

使用抽象数据类型定义的软件模块含定义、表示和实现三部分，封装在一起，对用户透明（提供接口），而不必了解实现细节。

2．下面程序段的时间复杂度是什么?

for（i=0；i<n；i++）

for（j=0；，j<m；j++）

A[i][j]=0；

答：赋值语句一共被执行了 m*n 次，所以该程序段的时间复杂度是 O（m*n）。

3．试举一例，说明对相同的逻辑结构，同一种运算在不同的存储方式下实现，其运算效率不同。

答：线性表中的插入、删除操作，在顺序存储方式下平均移动近一半的元素，时间复杂度为 O（n）；而在链式存储方式下，插入和删除时间复杂度都是 0（1）。

4．写出下面算法中带标号语句的频度。

TYPE

Ar=ARRAY[1．．n] OF datatype；

PROCEDURE perm（a：ar；k，n：integer）；

VAR x：datatype；　　i：integer；

BEGIN

（1）IF k=n

THEN BEGIN

（2）FOR i：=1 TO n DO

（3）write（a[i]）；

writein；

END

ELSE BEGIN

（4）FOR i：=k TO n DO

（5）A[i]：=a[i]+i*i；

（6）perm（a，k+1，n）；

END；

END；

设 k 的初值等于 1。

答：（1）这是一个递归调用，因 k 初值为 1，由语句（6）知，每次调用 k 增 1，故第（1）语句执行 n 次，所以频度是 n。

（2）这是 FOR 循环语句，在满足（1）的条件下执行，该语句进入循环体（3）n 次，加上最后一次判断出界，故执行了 n+1 次，所以频度是 n+1。

（3）这是循环体语句，共执行了 n 次，所以频度是 n。

（4）这是循环语句，当 k=1 时判断 n+1 次（进入循环体（5）n 次），k=2 时判断 n 次，最后一次 k=n-1 时判断 3 次，故执行次数是（n+1）+n+…+3=（n+4）（n-1）／2 次，所以频度是（n+4）（n-1）／2。

（5）语句（5）是（4）的循环体，每次比（4）少一次判断，故执行次数是 n+（n-1）+…+2=（n+2）（n-1）／2 次，所以频度是（n+2）（n-l）／2。

（6）这是循环体语句，共执行了 n-1 次，所以频度是 n-1。

5．阅读下列算法，指出算法 A 的功能和时间复杂性。

PROCEDURE A（h，g：pointer）；

／／h，g 分别为单循环链表（single linked circular list）中两个结点指针 PROCEDURE B（s，q：pointer）；
VAR p：pointer；
BEGIN
p=s：
WHILE P^·next<>q DO p=p^·next：
P^·next=s：
END；（of B）
BEGIN B（h，g）； B（g，h）； END；（Of A）
答：功能：将原单循环链表分解成两个单循环链表：其一包括结点 h 到结点 g 的前驱结点；另一个包括结点 g 到结点 h 的前驱结点。
时间复杂度：O（n）。
6．调用下列 C 函数 f（n），回答下列问题：
（1）试指出 f（n）值的大小，并写出，f（n）值的推导过程；
（2）假定 n=5，试指出，f（5）值的大小和执行，f（5）时的输出结果。
C 函数：

```
int f（int n）
{int i，j，k，silm=0；
for（i=1； i<n+1；i++）
{for（j=n；j>i-1；j--）
for（k=1；k<j+1；k++）
sum++；
printf（”sum=%d＼n”，sum）；
}
return（sum）；
}
```

答：（1）第一层 FOR 循环判断 n+1 次，往下执行 n 次，第二层 FOR 执行次数为（n+（n-1）+（n-2）+…+1），第三层循环体受第一层循环和第二层循环的控制，其执行次数如表 1-1 所示。

表 1-1 执行次数

i=	1	2	3	…	n
$j=n$	n	n	n	…	n
$j=n-1$	$n-1$	$n-1$	$n-1$	…	
…	…	…			
$j=3$	3	3	3		
$j=2$	2	2			
$j=1$	1				

执行次数为 f(n)=（1+2+…+，n）+（2+3+…+，n）+…+n=n*n（n+1）／2-n（n^2-1）／6。
（2）在 n=5 对，f（5）=55，执行过程中，输出结果为：sum=15 sum=29 sum=41 sum=50 sum=55

第 2 章　线性表

一、选择题
1．线性表是具有 n 个（　　）的有限序列（n>0）。

A．表元素　B．字符　C．数据元素　D．数据项　E．信息项

【答案】C 查看答案

【解析】一个线性表是 n 个数据元素的有限序列。至于每个数据元素的具体含义，在不同的情况下各不相同。

2．若某线性表最常用的操作是存取任一指定序号的元素和在最后进行插入和删除运算，则利用（　　）存储方式最节省时间。

A．顺序表　B．双链表　C．带头结点的双循环链表　D．单循环链表

【答案】A 查看答案

【解析】线性表采用顺序表，便于进行存取任一指定序号的元素；线性表采用链表，便于进行插入和删除操作。但该题是在最后进行插入和删除运算，所以利用顺序表存储方式最节省时间。

3．某线性表中最常用的操作是在最后一个元素之后插入一个元素和删除第一个元素，则采用（　　）存储方式最节省运算时间。

A．单链表　B．仅有头指针的单循环链表

C．双链表　D．仅有尾指针的单循环链表

【答案】D 查看答案

【解析】仅有尾指针的单循环链表，在最后插入元素和删除第一个元素都会用到这个尾指针。

4．若线性表最常用的操作是存取第 I 个元素及其前驱和后继元素的值，为节省时间应采用的存储方式（　　）。

A．单链表　B．双向链表　C．单循环链表　D．顺序表

【答案】D 查看答案

【解析】线性表采用顺序表，便于进行存取任一指定序号的元素。

5．静态链表中指针表示的是（　　）。

A．下一元素的地址

B．内存储器的地址

C．下一元素在数组中的位置

D．左链或右链指向的元素的地址

【答案】C 查看答案

【解析】静态链表的一般结构为：struct static_list{ ElemType data; int next;}

这种结构是预先分配一个较大的空间，类似于一次申请一个较大的数组，但是元素的增删操作都不会移动元素，只需要移动 next 成员就行。因此，静态链表中的指针实际上表示的就是下一个元素在数组中的位置。

6．链表不具有的特点是（　　）。

A．插入、删除不需要移动元素

B．可随机访问任一元素

C．不必事先估计存储空间

D．所需空间与线性长度成正比

【答案】B 查看答案

【解析】B 项是顺序表的特点。只要确定了顺序线性表的起始位置，线性表中的任一数据元素都可随机存取。

7．单链表中，增加一个头结点的目的是为了（　　）。

A．使单链表至少有一个结点

B．标识表结点中首结点的位置

C．方便运算的实现

D．说明单链表是线性表的链式存储

【答案】C 查看答案

【解析】单链表中增加一个头结点的目的是为了方便运算的实现，使得对第一个元素的操作与其它元素的操作相同。

8．对于一个线性表既要求能够进行较快速地的插入和删除，又要求存储结构能反映数据之间的逻辑关系，则应该用（　　）。

A．顺序存储方式　　B．链式存储方式　　C．散列存储方式　　D．以上均可以

【答案】B 查看答案

9．在双向链表指针 P 的结点前插入一个指针 q 的结点操作是（　　）。

A．p->11ink=q；q->Rlink=p；p->L1ink->Rlink=q；q->L1ink=q；

B．p->11ink=q；p->L1ink->Rlink=q；q->Rlink=p；q->L1ink=p->L1ink；

C．q->Rlink=p；q->L1ink=p->L1ink；p->L1ink->Rlink=q；p->L1ink=q；

D．q->11ink=p->11ink；q->Rlink=q；p->11ink=q；p->11ink=q；

【答案】C 查看答案

10．线性表的顺序存储结构是一种（　　）。

A．随机存取的存储结构　　B．顺序存取的存储结构

C．索引存取的存储结构　　D．Hash 存取的存储结构

【答案】A 查看答案

【解析】线性表包括顺序存储结构和链式存储结构，顺序存储结构能够随机存取表中的元素，但插入和删除操作较麻烦，链式存储结构不能随机访问表中的元素，但是能够表示元素之间的先后次序，而且插入和删除操作较容易。

二、判断题

1．在一个设有头指针和尾指针的单链表中，执行删除该单链表中最后一个元素的操作与链表的长度无关。（　　）

【答案】×查看答案

【解析】必须从头指针开始，查找到尾指针所指结点的前驱结点的指针。

2．顺序存储结构的主要缺点是不利于插入或删除操作。（　　）

【答案】√查看答案

【解析】因为顺序表的插入删除会移动大量的元素。

3．线性表采用链表存储时，结点和结点内部的存储空间可以是不连续的。（　　）

【答案】×查看答案

【解析】对于链式存储，数据元素之间的存储地址不一定是相邻的，即结点的存储空间可以是不连续的。而结点内部的存储空间需要是连续的，因为它是一个完整的数据。

4．静态链表就是一直不发生变化的链表。（　　）

【答案】×查看答案

【解析】用数组描述的链表，即称为静态链表。仍具有链式存储结构的主要优点。不是指不发生变化的的链表。

5．取线性表的第 i 个元素的时间同 i 的大小有关。（　　）

【答案】×查看答案

【解析】不一定，如果是顺序存储结构，它访问数据元素时的时间效率都是 O（1）。

6．为了很方便的插入和删除数据，可以使用双向链表存放数据。（　　）

【答案】√查看答案

【解析】链式存储结构便于数据的插入和删除，但只能顺序访问表中的元素。

三、填空题

1．当线性表的元素总数基本稳定，且很少进行插入和删除操作，但要求以最快的速度存取线性表中的元素时，应采用____存储结构。

【答案】顺序查看答案

【解析】顺序存储结构的存取操作比较方便，但插入和删除操作不如链式存储结构方便，而且需要连续的存储空间，由于该线性表的元素总数基本稳定，而且很少进行插入删除操作，为了更快的存取元素，顺序表更合适。

2．线性表 L=（a1，a2…．，an）用数组表示，假定删除表中任一元素的概率相同，则删除一个元素平均需要移动元素的个数是____。

【答案】（n-1）/ 2 查看答案

【解析】删除第一个元素需要移动 n-1 次，以此类推，删除最后一个元素需要移动 0 次。平均次数为(n-1)*n/n/2=(n-1)/2。

3．设单链表的结点结构为（data，next），next 为指针域，已知指针 px 指向单链表中 data 为 x 的结点，指针 py 指向 data 为 y 的新结点，若将结点 y 插入结点 x 之后，则需要执行以下语句：____；____;

【答案】py->next=px->next；px->next=py 查看答案

4．在单链表中设置头结点的作用是____。

【答案】方便运算查看答案

5．对于一个具有 n 个结点的单链表，在已知的结点半 p 后插入一个新结点的时间．复杂度为____，在给定值为 x 的结点后插入一个新结点的时间复杂度为____。

【答案】O（1）；O（n）查看答案

【解析】第一种情况只需直接修改指针的指向。第二种情况必须从头结点遍历找到 x 的结点。

6.根据线性表的链式存储结构中每一个结点包含的指针个数，将线性链表分成____和____；而又根据指针的连接方式，链表又可分成____和____。

【答案】单链表；双链表；（动态）链表；静态链表查看答案

【解析】线性表的链式存储结构根据每个结点包含的指针个数分为单链表和双链表，单链表只包含一个指针，指向后续元素，双链表包括两个指针，指向前一个元素和后续元素。根据指针的连接方式，链表可分为动态链表和静态链表。静态链表的指针指向下一个元素的编号，动态链表的指针指向下一个元素的物理位置。

7．在双向循环链表中，向 P 所指的结点之后插入指针 f 所指的结点，其操作是____、____、____、____。

【答案】f->next=p->next；f->prior=p；p->next->prior=f；p->next=f；查看答案

8．顺序存储结构是通过____表示元素之间的关系的；链式存储结构是通过____表示元素之间的关系的。

【答案】物理上相邻；指针查看答案

【解析】顺序存储结构是通过物理位置表示元素之间的关系的，链式存储结构通过指针表示元素之间的关系。

9．对于双向链表，在两个结点之间插入一个新结点需修改的指针共____个，单链表为____个。

【答案】4；2 查看答案

10．在单链表 L 中，指针 P 所指结点有后继结点的条件是____

【答案】P->next!=NULL 查看答案

【解析】指针所指节点的指针域所指向的元素非空，说明该指针所指节点有后继结点。

11．以下程序的功能是实现带附加头结点的单链表数据结点逆序连接，请填空完善之。

```
void reverse(pointer h)
 /* h为附加头结点指针 */
{ pointer p,q;
   p=h->next;  h->next=NULL;
while(   (1)   )
   {q=p; p=p->next; q->next=h->next; h->next=   (2)   ;    }
}
```

【答案】（1）p!=NULL //链表未到尾就一直进行查看答案

（2）q //将当前结点作为头结点后的第一元素结点插入

12．对单链表中元素按插入方法排序的 C 语言描述算法如下，其中 L 为链表头结点指针。请填充算法中标出的空白处，完成其功能。

```
typedef struct node
      {int data;  struct node *next;
      }linknode,*link;
void Insertsort(link L)
      { link p,q,r,u;
      p=L->next;   (1)   ;
      while(   (2)   )
        { r=L;  q=L->next;
          while(   (3)   && q->data<=p->data) {r=q; q=q->next;}
          u=p->next;   (4)   ;   (5)   ;  p=u;
        }
}
```

【答案】（1）L->next=NULL //置空链表，然后将原链表结点逐个插入到有序表中查看答案

（2）p!=NULL //当链表尚未到尾，p 为工作指针

（3）q!=NULL //查 P 结点在链表中的插入位置，这时 q 是工作指针

（4）p->next=r->next //将 P 结点链入链表中

（5）r->next=p //r 是 q 的前驱，u 是下个待插入结点的指针

四、应用题

1．线性表有两种存储结构：一是顺序表，二是链表。试问：

（1）如果有，2 个线性表同时并存，并且在处理过程中各表的长度会动态变化，线性表的总数也会自动地改变。在此情况下，应选用哪种存储结构?为什么?

（2）若线性表的总数基本稳定，且很少进行插入和删除，但要求以最快的速度存取线性表中的元素，那么应采用哪种存储结构?为什么?

答：（1）应选择链式存储结构。因为它可动态申请内存空间，不受表长度（即表中元素个数）的影响，插入、删除时间复杂度为 O（1）。

（2）应选择顺序存储结构。因为顺序表可以随机存取，时间复杂度为 O（1）。

2．阅读下面的算法，说明算法实现的功能。

```
node *link(node  *head1,   *head2)
      {node  *p,   *q;     p=head1;
       while (p->next!=head1)   p=p->next;
        q=head2;
       while (q->next!=head2)   q=q->next;
       p->next=head2;          q->next=head1;
       return(head1);
      }
```

答：本算法功能是将两个无头结点的循环链表合并为一个循环链表。head1 最后一个结点的链域指向 head2，head2 最后一个结点的链域指向 head1，head1 为结果循环链表的指针。

五、算法设计题

1．假设有两个按元素值递增次序排列的线性表，均以单链表形式存储。请编写算法将这两个单链表归并为一个按元素值递减次序排列的单链表，并要求利用原来两个单链表的结点存放归并后的单链表。

答：算法如下：

```
LinkedList Union(LinkedList la,lb)
//la,lb分别是带头结点的两个单链表的头指针,链表中的元素值按递增序排列
//本算法将两链表合并成一个按元素值递减次序排列的单链表
    { pa=la->next; pb=lb->next; //pa,pb分别是链表la和lb的工作指针
    la->next=null;              //la作为结果链表的头指针,先将结果链表初始化为空
    while(pa!=null && pb!=null) //当两链表均不为空时
      if(pa->data<=pb->data)
        { r=pa->next;           //将pa 的后继结点暂存于r
          pa->next=la->next;    //将pa结点链于结果表中,同时逆置
          la->next=pa;
          pa=r;                 //恢复pa为当前待比较结点
        }
      else
      {r=pb->next;              //将pb 的后继结点暂存于r
         pb->next=la->next;     //将pb结点链于结果表中,同时逆置
         la->next=pb;
         pb=r;                  //恢复pb为当前待比较结点
         }
      if(pa) pb=pa;             //避免再对pa写下面的while语句
      while(pb!=null)
       {r=pb->next; pb->next=la->next; la->next=pb; pb=r; }
    }//算法Union结束
```

2．已知两个链表 A 和 B 分别表示两个集合，其元素递增排列。编一函数，求 A 与 B 的交集，并存放于 A 链表中。

答：算法如下：

```
pa=la->next;pb=lb->next;//设工作指针pa和pb;
pc=la;                          //结果表中当前合并结点的前驱的指针
while(pa&&pb)
  if(pa->data==pb->data)//交集并入结果表中
   { pc->next=pa;pc=pa;pa=pa->next;
     u=pb;pb=pb->next;free(u);
   }
else if(pa->data<pb->data) {u=pa;pa=pa->next;free(u);} //释放结点空间
     else {u=pb; pb=pb->next; free(u);}
while(pa){ u=pa; pa=pa->next; free(u);}//释放结点空间
while(pb) {u=pb; pb=pb->next; free(u);}//释放结点空间
pc->next=null;//置链表尾标记
free(lb);    //注：本算法中也可对B表不作释放空间的处理
```

3．已知两个线性表 A，B 均以带头结点的单链表作存储结构，且表中元素按值递增有序排列。设计算法求出 A 与 B 的交集 C，要求 C 另开辟存储空间。，并同样以元素值的递增有序的单链表形式存储。

答：算法如下：

```
LinkedList Union(LinkedList A,B)
//线性表A和B以带头结点的单链表作为存储结构。本算法求A和B的交集C,C另辟空间
{pa=A->next;pb=B->next;//pa，pb是两链表的工作指针
     pc=C=(LinkedList)maloc(sizeof(LNode)); pc->data=MaxElemType //监视哨

while(pa&&pb)
  if(pa->data<pb->data) pa=pa->next;       //pa指针后移
  else if(pa->data>pb->data) pb=pb->next; //pb指针后移
       else      //处理交集元素
         if(pc->data==pa->data){pa=pa->next;pb=pb->next;} //删除重复元素
         else{ p=(LinkedList)maloc(sizeof(LNode));
               p->data=pa->data;
               pc->next=p;pc=p;
               pa=pa->next;pb=pb->next;
             } //交集元素并入结果表
  pc->next=null;  //置结果链表尾
  return C;}
```

4．已知递增有序的单链表 A，B 分别存储了一个集合，请设计算法以求出两个集合 A 和 B 的差集 A-B（即仅由在 A 中出现而不在 B 中出现的元素所构成的集合），并以同样的形式存储，同时返回该集合的元素个数。

答：算法如下：

```
void  Difference(LinkedList  A,B,*n)
//A和B均是带头结点递增有序的单链表,本算法求两集合的差集,*n是结果集合中元素个数,初始为0
{p=A->next;                    //p和q分别是链表A和B的工作指针
 q=B->next; pre=A;             //pre为A中p所指结点的前驱结点的指针
 while(p!=null && q!=null)
     if(p->data<q->data){pre=p;p=p->next;*n++;} // A链表中当前结点指针后移
     else if(p->data>q->data)q=q->next;       //B链表中当前结点指针后移
          else {pre->next=p->next;            //处理A,B中元素值相同的结点,应删除
                u=p; p=p->next; free(u);     //删除结点
               }   }
```

5. 设计算法将一个带头结点的单链表 A 分解为两个具有相同结构的链表 B、C，其中 B 表的结点为 A 表中值小于零的结点，而 C 表的结点为 A 表中值大于零的结点（链表 A 的元素类型为整型，要求 B、C 表利用 A 表的结点）。

答：算法如下：

```
void DisCreat1(LinkedList A)
//本算法将带头结点的单链表A分解成数据域值小于零和大于零的两个单链表B和C
  {B=A;
  C=(LinkedList )malloc(sizeof(LNode));//为C申请结点空间
  C->next=null       //C初始化为空表
  p=A->next;         //p为工作指针
  B->next=null;      //B表初始化
  while(p!=null)
     {r=p->next;      //暂存p的后继
     if(p->data<0)//小于0的放入B表
       {p->next=B->next; B->next=p; }//将小于0的结点链入B表
     else {p->next=C->next; C->next=p; }
     p=r;//p指向新的待处理结点
     }
}//算法结束
```

6. 试编写在带头结点的单链表中删除（一个）最小值结点的（高效）算法。delete（Linklist&L）

答：算法如下：

```
LinkedList Delete(LinkedList L)
//L是带头结点的单链表,本算法删除其最小值结点
 {p=L->next;    //p为工作指针,指向待处理的结点,假定链表非空
  pre=L;        //pre指向最小值结点的前驱
  q=p;          //q指向最小值结点,初始假定第一元素结点是最小值结点
 while(p->next)
   {if(p->next->data<q->data){pre=p;q=p->next;}     //查最小值结点
    p=p->next;      //指针后移
   }
 pre->next=q->next;//从链表上删除最小值结点
 free(q);return L;//释放最小值结点空间
}//结束算法Delete
```

7．设单链表的表头指针为 h，结点结构由 data 和 next 两个域构成，其中 data 域为字符型。写出算法 dc（h，n），判断该链表的前 n 个字符是否中心对称。例如 xyx，xyyx 都是中心对称。

答：算法如下：

```
int  dc(LinkedList  h,int n)
// h是带头结点的n个字符元素的单链表,本算法判断链表是否中心对称
  {char s[]; int i=1;//i记下结点个数, s是字符栈
   if(n<0) {printf("链表长度错误\n"); exit(0);}
   p=h->next;//p是链表的工作指针,指向待处理的当前元素
   for(i=1;i<=n/2;i++)// 链表前一半元素入栈
     {s[i]=p->data;p=p->next;}
   i--; //恢复最后的i值
   if(n%2==1)p=p->next;} //若n是奇数,后移过中心结点
   while(p!=null && s[i]==p->data){i--;p=p->next;}    //测试是否中心对称
   if(p==null)return(1); //链表中心对称
   else  return(0);        //链表不中心对称
}//算法结束
```

8．设线性表存于 A[1．．size]的前 num 各分量中，且递增有序。请设计一个算法，将 x 插入到线性表的适当位置上，以保持线性表的有序性，并在设计前说明设计思想，最后说明所设计算法的时间复杂度。

答：算法如下：

```
void  Insert(ElemType A[],int size, ElemType x)
//A是Size个元素空间但目前仅有num(num<size)个元素的线性表
//本算法将元素x插入到线性表中,并保持线性表的有序性
 {low=1;high=num; //题目要求下标从1开始
  while)low<=high)   //对分查找元素x的插入位置
    {mid=(low+high)/2;
     if(A[mid]==x){low=mid+1;break;}
     else if(A[mid]>x)high=mid-1 ;
          else  low=mid+1 ;
     }
   for(i=num;i>=low;i--) A[i+1]=A[i];//元素后移
   A[i+1]=x; //将元素x插入
  }算法结束
```

设计思想：算法中当查找失败（即线性表中无元素 X）时，变量 low 在变量 high 的右面（low=high+1）。移动元素从位置 low 开始，直到 num 为止。

时间复杂度：查找的复杂度为 O（logn），插入的时间复杂度为 O（n），若用顺序查找，则查找的时间复杂度亦为 O（n）。

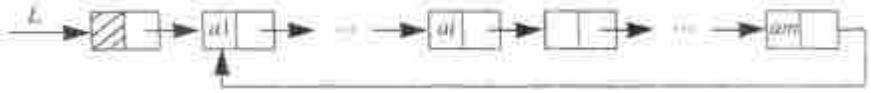
9．已知一个单链表中每个结点存放一个整数，并且结点数不少于 2，请设计算法以判断该链表中第二项起的每个元素值是否等于其序号的平方减去其前驱的值，若满足则返回 ture，否则返回 false。

答：算法如下：

```
int  Judge(LinkedList  la)
//la是结点的元素为整数的单链表。本算法判断从第二结点开始
//每个元素值是否等于其序号的平方减去其前驱的值,如是返回true;否则,返回false
{p=la->next->next;//p是工作指针,初始指向链表的第二项
 pre=la->next;     //pre是p所指结点的前驱指针
 i=2;              //i是la链表中结点的序号,初始值为2
 while(p!=null)
   if(p->data==i*i-pre->data){i++;pre=p;p=p->next;} //结点值间的关系符合题目要求
   else  break;                                     //当前结点的值不等于其序号的平方减去前驱的值
 if(p!=null)return(false);                         //未查到表尾就结束了
 else  return(true);                               //成功返回
}//算法结束
```

```
 p=la;//假设无头结点,初始p指向第一元素结点
 i=2;
 while(p->next!=null)                              //初始p->next指向第二项
   if(p->next->data= =i*i-p->data)
      {i++;p=p->next;}
 if(p->next!=null)return(false); //失败
 else return(true);                                //成功
```

10．已知 L 为链表的头结点地址，表中共有 m（m>3）个结点，从表中第 i 个结点（1<i<m）起到第 m 个结点构成一个循环部分链表，设计将这部分循环链表中所有结点顺序完全倒置的算法。



答：算法如下：

```
LinkedList PatternInvert1(LinkedList L,int  i,m)
//L是有m个结点的链表的头结点的指针。表中从第i(1<i<m)个结点到第m个结点构成循环部分链表
//本算法将这部分循环链表倒置
  {if(i<1|| i>=m || m<4){printf("%d,%d参数错误\n",i,m);exit(0);}
   p=L->next->next;              //p是工作指针,初始指向第二结点(已假定i>1)
   pre=L->next;                  //pre是前驱结点指针,最终指向第i-1个结点
```

```
j=1;                              //计数器
while(j<i-1)                      //查找第i个结点
  {j++;pre=p;p=p->next;}  //查找结束,p指向第i个结点
q=p;                          //暂存第i个结点的指针
p=p->next;                    //p指向第i+1个结点,准备逆置
j+=2;                         //上面while循环结束时,j=i-1,现从第i+1结点开始逆置
while(j<=m)
  {r=p->next;                 //暂存p的后继结点
  p->next=pre->next;  //逆置p结点
  pre->next=p;
  p=r;                        //p恢复为当前待逆置结点
  j++;                        //计数器增1
  }
q->next=pre->next;      //将原第i个结点的后继指针指向原第m个结点
```

11. 已知 L 为没有头结点的的单链表中第一个结点的指针，每个结点数据域存放一个字符，该字符可能是英文字母字符、数字字符或其他字符，编写算法构造三个以带头结点的单循环链表表示的线性表，使每个表中只含同一类字符（要求用最少的时间和最少的空间）。

答：算法如下：

```
void  OneToThree(LinkedList L,la,ld,lo)
L是不带头结点的单链表第一个结点的指针,链表中的数据域存放字符
  //本算法将链表L分解成含有英文字母字符、数字字符和其他字符的带头结点的三个循环链表
    {la=(LinkedList)malloc(sizeof(LNode));      //建立三个链表的头结点
    ld=(LinkedList)malloc(sizeof(LNode));
    lo=(LinkedList)malloc(sizeof(LNode));
    la->next=la;ld->next=ld;lo->next=lo;        //置三个循环链表为空表
    while(L!=null)       //分解原链表
     {r=L; L=L->next;   //L指向待处理结点的后继
     if(r->data>='a'&& r->data<='z' || r->data>='A'&& r->data<='Z')
         {r->next=la->next;  la->next=r;}        //处理字母字符
     else if(r->data>='0'&& r->data<='9')
           {r->next=ld->next;ld->next=r;}        //处理数字字符
     else {r->next=lo->next;lo->next=r;}         //处理其他符号
   }//结束while(L!=null)
}//算法结束
```

12. 在输入数据无序的情况下，建立一个数据值为整型的递增有序的顺序存储线性表 L，且要求当输入相同数据值时，线性表中不能存在数据值相同的数据元素，试写出其算法。

顺序存储结构的线性表描述为：

```
CONST  maxlen={线性表可能达到的最大长度};
TYPE   sqlisttp=RECORD
                  elem:array[1..maxlen] of integer;
                  last :0..maxlen
                END;
VAR L: sqlisttp;
```

答：算法如下：

```
FUNC  BinSearch(VAR a: sqlisttp;x: integer): integer;
  //在顺序表a中查找值为x的元素,查找成功返回0值,否则返回查找失败时的较大下标值
  low:=1;high:=a.last;found:=false;
  WHILE(low<=high)AND NOT found DO
    [mid:=(low+high)DIV 2;
     IF a.elem[mid]=x THEN found:=true
     ELSE IF a.elem[mid]>x THEN high:=mid-1 ELSE low:=mid+1;
    ]
  IF found=true THEN return(0)
  ELSE return(low);//当查找失败时,low=high+1
ENDF;//结束对分查找函数
PROC create(VAR L: sqlisttp)
  //本过程生成顺序表L
  L.last:=0;       //顺序表L初始化
  read(x);
  WHILE x<>9999 DO          //设x=9999时退出输入
    [k:=BinSearch(L,x); //去查找x元素
    IF k<>0                 //不同元素才插入
      THEN [FOR i:=L.last DOWNTO k DO L.elem[i+1]:=L.elem[i];
            L.elem[k]=x;L.last:=L.last+1;//插入元素x,线性表长度增1
           ]
     read(x);
    ]
ENDP;//结束过程creat
```

13. 设键盘输入 n 个英语单词，输入格式为 n，w_1，w_2，…，w_n，其中 n 表示随后输入英语单词个数，试编一程序，建立一个单向链表，实现：

（1）如果单词重复出现，则只在链表上保留一个。

（2）除满足（1）的要求外。链表结点还应有一个计数域，记录该单词重复出现的次数，然后输出出现次数最多的前 k（k<=n）个单词。

答：定义结点数据类型如下：

```
typedef  struct  node
  {int  freq;           //频度域,记单词出现的次数
   char  word[maxsize];//maxsize是单词中可能含有的最多字母个数
   struct  node  *next;
  }node, *LinkedList;
```

（1）算法如下：

```
LinkedList creat()
//建立有n(n>0)个单词的单向链表,若单词重复出现,则只在链表中保留一个
{LinkedList la;
 la=(LinkedList)malloc(sizeof(node));//申请头结点
 la->next=null;                          //链表初始化

for(i=1;i<=n;i++)          //建立n个结点的链表
  {scanf("%s",a);          //a是与链表中结点数据域同等长度的字符数组
   p=la->next;pre=la;      //p是工作指针,pre是前驱指针
   while(p!=null)
     if(strcmp(p->data,a)==0) {p->freq++;break;} //单词重复出现,频度增1
     else {pre=p;p=p->next;}    //指针后移
   if(p==null)                  //该单词没出现过,应插入
     {p=(LinkedList)malloc(sizeof(node));
      strcopy(p->data,a);p->freq=1;p->next=null;pre->next=p;
     }      //将新结点插入到链表最后
  }//结束for循环
return(la);
}//结束creat算法
```

（2）算法如下：

```
void CreatOut()
//建立有n个单词的单向链表,重复单词只在链表中保留一个,最后输出频度最高的k个单词
{LinkedList la;
 la=(LinkedList)malloc(sizeof(node));//申请头结点
 la->next=null;          //链表初始化
 for(i=1;i<=n;i++)       //建立n个结点的链表
   {scanf("%s",a);       //a是与链表中结点数据域同等长度的字符数组
    p=la->next;pre=la;//p是工作指针,pre是前驱指针
    while(p!=null)
      if(strcmp(p->data,a)==0)
         {p->freq++;             //单词重复出现,频度增1
         pre->next=p->next;  //先将p结点从链表上摘下,再按频度域值插入到合适位置
         pre=la; q=la->next;
         while(q->freq>p->freq) {pre=q; q=q->next; }
         pre->next=p; p->next=q; break; //将p结点插入到合适位置
         }
         else {pre=p;p=p->next;}          //指针后移
      if(p==null)                         //该单词没出现过,应插入到链表最后
         {p=(LinkedList)malloc(sizeof(node));
         strcopy(p->data,a);p->freq=1;p->next=null;pre->next=p;
      }//if 新结点插入
 }//结束for循环建表
 int k,i=0;
 scanf("输入要输出单词的个数%d",&k);
 p=la->next;
 while(p && i<k) //输出频度最高的k个单词
  {printf("第%3d个单词%s出现%3d次\n",++i,p->data,p->freq);
   p=p->next;
  }
if(!p)
   printf("给出的%d值太大\n",k);
}//结束算法
```

14．设有一头指针为 L 的带有表头结点的非循环双向链表，其每个结点中除有 pred（前驱指针），data（数据）和 next（后继指针）域外，还有一个访问频度域 freq。在链表被启用前，其值均初始化为零。每当在链表中进行一次 Locate（L，x）运算时，令元素值为 x 的结点中 freq 域的值增 1，并使此链表中结点保持按访问频度非增（递减）的顺序排列，同时最近访问的结点排在频度相同的结点的最后，以便使频繁访问的结点总是靠近表头。试编写符合上述要求的 Locate（L，x）运算的算法，该运算为函数过程，返回找到结点的地址，类型为指针型。

答：算法如下：

```
DLinkList  locate(DLinkedList L,ElemType x)
  // L是带头结点的按访问频度递减的双向链表
  //本算法先查找数据x,查找成功时结点的访问频度域增1,最后将该结点按频度递减插入链表中
{ DLinkedList p=L->next,q;     //p为L表的工作指针,q为p的前驱,用于查找插入位置
  while(p && p->data !=x) p=p->next; // 查找值为x的结点
  if(!p) {printf("不存在所查结点\n"); exit(0);}
  else { p->freq++;                    // 令元素值为x的结点的freq域加1
         p->next->pred=p->pred;        // 将p结点从链表上摘下
         p->pred->next=p->next;
         q=p->pred;                    // 以下查找p结点的插入位置
         while(q !=L && q->freq<p->freq) q=q->pred;
         p->next=q->next; q->next->pred=p;// 将p结点插入
         p->pred=q; q->next=p;
        }
  return(p);    //返回值为x的结点的指针
}  // 算法结束
```

15．给定（已生成）一个带表头结点的单链表，设 head 为头指针，结点的结构为（data，next），data 为整型元素，next 为指针，试写出算法：按递增次序输出单链表中各结点的数据元素，并释放结点所占的存储空间（要求：不允许使用数组作辅助空间）。

答：算法如下：

```
void  MiniDelete(LinkedList  head)
  //head是带头结点的单链表的头指针
  //本算法按递增顺序输出单链表各结点的值,并释放结点所占的存储空间
  {while(head->next!=null) //循环到仅剩头结点
     {pre=head;r=pre;       //pre为元素最小值结点的前驱结点的指针

   p=pre->next;           //p为工作指针
   while(p!=null)
    {if(p->data<pre->next->data)pre=r;        //记住当前最小值结点的前驱
     r=p; p=p->next;
    }
   printf(pre->next->data);                   //输出元素最小值结点的数据
   u=pre->next;pre->next=u->next; free(u);    //删除元素值最小的结点,释放结点空间
 }// while(head->next!=null)
free(head);}    //释放头结点
```

16．已知非空双向链表由 d 指出，结点结构为（11ink，data，rlink），请设计算法将链表中数据域值最大（假定唯一）的那个结点移至链表的最前面。要求：不得额外申请新的双链表结点。

答：算法如下：

```
DLinkedList  DListAddOne(DLinkedList d)
   //d是循环链表,本算法将链表中数据域值最大的结点移至链表的最前面
   {p=d->next;    //设链表有头结点
    q=p->next;    //q指向待处理结点
    while(q)
       {if(q->data>p->data) p=q; //p记数据域值最大的结点
        q=q->next;
       }
   p->prior->next=p->next; p->next->prior=p->prior;          //将p摘下
   p->next=d->next; p->prior=d; d->next->prior=p; d->next=p;//插入p结点
   return d;
}
```

17．已知 P 是指向单向循环链表最后一个结点的指针，试编写只包含一个循环的算法，将线性表（a_1，a_2，…，a_{n-1}，a_n）改造为（a_1，a_2，…，a_{n-1}，a_n，a_{n-1}，…，a_2，a_1）。

答：算法如下：

```
LinkedList  CyclInsert(LinkedList p)
  //本算法将线性表(a1,a2,...,an-1,an)改造为 (a1,a2,...,an-1,an, an-1,...., a2, a1)
  {q=p->next; //q指向a1结点
   t=(LNode*)malloc(sizeof(LNode)); t->data=q->data; r=t; //r记住a1结点的指针
   t->next=p->next; p->next=t; //先将a1结点放到正确位置
   q=q->next; //从 a2结点开始
   while(q!=p)
     {s=q->next;                    //暂存后继
      t=(LNode*)malloc(sizeof(LNode)); t->data=q->data;
      t->next=p->next; p->next=t; //对称放置
      q=s;                          //恢复待处理结点
     }

                          p=r; return p;
                       }
```

18．写算法将单链表 11 拆成二个链表，其中以 11 为头的链表保持原来向后的链接，另一个链表的头为 12，其链接方向与 11 相反，11 包含原链表的奇数序号的结点，12 包含原链表的偶数序号的结点。

答：算法如下：

```
LinkedList DisUnion(LinkedList L1,L2)
  //本算法将链表L1拆成L1和L2两个链表,L2链接方向与L1相反
{int i=0;
L2=(LNode*)malloc(sizeof(LNode));
 L2->next=null;      //空链表
  p=L1->next; pre=L1;
  while(p)
    {i++;
     if(i%2) {pre->next=p; pre=p; p=p->next;}          //奇数序号结点在L1中
     else{s=p->next; p->next=L2->next; L2->next=p; p=s;}//偶数序号结点逆置插入到L2中
    }
  pre->next=null; return L2;//置L1表尾
}
```

第 3 章　栈和队列

一、选择题

1．对于栈操作数据的原则是（　　）。

A．先进先出　　B．后进先出　　C．后进后出　　D．不分顺序

【答案】B 查看答案

【解析】先进先出是队列操作数据的原则。先进后出是栈操作数据的原则，栈限定在表尾进行插入和删除。

2．有六个元素 6，5，4，3，2，1 顺序入栈，下列不是合法的出栈序列的是（　　）。

A．543612　　B．453126　　C．346521　　D．234156

【答案】C 查看答案

【解析】根据栈的后进先出的特点，对于 C 选项中前两个元素得出栈顺序可以看出，4 在 5 和 6 前先出栈，又根据入栈顺序，4 在 5 和 6 后入栈，因此 4 出栈时，5 和 6 必定在栈内，且 5 在 6 之上，所以出栈时 5 要比 6 先出栈。

3．向一个栈顶指针为 h 的带头结点的链栈中插入指针 S 所指的结点时，应执行（　　）。

A．h->next=s；　　B．s->next=h；

C．s->next=h；h->next=s；　　D．s->next=h-next；h->next=s；

【答案】D 查看答案

【解析】本题是向一个链栈中插入结点，可从头结点后插入。先将 s 结点指向第一个头结点之后的结点之前，再将头结点指向 s 结点。

4．输入序列为 ABC，可以变为 CBA 时，经过的栈操作为（　　）。

A．push，pop，push，pop，push，pop　　B．push，push，push，pop，pop，pop

C．push，push，pop，pop，push，pop　　D．push，pop，push，push，pop，pop

【答案】B 查看答案

【解析】根据输入序列和输出序列可知，输入序列全部进栈，然后再出栈。从中可以看出，push 的数目始终大于等于 pop 的数目。

5．若一个栈以向量 V[1…，n]存储，初始栈顶指针 top 为 n+1，则下面 X 入栈的正确操作是（　　）。

A．top：=top+1；V[top]：=x　　B．V[top]：=x；top：=top+1

C．top：=top-1；V[top]：=x　　D．V[top]：=x；top：=top-1

【答案】C 查看答案

【解析】题中初始栈顶指针 top 为 n+1，而栈顶指针又位于最大下标以上，此时入栈应进行先减一操作。

6．执行完下列语句段后，f 值为（　　）。

```
int  f(int x)
{ return  ((x>0) ? x* f(x-1):2);}
int i  ;
i =f(f(1));
```

A．2　B．4　C．8　D．无限递归

【答案】B 查看答案

【解析】该程序使用了递归调用，由题知，f(0)=2；f(1)=1*f(0)=2；f(2)=2*f(1)=4；所以结果为 4。

7．设计一个判别表达式中左、右括号是否配对出现的算法，采用（　　）数据结构最佳。

A．线性表的顺序存储结构　B．队列　C．线性表的链式存储结构　D．栈

【答案】D 查看答案

【解析】用栈更合适，如果是左括号，进栈；如果是右括号，看栈顶是不是左括号，如果是，则左括号出栈；否则不配对（可以直接结束算法）。处理完所有符号号，如果栈为空则配对成功。

8．用不带头结点的单链表存储队列，其队头指针指向队头结点，队尾指针指向队尾结点，则在进行出队操作时（　　）。

A．仅修改队头指针

B．仅修改队尾指针

C．队头、队尾指针都可能要修改

D．队头、队尾指针都要修改

【答案】C 查看答案

【解析】用不带头结点的单链表存储队列，一般删除操作仅修改队头指针，但当队列中只有一个结点时，进行删除操作要将队头、队尾指针都修改成 NULL。

9．对于循环队列（　　）。

A．无法判断队列是否为空

B．无法判断队列是否为满

C．队列不可能满

D．以上说法都不是

【答案】D 查看答案

【解析】，循环队列也会出现队列满的情况，并且循环队列也可以判断是否为空或满。至少可以通过两种方法进行判断：①另设一个布尔变量来区别队列是空还是满；②队满时，（rear+1）==font。

10．循环队列 A[0．．m-1]存放其元素值，用 front 和 rear 分别表示队头和队尾，则当前队列中的元素数是（　　）。

A．（rear-front+m）%m　B．rear-front+1　C．rear-front-1　D．rear-front

【答案】A 查看答案

【解析】对于循环队列，需要深刻理解队头（font）和队尾（rear）的概念，在队头进行出队操作，在队尾进行进队操作。rear-front 可能为正也可能为负，为正时元素个数=（rear-front）；如果为负则元素的个数=（rear-front+m），所以统一的公式就是（rear-front+m）%m。

11．循环队列存储在数组 A[0．．m]中，则入队时的操作为（　　）。

A．rear=rear+1

B．rear=（rear+1）mod（m-1）

C．rear=（rear+1）mod m

D．rear=（rear+1）mod（m+1）

【答案】D 查看答案

12．若用一个大小为 6 的数组来实现循环队列，且当前 rear 和 front 的值分别为 0 和 3，当从队列中删除一个元素，再加入两个元素后，rear, front 的值分别为多少?（　　）

A．1 和 5 B.2 和 4　C．4 和 2　D．5 和 1

【答案】B 查看答案

【解析】入队操作的主要步骤：rear=（rear+1）%6，加入一个后，rear=（0+1）%6=1，再加入一个后，rear=（1+1）%6=2。出队操作的主要步骤：front=（front+1）%6。删除一个后，front=（3+1）%6=4。

13．最大容量为 n 的循环队列，队尾指针是 rear，队头：front，则队空的条件是（　　）。

A．（rear+1）MOD n=front

B．rear=front

C．rear+1=front

D．（rear-1）MOD n=front

【答案】B 查看答案

【解析】循环队列队空的条件是：rear=front。循环队列队满的条件，通常采用（rear+1）%MAXQSIZE=front 来判定队满，其中 MAXQSIZE 表示队列的长度。

14．栈和队的共同点是（　　）。

A．都是先进后出

B．都是后进先出

C．只允许在端点处插入和删除元素

D．没有共同点

【答案】C 查看答案

【解析】栈和队列的区别是栈是先进后出的数据结构，队列是先进先出的数据结构，栈和队列的共同点是都只能在端点处插入和删除元素。

15．设栈 S 和队列 Q 的初始状态为空，元素 el，e2，e3，e4，e5 和 e6 依次通过栈 S，一个元素出栈后即进队列 Q，若 6 个元素出队的序列是 e2，e4，e3，e6，e5，e1，则栈 S 的容量至少应该是（　　）。

A．6　B．4　C．3　D．2

【答案】C 查看答案

16.4 个圆盘的 Hanoi 塔，总的移动次数为（　　）。

A．7　B．-8　C．15　D．16

【答案】C 查看答案

【解析】Hanoi 问题总移动次数为：2^n-1 次。

17．当字符序列 t3_作为图 3-1 输入时，输出长度为 3 的且可用作 C 语言标识符的序列的有（　　）。

A．4 个　B．5 个　C．3 个　D．6 个

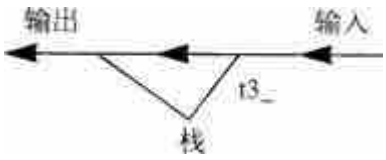


图 3-1

【答案】C 查看答案

【解析】首先需要明白 C 语言标识符的命名规则。数字不能作为标识符的开头，因此第一个字符只能为 t 或者下划线。若首字符为 t，有两种结果 t3_和 t_3，若首字符为_，则只有一种结果_3t。因此总共有 3 种结果。

18．和顺序栈相比，链栈有一个比较明显的优势是（　　）。

A．通常不会出现栈满的情况

B．通常不会出现栈空的情况

C．插入操作更容易实现

D．删除操作更容易实现

【答案】A 查看答案

19．执行（　　）操作时，需要使用队列做辅助存储空间。

A．查找哈希（Hash）表

B．广度优先搜索网

C．前序（根）遍历二叉树

D．深度优先搜索网

【答案】B 查看答案

【解析】查找哈希表不需要辅助存储空间，前序遍历二叉树和深度优先搜索网需要使用栈做辅助存储空间，广度优先搜索树需要队列做辅助存储空间。

二、判断题

1．即使对不含相同元素的同一输入序列进行两组不同的合法的入栈和出栈组合操作，所得的输出序列也一定相同。（　　）

【答案】×查看答案

【解析】栈只是一种先入后出的存储结构。对于出栈、入栈的元素不进行修改,因此，输入序列的元素不相同，不可能得到相同的输出序列。

2．有 n 个数顺序（依次）入栈，出栈序列有 Cn 种，Cn=[1 / （n+1）]*（2n）! / [（n!）*（n!）]。（　　）

【答案】√查看答案

3．设栈采用顺序存储结构。若已有 i-1 个元素入栈，则将第 i 个元素入栈时，入栈算法的时间复杂性为 O（i）。（　　）

【答案】×查看答案

【解析】由于该栈采用顺序存储结构，时间复杂度应该为 O（1）。

4．栈的输入序列是 1，2，…，n，输出序列是 a_1，a_2，…，a_n若 a_i=n（1≤i≤n）则有：a_i>a_{i+1}…>a_n。（　　）

【答案】×查看答案

【解析】出栈序列不一定满足 a_i>a_{i+1}…>a_n，比如 1 进栈，然后出栈，a_1=1。a_1<a_i。

5．栈和队列都是限制存取点的线性结构。（　　）

【答案】√查看答案

6．在链队列中，即使不设置尾指针也能进行入队操作。（　　）

【答案】√查看答案

【解析】因为存在头指针，根据链表的性质，根据头指针可以找到为指针。

7.队列是一种插入与删除操作分别在表的两端进行的线性表，是一种先进后出型结构。（　　）

【答案】×查看答案

【解析】队列是一种先入先出型结构，而栈才是先进后出的线性结构。

8．通常使用队列来处理函数或过程的调用。（　　）

【答案】×查看答案

【解析】经常使用栈来处理函数或过程的调用。

9．队列逻辑上是一个下端和上端既能增加又能减少的线性表。（　　）

【答案】×查看答案

【解析】队列只在下端（队尾）增加，在上端（队头）减少。

10．设尾指针的循环链表表示队列，则入队和出队算法的时间复杂度均为 O（1）（　　）。

【答案】√查看答案

【解析】入队和出队操作分别在队尾和队头进行，设有尾指针的循环链表对头和尾元素的操作的时间复杂度是 O（1）。

11．循环队列也存在空间溢出问题。（　　）

【答案】√查看答案

【解析】循环队列的存储空间也是有限的，因此也存在空间溢出问题。

12．队列和栈都是运算受限的线性表，只允许在表的两端进行运算。（　　）

【答案】×查看答案

【解析】栈只能在一端进行操作，队列可以在表的两端进行运算。

三、填空题

1．栈是_____的线性表，其运算遵循_____的原则。

【答案】操作受限（或限定仅在表尾进行插入和删除操作）；后进先出查看答案

2．在一个具有 n 个单元的顺序栈中，假定以地址高端（即下标为 n 的单元）作为栈底，以 top 作为栈顶指针，则当向栈中压入一个元素时，top 的变化是 top=_____。

【答案】top-1 查看答案

【解析】由于栈底在地址高端，栈中压入一个元素时，栈顶向地址底端移动一个单位，所以 top-1。

3．设有一个空栈，栈顶指针为 1000H（十六进制），现有输入序列为 l，2，3，4，5，经过 PUSH，PUSH，POP，PUSH，POP，PUSH，PUSH 之后，输出序列是_____，而栈顶指针值是_____。设栈为顺序栈，每个元素占 4 个字节。

【答案】23；100CH 查看答案

4．当两个栈共享一存储区时，栈利用一维数组 stack（1，，l）表示，两栈顶指针为 top[1]与 top[2]，则当栈 1 空时，top[1]为_____，栈 2 空时，top[2]为_____，栈满时为_____。

【答案】0；n+1；top[1]+l=top[2]查看答案

【解析】共享栈的栈底在共享存储区的两端，当栈满时栈顶相邻。

5．在进行入栈运算时应先判别栈是否（1）；在进行出栈运算时应先判别栈是否（2）；当栈中元素为 n 个，进行入栈运算时发生上溢，则说明该栈的最大容量为（3）。为了增加内存空间的利用率和减少溢出的可能性，由两个栈共享一片连续的空间时，应将两栈的（4）分别设在内存空间的两端，这样只有当（5）时才产生溢出。

【答案】满 ；空 ；n ；栈底 ；两栈顶指针相邻（即值之差的绝对值为 1）查看答案

6．有五个数据依次入栈：1，2，3，4，5。在各种出栈的序列中，以 3，4 先出栈的序列有_____。（3 在 4 之前出栈）

【答案】3 个查看答案

【解析】以 3,4 先出栈的序列有 34521、34215、34251 共 3 个。

7．顺序栈用 data[1．．n]存储数据，栈顶指针是 top，则值为 x 的元素入栈的操作是_____。

【答案】if（top!=n）data[++top]=x；查看答案

【解析】先判断栈是否满，如果不满，元素入栈。否则返回溢出信息。

8．表达式 23+（（12*3-2）／4+34,5／7）+108／9 的后缀表达式是_____。

【答案】23.12.3*2-4 / 34.5*7 / ++108.9 / +（表达式中的点（．）是数分隔符，如 23.12.3 是三个数）查看答案

9．循环队列的引入，目的是为了克服_____。

【答案】假溢出时大量移动数据元素查看答案

【解析】用数组实现队列时，如果不移动，随着数据的不断读写，会出现假满队列的情况。即尾数组已满但头数组还是空的。循环队列也是一种数组，引入循环队列，有效克服假溢出大量移动数据元素的问题。

10．在循环队列中，队列长度为 n，存储位置从 0 到，n-1 编号，以 rear 指示实际的队尾元素，现要在此队列中插入一个新元素，新元素的位置是（　　）。

【答案】rear=（rear+1）%n 查看答案

11．已知链队列的头尾指针分别是 f 和 r，则将值 x 入队的操作序列是_____。

【答案】S=（LinkedList*）ma11oc（sizeof（LNode））；s->data=x；s->next=r->next；r->next=s；r=s；查看答案

【解析】队列采用链式存储结构，先分配一个节点的内存，然后在队尾添加该节点。

12．已知一循环队列的存储空间为[m. . n]，其中 n>m，队头和队尾指针分别为 front 和 rear，则此循环队列判满的条件是（　　）

【答案】（rear+1）%（n-m+1）==front 查看答案

13．用循环链表表示的队列长度为 n，若只设头指针，则出队和入队的时间复杂度分别是_____和_____；若只设尾指针，则出队和入队的时间复杂度分别是_____和_____。

【答案】O（1）；O（n）；O（1）；O（1）查看答案

【解析】队列的出队操作即删除队头的元素，队列的入队操作即在队尾添加元素，循环链表只设头指针，出队时，只要把头结点的下一个结点删除就好了，入队时，要把新的结点插入队尾，必须把队列遍历，找到队尾指针，才能插入。循环队列只设尾指针，出队时只要把为指针的下一个结点或者下下个结点删除即可，入队时，只要在尾指针的后面插入新的结点，并更新尾结点即可。

14．下面程序的功能是用递归算法将一个整数按逆序存放到一个字符数组中。如 123 存放成 321。请填空：

```
#include<stdio.h>
void convert(char *a, int n)
{   int i;
    if(i=n/10)  convert(_____,i);
    *a=_____;
}
main(     )
     {int number;  char str[10]="";
     scanf("%d",&number);
     convert(str,number);     puts(str);
}
```

【答案】a+1；n%10 查看答案

【解析】通过递归算法，首先找到最高位的值，将其放到 str 对应的数组中，依次反向获取从高位到地位的值，将其放到数组中，完成了将整数逆序放到一个字符数组中。

四、应用题

1．假设以 S 和 X 分别表示入栈和出栈操作，则对初态和终态均为空的栈操作可由 S 和 X 生成的序列表示（如 SXSX）。

（1）试指出判别给定序列是否合法的一般规则；
（2）两个不同合法序列（对同一输入序列）能否得到相同的输出元素序列?如能得到，请举列说明。
答：（1）通常有两条规则。第一是给定序列中 S 的个数和 X 的个数相等；第二是从给定序列的开始，到给定序列中的任一位置，S 的个数要大于或等于 X 的个数。
（2）可以得到相同的输出元素序列。例如，输入元素为 A，B，C，则两个输入的合法序列 ABC 和 BAC 均可得到输出元素序列 ABC。对于合法序列 ABC，使用 SXSXSX 操作序列；对于合法序列 BAC，使用 SSXXSX 操作序列。
2．如果输入序列为 123456，试问能否通过栈结构得到以下两个序列：435612 和 135426；请说明为什么不能或如何才能得到。
答：输入序列为 123456，不能得出 435612，其理由是，输出序列最后两元素是 12，前面 4 个元素（4356）得到后，栈中元素剩 12，且 2 在栈顶，栈底元素 1 不可能在栈顶元素 2 之前出栈。得到 135426 的过程如下：1 入栈并出栈，得到部分输出序列 1；然后 2 和 3 入栈，3 出栈，部分输出序列变为 13；接着 4 和 5 入栈，5、4 和 2 依次出栈，部分输出序列变为 13542；最后 6 入栈并出栈，得到最终结果 135426。
3．设输入序列为 2，3，4，5，6，利用一个栈能得到序列 2，5，3，4，6 吗?栈可以用单链表实现吗?
答：不能得到序列 2，5，3，4，6。因为根据输入序列，2 进栈之后，2 出栈，3，4，5 依次进栈。5 出栈，此时栈中剩下 3，4。因为 4 在栈顶，所以 4 应该比 3 先出栈，不能得到提供的序列。栈可以用单链表实现，这就是链栈。由于栈只在栈顶操作，所以链栈通常不设头结点。
4．队列可以用循环单链表来实现，故可以只设置一个头指针或者只设置一个尾指针。请你分析对于循环单链表实现的队列，用哪种方案更合适。
答：循环单链表若只设头指针，则出队操作时间复杂度是 O（1），而如对操作时间复杂度是 O（n）；若只设尾指针，则出队和入队操作时间复杂度都是 O（1）。因此，用循环单链表来实现队列，设置一个尾指针更合适。
5．用一个数组 S（设大小为 MAX）作为两个堆栈的共享空间。请说明共享方法，栈满 / 栈空的判断条件，并用 C 语言或 PASCAL 语言设计公用的入栈操作 push（i，x），其中 i 为 0 或 1，用于表示栈号，x 为入栈值。
答：两栈共享一向量空间（一维数组），栈底设在数组的两端，两栈顶相邻时为栈满。设共享数组为 S[MAX]，则一个栈顶指针为一 l，另一个栈顶指针为 MAX 时，栈为空。用 C 语言写的入栈操作 push（i，x）如下：

```
const  MAX=共享栈可能达到的最大容量
typedef   struct  node
   {elemtype  s[MAX];
   int top[2];
   }anode;
anode ds;
int push(int I,elemtype x)
   //ds为容量有MAX个类型为elemtype的元素的一维数组，由两个栈共享其空间。I的值为0或1
   //x为类型为elemtype的元素。本算法将x压入栈中。如压栈成功，返回1；否则，返回0
{if(ds.top[1]-ds.top[0]==1){printf("栈满\n");return(0);}
 switch(i)
  {case  0: ds.s[++ds.top[i]]=x;break;
   case  1: ds.s[--ds.top[i]]=x;
  } return(1);//入栈成功
  }
```

6．画出对算术表达式 A-B*C／D—E↑F 求值时操作数栈和运算符栈的变化过程。

答：设操作数栈是 opnd，运算符栈是 optr,对算术表达式 A-B*C／D-E↑F 求值，过程如表 3-1 所示。

表 3-1 操作数栈和运算符栈的变化过程

步骤	*opnd*栈	*optr*栈	输入字符	主要操作
初始		#	A－B*C/D－E↑F#	PUSH(OPTR,'#')
1	A	#	A－B*C/D－E↑F#	PUSH(OPND, A)
2	A	#－	－B*C/D－E↑F#	PUSH(OPTR,'－')
3	AB	#－	B*C/D－E↑F#	PUSH(OPND, B)
4	AB	#－*	*C/D－E↑F#	PUSH(OPTR,'*')
5	ABC	#－*	C/D－E↑F#	PUSH(OPND, C)
6	AT(T=B*C)	#－/	/D－E↑F#	PUSH(OPND, POP(OPND)*POP(OPND)) PUSH(OPTR,'/')
7	ATD	#－/	D－E↑F#	PUSH(OPND, D)
8	AT(T=T/D) T(T=A－T)	#－ #－	－E↑F#	x=POP(OPND); y=POP(OPND) PUSH(OPND, y'x'); x=POP(OPND); y=POP(OPND); PUSH(OPND, y－x) PUSH(OPTR,'－')
9	TE	#－	E↑F#	PUSH(OPND, E)
10	TE	#－↑	↑F#	PUSH(OPTR,'↑')
11	TEF	#－↑	F#	PUSH(OPND, F)
12	TE TS(S=E↑F) R(R=T－S)	 #－ #	#	x=POP(OPND) y=POP(OPND) POP(OPTR) PUSH(OPND, y↑x) x=POP(OPND) y=POP(OPND) POP(OPTR) PUSH(OPND, y－x)

7．在某程序中，有两个栈共享一个一维数组空间 SPACE[N]、SPACE[0]、SPACE[N-1]分别是两个栈的栈底。

（1）对栈 1、栈 2，试分别写出（元素 x）入栈的主要语句和出栈的主要语句。

（2）对栈 1、栈 2，试分别写出栈满、栈空的条件。

答：（1）入栈主要语句

```
if(top2-top1==1) {printf("栈满\n"); exit(0);}
case1:top1++;SPACE[top1]=x;      //设x为入栈元素
Case2:top2--;SPACE[top2]=x;
```

出栈主要语句

```
case1: if(top1==-1) {printf("栈空\n");exit(0);}
       top1--;return(SPACE[top1+1]);     //返回出栈元素
Case2: if(top2==N){printf("栈空\n");exit(0);}
       top2++;return(SPACE[top2-1]);     //返回出栈元素
```

（2）

```
栈满条件：top2-top1=1
栈空条件：top1=-1并且top2=N    //top1=-1为左栈空,top2=N为右栈空
```

8．一个循环队列的数据结构描述如下：

```
TYPE sequeuetp=RECORD
                 elem: ARRAY[1..MAXSIZE] OF elemtp;
                 front,rear: 0..MAXSIZE;
               END;
```

给出循环队列的队空和队满的判断条件，并且分析一下该条件对队列实际存储空间大小的影响，如果为了不损失存储空间，你如何改进循环队列的队空和队满的判断条件?

答：（1）队空：

```
s.front=s.rear;                    //设s是sequeuetp类型变量
```

（2）队满：

```
(s.rear+1)MOD MAXSIZE=s.front    //数组下标为0.. MAXSIZE-1
```

这种判断方法，会“牺牲一个存储单元”。为了不损失存储空间，可以通过设置标志位的方式来进行队空和队满的判断。设标记 tag，tag 等于 0 情况下，若删除时导致 front=rear 为队空；tag=1 情况下，若因插入导致 front=rear 则为队满。

五、算法设计题

1．设有两个栈 S_1，S_2都采用顺序栈方式，并且共享一个存储区【0. . maxsize-1】，为了尽量利用空间，减少溢出的可能，可采用栈顶相向，迎面增长的存储方式。试设计 S_1，S_2有关入栈和出栈的操作算法。

答：栈的定义：

```
#define maxsize    两栈共享顺序存储空间所能达到的最多元素数
#define ElemType int       //假设元素类型为整型
typedef struct
{ElemType stack[maxsize];//栈空间
 int top[2];               //top为两个栈顶指针
}stk;
stk s;                     //s是如上定义的结构类型变量，为全局变量
```

（1）入栈操作：

```
int push(int i,int x)
//入栈操作。i为栈号,i=0表示左栈s1,i=1表示右栈s2,x是入栈元素。入栈成功返回1,否则返回0
  {if(i<0||i>1){printf("栈号输入不对\n");exit(0);}
   if(s.top[1]-s.top[0]==1) {printf("栈已满\n");return(0);}
  switch(i)
    {case 0: s.stack[++s.top[0]]=x; return(1); break;
     case 1: s.stack[--s.top[1]]=x; return(1);
    }
  }//push
```

（2）出栈操作

```
ElemType pop(int i)
//出栈算法。i代表栈号,i=0时为s1栈,i=1时为s2栈。出栈成功返回出栈元素,否则返回-1
{if(i<0 || i>1){printf("栈号输入错误\n");exit(0);}
   switch(i)
    {case 0: if(s.top[0]==-1) {printf("栈空\n");return(-1);}
             else return(s.stack[s.top[0]--]);
    case 1: if(s.top[1]==maxsize {printf("栈空\n"); return(-1);}
             else return(s.stack[s.top[1]++]);
}//switch       }//算法结束
```

2．设从键盘输入一整数的序列：a_1，a_2，a_3，…，a_n，试编写算法实现：用栈结构存储输入的整数，当 $a_i \neq -1$ 时，将 a_i 入栈；当 $a_i=-1$ 时，输出栈顶整数并出栈。算法应对异常情况（入栈满等）给出相应的信息。

答：算法如下：

```
#define maxsize 栈空间容量
 void InOutS(int s[maxsize])
      //s是元素为整数的栈,本算法进行入栈和出栈操作
 {int top=0;                  //top为栈顶指针,定义top=0时为栈空
  for(i=1; i<=n; i++)         //n个整数序列进行处理
    {scanf("%d",&x);          //从键盘读入整数序列
     if(x!=-1)                // 读入的整数不等于-1时入栈
      if(top==maxsize-1){printf("栈满\n");exit(0);}else s[++top]=x; //x入栈
      else    //读入的整数等于-1时出栈
    {if(top==0){printf("栈空\n");exit(0);} else printf("出栈元素是%d\n",s[top--]);}
} }//算法结束。
```

3．设表达式以字符形式已存入数组 E 中，'#'为表达式的结束符，试写出判断表达式中括号（'（'和'）'）是否配对的 C 语言描述算法：EXYX（E）（注：算法中可调用栈操作的基本算法）。

答：算法如下：

```
Int EXYX(char E[],int n)
//E[]是有n字符的字符数组,存放字符串表达式,以'#'结束。本算法判断表达式中圆括号是否匹配
  {char s[30];                  //s是一维数组,容量足够大,是用于存放括号的栈
   int top=0;                   //top用作栈顶指针
   S[top]= '#';                 //'#'先入栈,用于和表达式结束符号'#'匹配
   int i=0;                     //字符数组E的工作指针
   while(E[i]!= '#')            //逐字符处理字符表达式的数组
     Switch(E[i])
       {case'(':  s[++top]='('; i++ ;  break ;
          case')':  if(s[top]=='('){top--; i++; break;}
                    else{printf("括号不配对\n");exit(0);}
          case'#':  if(s[top]=='#'){printf("括号配对\n");return (1);}
                    else {printf(" 括号不配对\n");return (0);} //括号不配对
          default :     i++;     //读入其他字符,不进行处理

  }//switch  }//算法结束
```

4．假设以 I 和 O 分别表示入栈和出栈操作。栈的初态和终态均为空，入栈和出栈的操作序列可表示为仅由 I 和 O 组成的序列，称可以操作的序列为合法序列，否则称为非法序列。

（1）下面所示的序列中哪些是合法的?

A．IOIIOIOO　B．IOOIOIIO　C．IIIOIOIO　D．IIIOOIO

（2）通过对（1）的分析，写出一个算法，判定所给的操作序列是否合法。若合法，返回 true，否则返回 false（假定被判定的操作序列已存入一维数组中）。

答：（1）A 和 D 是合法序列，B 和 C 是非法序列。

（2）设被判定的操作序列已存入一维数组 A 中，算法如下：

int Judge（char A[]）／／判断字符数组 A 中的输入输出序列是否是合法序列。

{i=0 ／／i 为下标

```
  j=k=0;                        //j和k分别为I和字母O的个数
  while(A[i]!='\0')
    {switch(A[i])
      {case'I': j++; break; //入栈次数增1
       case'O': k++; if(k>j){printf("序列非法\n");exit(0);}
      }
     i++; //不论A[i]是'I'或'O',指针i均后移
    }
   if(j!=k) {printf("序列非法\n");return(false);}
   else {printf("序列合法\n");return(true);}
  }//算法结束
```

5．编程：假设以数组 Q[m]存放循环队列中的元素，同时以 rear 和 length 分别指示环形队列中的队尾位置和队列中所含元素的个数。试给出该循环队列的队空条件和队满条件，并写出相应的初始化（initqueue），插入（enqueue）和删除（dequeue）元素的操作。

答：定义队列：

```
typedef struct
{ElemType  Q[m];          // 循环队列占m个存储单元
 int  rear,length;        // rear指向队尾元素,length为元素个数
}SeQueue;
```

（1）设 cq 是 seQueue 类型变量，则当 cq. length==0 时队列空，当 cq.1ength==m 时队列满。

（2）队列的初始化：

```
SeQueue initqueue(SeQueue cq)
//cq为循环队列,本算法进行队列初始化
{ cq.rear=0; cq.length=0; return cq;
} // 算法结束
```

（3）队列的插入：

```
SeQueue  enqueue(SeQueue cq,ElemType x)
//cq是已如上定义的循环队列,本算法将元素x入队
{if(cq.length==m) return(0);          // 队满
 else {cq.rear=(cq.rear+1) % m;       // 计算插入元素位置
       cq.Q[cq.rear]=x;               // 将元素x入队列
       cq.length++;                   // 修改队列长度
       }
 return (cq);
} // 算法结束
```

（4）队列的删除：

```
ElemType  dlqueue(SeQueue cq)
// cq是已如上定义的循环队列,本算法是出队算法,且返回出队元素
  {if(cq.length==0) return(0);                    // 队空
   else {front=(cq.rear-cq.length+M+1) % m;       // 出队元素位置
         cq.length--;                             // 修改队列长度
         return (cq.Q[front]);                    // 返回队头元素
}//else}  // 算法结束
```

6．在一个循环链队中只有尾指针（记为 rear，结点结构为数据域 data，指针域 next），请给出这种队列的入队和出队操作的实现过程。

答：算法如下：

```
void EnQueue (LinkedList rear, ElemType x)
// rear是带头结点的循环链队列的尾指针，本算法将元素x插入到队尾
{ s= (LinkedList) malloc (sizeof(Lnode)); //申请结点空间
  s->data=x;  s->next=rear->next;          //将s结点链入队尾
  rear->next=s;  rear=s;                   //rear指向新队尾
}
void DeQueue (LinkedList  rear)
// rear是带头结点的循环链队列的尾指针
//本算法执行出队操作，成功输出队头元素；否则给出出错信息
{ if(rear->next==rear)  { printf("队空\n"); exit(0);}
s=rear->next->next;                  //s指向队头元素
rear->next->next=s->next;            //队头元素出队
printf ("出队元素是",s->data);
if(s==rear) rear=rear->next;         //空队列
free(s);
}
```

7．设整数序列 a_1，a_2，a_3，…，a_n，给出求解最大值的递归程序。

答：算法如下：

```
int MaxValue (int a[],int n)
//设整数序列存于数组a中，共有n个，本算法求解其最大值
{if(n==1) max=a[1];
 else if a[n]>MaxValue(a,n-1) max=a[n];
      else max=MaxValue(a,n-1);
 return(max);
 }
```

8．线性表中元素存放在向量 A（1，…，，l）中，元素是整型数。试写出递归算法求出 A 中的最大和最小元素。

答：算法如下：

```
Int MinMaxValue(int A[],int n,int *max,int *min)
//一维数组A中存放有n个整型数，本算法递归的求出其中的最小数和最大数
{if(n>0)
   {if(*max<A[n-1]) *max=A[n-1];
    if(*min>A[n-1]) *min=A[n-1];
    MinMaxValue(A,n-1,max,min);
  } }//算法结束
```

9．试将下列递归过程改写为非递归过程。

```
void  test(int  &sum)
{ int  x;
  scanf(x);
  if(x=0) sum=0 else {test(sum); sum+=x;}
  printf(sum);
}
```

答：算法如下：

```
int test;
 {int x,sum=0,top=0,s[];
  scanf("%d",&x)
  while(x<>0)
    {s[++top]=a; scanf("%d",&x); }
  printf(sum: 5);
  while(top)
    {sum+=s[top--]; printf(sum:5); }
  };
```

10．对于任意的无符号的十进制整数 m，写出将其转换为十六进制整数的算法（转换仅要求能够输出正确的十六进制的整数即可）。

答：算法如下：

```
void convert(unsigned int m)
  //本算法将无符号十进制整数m转换为十六进制整数
(while(m/16)
   {n=m%16;
    switch(n)
       {case 0:case 1: case 2:case 3: case 4:case 5: case 6:case 7:
        case 8:case 9: printf("%c",n+48); break;
        case 10: case 11: case 12:case 13: case 14:case 15:
                 printf("%c",n+55);
        };  m=m/16;
    }; printf("\n");
}
```

本算法的递归描述如下：

```
void convert(unsigned int m)
  //本算法将无符号十进制整数m转换为十六进制整数
{if(m)
   {n=m%16;
    switch(n)
       {case 0:case 1: case 2:case 3: case 4:case 5: case 6:case 7:
        case 8:case 9: printf("%c",n+48); break;
```

```
        case 10: case 11: case 12:case 13: case 14:case 15:
                printf("%c",n+55);
        }
    convert(m/16);
}//if  }//end
```

第 4 章　串

一、选择题

1．下面关于串的叙述中，不正确的是(　　)。

A．串是字符的有限序列

B．空串是由空格构成的串

C．模式匹配是串的一种重要运算

D．串既可以采用顺序存储，也可以采用链式存储

【答案】B 查看答案

【解析】空格构成的串称空格串。空串用 φ 表示。零个字符的串称为空串，空格也是一个字符，因此 B 项不正确。

2．设有两个串 S1 和 S2，求 S2 在 S1 中首次出现的位置的运算称作（　　）。

A．求子串　　B．判断是否相等　　C．模型匹配　　D．连接

【答案】C 查看答案

【解析】常用的串的基本操作有七种，INDEX(s,t)是其中的定位函数，这种运算就是所说的模式匹配。

3．已知串 S=′aaab′，其 Next 数组值为（　　）。

A．0123　　B．1123　　C．123l　　D．1211

【答案】A 查看答案

【解析】KMP 算法的 next 数组建立的原则

$$next[j]=\begin{cases}0 & \text{当 } j=1 \text{ 时}\\ \max\{k \mid 1<k<j \text{ 且 } 'p_1\cdots p_{k-1}'='p_{j-k+1}\cdots p_{j-1}'\} & \text{当此集合不空时}\\ 1 & \text{其它情况}\end{cases}$$

4．若串 S=′software′，其子串的数目是（　　）。

A．8　　B．37　　C．36　　D．9

【答案】B 查看答案

【解析】子串的定义是：串中任意个连续的字符组成的子序列，并规定空串是任意串的子串，任意串是其自身的子串。若字符串长度为 n（n>O），长为 n 的子串有 l 个，长为 n-1 的子串有 2 个，长为 n-2 的子串有 3 个，……，长为 1 的子串有 n 个。由于空串是任何串的子串，所以本题的答案为：8*（8+1）／2 十 l=37。故选 B。

5．串的长度是指（　　）。

A．串中所含不同字母的个数　　B．串中所含字符的个数

C．串中所含不同字符的个数　　D．串中所含非空格字符的个数

【答案】B 查看答案

【解析】串中字符的数目 n 称为字符的长度，不必考虑其中单个字符是否相等。

6．串是一种特殊的线性表，其特殊性体现在（　　）。

A．数据元素是一个字符　　B．可以顺序存储

C．数据元素可以是多个字符　　D．可以链接存储

【答案】A 查看答案

7．在下列表述中，正确的是（　　）

A．含有一个或多个空格字符的串称为空格串

B．对 n（n>0）个顶点的网，求出权最小的 n-1 条边便可构成其最小生成树

C．选择排序算法是不稳定的

D．平衡二叉树的左右子树的结点数之差的绝对值不超过 l

【答案】C 查看答案

【解析】平衡二叉树的左右子树的深度之差的绝对值不超过 l。求最小生成树时，每次挑最小权值边，是要求该边的两点都在不同的连通分量上的。

二、判断题

1.KMP 算法的特点是在模式匹配时指示主串的指针不会变小。（　　）

【答案】√查看答案

【解析】KMP 算法是一种字符串匹配的算法，KMP 算法的关键是利用匹配失败后的信息，尽量减少模式串与主串的匹配次数以达到快速匹配的目的。具体实现就是实现一个 next()函数，函数本身包含了模式串的局部匹配信息。

2．设模式串的长度为 m，目标串的长度为 n，当 n≈m 且处理只匹配一次的模式时，朴素的匹配（即子串定位函数）算法所花的时间代价可能会更为节省。（　　）

【答案】√查看答案

【解析】因为两个字符串的长度几乎相等，因此，在进行相应的匹配时，只需要定位父串的前几位即可。

3．串是一种数据对象和操作都特殊的线性表。（　　）

【答案】√查看答案

【解析】串是一种操作特殊的线性表，其特殊性主要体现在数据元素是一个字符。在内存中，一份文本都可以看做是一个字符串，而每一行都可以看做是其子串。

4．两个长度不相同的串有可能相等。（　　）

【答案】×查看答案

【解析】两个字符串相等，只有当两个字符串的长度相等，并且各个对应位置的字符相等才相等。

5．串长度是指串中不同字符的个数。（　　）

【答案】×查看答案

【解析】不是，串长度是指串中字符的总个数。需要注意的是，在计算字符串的长度时，不要漏掉了空格。

三、填空题

1．空格串是指____，其长度等于____。

【答案】由空格字符（ASCII 值 32）所组成的字符串；空格个数查看答案

2．组成串的数据元素只能是____。

【答案】字符查看答案

3．一个字符串中____称为该串的子串。

【答案】任意个连续的字符组成的子序列查看答案

4.INDEX（′DATASTRUCTURE′，′STR′）=____。

【答案】5 查看答案

5．设正文串长度为 n，模式串长度为 m，则串匹配的 KMP 算法的时间复杂度为____。

【答案】O（m+n）查看答案

6．模式串 P=′abaabcac′的 next 函数值序列为____。

【答案】01122312 查看答案

7．设 T 和 P 是两个给定的串，在 T 中寻找等于 P 的子串的过程称为____，又称 P 为____。

【答案】模式匹配；模式串查看答案

8．串是一种特殊的线性表，其特殊性表现在____；串的两种最基本的存储方式是____、____；两个串相等的充分必要条件是____。

【答案】其数据元素都是字符；顺序存储；链式存储；串的长度相等且两串中对应位置的字符也相等查看答案

9．已知 U=′xyxyxyxxyxy′；t=’xxy′；ASSIGN（S，U）；ASSIGN（V，SUBSTR（S，INDEX（S，t），LEN（t）+1））:ASSIGN（m，′ww′），求 REPLACE（S，V，m）=____。

【答案】′xyxyxywwy′查看答案

10．实现字符串拷贝的函数 strcpy 为：

```
void strcpy(char *s , char *t) / *copy t to s* /
{ while (_____________)
}
```

【答案】s++=*t++或（*s++=*t++）!=′＼0′查看答案

11．完善算法：求 KMP 算法．next 数组。

```
PROC get _next(t:string,VAR next:ARRAY[1..t.len] OF integer);
BEGIN
 j:=1; k:=__(1)__;  next[1]:=0;
 WHILE j<t.len DO
 IF k=0 OR t.ch[j]=t.ch[k] THEN BEGIN j:=j+1; k:=k+1; next[j]:=k;END
                           ELSE k:=__(2)__;
END;
```

【答案】0；next[k]查看答案

12．试利用下列栈和串的基本操作完成下述填空题。

initstack（S）　置 S 为空栈；

push（S，X）　元素 X 入栈；

pop（S）　出栈操作；

gettop（S）　返回栈顶元素；

sempty（S）　判栈空函数；

setnu11（St）　置串 St 为空串；

length（st）　返回串 st 的长度；

equal（S_1，S_2）　判串 S_1并 S_2是否相等的函数；

concat（S_1，S_2）　返回联接 S_1和 S_2之后的串；

sub（S，i，1）　返回 S 中第 i 个字符；

empty（st）　判串空函数

FUNC invert（pre：string；VAR exp：string）：boolean；

{若给定的表达式的前缀式 pre 正确，本过程求得和它相应的表达式 exp 并返回 true，否则 exp 为空串，并返回 false。已知原表达式中不包含括弧，opset 为运算符的集合。）

```
VAR  s:stack;   i,n:integer;   succ:boolean;   ch: char;
BEGIN
  i:=1;  n:=length(pre);   succ:=true;
  __(1)__;__(2)__;
  WHILE  (i<n)  AND  succ  DO
   BEGIN ch:=sub(pre,i,1);
      IF__(3)__THEN__(4)__
      ELSE IF__(5)__THEN__(6)__
           ELSE  BEGIN
                   exp:=concat(__(7)__,__(8)__);
                   exp:=concat(__(9)__,__(10)__);
                   __(11)__;
                 END;
      i:=i+1
   END;
  IF__(12)__THEN
    BEGIN exp:=concat(exp,sub(pre,n,1)); invert:=true END
  ELSE  BEGIN setnull(exp); invert:=false  END
END;
```

注意：每个空格只填一个语句。

【答案】（1）initstack（S）　/ / 栈 s 初始化为空栈查看答案

（2）setnu11（exp）　/ / 串 exp 初始化为空串

（3）ch in opset　/ / 判取出字符是否是操作符

（4）push（s，ch）　/ / 如 ch 是运算符，则入操作符栈 s

（5）sempty（s）　//判栈 s 是否为空

（6）succ：=false　/ / 若读出 ch 是操作数且栈为空，则按出错处理

（7）exp

（8）ch　/ / 若 ch 是操作数且栈非空，则形成部分中缀表达式

（9）exp

（10）gettop（s）　/ / 取栈顶操作符

（11）pop（s）　/ / 操作符取出后，出栈

（12）sempty（s）　/ / 将 pre 的最后一个字符（操作数）加入到中缀式 exp 的最后

四、应用题

1．两个字符串 sl 和 s2 的长度分别为 m 和 n。求这两个字符串最大共同子串算法的时间复杂度为 T（m，n）。估算最优的 T（m，n），并简要说明理由。

答：最优的 T（m，n）是 D（n）。串 S2 是串 Sl 的子串，且在 S1 中的位置是 1。开始求出最大公共子串的长度恰是串 S2 的长度。一般情况下，T（m，n）=D（m*n）。

2.KMP 算法（字符串匹配算法）较 Brute（朴素的字符串匹配）算法有哪些改进?

答：朴素的模式匹配（Brute．Force）时间复杂度是 0（m*n），KMP 算法有一定改进，时间复杂度达到 O（m+n）。主要优点是主串指针不回溯。当主串很大不能一次读入内存且经常发生部分匹配时，KMP 算法的优点更为突出。

3．给出模式串′abacabaaad′在 KMP 算法中的 next 和 nextval 数组。

答：模式串的 next 函数定义如下：

$$next[j]=\begin{cases}0, \text{当 } j=1 \text{时}\\ \max\{k \mid 1<k<j \text{且} ' p_1\cdots p_{k-1} ' = ' p_{j-k+1}\cdots p_{j-1} '\}, \text{当此集合不空时}\\ 1, \text{其他情况}\end{cases}$$

根据此定义，可求解模式串′abacabaaad′的 next 和 nextval 值如下：

j	1	2	3	4	5	6	7	8	9	10
模式串	a	b	a	c	a	b	a	a	a	d
$next[j]$	0	1	1	2	1	2	3	4	2	2
$nextval[j]$	0	1	0	2	0	1	0	4	2	2

4．设目标为 t=′abcaabbabcabaacbacba′，模式为 P=′abcabaa′

（1）计算模式 p 的 nextval 函数值；

（2）不写出算法，只画出利用 KMP 算法进行模式匹配时每一趟的匹配过程。

答：（1）P 的 nextval 函数值为 0110132（P 的 next 函数值为 0111232）。

（2）利用 KMP（改进的 nextval）算法，每趟匹配过程如下：

第一趟匹配：abcaabbabcabaacbacba

abcab（i=5，j=5）

第二趟匹配：abcaabbabcabaacbacba

abc（i=7，j=3）

第三趟匹配：abcaabbabcabaacbacba

a（i=7，j=1）

第四趟匹配：abcaabbabcabaac bacba

（成功）　abcabaa（i=15，j=8）

5．模式匹配算法是在主串中快速寻找模式的一种有效的方法，如果设主串的长度为 m，模式的长度为 n，则在主串中寻找模式的 KMP 算法的时间复杂性是多少?如果某一模式 P=′abcaacabaca′，请给出它的 next 函数值及 next 函数的修正值 nextval 之值。

答：KMP 算法的时间复杂性是 O（m+n）。p 的 next 和 nextval 值分别为 01112212321 和 O1102201320。

6.s 是字符数组，s[0]中存放的是该字符串的有效长度，假设 S[1．.7]中字符串的内容为″abcabaa″，说明下列程序的功能及执行结果。

```
#define  len  8
int k,  n[len];
char s[len]="7abcabaa";
void  unknown3(char T[])
  {int  i,  j;
  i=1;  n[1]=0;   j=0;
  while (i<len)
    {if (j==0 || T[i]==T[j])
      {++i;   ++j;
      if(T[i]!=T[j]) n[i]=j; else  n[i]=n[j];
      }
     else  j=n[j];
    }
  }
main()
{unknown3(s); for (k=1;k<len; k++) printf("%d",n[k];)
```

答：本程序的功能是求字符串的 nextval 函数，程序执行结果是 0110132。

7．在改进了的（无回溯）字符串模式匹配中，要先求 next 数组的值。下面是求 nextval 值的算法。

```
TYPE SAR=ARRAY[1..m] OF INTEGER;
     PTY=ARRAY[1..m] OF CHAR;
PROCEDURE next2(P:PTY;VAR NEXTVAL:SAR);
{在模式P中求nextval数组的值}
1 BEGIN
2 J:=1;NEXTVAL[1]:=0;K:=0
3 REPEAT
4   IF (K=0) OR (P[J]=P[K])
5     THEN [ J:=J+1;K:=K+1;
6            IF P[J]=P[K]
7              THEN NEXTVAL[J]:=NEXTVAL[K]
8              ELSE NEXTVAL[J]:=K ]
9     ELSE K:=NEXTVAL[K]
10  UNTIL J=m
11 END;
```

算法中第 4 行有 P[J]=P[K]，第六行中也有 P[J]=P[K]。两处比较语句相同。请分析说明此两处比较语句的含义是什么?分析此算法在最坏情况下的时间复杂度是多少?

答：第 4 行的 P[J]=P[K]语句是测试模式串的第 J 个字符是否等于第 K 个字符，如是，则指针 J 和 K 均增加 1，继续比较。第 6 行的 p[J]=p[K]语句的意义是，当第 J 个字符在模式匹配中失配时，若第 K 个字符和第 J 个字符不等，则下个与主串匹配的字符是第 K 个字符；否则，若第 K 个字符和第 J 个字符相等，则下个与主串匹配的字符是第 K 个字符失配时的下一个（即 NEXTVAL[K]）。该算法在最坏情况下的时间复杂度 O（m^2）。

8．在模试匹配 KMP 算法中所用失败函数的定义中，为何要求 $P_1P_2……P_j$为 $P_1P_2……P_j$两头匹配的真子串?且为最大真子串?

答：失败函数（即 next）的值只取决于模式串自身，若第 j 个字符与主串第 i 个字符失配时，假定主串不回溯，模式串用第 k（即 next[j]）个字符与第 i 个相比，有′$p_1 \ldots P_{k-1}$′=′$P_{j-k+1} \ldots p_{j-1}$′，为了不因模式串右移与主串第 i 个字符比较而丢失可能的匹配，对于上式中可能存在的多个 k 值，应取其中最大的一个。这样，因 j-k 最小，即模式串向右滑动的位数最小，避免因右移造成可能匹配的丢失。

9．如果两个串含有相等的字符，能否说它们相等?

答：仅从两串含有相等的字符，不能判定两串是否相等，两串相等的充分必要条件是两串长度相等且对应位置上的字符相同（即两串串值相等）。

五、算法设计题

1．输入一个字符串，内有数字和非数字字符，如：ak123x4561796073029ef4563。将其中连续的数字作为一个整体，依次存放到一数组 a 中，例如 123 放入 a[0]，456 放入 a[1]，……。编程统计其共有多少个整数，并输出这些数。

答：算法如下：

```
int CountInt()
  // 从键盘输入字符串,连续的数字字符算作一个整数,统计其中整数的个数
  {char ch; int i=0,a[];              // 整数存储到数组a,i记整数个数
    scanf("%c",&ch);                  // 从左到右读入字符串
    while(ch!='#')                    //'#'是字符串结束标记
     if(isdigit(ch))                 // 是数字字符
       {num=0;                       // 数初始化
        while(isdigit(ch)&& ch!='#')// 拼数
          {num=num*10+'ch'-'0';
           scanf("%c",&ch);
          }
        a[i++]=num;
          if(ch!='#')scanf("%c",&ch); // 若拼数中输入了'#',则不再输入
        }
    else scanf("%c",&ch); //输入非数字且非#时,继续输入字符
  printf("共有%d个整数,它们是:",i);
  for(j=0;j<i;j++)
  {printf("%6d",a[j]);
    if((j+1)%10==0)printf("\n");} // 每10个数输出在一行上
}// 算法结束
```

2．以顺序存储结构表示串，设计算法。求串 S 中出现的第一个最长重复子串及其位置并分析算法的时间复杂度。

答：算法如下：

```
int LongestString(char s[])
   //串用一维数组s存储，本算法求最长重复子串，返回其长度
{int index=0,max=0;          //index记最长的串在s串中的开始位置，max记其长度
 int length=1,i=0,start=0;   //length记局部重复子串长度，i为字符数组下标
 while(s[i]!='\0')
   if(s[i]==s[i+1]) {i++; length++;}
   else //上一个重复子串结束
      {if(max<length) {max=length; index=start;} //当前重复子串长度大，则更新max
       i++;start=i;length=1;                     //初始化下一重复子串的起始位置和长度
      }
 printf("最长重复子串的长度为%d,在串中的位置%d\n",max,index);
 return(max);
}//算法结束
```

时间复杂度：算法的时间复杂度为O（n），每个字符与其后继比较一次。

3．假设串的存储结构如下所示，编写算法实现串的置换操作。

```
TYPE strtp =RECORD
              ch: ARRAY[1..maxlen]  OF  char;
              curlen:0..maxlen
             END;
```

答：算法如下：

```
int replace(strtp s,t,int i,j)
//s和t是用一维数组存储的串，本算法将s串第i个字符开始连续j个字符用t串置换，操作成功返回1，否则返回0
表示失败
{if(i<1 || j<0 || t.curlen+s.curlen-j>maxlen)

    {printf("参数错误\n");exit(0);}                    //检查参数及置换后的长度的合法性
 if(j<t.curlen) //若s串被替换的子串长度小于t串长度，则s串部分右移
     for(k=s.curlen-1;k>=i+j-1;k--) s.ch[k+t.curlen-j]=s.ch[k];
 else if(j>t.curlen)                                   //s串中被替换子串的长度小于t串的长度
         for(k=i-1+j;k<=s.curlen-1;k++) s.ch[k-(j-t.curlen)]=s.ch[k];
   for(k=0;k<t.curlen;k++) s.ch[i-1+k]=t.ch[k];  //将t串复制到s串的适当位置
  s.curlen=s.curlen+(t.curlen-j);
 }//算法结束
```

```
void replace(string s,t,v)
//本算法是串的置换操作,将串s中所有非空串t相等且不重叠的子串用v代替
{i=index(s,t);   //判断s是否有和t相等的子串
 if(i!=0)        //串s中包含和t相等的子串
  {creat(temp,");              //creat操作是将串常量(此处为空串)赋值给temp
   m=length(t);n=length(s);   //求串t和s的长度
   while(i!=0)
      {assign(temp,concat(temp,substr(s,1,i-1),v));//用串v替换t形成部分结果
       assign(s,substr(s,i+m,n-i-m+1));            //将串s中串后的部分形成新的s串
       n=n-(i-1)-m;                                //求串s的长度
       i=index(s,t);                               //在新s串中再找串t的位置
}
assign(s,contact(temp,s));   //将串temp和剩余的串s连接后再赋值给s
}//if结束
}//算法结束
```

4．已知字符串 Sl 中存放一段英文，写出算法 format（sl，s2，s3，n），将其按给定的长度 n 格式化成两端对齐的字符串 S2，其多余的字符送 S3。

答：算法如下：

```
void format (char *s1,*s2,*s3)
 //将字符串s1拆分成字符串s2和字符串s3,要求字符串s2长度为n且两端对齐
{char *p=s1, *q=s2;
 int i=0;
 while(*p!= '\0' && *p==' ') p++;//滤掉s1左端空格
 if(*p=='\0') {printf("字符串s1为空串或空格串\n");exit(0); }
 while( *p!='\0' && i<n){*q=*p; q++; p++; i++;}//字符串s1向字符串s2中复制
 if(*p =='\0'){ printf("字符串s1没有%d个有效字符\n",n); exit(0);}

    if(*(--q)==' ' )   //若最后一个字符为空格,则需向后找到第一个非空格字符
     {p-- ;            //p指针也后退
      while(*p==' '&&*p!='\0') p++;//往后查找一个非空格字符作为串s2的尾字符
      if(*p=='\0') {printf("s1串没有%d个两端对齐的字符串\n",n); exit(0);}
      *q=*p;           //字符串s2最后一个非空字符
      *(++q)='\0';     //置s2字符串结束标记
      }
    *q=s3;p++;         //将s1串其余部分送字符串s3
    while(*p!= '\0') {*q=*p; q++; p++;}
    *q='\0';           //置串s3结束标记
   }
```

5．串以静态存储结构存储，结构如下所述，试实现串操作 equal 算法。

```
CONST   maxlen=串被确认的最大长度
TYPE   strtp=RECORD
                ch:ARRAY[1..maxlen] OF char;
                curlen:0..maxlen
              END;
```

答：算法如下：

```
int equal(strtp s,strtp t)
//本算法判断字符串s和字符串t是否相等，如相等返回1，否则返回0
{if(s.curlen!=t.curlen) return (0);
for(i=0; i<s.curlen;i++)      //在类C中，一维数组下标从零开始
  if(s.ch[i]!= t.ch[i]) return (0);
return (1);    //两串相等
}//算法结束
```

6．编写程序，统计在输入字符串中各个不同字符出现的频度并将结果存入文件（字符串中的合法字符为 A～Z 这 26 个字母和 0～9 这 10 个数字）。

答：算法如下：

```
void Count()
//统计输入字符串中数字字符和字母字符的个数
 {int i,num[36];
 char ch;
  for(i=0;i<36;i++)num[i]=0;// 初始化
  while((ch=getchar())!='#')   //'#'表示输入字符串结束
     if(ch>='0'&& ch<='9'){i=ch-48;num[i]++;}          // 数字字符
     else if(ch>='A'&& ch<='Z'){i=ch-65+10;num[i]++;}// 字母字符
  for(i=0;i<10;i++)   // 输出数字字符的个数
       printf("数字%d的个数=%d\n",i,num[i]);
  for(i=10;i<36;i++)// 求出字母字符的个数
       printf("字母字符%c的个数=%d\n",i+55,num[i]);
}// 算法结束。
```

7．写出一个递归算法来实现字符串逆序存储。

答：算法如下：

```
void InvertStore(char A[])
  //字符串逆序存储的递归算法
{char ch;
 static int i = 0;   //需要使用静态变量
 scanf ("%c",&ch);
```

```
    if(ch!= '.')          //规定'.'是字符串输入结束标志
        {InvertStore(A);
        A[i++] = ch; //字符串逆序存储
      }
    A[i] = '\0';          //字符串结尾标记
  }//结束算法InvertStore
```

8．若 x 和 y 是两个采用顺序结构存储的串，编写一个比较两个串是否相等的函数。

答：算法如下：

```
(int StringEqual(STRING x, STRING y)
  //本算法判断两个顺序存储的串x和y是否相等，相等返回1，否则返回0
  (if(x.length!=y.length) return(0);
   p=x.str; q=y.str;
   while(p && q && *p==*q) {p++; q++;}   //对应字符相等，指针后移
   if(!p && !q) return(1); else return(0);
  }
```

9．试设计一个 C 语言算法（或 C 语言程序）：用单链表做存储结构，以回车符为结束标志，输入一个任意长度的字符串，然后判断该字符串是否为“回文”（正向读和反向读时，串值相同的字符串称为“回文”），输出信息“Yes”或“N0”；最后删除字符串并释放全部空间。例如：

若输入“ABCD12321DCBA”是回文，则输出“Yes”；

若输入“ABCD123DCBA”，不是回文，则输出“N0”。

要求：定义相关数据类型，不得使用数组（顺序表）做字符串的存储结构和辅助存储空间。假定字符串的长度为 n，试分析上述算法的时间复杂度。

答：算法如下：

```
int Symmetry(Linkedlist head,int n)
  //本算法判断数据域为字符且长为n的单链表是否是"回文"，返回1或0表示成功或失败
  {char s[];                  //字符栈，容量足够大
  p=head->next                //设链表带头结点
  int i=0;
  while(i<=n/2)               //前一半字符入栈，链表指针后移
    {s[i]=p->data; p=p->next; i++;}
  if(n%2==1) p=p->next;   //若链表有奇数个结点，则跳过中间结点
  while(p)
     if(p->data==s[i]) {p=p->next; i--;}
     else break;              //不是回文
  if(!p && i==0) return(1);
   else return(0);
  }
```

第 5 章　数组和广义表

一．选择题

1．设有一个 10 阶的对称矩阵 A，采用压缩存储方式，以行序为主存储，a_{11}为第一元素，其存储地址为 1，每个元素占一个地址空间，则 a_{85}的地址为（　　）。

A．13　　B．33　　C．18　　D．40

【答案】B 查看答案

【解析】对于对称矩阵,$a_{i,j}=a_{j,i}$。为了节省存储空间，为多个相同的元素只分配一个存储空间。对于对称矩阵，元素下表之间的对应关系为：当 i>=j 时，k=i(i-1)/2+j -1；当 i< =j 时，k=j(j-1)/2+i-1。其中 k 相当于地址空间的标号，i 为行号，j 为列号。因为第一个元素存储地址为 1，所以最后计算的 k 需要加 1。所以 a_{85}的存储位置为 8*（8-1）/2+5=33。

2．设有数组 A[i，j]，数组的每个元素长度为 3 字节，i 的值为 1 到 8，j 的值为 1 到 10，数组从内存首地址 BA 开始顺序存放，当用以列为主存放时，元素 A[5，8]的存储首地址为（　　）。

A．BA+141　　B．BA+180　　C．BA+222　　D．BA+225

【答案】B 查看答案

【解析】在计算中，可以考虑按照列存放时，A[5,8]在内存的位置，比较容易计算元素的首地址。比如 A[5，8]顺序存放时，它是第 7*8+5=61 个元素，由于首地址为 BA，所以它的存储首地址为 BA+（61-1）*3=180+BA。

3．数组通常具有的两种基本操作是（　　）。

A．查找和修改　　B．查找和索引　　C．索引和修改　　D．建立和删除

【答案】A 查看答案

【解析】数组中的元素是顺序存放的，通过下标可以很好地查找数组元素，同时通过对应的指针可以修改数组元素的值，因此数组通常具有的两种基本操作是查找和修改。根据数组的性质，数组通常具有的两种基本运算是排序和查找。

4．将一个 A[1．.100，1．.100]的三对角矩阵，按行优先存入一维数组 B[1．.298]中，A 中元素 A_{6665}（即该元素下标 i=66，j=65），在 B 数组中的位置 K 为（　　）。

A．198　　B．195　　C．197

【答案】B 查看答案

【解析】将对角矩阵 a[i][j]存入 b[k]，三对角矩阵压缩地址计算公式如下：k=2i+j-2。

5．稀疏矩阵一般的压缩存储方法有两种，即（　　）。

A．二维数组和三维数组　　B．三元组和散列　　C．三元组和十字链表　　D．散列和十字链表

【答案】C 查看答案

【解析】稀疏矩阵一般的压缩方法为三元组表和十字链表。三元组表就是将非零元素及其对应的行和列构成一个三元组（行标，列标，值）。十字链表相比三元组表而言，主要是对每个结点增加了两个链域。如果数组经常运算时，会产生大量数据元素的移动，此时，采用链表存储结构更为恰当。

6.若对 n 阶对称矩阵 A 以行序为主序方式将其下三角形的元素(包括主对角线上所有元素)依次存放于一维数组 B[1…（n（n+1））/ 2]中，则在 B 中确定 a_{ij}（i<j）的位置 k 的关系为（　　）。

A．i*（i-1）/ 2+j　　B．j*（j-1）/ 2+i

C．i*（i+1）/ 2+j　　D．j*（j+1）/ 2+i

【答案】B 查看答案

【解析】将 n 阶对称矩阵存人一维数组中，一维数组的大小需为 n（n+1）/ 2。对 n 阶对称矩阵 A 以行序为主序方式将其下三角形的元素（包括主对角线上所有元素）依次存放于一维数组 B[1…（n（n+1））/ 2]中，当 i<j 时，i 与 k 的关系为 j*（j-1）/ 2+i。

7．设二维数组 A[1．．m，1．．n]（即 m 行 n 列）按行存储在数组 B[1．．m*n]中，则二维数组元素 A[i，j]在一维数组 B 中的下标为（　　）。

A．（i-1）*n+j　B．（i-1）*n+j-1　C．i*（j-1）　D．j*m+i-1

【答案】A 查看答案

【解析】前 i-1 的元素个数为（i-1）*n，所以二维数组元素 A[i，j]在一维数组 B 中的下标为（i-1）*n+j。需要注意数组 B 的下标是从 0 开始，还是从 1 开始。

8. 设有一个 n 行 n 列的对称矩阵 A，将其下三角部分按行存放在一个一维数组 B 中，A[0][0]存放于 B[0]中，那么第 i 行的对角元素 A[i][i]存放于 B 中（　　）处。

A．（i+3）*i / 2　B．（i+1）*i / 2　C．（2n-i+1）*i / 2　D．（2n-i-1）*i / 2

【答案】A 查看答案

【解析】A[i][i]中列标不大于行标，又 A[0][0]存放在 B[0]中，所以 A[i][i]存放的位置为 i*(i+1)/2+i+1-1=i*(i+3)/2。

9．有一个 100*90 的稀疏矩阵，非 0 元素有 10 个，设每个整型数占 2 字节，则用三元组表示该矩阵时，所需的字节数是（　　）。

A．60　B．66　C．18000　D．33

【答案】B 查看答案

【解析】如果是全部，则是需要 100*90*2 个字节；但是用三元组表示的话，只需要记录非零数据的 X 坐标，Y 坐标，数值即可，就是每个非零数字需要占用三个整数的空间，即 2*3=6 字节，10 个非零整数则是 2*3*10=60 字节；如果问有效元素占的空间大小，则选 A 项，但是如果从整体来看，应该多一个用来记录矩阵 宽(100)、高(90)、默认值(0) 的元素，所以还应该多算 6 个字节。所以全部为 66 字节，选 B 项。

10．数组 A[0．.4，-l．．-3，5．.7]中含有元素的个数（　　）。

A．55　B．45　C．36　D．16

【答案】B 查看答案

【解析】该数组为三维数组。其个数为 5*3*3=45。

11．用数组 r 存储静态链表，结点的 next 域指向后继，工作指针 j 指向链中结点，使 j 沿链移动的操作为（　　）。

A．j=r[j].next　B．j=j+l　C．j=j->next　D．j=r[j]->next

【答案】A 查看答案

【解析】因为是用数组存储，这里所说的工作指针 j 相当于数组的下标，结点是存储一个值域和 next 域，next 域就是存放下一个结点的下表，所以只要将 next 域中的值赋给 j 就可以实现 j 沿链移动。

12．对矩阵压缩存储是为了（　　）。

A．方便运算　B．方便存储　C．提高运算速度　D．减少存储空间

【答案】D 查看答案

【解析】压缩存储也就是对那些没用的元素不进行存储或者对那些具有一定规律的相同元素放在一个存储空间，目的就是为了节省空间。

13．一个非空广义表的表尾（　　）。

A．不能是子表　B．只能是子表　C．只能是原子　D．是原子或子表

【答案】B 查看答案

【解析】广义表的定义是一个递归定义，当广义表非空时，称第一个元素是它的表头，称其余元素构成的表称为它的表尾。因此一个非空广义表的表尾只能是子表。

14．已知广义表 LS=（（a，b，c），（d，e，f）），用 head 和 tail 数取出 LS 中原子 e 的运算是（　　）。

A．head（tail（LS））　　B．tail（head（LS））
C．head（tail（head（tail（LS））））　　D．head（tail（tail（head（LS））））
【答案】C 查看答案
【解析】head 操作就是得到广义表中第一个的原子。tail 操作就是得到除第一个原子外剩下元素构成的表。tail(LS)得到((d,e,f)),head(tail(LS))得到(d,e,f),tail(head(tail(LS)))得到（e,f）, head（tail（head（tail（LS））））得到 e。
15．广义表 A=（a，b，（c，d），（e，（f,g））），则式子 Head（Tail（Head（Tail（Tail（A））））））的值为（　　）。
A．（g）　　B．（d）　　C．c D．d
【答案】D 查看答案
【解析】head 操作就是得到广义表中第一个的原子。tail 操作就是得到除第一个原子外剩下元素构成的表。也就是 tail 得到的元素需要在外层再加一个()。
16．将线性表的数据元素进行扩充，允许带结构的线性表是（　　）。
A．串　　B．树　　C．广义表　　D．栈
【答案】C 查看答案
【解析】串、树、栈中的数据元素都是属于非结构的原子类型，元素的值是不可分解的。数组和广义表都是允许带结构的线性表。
二、判断题
1．数组不适合作为任何二叉树的存储结构。（　　）
【答案】×查看答案
【解析】对于完全二叉树，因为其 n 个结点的位置已经固定，所以用一维数组作为存储结构是高效率的（存储密度大）。
2．从逻辑结构上看，n 维数组的每个元素均属于 n 个向量。（　　）
【答案】√查看答案
【解析】对于每一维，都有一个向量共用这个元素，比如我们最常见的二维数组，对于每个元素，在行和列都有一个向量共用这个元素。
3．稀疏矩阵压缩存储后，必会失去随机存取功能。（　　）
【答案】√查看答案
【解析】稀疏矩阵在压缩存储后，必回失去随机存储的功能。因为在这个矩阵中，非零元素的分布是没有规律的，为了压缩存储，就将每一个非零元素的值和它所在的行、列号做为一个结点存放在一起，这样的结点组成的线性表中叫三元组表，它已不是简单的向量，所以无法用下标直接存取矩阵中的元素。
4．数组是同类型值的集合。（　　）
【答案】×查看答案
【解析】数组是具有相同性质的数据元素的集合，数据元素不仅有值，还有下标。因此，可以说数组是元素值和下标构成的偶对的有穷集合。
5．数组可看成线性结构的一种推广，因此与线性表一样，可以对它进行插入，删除等操作。（　　）
【答案】×查看答案
【解析】数组在维数和界偶确定后，其元素个数已经确定，不能进行插入和删除运算。
6．一个稀疏矩阵 A_{m*n}采用三元组形式表示，若把三元组中有关行下标与列下标的值互换，并把 m 和 n 的值互换，则就完成了 A_{m*n}的转置运算。（　　）
【答案】×查看答案

【解析】稀疏矩阵转置后，除行列下标及行列数互换外，还必须确定该元素转置后在新三元组中的位置。

7．二维以上的数组其实是一种特殊的广义表。（　　）

【答案】√查看答案

【解析】广义表是线性表的推广。广义表中的元素还有可能是广义表。对于维数大于二的数组，它在某一维上的元素还是数组。符合广义表的定义，因此二维以上的数组其实是一种特殊的广义表。

8．广义表的取表尾运算，其结果通常是个表，但有时也可是个单元素值。（　　）

【答案】×查看答案

【解析】广义表的取表尾运算，是非空广义表除去表头元素，剩余元素组成的表，不可能是原子。

9．若一个广义表的表头为空表，则此广义表亦为空表。（　　）

【答案】×查看答案

【解析】广义表的表头就是广义表的第一个元素。只有非空广义表才能取表头。

10．对长度为无穷大的广义表，由于存储空间的限制，不能在计算机中实现。（　　）

【答案】√查看答案

【解析】可以有长度无穷大的广义表，只是在计算机中不能实现。

11．一个广义表可以为其他广义表所共享。（　　）

【答案】√查看答案

【解析】因为广义表中的元素还可能是表，所以对于一个广义表可以为其他广义表所共享。

12．广义表（（（a，b，C），d，e，f））的长度是 4。（　　）]

【答案】×查看答案

【解析】长度为 1。因为只含一个元素即子表（（（a，b，C），d，e，f））。

三、填空题

1．数组的存储结构采用_____存储方式。

【答案】顺序存储结构查看答案

【解析】数组本身的存储结构是线性的，也就是说它是连续存储的。

2．设二维数组 A 的行和列的下标范围分别为[0：8]和[0：10]，每个元素占 2 个单元，按行优先顺序存储，第一个元素的存储起始位置为 b，则存储位置为 b+50 处的元素为_____。

【答案】A[2][3]查看答案

【解析】令这个元素的行标为 i，列标为 j。则它的存储位置是（11*i+j+1-1）*2+b。当其值为 b+50 时，则 i=2,j=3。

3．设数组 a[1．.50，1．.80]的基地址为 2000，每个元素占 2 个存储单元，若以行序为主序顺序存储，则元素 a[45，68]的存储地址为__（1）____；若以列序为主序顺序存储，则元素 a[45，68]的存储地址为___（2）____。

【答案】9174；8788 查看答案

【解析】设一个元素的行标为 i，列标为 j。若以行序为主存储顺序，则它的存储地址为 2000+（(i-1)*80＋ｊ－１）＊２。若以列序为主存储顺序，则它的存储地址为 2000+((j-1)*50+i-1)*2。

4．已知二维数组 A[1．.10，0．.9]中每个元素占 4 个单元，在按行优先方式将其存储到起始地址为 1000 的连续存储区域时，A[5，9]的地址是：_____。

【答案】1196 查看答案

【解析】设元素的行标为 i，列标为 j。则它的存储位置为：1000+[(i-1)*10+(j-0)]*4

5．设数组 A[0．.8，1．.10]，数组中任一元素 A[i，j]均占内存 48 个二进制位，从首地址 2000 开始连续存放在主内存里，主内存字长为 16 位，那么

（1）存放该数组至少需要的单元数是_____；
（2）存放数组的第 8 列的所有元素至少需要的单元数_____；
（3）数组按列存储时，元素 A[5，8]的起始地址是_____。
【答案】270；27；2204 查看答案
【解析】数组的元素个数为 9*10=90，因为每个元素占内存 48 个二进制位，即 6 个字节。故总需要 90*6=540 个字节，因为主内存字长为 16 位，即 2 个字节，所以至少需要 540/2=270 个单元数。第 8 列有 9 个元素，共占 9*6=54 个字节，因此至少需要 54/2=27 个单元数。由题知，每个元素占 3 个单元。按列存储时，A[5，8]的起始地址为 2000+[(8-1)*9+(5-0)]*3=2204。
6．设有一个 10 阶对称矩阵 A 采用压缩存储方式（以行为主序存储：a_{11}=1），则 a_{85}的地址为_____。
【答案】33 查看答案
【解析】设存储的元素的行标为 i，列标为 j。若 i>=j，则 $a_{i,j}$的地址为 1+2+…+i-1+j=i(i-1)/2+j。若 i<j。则 $a_{i,j}$的地址为 j(j-1)/2+i。将 i=8,j=5 代入得 33。
7. 假设一个 15 阶的上三角矩阵 A 按行优先顺序压缩存储在一维数组 B 中，则非零元素 $A_{9,9}$在 B 中的存储位置 k=_____。（注：矩阵元素下标从 1 开始）
【答案】93 查看答案
【解析】对于上三角矩阵，k=(i-1)(2n-i+2)/2+(j-i)+1。将 i=j=9,n=15 代入得 93。
8．当广义表中的每个元素都是原子时，广义表便成了_____。
【答案】线性表查看答案
【解析】如果每个元素都是原子，则元素不可分。此时的元素是只有一对一的关系，所以广义表变成了线性表。
9．设广义表 L=（（），（））， 则 head（L）是____（1）____tail（L）是____（2）____；L 的长度是____（3）____；深度是____（4）____。
【答案】（）；（（））；2；2 查看答案
【解析】广义表的表头是表的第一个元素，表尾是除了第一个元素外其余的所有的元素构成的表；表的长度指表中元素的个数；表的深度指展开后括号的层数。
四、应用题
1．三维数组 A[1．.10，-2．.6，2．.8]的每个元素的长度为 4 个字节，试问该数组要占多少个字节的存储空间?如果数组元素以行优先的顺序存储，设第一个元素的首地址是 100，试求元素 A[5，0，7]的存储首地址。
答：数组占的存储字节数=10*9*7*4=2520；A[5，0，7]的存储地址=100+[4*9*7+2*7+5]*4=1184
2．设 m×n 阶稀疏矩阵 A 有 t 个非零元素，其三元组表示为 LTMA[1．．（t+1），1．.3]，试问：非零元素的个数 t 达到什么程度时用 LTMA 表示 A 才有意义?
答：稀疏矩阵 A 有 t 个非零元素，加上行数 mu、列数 nu 和非零元素个数 tu（也算一个三元组），共占用三元组表 LTMA 的 3（t+1）个存储单元，用二维数组存储时占用 m*n 个单元，只有当 3（t+1）<m*n 时，用 LTMA 表示 A 才有意义。解不等式得 t<m*n / 3-1。
3．对一个有 t 个非零元素的 A_{mn}矩阵，用 B[0．．t][1．.3]的数组来表示，其中第 0 行的三个元素分别为 m，n，t，从第一行开始到最后一行，每行表示一个非零元素；第一列为矩阵元素的行号，第二列为其列号，第三列为其值。对这样的表示法，如果需要经常进行该操作-确定任意一个元素 A[i][j]在 B 中的位置并修改其值，应如何设计算法可以使时间得到改善?
答：题中矩阵非零元素用三元组表存储，查找某非零元素时，按常规要从第一个元素开始查找，属于顺序查找，时间复杂度为 O（n）。若使查找时间得到改善，可以建立索引，将各行行号及各行第一个非零元素在数组 B 中的位置（下标）放入一向量 C 中。若查找非零元

素，可先在数组 C 中用折半查找到该非零元素的行号，并取出该行第一个非零元素在 B 中的位置，再到 B 中顺序（或折半）查找该元素，这时时间复杂度为 O（logn）。

4．特殊矩阵和稀疏矩阵哪一种压缩存储后失去随机存取的功能?为什么?

答：特殊矩阵指值相同的元素或零元素在矩阵中的分布有一定规律，因此可以对非零元素分配单元（对值相同元素只分配一个单元），将非零元素存储在向量中，元素的下标 i 和 j 和该元素在向量中的下标有一定规律，可以用简单公式表示，仍具有随机存取功能。而稀疏矩阵是指非零元素和矩阵容量相比很小（t<<m*n），且分布没有规律。用十字链表作存储结构自然失去了随机存取的功能。即使用三元组表的顺序存储结构，存取下标为 i 和 j 的元素时，要扫描三元组表，下标不同的元素，存取时间也不同，最好情况下存取时间为 O（1），最差情况下是 O（n），因此也失去了随机存取的功能。

5．已知 n 阶下三角矩阵 A（即当 i<j 时，有 $a_{ij}=0$），按照压缩存储的思想，可以将其主对角线以下所有元素（包括主对角线上元素）依次存放于一维数组 B 中，请写出从第一列开始采用列序为主序分配方式时在 B 中确定元素 a_{ij} 的存放位置的公式。

答：2 阶下三角矩阵元素 A[il[j]（1≤i，j≤n，i≥j）。第 1 列有 n 个元素，第 j 列有 n-j+1 个元素，第 1 列到第 j-1 列是梯形，元素数为（n+（n-j+2）（j-1）/ 2，而 a_{ij} 在第 j 列上的位置是为 i-j+1。所以 n 阶下三角矩阵 A 按列存储，其元素 a_{ij} 在一维数组 B 中的存储位置 k 与 i 和 j 的关系为：

$$k=(n+(n-(j-1)+1)(j-1)/2+(i-j+1)=(2n-j)(j-1)/2+i$$

6．假定有下列，n×n 矩阵（n 为奇数）

$$A=\begin{bmatrix} a_{11} & 0 & \cdots & 0 & a_{1n} \\ 0 & a_{22} & \cdots & a_{2,n-1} & 0 \\ & & \cdots & & \\ 0 & \cdots & a_{(n+1)/2,(n+1)/2} & \cdots & 0 & 0 \\ & & \cdots & & \\ a_{n1} & 0 & \cdots & 0 & a_{nn} \end{bmatrix}$$

如果用一维数组 B 按行主次序存储 A 的非零元素，问：

（1）A 中非零元素的行下标与列下标的关系；

（2）给出 A 中非零元素 a_{ij} 的下标（i，j）与 B 中的下标 R 的关系；

（3）假定矩阵中每个元素占一个存储单元且 B 的起始地址为 A_0。，给出利用 a_{ij} 的下标（i，j）定位在 B 中的位置公式。

答：（1）主对角线上元素的坐标是 i=j，副对角线上元素的坐标 i 和 j 有 i+j=n+1 的关系，所以 A 中非零元素的行下标和列下标的关系是 i=j 或 i=n+1-j（1≤i，j≤n）。

（2）非零元素分布在两条主、副对角线上，除对角线相交处一个元素（下称“中心元素”）外，其余每行都有两个元素。主对角线上的元素，在向量 B 中存储的下标是 k=2i-1（i=j，1≤i，j≤n，1≤k≤2n-1）。副对角线上的元素，在中心元素前，在向量 B 中存储的下标是 k=2i（i<>j，1≤i，j≤n / 2）；在中心元素后，其下标如下：

$$k=2(i-1)(i\neq j,\ n/2+1\leqslant i,\ j\leqslant n,\ 1\leqslant k\leqslant 2n-1)$$

（3）a_{ij} 在 B 中的位置如下：

$$k=\begin{cases} A_0+2(i-1) & (i=j,\ 1\leqslant i,\ j\leqslant n) \\ A_0+2(i-1)+1 & (i\neq j,\ 1\leqslant i,\ j\leqslant n/2) \\ A_0+2(i-1)-1 & (i\neq j,\ n/2+1<i,\ j\leqslant n) \end{cases}$$

7．简述广义表属于线性结构的理由。

答：广义表中的元素，可以是原子，也可以是子表，即广义表是原子或子表的有限序列，满足线性结构的特性：在非空线性结构中，只有一个称为“第一个”的元素，只有一个称为“最后一个”的元素，第一元素有后继而没有前驱，最后一个元素有前驱而没有后继，其余每个元素有唯一前驱和唯一后继。从这个意义上说，广义表属于线性结构。

8．下列广义表，可以唯一对应一棵二叉树的有（　　）。并归纳出唯一对应的条件。

（1）（A（B（D，E），C（F））） （2）（A（B（D，E），C）） （3）（A）

（4）（A（B（C，D（E））））　（5）（）

答：唯一对应一棵二叉树的有（2）、（3）和（5）。唯一对应的条件：空表、只有一个元素的表、每个子表个数是零或是 2 的表。

五、算法设计题

1．以三元组表存储的稀疏矩阵 A，B 非零元个数分别为 m 和 n。试用类 PASCAL 语言编写时间复杂度为 O（m+n）的算法将矩阵 B 加到矩阵 A 上去。A 的空间足够大，不另加辅助空间。要求描述所用结构。

答：算法如下：

```
CONST maxnum=大于非零元素个数的某个常量
TYPE tuple=RECORD
                  i,j: integer; v: elemtp;
            END;
sparmattp=RECORD
             mu,nu,tu: integer;
             data: ARRAY[1..maxnum] OF tuple;
           END;
PROC AddMatrix(VAR A: sparmattp;B: sparmattp);
//本算法实现以三元组表存储的各有m和n个非零元素两个稀疏矩阵相加,结果放到A中
 L:=m;p:=n;k:=m+n;// L,p为A,B三元组表指针,k为结果三元组表指针(下标)
WHILE(L>1)AND(p>1)DO
  [CASE  // 行号不等时,行号大者的三元组为结果三元组表中一项
    A.data[L].i>B.data[p].i: A.data[k]:=A.data[L];L:=L-1;// A中当前项为结果项
    A.data[L].i<B.data[p].i: A.data[k]:=B.data[p];p:=p-1;//B中当前项为结果当前项
    A.data[L].i=B.data[p].i;
    CASE  //行号相等时,比较列号
     A.data[L].j>B.data[p].j: A.data[k]:=A.data[L];L:=L-1;
     A.data[L].j<B.data[p].j: A.data[k]:=B.data[p];p:=p-1;
     A.data[L].j=B.data[p].j: IF A.data[L].v+B.data[p].v≠0 THEN
                              [A.data[L].v=A.data[L].v+ B.data[p].v;
                              A.data[k]:= A.data[L];
                              ]
                              L:=L-1;p:=p-1;
     ENDC;      //结束行号相等时的处理
  ENDC;   //结束行号比较处理
  k:=k-1; //结果三元组表的指针前移(减1)
]//结束WHILE循环。
WHILE p>0 DO[A.data[k]:=B.data[p];k:=k-1;p:=p-1;]   //处理B的剩余部分
WHILE L>0 DO[A.data[k]:=A.data[L];k:=k-1;L:=L-1;]   //处理A的剩余部分
IF k>1 THEN     //稀疏矩阵相应元素相加时,有和为零的元素,因而元素总数<m+n
 [FOR p:=k+1 TO m+n DO A[p-k]:=A[p];// 三元组前移,使第一个三元组的下标为1
  A.tu=m+n-k;]                          // 修改结果三元组表中非零元素个数
ENDP;  // 结束addmatrix
```

2．请编写完整的程序。如果矩阵 A 中存在这样的一个元素 A[i，j]满足条件：A[i，j]是第 i 行中值最小的元素，且又是第 j 列中值最大的元素，则称之为该矩阵的一个马鞍点。请编程计算出 m*n 的矩阵 A 的所有马鞍点。

答：算法如下：

```
int m=10, n=10;
 void Saddle(int A[m][n])
  //A是m*n的矩阵,本算法求矩阵A中的马鞍点
  {int i,j, max[n]={0}, //max数组存放各列最大值元素的行号,初始化为行号0
       min[m]={0};      //min数组存放各行最小值元素的列号,初始化为列号0
   for(i=0;i<m;i++)     //选各行最小值元素和各列最大值元素
      for(j=0;j<n;j++)
         {if(A[max[j]][j]<A[i][j]) max[j]=i;  //修改第j列最大元素的行号
          if(A[i][min[i]]>A[i][j]) min[i]=j;  //修改第i行最小元素的列号
         }
  for(i=0;i<m;i++)
   {j=min[i];                 //第i行最小元素的列号
    if(i==max[j])printf("A[%d][%d]是马鞍点,元素值是%d",i,j,A[i][j]);  //是马鞍点
   }
}// Saddle
```

3．给定一个整数数组 b[0．．N 一 1]，b 中连续的相等元素构成的子序列称为平台。试设计算法，求出 b 中最长平台的长度。

答：算法如下：

```
void Platform (int b[], int N)
 //求具有N个元素的整型数组b中最长平台的长度。
   {l=1;k=0;j=0;i=0;
    while(i<N-1)
     {while(i<N-1 && b[i]==b[i+1]) i++;
      if(i-j+1>1) {l=i-j+1;k=j;} //局部最长平台
      i++; j=i;                  //新平台起点
     }
    printf("最长平台长度%d,在b数组中起始下标为%d",l,k);
}// Platform
```

4．给定 n×m 矩阵 A[a．．b，c．．d]，并设

$$A[i,j] \leqslant A[i,j+1] \quad (a \leqslant i \leqslant b,\ c \leqslant j \leqslant d-1)$$

A[i，j]≤A[i+1，j]（a≤i≤b-1，c≤j≤d）。设计一算法判定 x 的值是否在 A 中，要求时间复杂度为 O（m+n）。

答：算法如下：

```
void search(ElemType A[][], int a,b,c,d, ElemType x)
 //n*m矩阵A,行下标从a到b,列下标从c到d,本算法查找x是否在矩阵A中
 {i=a; j=d; flag=0;       //flag是成功查到x的标志
  while(i<=b && j>=c && !flag)
    if(A[i][j]==x) flag=1;
    else if(A[i][j]>x) j--; else i++;
   if(flag) printf("A[%d][%d]=%d\n",i,j,x);          //假定x为整型
   else printf("矩阵A中无%d 元素\n",x);
  }算法search结束。
```

5．编写算法，将自然数 l～n^2按“蛇形”填 n×n 矩阵中。例（1～4^2）如图 5-1 所示（用程序实现）。

1	3	4	10
2	5	9	11
6	8	12	15
7	13	14	16

图 5-1

答：算法如下：

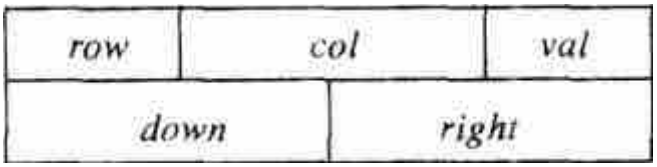
```
void Snake_Number(int A[n][n],int n)
  //将自然数1..n×n,按"蛇形"填入n阶方阵A中
{i=0; j=0; k=1;                          //i,j是矩阵元素的下标,k是要填入的自然数
while(i<n && j<n)
  {while(i<n && j>-1)                    //从右上向左下填数
     {A[i][j]=k++; i++ ;j--;}
      if((j<0)&&(i<n)) j=0;              //副对角线及以上部分的新i,j坐标
      else {j=j+2; i=n-1;}               //  副对角线以下的新的i,j坐标
   while(i>-1 && j<n)                    //从左下向右上
     {A[i][j]=k++; i--; j++;}
   if(i<0 && j<n) i=0;
     else{i=i+2; j=n-1;}
  }//最外层while
}//Snake_Number
```

6．设计将数组 A[n]中所有的偶数移到奇数之前的算法。要求不增加存储空间，且时间复杂性为 O（n）。

答：算法如下：

```
void Arrange(int A[],int n)
//n个整数存于数组A中,本算法将数组中所有偶数排在奇数之前
 {int i=0,j=n-1,x;   //用类C语言编写,数组下标从0开始
  while(i<j)
   {while(i<j && A[i]%2==0)  i++;
    while(i<j && A[i]%2==1)  j--;
    if(i<j) {x=A[i]; A[i++]=A[j]; A[j--]=x; }//交换A[i]与A[j]
   }
}//算法Arrange结束
```

7．编写算法打印出由指针 Hm 指向总表头的以十字链表形式存储的稀疏矩阵中每一行的非零元的个数。注意：行、列及总表头结点的形式为：



它们已用 val 域链接成循环链表。非零元的结点形式也同上，每一行（列）的非零元由 right（down）域把它们链接成循环链表，该行（列）的表头结点即为该行（列）循环链表的表头。

答：算法如下：

```
int MatrixNum(Olink Hm)
//输出由Hm指向的十字链表中每一行的非零元素个数
{Olink rch=Hm->uval.next, p;
 int A[]; i=1;              //数组A记各行非零元个数,i记行号
 while(rch!=Hm)             //循环完各行列表头
  {p=rch->right; num=0;  //p是稀疏矩阵行内工作指针,num记该行非零个数
   while(p!=rch)            //完成行内非零元的查找
    {printf("M[%d][%d]=%d",p->row,p->col,p->uval.e);
     num++;p=p->right; printf("\n");//指针后移
    }
    A[i++]=num;             //存该行非零元个数
    rch=rch->uval.next; //移到下一行列表头
    }
    num=0;
    for(j=1;j<i;j++)        //输出各行非零元个数
       {num+=A[j]; printf("第%d行非零元个数为%d\n",j,A[j]); }
    return(num);//稀疏矩阵非零元个数
}算法结束
```

8．已知两个定长数组，它们分别存放两个非降序有序序列，请编写程序把第二个数组序列中的数逐个插入到前一个数组序列中，完成后两个数组中的数分别有序（非降序）并且第一数组中所有的数都不大于第二个数组中的任意一个数。注意，不能另开辟数组，也不能对任意一个数组进行排序操作。

例如：

第一个数组为：4，12，28
第二个数组为：1，7，9，29，45
输出结果为：
l，4，7……第一个数组
9，12，28，29，45……第二个数组
答：算法如下：

```
void ReArranger (int A[],B[],m,n)
//A和B是各有m个和n个整数的非降序数组，本算法将B数组元素逐个插入到A中
//使A中各元素均不大于B中各元素，且两数组仍保持非降序排列
{ while(A[m-1]>B[0])
  {x=A[m-1];A[m-1]=B[0]; //交换A[m-1]和B[0]
   j=1;
   wkile(j<n && B[j]<x) B[j-1]=B[j++];     //寻找A[m-1]的插入位置
   B[j-1]=x;
   x=A[m-1];i=m-2;
   wkile(i>=0 && A[i]>x) A[i+1]=A[i--];    //寻找B[0]的插入位置
   A[i+1]=x;
  }
}算法结束
```

9．设 A[1．.100]是一个记录构成的数组，B[1．.100]是一个整数数组，其值介于 l～100 之间，现要求按 B[1．.100]的内容调整 A 中记录的次序，比如当 B[1]=11 时，则要求将 A[1]的内容调整到 A[11]中去。规定可使用的附加空间为 O（1）。
答：算法如下：

```
void CountSort(rectype A[],int B[])
//A是100个记录的数组，B是整型数组，本算法利用数组B对A进行计数排序
 {int i,j,n=100;
  i=1;
  while(i<n)
   {if(B[i]!=i)   //若B[i]=i则A[i]正好在自己的位置上，则不需要调整
     { j=i;
       while(B[j]!=i)
       { k=B[j]; B[j]=B[k]; B[k]=k;     // B[j]和B[k]交换
         r0=A[j];A[j]=A[k]; A[k]=r0; } //r0是数组A的元素类型，A[j]和A[k]交换
   i++;} //完成了一个小循环，第i个已经安排好
}//算法结束
```

10．设 A 和 B 均为下三角矩阵，每一个都有 n 行 n 列。因此在下三角区域中各有 n（n+1）／2 个无素。另设有一个二维数组 C，它有 n 行 n+1 列。试设计一个方案，将两个矩阵 A 和 B 中的下三角区域元素存放于同一个 C 中。要求将 A 的下三角区域中的元素存放于 C 的下三角区域中，B 的下三角区域中的元素转置后存放于 C 的上三角区域中。并给出计算 A 的矩阵元素 a_{ij}，和 B 的矩阵元素 b_{ij}在 C 中的存放位置下标的公式。
答：算法如下：

```
void UnionTrans(int A[][],B[][],C[][],n)
//本算法将n阶方阵的下三角矩阵A和B置于C中，矩阵B要逆置
 {for(i=0;i<n;i++)
 {for(j=0;j<=i;j++)    C[i][j]=A[i][j];
  for(j=i+1;j<=n;j++) C[i][j]=B[j-1][i];
 }
}//算法结束
```

```
A[i][j]=C[i][j];   { 0≤i <n, 0≤j≤i}
B[i][j]=C[j][i+1];   { 0≤i <n, i≤j≤n}
```

11．设稀疏矩阵 $M_{m\times n}$ 中有 t 个非零元素，用三元组顺序表的方式存储。请设计一个算法，计算矩阵 M 的转置矩阵 N，要求转置算法的时间复杂度为 O（n+t）。

答：算法如下：

```
TSMatrix TransMatrix(TSMatrix M,TSMatrix N)
{//采用三元组表方式存储,按列序实现矩阵的转置
  N.m=M.n;    N.n=M.m;    N.len=M.len;   //行数、列数和非零元素个数
  if(N.len){q=1;    //设置N中第一个非零元素从下标1开始存储
            for(j=1;j<=M.n;j++)                  //按列,共M.n列
                for(p=1;p<=M.len; ++p)           //在M.len个元素中查找
                    if(M.data[p].col==j) //转置
                      {N.data[q].row=M.data[p].col;
                       N.data[q].col=M.data[p].row;
                       N.data[q].e=M.data[p].e; q++;
                      }
            }
return N;
}//TransMatrix
  TSMatrix FastTransMatrix(TSMatrix M, TSMatrix N)
{//三元组表上实现矩阵的快速转置的算法
N.m=M.n;  N.n=M.m;  N.len=M.len;
 if(M.len)
   {for(j=1;j<=M.n;j++) numb[j]=0;                //矩阵M每一列非零元初始化为零
    for(t=1;t<=M.len;t++)numb[M.data[t].col]++;//求矩阵M每一列的非零元个数
    pos[1]=1;              //第1列第一个非零元在转置后的三元组中下标是1
    for(j=2;j<=M.n;j++)   //求M.data第j列第一个非零元在N.data中的序号
        pos[col]=pos[col-1]+num[col-1];
    for(p=1;p<=M.len;p++)//求转置矩阵N的三元组表
        {j=M.data[p].col;  q=pos[j];
         N.data[q].row=M.data[p].col; N.data[q].col=M.data[p].row;
         N.data[q].e=M.data[p].e;          pos[j]++; //同列下一非零元素位置
         }
   }
 return N;
}
```

第 6 章　树和二叉树

一、选择题

1．已知一算术表达式的中缀表达式为 a-（b+c / d）*e，其后缀形式为（　　）。

A．-a+b*c / d　　B．-a+b*cd / e　　C．-+*abc / de　　D．abcd / +e*-

【答案】D 查看答案

【解析】后缀表达式：在程序语言中，运算符位于两个操作数后面的表达式。

2．每个结点的度或者为 0 或者为 2 的二叉树称为正则二叉树。n 个结点的正则二叉树中有（　　）个叶子。

A. $\lceil \log_2 n \rceil$　　B. $\frac{n-1}{2}$　　C. $\lceil \log_2(n+1) \rceil$　　D. $\frac{n+1}{2}$

【答案】D 查看答案

【解析】二叉树结点总数 $n=n_0+n_1+n_2$（n_0，n_1，n_2分别代表度为 0，度为 1，度为 2 的结点数）。又在非空二叉树中：$n_0=n_2+1$，且本题所给树为正则二叉树，$n_1=0$， 所以 $n=2*n_0-1$ ，因此 $n_1=（n+1）/2$ 。

3．在下述结论中，正确的有（　　）。

①只有一个结点的二叉树的度为 0；②二叉树的度为 2；③二叉树的左右子树可任意交换，④深度为 K 的完全二叉树的结点个数小于或等于深度相同的满二叉树。

A．①②③　　B．⑦③④　　C．②④　　D．①④

【答案】D 查看答案

【解析】只有根节点的二叉树的度为 0，二叉树的左右子树隐含着他们的位置关系。因此，②③均错。

4．在一棵三元树中度为 3 的结点数为 2 个，度为 2 的结点数为 1 个，度为 1 的结点数为 2 个，则度为 0 的结点数为（　　）个。

A．4　　B．5　　C．6　　D．7

【答案】C 查看答案

【解析】设度为 0 的结点数为 x，则度为 3 的树总结点数 n=度为 0 的结点数+度为 1 的结点数+度为 2 的结点数+度为 3 的结点数=x+2+1+2=x+5；从每个结点所指向的结点数的和的角度来计算度为 3 的树总结点数 n=2×3+1×2+2×1+1=11。两种方法所计算出来的 n 相等，所以 x=6。

5．有关二叉树下列说法正确的是（　　）。

A．二叉树的度为 2

B．一棵二叉树的度可以小于 2

C．二叉树中至少有一个结点的度为 2

D．二叉树中任何一个结点的度都为 2

【答案】B 查看答案

【解析】树的度=MAX(结点 1 的度,结点 2 的度,结点 3 的度,...,结点 n 的度)。二叉树之所以称为二叉树，是因为二叉树中节点的度最大是 2，也可以小于 2。

6．一个具有 1025 个结点的二叉树的高 h 为（　　）。

A．11　　B．10　　C．11 至 1025 之间　　D．10 至 1024 之间

【答案】C 查看答案

【解析】当一棵树是完全二叉树时，其高度最低，此时高度为 11，当一棵树的结点在一条线上时，此时最高，这时二叉树的高度是 1025。

7．一棵二叉树高度为 h，所有结点的度或为 0 或为 2，则这棵二叉树最少有（　　）个结点。

A．2h　　B．2h-1　　C．2h+1　　D．h+1

【答案】B 查看答案

【解析】此树满足哈夫曼树，除根节点外每层有两个节点。

8．有 n（n>0）个分支结点的满二叉树的深度是（　　）。

A．n^2-1　　B．$\log_2（n+1）+1$　　C．$\log_2（n+1）$　　D．$\log_2（n一1）$

【答案】C 查看答案

【解析】满二叉树的结点总数=分支的结点总数+非分支的结点总数。由于此树为满二叉树，所以非分支的结点总数为 1，所以满二叉树共有 n+1 个结点，所以满二叉树的深度为 $\log_2（n+1）$。

9．将有关二叉树的概念推广到三叉树，则一棵有 244 个结点的完全三叉树的高度为（　　）。

A．4　　B．5　　C．6　　D．7

【答案】C 查看答案

【解析】若二叉树中最多只有最下面两层的结点的度数可以小于 2，并且最下面一层的叶结点都依次排列在该层最左边的位置上，则这样的二叉树称为完全二叉树。具有 n 个（n>0）结点的完全二叉树的高度为$\lceil \log_2(n+1) \rceil$或$\lfloor \log_2 n \rfloor+1$；由完全二叉树类推到完全三叉树可知，n 个结点的完全三叉树的高度为$\lceil \log_3(n+1) \rceil$或$\lfloor \log_3 n \rfloor+1$。

10．在下列存储形式中，哪一个不是树的存储形式？（　　）

A．双亲表示法　B．孩子链表表示法　C．孩子兄弟表示法　D．顺序存储表示法

【答案】D 查看答案

【解析】顺序存储就是利用一段连续的存储单元依次存储线性表中的元素。树中某个结点的孩子可以有多个，这就意味着，无论用哪种顺序将树中所有的结点存储到数组中，结点的存储位置都无法直接反映逻辑关系。因此简单的顺序存储表示不能满足树的基本要求。常用的三种树的表示法为：双亲表示法、孩子链表示法、孩子兄弟表示法。

11．已知一棵二叉树的前序遍历结果为 ABCDEF，中序遍历结果为 CBAEDF，则后序遍历结果为（　　）。

A．CBEFDA　B．FEDCBA　C．CBEDFA　D．不定

【答案】A 查看答案

【解析】由前序结果可知 A 为根节点，再由中序遍历结果知 BC 为 A 的左孩子，且 C 为 B 的左孩子结点，到此可排除 B 项，按照这种逻辑依次推理，便可得出结果对于该类型题目，可以先根据前序遍历结果和中序遍历结果画出二叉树，然后后序遍历二叉树得到后序遍历序列。

12．某二叉树结点的中序序列为 BDAECF，后序序列为 DBEFCA，则该二叉树对应的森林包括（　　）棵树。

A．1　B．2　C．3　D．4

【答案】C 查看答案

【解析】由两序列可知，A 为根节点，ECF 为右子树，C 为右子树的根，F 为 C 的右孩子。再由二叉树和森林的对应关系可知该二叉树对应的森林包括 3 棵树。根据中序序列和后序序列画出二叉树，根据二叉树得出对应的森林包含的树的棵数。

13．将一棵树 t 转换为孩子兄弟链表表示的二叉树 h，则 t 的后序遍历是 h 的（　　）。

A．前序遍历　B．中序遍历　c．后序遍历

【答案】B 查看答案

【解析】树的后序遍历恰好对应于二叉树的中序遍历。

14．对任意一棵树，设它有 n 个结点，这 n 个结点的度数之和为（　　）。

A．n　B．n-2　C．n-1　D．n+1

【答案】C 查看答案

【解析】每个结点（除根节点外）都是一个分支，即所有结点的度数之和等于分支个数等于总的结点数减一，即 n-1。

15．一棵非空的二叉树的前序序列和后序序列正好相反，则该二叉树一定满足（　　）。

A．其中任意一个结点均无左孩子

B．其中任意一个结点均无右孩子

C．其中只有一个叶结点

D．其中度为 2 的结点最多为一个

【答案】C 查看答案

【解析】前序序列是“根左右”，后序序列是“左右根”，若要这两个序列相反，只有单支树才有可能，所以本题的 A 项和 B 项均对，单支树的特点是只有一个叶结点，故 C 项是最合适的。A 项或 B 项都不全。

16. 若 X 是二叉中序线索树中一个有左孩子的结点，且 X 不为根，则 X 的前驱为（　　）。

A. X 的双亲　　B. X 的右子树中最左的结点

C. X 的左子树中最右的结点　　D. X 的左子树中最右的叶结点

【答案】C 查看答案

【解析】中序线索，只有把其左子树最右结点遍历完后，才会遍历自己，所以 X 的前驱为 X 的左子树中最右的结点。

17. 引入二叉线索树的目的是（　　）。

A. 加快查找结点的前驱或后继的速度

B. 为了能在二叉树中方便地进行插入与删除

C. 为了能方便地找到双亲

D. 使二叉树的遍历结果唯一

【答案】A 查看答案

【解析】二叉线索树有指向前驱和后继的指针，因此加快了查找前驱和后继结点的速度。

18.n 个结点的线索二叉树上含有的线索数为（　　）。

A. 2n　　B. n-1　　C. n+1　　D. n

【答案】C 查看答案

【解析】线索二叉树是利用二叉树的空链域加上线索，n 个结点的二叉树有 n+1 个空链域。

19. 二叉树在线索化后，仍不能有效求解的问题是（　　）。

A. 前序线索二叉树中求前序后继　　B. 中序线索二叉树中求中序后继

C. 中序线索二叉树中求中序前驱　　D. 后序线索二叉树中求后序后继

【答案】D 查看答案

【解析】后序线索二叉树求后序后继要分 3 种情况，比较复杂，不是仅仅线索化后就能求解的，算法上还要要分情况讨论。

20. 由 3 个结点可以构造出多少种不同的有向树?（　　）

A. 2　　B. 3　　C. 4　　D. 5

【答案】A 查看答案

【解析】满足以下条件的有向图称为有向树：①有且仅有一个结点的入度为 0；②除树根外结点的入度为 1；③从树根到任一结点有一有向通路。

21. 下述二叉树中，哪一种满足性质：从任一结点出发到根的路径上所经过的结点序列按其关键字有序（　　）。

A. 二叉排序树　　B. 哈夫曼树　　C. AVL 树　　D. 堆

【答案】D 查看答案

【解析】堆的定义：

n 个关键字序列 K_1，K_2，…，K_n称为堆，当且仅当该序列满足如下性质（简称为堆性质）：

（1）$K_i \le K_{2i}$，且 $K_i \le K_{2i+1}$或

（2）$K_i \ge K_{2i}$且 $K_i \ge K_{2i+1}$（$1 \le i \le [n/2]$）

满足第（1）种情况的堆，称为小顶堆；满足第（2）种情况的堆，称为大顶堆。

由堆的定义可知堆可以满足上述性质。

22. 一棵哈夫曼树共有 215 个结点，对其进行哈夫曼编码，共能得到（　　）个不同的码字。

A. 107　　B. 108　　C. 214　　D. 215

【答案】B 查看答案

【解析】此题可转化为一棵哈夫曼树共有 215 个结点，共有多少叶子结点。又有 n0=n2+1，所以 215=n0+n2=2*n2+1， n2=107， n0=108。也就是说若对其进行哈夫曼编码，共能得到 108 个码字。

23．设 X 是树 T 中的一个非根结点，B 是 T 所对应的二叉树。在 B 中，X 是其双亲的右孩子，下列结论正确的是（　　）。

A．在树 T 中，X 是其双亲的第一个孩子

B．在树 T 中，X 一定无右兄弟

C．在树 T 中，X 一定是叶结点

D．在树 T 中，X 一定有左兄弟

【答案】D 查看答案

【解析】由树和二叉树的转换关系可知，X 一定有左兄弟，X 是其双亲的第二个孩子，不能确定在树 T 中，X 是否有右兄弟，是否是叶结点。

二、判断题

1．对于有 n 个结点的二叉树，其高度为 $\log_2 n$。（　　）

【答案】×查看答案

【解析】例如 n 结点的单枝树，高度就为 n。

2．深度为 k 的二叉树中结点总数小于等于 2^k-1。（　　）

【答案】√查看答案

【解析】深度为 K 的二叉树，当结点数最多时为满二叉树，此时结点数为 2^k-1。

3．一个树形的叶结点，在前序遍历和后序遍历下，皆以相同的相对位置出现。（　　）

【答案】√查看答案

【解析】树的前序遍历和后序遍历叶子节点的相对次序是不变的，都遵循左右的次序。

4．用一维数组存储二叉树时，总是以前序遍历顺序存储结点。（　　）

【答案】×查看答案

【解析】后序遍历、中序遍历也可以遍历一维数组存储的二叉树。

5．中序遍历一棵二叉排序树的结点就可得到排好序的结点序列。（　　）

【答案】√查看答案

【解析】二叉排序树是的父结点和左右子树的值的大小是确定的。

6．任何二叉树的后序线索树进行后序遍历时都必须用栈。（　　）

【答案】×查看答案

【解析】任何结点至多只有左子树的二叉树的遍历就不需要栈。

7．一般来说，若深度为 k 的 n 个结点的二叉树只有最小路径长度，那么从根结点到第 k-1 层具有的最多结点数为 $2^{k-1}-1$，余下的 $n-2^{k-1}+1$ 个结点在第 k 层的任一位置上。（　　）

【答案】√查看答案

【解析】求最小路径长度，即构成哈夫曼树，当哈夫曼树为 k-1 层全满时，此时从根结点到第 k-1 层具有最大的结点数为 $2^{k-1}-1$。

8．一棵树中的叶子数一定等于与其对应的二叉树的叶子数。（　　）

【答案】×查看答案

【解析】一颗树的叶子树与它对一个的二叉树的叶子树没有直接联系。不妨举例：假设一个有三个结点的二叉树，层数为 2。则它的叶结点数为 2。将其按规则转为对应的二叉树时，则它的叶结点数为 1。

9．树形结构中元素之间存在一对多的关系。（　　）

【答案】√查看答案

【解析】树形结构是非线性结构，存在一对多的关系。

10．二叉树是一般树的特殊情形。（　　）
【答案】×查看答案
【解析】树与二叉树是两种不同的树形结构，二叉树中的孩子结点有着严格左右之分的。
11．哈夫曼树度为 1 的结点数等于度为 2 和 0 的结点数之差。（　　）
【答案】×查看答案
【解析】哈夫曼树不存在度数为 1 的结点。度数为 2 和 0 的结点数之差为 1。
12．若从二叉树的任一结点出发，到根的路径上所经过的结点序列按其关键字有序，则该二叉树一定是哈夫曼树。（　　）
【答案】×查看答案
【解析】在含有 N 个带权叶子结点的二叉树中，其中带权路径长度最小的二叉树称为哈夫曼树，也叫做最优二叉树。关键字有序，可能叶子结点部分的关键字最大，根结点的关键字部分最小，此时就不是哈夫曼树。
13．二叉树中序线索化后，不存在空指针域。（　　）
【答案】×查看答案
【解析】非空二叉树中序遍历第一个结点无前驱，最后一个结点无后继，这两个结点的前驱线索和后继线索为空指针。
14．哈夫曼树无左右子树之分。（　　）
【答案】×查看答案
【解析】哈夫曼树就是一棵二叉树。
三、填空题
1．二叉树由__（1）__，__（2）__，__（3）__三个基本单元组成。
【答案】根结点；左子树；右子树查看答案
2．在二叉树中，指针 p 所指结点为叶结点的条件是_____。
【答案】p->lchild==null&&p->rchlid==null 查看答案
【解析】叶子节点的左右孩子都不存在。
3．中缀式 a+b*3+4*（c-d）对应的前缀式为__（1）__，若 a=1，b=2，c=3，d=4，则后缀式 db / cc*a-b*+的运算结果为__（2）__。
【答案】++a*b3*4-cd；18 查看答案
【解析】中缀式相当于中序遍历，前缀式相当于前序遍历，后缀式相当于后序遍历。
4．深度为 H 的完全二叉树至少有__（1）__个结点；至多有__（2）__个结点；H 和结点总数 N 之间的关系是__（3）__。
【答案】2^{H-1}；2^{H}-1；H=ë$\log_2$N」+1 查看答案
5．在顺序存储的二叉树中，编号为 i 和 j 的两个结点处在同一层的条件是_____。
【答案】ë$\log_2$s」=ë$\log_2$t」。查看答案
【解析】用顺序存储结构存储二叉树时，要按完全二叉树的形式存储，非完全二叉树存储时，要加“虚结点”。设编号为 i 和 j 的结点在顺序存储中的下标为 s 和 t，则结点 i 和 j 在同一层上的条件是 ë$\log_2$s」=ë$\log_2$t」。
6．一棵有 n 个结点的满二叉树有__（1）__个度为 1 的结点、有__（2）__个分支（非终端）结点和__（3）__个叶子，该满二叉树的深度为__（4）__。
【答案】0；（n-1）/ 2 或 ën / 2」；（n+1）/ 2；ë$\log_2$n」+1 查看答案
【解析】满二叉树没有度为 1 的结点，度为 0 的结点等于度为 2 的结点个数+1。
7．设有 N 个结点的完全二叉树顺序存放在向量 A[1：N]中，其下标值最大的分支结点为_____。
【答案】ëN / 2」查看答案

【解析】最大的分支结点是最后一个叶子结点的父结点。

8．一个有 2001 个结点的完全二叉树的高度是_____。

【答案】11 查看答案

【解析】完全二叉树的高度 H=ëlog$_2$N」+1

9. 如某二叉树有 20 个叶结点，有 30 个结点仅有一个孩子，则该二叉树的总结点数为_____。

【答案】69 查看答案

【解析】二叉树叶结点数为 20，则度为 2 的结点数为 19，所以总的结点数为 20+19+30=69。

10．对于一个具有 n 个结点的二叉树，当它为一棵__（1）__二叉树时具有最小高度，当它为一棵__（2）__时，具有最大高度。

【答案】完全；只有一个叶结点的二叉树查看答案

11．每一棵树都能唯一地转换为它所对应的二叉树。若已知一棵二叉树的前序序列是 BEFCGDH，中序序列是 FEBGCHD，则它的后序序列是__（1）__。设上述二叉树是由某棵树转换而成，则该树的前序序列是__（2）__。

【答案】FEGHDCB；BEF 查看答案

【解析】树的前序序列对应二叉树的前序序列，该二叉树转换成森林时含三棵树，其第一棵树的前序是 BEF。

12．二叉树的前序序列和中序序列相同的条件是_____。

【答案】空树或任何结点至多只有右子树的二叉树查看答案

【解析】前序遍历的顺序为根左右，中序遍历的顺序为左根右，因此若中序遍历和前序遍历序列相同，则任何结点都没有左子树。

13．一棵左子树为空的二叉树在前序线索化后，其中的空链域的个数为_____。

【答案】2 查看答案

【解析】只有根结点的做指针为空和最右边的叶结点的右指针为空。

14．设 n_0为哈夫曼树的叶结点数目，则该哈夫曼树共有_____个结点。

【答案】$2n_0$-1 查看答案

【解析】哈夫曼树只有度为 0 和 2 的节点。

15．阅读下列程序说明和程序，填充程序中的_____。

【程序说明】本程序完成将二叉树中左、右孩子交换的操作。交换的结果如下所示（编者略）。本程序采用非递归的方法，设立一个堆栈 stack 存放还没有转换过的结点，它的栈顶指针为 tp。交换左、右子树的算法为：

（1）把根结点放入堆栈。

（2）当堆栈不空时，取出栈顶元素，交换它的左、右子树，并把它的左、右子树分别入栈。

（3）重复（2）直到堆栈为空时为止。

```
typedef  struct  node  *tree;
struct node{int data; tree lchild,rchild;}
exchange(t) tree t;
{tree  r,p; tree  stack [500]; int  tp=0;
  __(1)__
 while (tp>=0)
  {__(2)__
   if(__(3)__)
      {r=p->lchild; p->lchild=p->rchild; p->rchild=r;
      stack[__(4)__]=p->lchild; stack[++tp]=p->rchild;
      }
     }}
```

【答案】stack[tp]=t；p=stack[tp--]；P；++tp 查看答案

【解析】本题主要使用堆栈完成了二叉树左右子树交换的操作。首先根结点进栈，然后判断栈是否为空，如果不为空，则取栈顶元素，交换取出结点的左右指针。并将左右指针分别进栈，重复这一操作。完成二叉树左右孩子的交换。

16．如下的算法分别是后序线索二叉树求给定结点 node 的前驱结点与后继结点的算法，请在算法空格处填上正确的语句。设线索二叉树的结点数据结构为（lflag，left，data，right，rflag），其中：lflag=0，left 指向其左孩子，lflag=1，left 指向其前驱；rflag=0，right 指向其右孩子，rflag=1，right 指向其后继。

```
prior(node,x)
  { if(node !=null)
   if(   (1)   ) *x=node->right; else *x=node->left;
  }
next(bt, node, x)     /*bt是二叉树的树根*/
  {   (2)   ;
   if(node!=bt  &&  node!=null)
     if(node->rflag)   (3)    ;
     else {do t=*x;   (4)   ;
           while(*x==node);
           *x=t;
     }
  }
```

【答案】node->rflag==0；*x=bt；*x=node->right；prior（t，X）查看答案

17．以下是用类 C 语言写出的算法，该算法将以二叉链表存储的二叉树中的叶结点按从左到右的顺序链成一个带头结点的双向循环链表，链接时，结点的 Lchild 域作为前链域，指向结点的直接前驱，结点的 Rchild 域作为后链域，指向结点的直接后继。算法中，使用一个顺序栈 stack，栈顶指针为 top，P，t 为辅助指针，head 为双向循环链表的头指针。试填充算法中的空格，使算法完整。

```
void leafchain(BiTree &bt)
  {p=(BiTree)malloc(sizeof(BiTNode));
   if(!p){printf("OVERFLOW\n";exit(1); }
   head=p; top=0;
   if(bt)
     {top++; stack[top]=bt;
      while(top)
        {t=stack[top]; top--;
        if(!t->Lchild && !t->Rchild){   (1)   ;   (2)   ;   (3)   ;}
        else {if(   (4)   ){top++; stack[top]=   (5)   ; }
              if(   (6)   ){top++; stack[top]=   (7)   ; }
              }
        }
   (8)   ;   (9)   ;   }  }
```

【答案】p->Rchild=t; t->Lchild=p; p=t; t->Rchild!=null; t->Rchild; t->Lchild!=null; t->Lchild; p->Rchild=head; head->Lchild=p 查看答案

四、应用题

1．从概念上讲，树、森林和二叉树是三种不同的数据结构，将树、森林转化为二叉树的基本目的是什么?并指出树和二叉树的主要区别。

答：（1）基本目的

树的孩子兄弟链表表示法和二叉树的二叉链表表示法本质是一样的，只是解释不同，也就是说树（树是森林的特例，即森林中只有一棵树的特殊情况）可用二叉树唯一表示，并可使用二叉树的一些算法去解决树和森林中的问题。

（2）主要区别

一是二叉树的度至多为 2，树无此限制；二是二叉树有左右子树之分，即使在只有一个分支的情况下，也必须指出是左子树还是右子树，树无此限制；三是二叉树允许为空，树一般不允许为空（有些书上考虑到与二叉树的转换，允许树为空）。

2．设有一棵算术表达式树，用什么方法可以对该树所表示的表达式求值?

答：有两种方法，具体如下：

（1）对该算术表达式（二叉树）进行后序遍历，得到表达式的后序遍历序列，再按后缀表达式求值。

（2）递归求出左子树表达式的值，再递归求出右子树表达式的值，最后按根结点运算符（+、一、*、 / 等）进行求值。

3．将算术表达式（（a+b）+c*（d+e）+f）*（g+h）转化为二叉树。

答：该算术表达式转化的二叉树如图 6-1 所示。

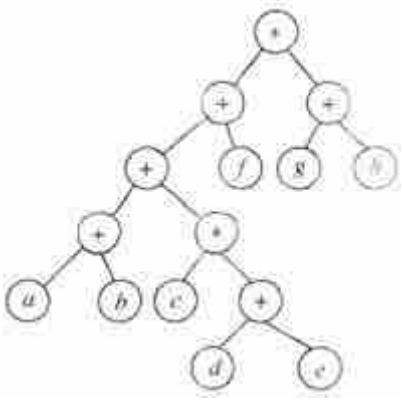


图 6-1

4．设度为 m 的树采用多重链表存储。每个结点有 m+1 个域，其中有 1 个数据域，m 个指向孩子的指针。则空指针的数目是多少?说明这种存储方式的利弊。

答：（1）空指针数目：n（n>0）个结点的 m 度树共有 nm 个链域，除根结点外，每个结点均有一个指针所指，故该树的空链域有 nm-（n-1）=n（m-1）+1 个。

（2）利弊：这种存储结构统一，便于处理但空链域造成存储效率低。

5．证明任一结点个数为 n 的二叉树的高度至少为 0（logn）。

答：证明：最低高度二叉树的特点是，除最下层结点个数不满外，其余各层的结点数都应达到各层的最大值。设 n 个结点的二叉树的最低高度是 h，则 n 应满足 $2^{h-1} \le n \le 2^h$。解此不等式，并考虑 h 是整数，则有 $h=\ddot{e}\log_2 n\rfloor+1$，即任一结点个数为 n 的二叉树的高度至少为 $0(\log_2 n)$。

6．在一棵表示有序集 S 的二叉搜索树（binary search tree）中，任意一条从根到叶结点的路径将 S 分为 3 部分：在该路径左边结点中的元素组成的集合 S1；在该路径上的结点中的元

素组成的集合 S2；在该路径右边结点中的元素组成的集合 S3；S=S1∪S2∪S3。若对于任意的 a∈S1，b∈S2，c∈S3 是否总有 a≤b≤c?为什么?

答：该结论不成立。例如，本题中对于任一 a∈S1，可在 S2 中找到 a 的最近祖先 f。a 在 f 的左子树上。对于从 a 到根结点路径上的所有 b∈S2，有可能 f 在 b 的右子树上，因而 a 也就在 b 的右子树上，这时 a>b，因此 a<b 不成立。同理可以证明 b<c 不成立。而对于任何 a∈ES1，c∈S3 均有 a<c。

7．对于具有 n 个叶结点且所有非叶结点都有左、右孩子的二叉树。

（1）试问这种二叉树的结点总数是多少?

（2）试证明 $\sum_{i=1}^{n} 2^{-(l_i-1)}=1$。其中：$l_i$表示第 i 个叶结点所在的层号（设根结点所在层号为 1）。

答：（1）根据二叉树中度为 2 的结点个数等于叶结点个数减 1 的性质，故具有 n 个叶结点且非叶子结点均有左子树的二叉树的结点数是 2n-1。

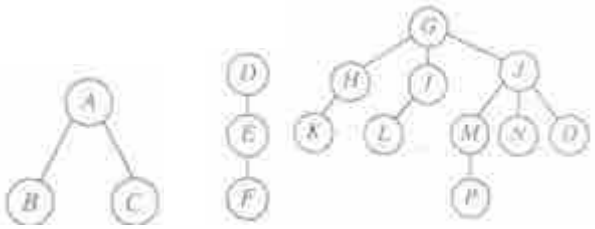
（2）证明：当 i=1 时，$2^{-(1-1)}=2^0=1$，公式成立。设当 i=n-1 时公式成立，证明当 i=n 时公式仍成立。

设某叶结点的层号为 t，当将该结点变为内部结点，从而再增加两个叶结点时，这两个叶结点的层号都是 t+1，对于公式的变化，是减少了一个原来的叶结点，增加了两个新叶结点，反映到公式中，因为 $2^{-(t-1)}=2^{-(t+1-1)}+2^{-(t+1-1)}$，所以结果不变，这就证明当 i=n 时公式仍成立。证毕。

8．试证明：同一棵二叉树的所有叶结点，在前序序列、对称序序列以及后序序列中都按相同的相对位置出现（即先后顺序相同），例如前序 abc，后序 bca，对称序 bac。

答：证明：前序遍历是“根左右”，中序遍历是“左根右”，后序遍历是“左右根”。三种遍历中只是访问“根”结点的时机不同，对左右子树均是按左右顺序来遍历的，因此所有叶子都按相同的相对位置出现。

9．将下列由三棵树组成的森林转换为二叉树（只要求给出转换结果）。



答：森林转换为二叉树分以下三步：

（1）连线（将兄弟结点相连，各树的根看作兄弟）。

（2）切线（保留最左边子女为独生子女，将其他子女分支切掉）。

（3）旋转（以最左边树的根为轴，顺时针向下旋转 45 度）。

所以由上面三棵树转换得到的二叉树如图 6-2 所示：

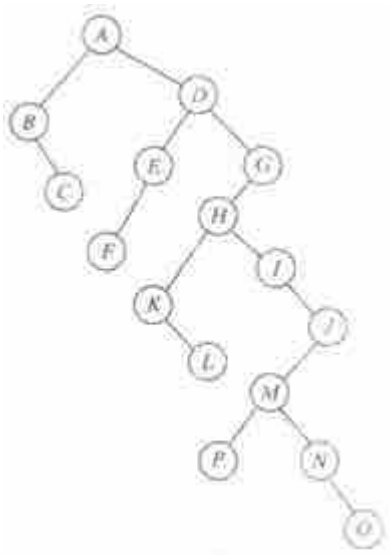


图 6-2

10．已知一棵二叉树的后序遍历序列为 EICBGAHDF，同时知道该二叉树的中序遍历序列为 CEIFGBADH，试画出该二叉树。

答：该二叉树如图 6-3 所示：

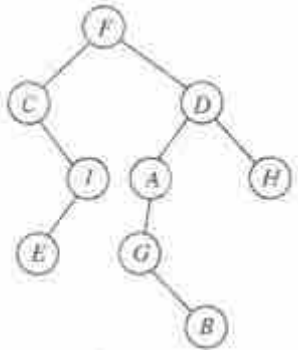


图 6-3

11．画出同时满足下列两个条件的两棵不同的二叉树。

（1）按前序遍历二叉树的顺序为 ABCDE。

（2）高度为 5 其对应的树（森林）的高度最大为 4。

答：（1）满足条件的二叉树如图 6-4 所示：

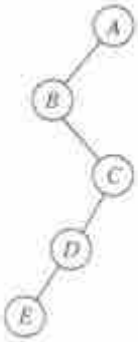


图 6-4

（2）满足条件的二叉树如图 6-5 所示：

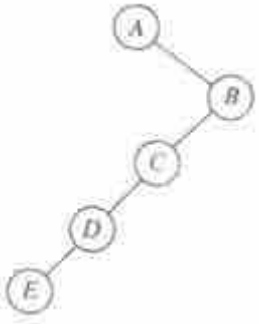


图 6-5

12．证明：在二叉树的三种遍历序列中，所有叶结点间的先后关系都是相同的。要求每步论断都指出根据。

答：证明：前序遍历是“根左右”，中序遍历是“左根右”，后序遍历是“左右根”。若将“根”去掉，三种遍历就剩“左右”。三种遍历中的差别就是访问根结点的时机不同。二叉树是递归定义的，对左右子树均是按左右顺序来遍历的，因此所有叶结点间的先后关系都是相同的。

13．在二叉树的 Llink-Rlink 存储表示中，引入“线索”的好处是什么?

答：在二叉链表表示的二叉树中，引入线索的目的主要是便于查找结点的前驱和后继。因为若知道各结点的后继，二叉树的遍历就非常简单。利用二叉链表结构查结点的左右子女非常方便，但其前驱和后继是在遍历中形成的。为了将非线性结构二叉树的结点排成线性序列，利用结点的空链域，左链为空时作为前驱指针，右链为空时作为后继指针。再引入左右标记 ltag 和 rtag，规定：当 ltag=0 时，lchild 指向左子树，当 ltag=1 时，Ichild 指向前驱；当 rtag=0 时，rchild 指向右子树，当 rtag=1 时，rchild 指向后继。这样，在线索二叉树（特别是中序线索二叉树）上遍历就消除了递归，也不使用栈（利用后序线索二叉树查后继仍需要栈）。

14．设二叉树 BT 的存储结构如表 6-1：

表 6-1 二叉树 BT 的存储结构

Lchild	0	0	2	3	7	5	8	0	10	1
Data	J	H	F	D	B	A	C	E	G	I
Rchild	0	0	0	9	4	0	0	0	0	0

其中 BT 为树根结点的指针，其值为 6，Lchild、Rchild 分别为结点的左、右孩子指针域 data 为结点的数据域。试完成下列各题：

（1）画出二叉树 BT 逻辑结构；

（2）写出按前序、中序、后序遍历该二叉树所得到的结点序列；

（3）画出二叉树的后序线索树。

答：（1）二叉树的逻辑结构如图 6-6 所示：

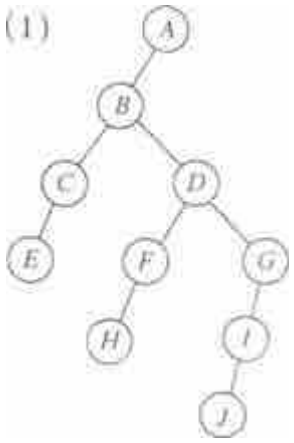


图 6-6

（2）前序序列：A B C E D F H G I J

中序序列：E C B H F D J I G A

后序序列：E C H F J I G D B A

（3）二叉树的后序线索树如图 6-7 所示：

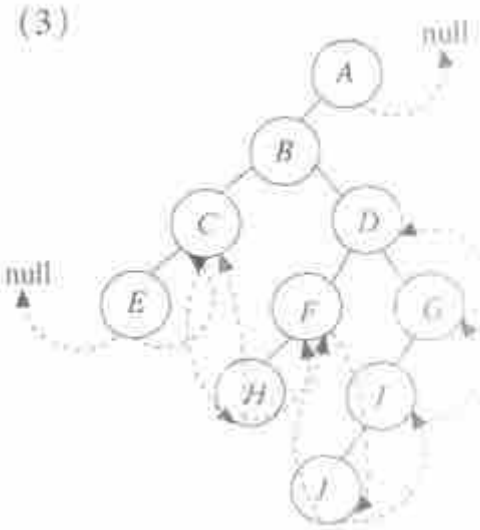


图 6-7

15．对于后序线索二叉树，怎样查找任意结点的直接后继?对于中序线索二叉树，怎样查找任意结点的直接前驱?

答：（1）后序线索树中结点的后继的方法如下：根结点无后继；当结点的 rtag=1 时，其右线索指向后继；当结点的 rtag=0 且是其双亲的右孩子，或是双亲的左孩子且双亲无右孩子时，其双亲是该结点的后继；当结点是其双亲的左孩子且双亲有右孩子时，其双亲结点右子树中最左下的叶结点是其后继。

（2）对中序线索二叉树的某结点，若其左标记等于 1，则左孩子为线索，指向直接前驱；否则，其前驱是其左子树上按中序遍历的最后一个结点。

16．设有正文 AADBAACACCDACACAAD，字符集为 A，B，C，D，设计一套二进制编码，使得上述正文的编码最短。

答：字符 A，B，C，D 出现的次数为 9，1，5，3。其哈夫曼编码如下：A:1，B：000，C：01，D：001。

17．若森林共有 n 个结点和 b 条边（b<n），则该森林中有多少棵树?

答：森林的 n 个结点开始可看作是 n 个连通分量，加入一条边将减少一个连通分量。因为树可以定义为无环的图，故加入 b 条边将减少 b 个连通分量，因而 n 个结点 b 条边的森林有 n-b 棵树。

五、算法设计题

1．假设一个仅包含二元运算符的算术表达式以链表形式存储在二叉树 BT 中，写出计算该算术表达式值的算法。

答：算法如下：

```
typedef struct node
   {ElemType  data;
    char optr;  //只取'+', '-', '*','/'
    struct node *lchild,*rchild }BiNode,*BiTree;
```

```
float PostEval(BiTree bt)        // 以后序遍历算法求以二叉树表示的算术表达式的值
  {float lv,rv;
   if(bt!=null)
     {lv=PostEval(bt->lchild); // 求左子树表示的子表达式的值
      rv=PostEval(bt->rchild); // 求右子树表示的子表达式的值
      switch(bt->optr)
       {case '+': value=lv+rv; break;
        case '-': value=lv-rv;break;
        case '*': value=lv*rv;break;
        case '/': value=lv/rv;
       } }
return(value); }
```

2．已知深度为 h 的二叉树，以一维数组 BT[0．．2^h-2]作为其存储结构，试编写一算法，求该二叉树中叶结点的个数，为简单起见，设二叉树中元素结点为非负整数，要求写出算法基本思想及相应的算法。

答：算法如下：

```
int Leaves(int BT[],int n)
  //计算深度为h，以一维数组BT作为其存储结构的二叉树的叶结点数，n为数组长度
 {int num=0;    //记叶结点数
  for(i=0;i<n;i++)
   if(i<=(n-1)/2)
     {if(BT[2*i]<0 && 2*i+1<n && BT[2*i+1]<0) num++;}//若结点无孩子，则是叶子
   else if(BT[i]>0) num++;      //存储在数组后一半的元素是叶结点
 return num;
}//结束Leaves
```

3．假定用两个一维数组 L【N】和 R【N】作为有 N 个结点 1，2，…，N 的二叉树的存储结构。L[i]和 R[i]分别指示结点 i 的左儿子和右儿子，L[i]=0（R[i]=0）表示 i 的左（右）儿子为空。试写一个算法，由 L 和 R 建立一个一维数组 T[n]，使 T[i]存放结点 i 的父亲；然后再写一个判别结点 u 是否为结点 V 的后代的算法。

答：算法如下：

```
int Generation (int U,V,N,L[],R[],T[])
 //L[]和R[]是含有N个元素且指示二叉树结点i左儿子和右儿子的一维数组
 //本算法据此建立结点i的双亲数组T，并判断结点U是否是结点V的后代
   {for(i=1;i<=N;i++)  T[i]=0;                    //T数组初始化
    for(i=1;i<=N;i++) if(L[i]!=0) T[L[i]]=i; //若结点i的左子女是L，则结点L的双亲是结点i
    for(i=1;i<=N;i++)  if(R[i]!=0) T[R[i]]=i;//若结点i的右子女是R，则R的双亲是i
    int parent=U;                                  //判断U是否是V的后代
    while(parent!=V && parent!=0) parent=T[parent];
    if(parent==V) {printf("结点U是结点V的后代"); return(1);}
    else{ printf("结点U不是结点V 的后代");return(0);}
    }结束Generation
```

4．设计算法将一棵以二叉链表存储的二叉树按顺序方式存储到一维数组中（注：按层从上到下，由左到右）。

答：算法如下：

```
typedef struct {BiTree data; int num;}tnode    // num是结点在一维数组中的编号
tnode  Q[maxsize];    //队列,容量足够大
void  BiToSeqt(BiTree t,ElemType seq[])
        //本算法将二叉树的二叉链表存储结构转换为顺序存储结构seq
       {int front=0,rear=0;
        tnode q; Bitree p;
        if(!t) exit(0);
        for(i=1;i<maxsize;i++) seq[i]='#';//初始化,#代表虚结点
        tq.data=t; tq.num=1; Q[++rear]=tq;   //根结点入队
        while(front<rear)
           {tq=Q[++front]; p=tq.data; i=tq.num; seq[i]=p->date; //存入顺序存储结构
           if(p->lchild){tq.data=p->lchild; tq.num=2*i; Q[++rear]=tq;}//左子女入队
           if(p->rchild){tq.data=p->rchild; tq.num=2*i+1; Q[++rear]=tq;}//右子女入队
       } }
```

5．假设以双亲表示法作树的存储结构，写出双亲表示的类型说明，并编写求给定的树的深度的算法（注：已知树中的结点数）。

答：算法如下：

```
int Depth(PTree t)      //求以双亲表示法作为存储结构的树的深度
{int maxdepth=0;
 for(i=1;i<=t.n;i++)
    {temp=0; f=i;
     while(f>0) {temp++; f=t.nodes[f].parent; }   // 深度加1,并取新的双亲
     if(temp>maxdepth)  maxdepth=temp;            //最大深度更新
    }
 return(maxdepth);//返回树的深度
} //结束Depth
```

6．编写算法，求二叉树的宽度。

答：算法如下：

```
int Width(BiTree bt)                    //求二叉树bt的最大宽度
  {if(bt==null) return (0);             //空二叉树宽度为0
   BiTree Q[];                          //Q是队列,元素为二叉树结点指针,容量足够大
   front=1;rear=1;                      //front为队头指针,rear为队尾指针
   last=1;                              //last为同层最右结点在队列中的位置
   temp=0; maxw=0;                      //temp记当前层宽度, maxw记最大宽度
   Q[rear]=bt;                          //根结点入队
   while(front<=last)
       {p=Q[front++]; temp++; //同层元素数加1
        if(p->lchild!=null)  Q[++rear]=p->lchild;    //左子女入队
        if(p->rchild!=null)  Q[++rear]=p->rchild;    //右子女入队
        if(front>last)                //一层结束
           {last=rear;                //last指向下层最右元素
            if(temp>maxw) maxw=temp;//更新当前最大宽度
             temp=0;
            }//if }//while
        return (maxw);
}//结束Width
```

7．当一棵有 n（n≤100）个结点的二叉树按顺序存储方式存储在 bt[1．．n]中时，试写一个算法，求出二叉树中结点值分别为 X 和 Y 的两个结点的最近公共祖先结点的值。

答：算法如下：

```
void Ancestor(ElemType bt[],x,y,int n)
//二叉树顺序存储在数组bt[1..n]中,本算法求结点i和j的最近公共祖先结点的值
  {int i,j;
   while(i!=j)
     if(i>j) i=i/2; //下标为i的结点的双亲结点的下标
     else j=j/2;    //下标为j的结点的双亲结点的下标
  printf("所查结点的最近公共祖先的下标是%d,值是%d",i,A[i]);//设元素类型为整型
}// Ancestor
```

8．已知一棵高度为 K 具有 n 个结点的二叉树，按顺序方式存储。

（1）编写用前序遍历树中每个结点的非递归算法；

（2）编写将树中最大序号叶结点的祖先结点全部打印输出的算法。

答：（1）算法如下：

```
int m=2^r-1;  //全局变量
void PreOrder(ElemType bt[],i )
//递归遍历以顺序方式存储的二叉树bt, i是根结点下标(初始调用时为1)
 {int top=0,s[];  //top是栈s的栈顶指针,栈容量足够大
  while(i<=m||top>0)
   {while(i<=m)
     {if(bt[i]!=0) printf(bt[i]);                    //访问根结点,设虚结点以0表示
      if(bt[i]!=0 && 2*i+1<=m) s[++top]=2*i+1;     //右子女的下标位置入栈
      i=2*i;                                        //沿左子女向下
      }
     if(top>0) i=s[top--];                          //取出栈顶元素
  }//while }//结束PreOrder
```

(2)算法如下：

```
void Ancesstor(ElemType bt[])      //打印最大序号叶结点的全部祖先
  {c=m;
   while(bt[c]==0) c--;//找最大序号叶结点,该结点存储时在最后
   f=c/2;          //c的双亲结点f
   while(f!=0)   //从结点c的双亲结点直到根结点,路径上所有结点均为祖先结点
     {printf(bt[f]); f=f/2; }//逆序输出,最老的祖先最后输出
   }//结束
```

9．在一棵以二叉链表表示的二叉树上，试写出按层次顺序遍历二叉树的方法，统计树中具有度为 1 的结点数目的算法。

答：算法如下：

```
int Level(BiTree bt) //层次遍历二叉树,并统计度为1的结点的个数
{int num=0; //num统计度为1的结点的个数
 if(bt){QueueInit(Q); QueueIn(Q,bt);//Q是以二叉树结点指针为元素的队列
        while(!QueueEmpty(Q))
         {p=QueueOut(Q); printf(p->data);      //出队,访问结点
          if(p->lchild && !p->rchild ||!p->lchild && p->rchild)num++;//度为1的结点
          if(p->lchild) QueueIn(Q,p->lchild); //非空左子女入队

             if(p->rchild) QueueIn(Q,p->rchild); //非空右子女入队
           }  }//if(bt)
    return(num); }//返回度为1的结点的个数
```

10．设 T 是一棵满二叉树，写一个把 T 的后序遍历序列转换为前序遍历序列的递归算法。

答：算法如下：

```
void PostToPre(ElemType post[],pre[],int l1,h1,l2,h2)
  //将满二叉树的后序序列转为前序序列,l1、h1、l2、h2是序列初始和最后结点的下标。
    {if(h1>=l1)
     {pre[l2]=post[h1];      //根结点
     half=(h1-l1)/2;         //左子树或右子树的结点数
     PostToPre(post,pre,l1,l1+half-1,l2+1,l2+half)   //将左子树后序序列转为后序序列
     PostToPre(post,pre,l1+half,h1-1,l2+half+1,h2)   //将右子树后序序列转为后序序列
     }  }// PostToPre t
```

11．已知一棵二叉树的前序遍历序列和中序遍历序列分别存于两个一维数组中，试编写算法建立该二叉树的二叉链表。

答：算法如下：

```
void PreInCreat(BiTree root,ElemType pre[],in[],int l1,h1,l2,h2)
//根据二叉树前序序列pre和中序序列in建立二叉树。l1、h1和l2、h2是两个序列首、尾元素下标
{root=(BiTree)malloc(sizeof(BiNode)); //申请结点
 root->data=pre[l1];  //pre[l1]是根
 for(i=l2;i<=h2;i++) if(in[i]==pre[l1]) break;//在中序序列中,根结点将树分成左右子树
 if(i==l2) root->lchild=null; //无左子树
 else PreInCreat(root->lchild,pre,in,l1+1,l1+(i-l2),l2,i-1); //递归建立左子树
 if(i==h2) root->rchild=null;  //无右子树
 else PreInCreat(root->rchild,pre,in,l1+(i-l2)+1,h1,i+1,h2)    //递归建立右子树
}//结束PreInCreat
```

12．已知一具有 n 个结点的二叉树的中序遍历序列与后序遍历序列分别存放于数组 IN[1：n]和 POST[1：n]中（设该二叉树各结点的数据值均不相同）。请写一建立该二叉树的二叉链表结构的非递归算法。该二叉链表的链结点结构为（lchild，data，rchild），其中 data 为数据域，lchild 与 rhild 分别为指向该结点左、右孩子的指针域（当孩子结点不存在时，相应指针域为空，用 nil 表示）。

答：算法如下：

```
void InPostCreat(ElemType IN[],POST[],int l1,h1,l2,h2)
 //由二叉树的中序序列IN[]和后序序列POST[]建立二叉树
 //l1、h1和l2、h2分别是中序序列和后序序列第一和最后元素的下标,初始调用时,l1=l2=1,h1=h2=n
{typedef struct {int l1,h1,l2,h2; BiTree t; }node;
 node s[],p;//s为栈,容量足够大
 BiTree bt=(BiTree)malloc(sizeof(BiNode)); int top=0,i;
 p.l1=l1; p.h1=h1; p.l2=l2; p.h2=h2; p.t=bt; s[++top]=p;//初始化
 while(top>0)
  {p=s[top--]; bt=p.t; l1=p.l1; h1=p.h1; l2=p.l2; h2=p.h2;//取出栈顶数据
   for(i=l1;i<=h1;i++) if(IN[i]==POST[h2]) break;//在中序序列中查等于POST[h2]的结点
   bt->data=POST[h2];              //根结点的值
   if(i==l1) bt->lchild=null;     //bt无左子树
   else    //将建立左子树的数据入栈
      {bt->lchild=(BiTree)malloc(sizeof(BiNode)); p.t=bt->lchild;
       p.l1=l1; p.h1=i-1; p.l2=l2; p.h2=l2+i-l1-1; s[++top]=p;
      }
   if(i==h1) bt->rchild=null;     //bt无右子树
     else {bt->rchild=(BiTree)malloc(sizeof(BiNode)); p.t=bt->rchild;
           p.l1=i+1; p.h1=h1; p.l2=l2+i-l1; p.h2=h2-1; s[++top]=p; //右子树数据入栈
          }
   }// while(top>0)
}结束InPostCreat
```

13．已知二叉树 T，试写出复制该二叉树的算法（t→T）。

答：算法如下：

```
BiTree Copy(BiTree t)    //复制二叉树t的非递归算法
```

```
{typedef struct{BiTree t,bt} node;
 node Q[maxsize];      //Q是二叉树的结点指针的队列,容量足够大
  if(!t) {bt=null; return bt;}
  else
    {QueueIn(Q,(t,bt));
     while(!QueueEmpty(Q))
       {(t,bt)=QueueOut(Q);
        bt=(BiNode*)malloc(sizeof(BiNode)); bt->data=t->data;
        if(t->lchild)QueueIn(Q,(t->lchild,bt->lchild));else bt->lchild=null;
        if(t->rchild)QueueIn(Q,(t->rchild,bt->rchild));else bt->rchild=null;
        }//while }//else
      return bt
}//结束本题
```

14．（1）试分别找出满足下列条件的所有二叉树：（a）前序序列和中序序列相同：（b）前序序列和后序序列相同；（c）中序序列和后序序列相同。

（2）已知非空二叉树的结点结构为（lchild，data，rchild），设计算法：从右向左依次将所有叶子的数据值放到向量（假定向量的空间大于叶子的总个数）中。

答：（1）满足条件的二叉树如下：

（a）若前序序列与中序序列相同，则或为空树，或为任一结点至多只有右子树的二叉树。

（b）若前序序列与后序序列相同，则或为空树，或为只有根结点的二叉树。

（c）若中序序列与后序序列相同，则或为空树，或为任一结点至多只有左子树的二叉树。

（2）算法如下：

```
ElemType a[maxsize]; int i=0;  //全局变量
void InOrder(BiTree bt)
  //从右向左依次将二叉树bt的所有叶子的数据值放到a向量中
  {if(bt){InOrder(bt->rlchild);            //中序遍历右子树
          if(!bt->lchild && !bt->rchild) a[++i]=bt->data; //叶结点
          InOrder(bt->rchild);             //中序遍历左子树
  }    } //InOrder
```

15．编写一算法，利用叶结点中的空指针域将所有叶结点链接为一个带有头结点的双链表，算法返回头结点的地址。

答：算法如下：

```
BiTree head=null,pre; //全局变量链表头指针head,pre
void CreatLeafList(BiTree bt) //将BiTree树中的所有叶结点链成带头结点的双链表
   {if(bt)          //若bt不空
    {CreatLeafList(bt->lchild);                //中序遍历左子树
     if(bt->lchild==null && bt->rchild==null) //叶结点
       if(head==null)//第一个叶结点
         {head=(BiNode*)malloc(sizeof(BiNode));//生成头结点
          head->lchild=null; head->rchild=bt;  //头结点的左链空,右链指向第一个结点
          bt->lchild=head; pre=bt;//第一个叶结点左链指向头结点,pre指向当前叶结点
         }
       else{pre->rchild=bt; bt->lchild=pre; pre=bt;} //当前叶结点链入双链表
     CreatLeafList(bt->rchild);  //中序遍历右子树
     pre->rchild=null;     //最后一个叶结点的右链置空(链表结束标记)
   }//if(bt)  }//结束CreatLeafList;
```

16．试为二叉树写出一个建立三叉链表的算法，并在此三叉链表中删去每一个元素值为 x 的结点，以及以它为根的子树，且释放相应存储空间。二叉树的三叉链表的描述为：

```
TYPE bitreptr=^nodetp;
     nodetp=RECORD data:char; lchild,rchild,parent:bitreptr END;
```

VAR bt：bitreptr；{二叉树根结点的指针}

答：算法如下：

```
BiTree creat()// //生成三叉链表的二叉树(题目给出PASCAL定义,下面用类C语言书写)
 {BiTree p,Q[],root;  //Q是二叉树结点指针的一维数组,容量足够大
  char ch; int rear=0; //一维数组最后元素的下标
  scanf(&ch);
  while(ch!='#')
    {p=null;
     if(ch!='$'){p=(BiTree)malloc(sizeof(nodetp));
                 p->data=ch; p->lchild=p->rchild=null;
                }
     Q[++rear]=p;  //元素或虚结点
     if(p){if(rear==1) {root=p;root->parent=null; } //根结点
        else{Q[rear]->parent=Q[rear/2]; //双亲结点和子女结点用指针链上
             if(rear%2==0) Q[rear/2]->lchild=Q[rear]; else Q[rear/2]->rchild=Q[rear];
        }
     scanf("%c",&ch);
    }//while
  return(root);
}//结束creat
```

17．设二叉树用二指针结构存储（可以是动态存储结构），元素值为整数，且元素值无重复，请编写子程序，求出以元素值等于某个给定的整数的结点为根的子树中的各个叶结点。

答：算法如下：

```
BiTree Search(BiTree bt,ElemType x) //在二叉树t中查找结点值等于x的结点
 {if(bt){Search(bt->lchild,x);
         Search(bt->rchild,x);
         if(bt->data==x) return bt;
  }//if  }//结束
void Count(BiTree t,int *n0,) //统计以t为根结点的子树的叶结点数n0
 {if(t)
     {Count(bt->lchild, &n);
     Count(bt->rchild, &n);
     if(!t->lchild && !t->rchild)    //叶结点
       {printf(t->data); *n0++; }  //输出并计数
     }
}//结束 Count
```

18．已知二叉树采用二叉链表存储，设计算法以输出二叉树 T 中根结点到每个叶结点的路径。

答：算法如下：：

```
void PrintPath(BiTree bt,p) //打印从根结点bt到结点p之间路径上的所有结点
 {BiTree q=bt,s[];  //s是元素为二叉树结点指针的栈,容量足够大
  int top=0; tag[];//tag是数组,元素值为0或1,访问左、右子树的标志,tag和s同步
  if(q==p) {printf(q->data); return;} //根结点就是所找结点
  while(q!=null || top>0)
    {while(q!=null)   //左子女入栈,并置标记
       if(q==p)       //找到结点p,栈中元素均为结点p的祖先
         {printf("从根结点到p结点的路径为\n");
          for(i=1;i<=top;i++) printf(s[i]->data);printf(q->data); return;

          }
        else {s[++top]=q; tag[top]=0; q=q->lchild;} //沿左分支向下
   while(top>0 && tag[top]==1)) top--;              //本题不要求输出遍历序列,这里只出栈
   if(top>0) {q=s[top]; q=q->rchild; tag[top]=1; }//沿右分支向下
 }//while(q!=null || top>0)
}//结束算法PrintPath

if(q->lchild==null&&q->rchild==null) //q为叶结点
  {printf("从根结点到p结点的路径为\n"); //输出从根到叶子q的路径上的所有祖先
   for(i=1;i<=top;i++) printf(s[i]->data); printf(q->data);printf("\n");
}
```

19．已知指针 P 指向带表头的中根次序线索二叉树中的某结点，试写一算法 FFA（P，q），该算法寻找结点 P 的父亲结点 q。设线索二叉树的结点结构、表头结点结构和空树结构分别为（LTAG，LLINK，INFO，RLINK，RTAG），且规定线索树的最左下结点的 LLINK 域和最右下结点的 RLINKt 域指向表头。

答：算法如下：

```
     void FFA(BiThrTree t,p,q)//在中序线索树t上,求结点p的双亲结点q
      {q=p;  //暂存

while(q->RTAG==0) q=q->RLINK; //找p的中序最右下的结点
q=q->RLINK;                   //顺右线索找到q的后继(p的祖先结点)
if(q==t)
  {q=t->LLINK; //若后继是头结点,则转到根结点
   if(q==p) {printf("根结点无双亲\n");return; }
   {if(q->LLINK==p) return(q); else q=q->LLINK; //准备到左子树中找p
   while(q->RLINK!=p) q=q->RLINK;return(q);
  } //找最右结点的过程中回找到p
}//else  }//结束FFA
```

20．试编写一算法对二叉树按前序线索化。

答：算法如下：

```
BiThrTree pre=null;//设置前驱
void PreOrderThreat(BiThrTree BT)
  //对以线索链表为存储结构的二叉树BT进行前序线索化
{if(BT!=null)
  {if(BT->lchild==null){BT->ltag=1; BT->lchild=pre;}//设置左线索
   if(pre!=null && pre->rtag==1) pre->rchild=BT;      //设置前驱的右线索
   if(BT->rchild==null) BT->rtag=1;                   //为建立右链做准备
   pre=BT;//前驱后移
   if(BT->ltag==0) PreOrderThreat(BT->lchild);        //左子树前序线索化
   PreOrderThreat(BT->rchild);                        //右子树前序线索化
  }//if(BT!=null) }//结束PreOrderThreat
```

21．写出算法，求出中序线索二叉树中给定值为 X 的结点之后继结点，返回该后继结点的指针。线索树中结点结构为：（ltag，lc，data，rC，rtag）。其中，data 存放结点的值；lc、rc 为指向左、右孩子或该结点前驱、后继的指针，ltag、rtag 为标志域，若值为 0，则 lc、rc 为指向左、右孩子的指针；若值为 1，则 lc、rc 为指向其前驱、后继结点的指针。

答：算法如下：

```
BiThrTree InOrder(BiThrTree T,ElemType x)
  //先在带头结点的中序线索二叉树T中查找给定值为x的结点,假定值为x的结点存在
{p=T->lchild;//设p指向二叉树的根结点
 while(p!=T)
  {while(p->ltag==0 && p->data!=x) p=p->lc;
   if(p->data==x) return(p);
   while(p->rtag==1 && p->rc!=T) {p=p->rc; if(p->data== x) return(p);}
   p=p->rc;
  }//while  }//结束InOrder
BiThrTree AfterXNode(BiThrTree T) //在中序线索二叉树T中,求给定值为x的结点的后继结点
 {BiThrTree p=InOrder(T,x);       //首先在T树上查找给定值为x的结点,由p指向
  if(p->rtag==1) return(p->rc);   //若p的右标志为1,则p的rc指针指向其后继
  else {q=p->rc; while(q->ltag==0)q=q->lc; return(q); }
  //结点p的右子树中最左边的结点是结点p的中序后继
}//结束AfterXnode
```

22．写出按后序序列遍历中序线索树的算法。

答：算法如下:

```
BiThrTree LeftMost(BiThrTree t) //求结点t最左子孙的左线索
     {BiThrTree p=t;
      while(p->ltag==0) p=p->lchild; //沿左分支向下
      if(p->lchild!=null) return(p->lchild); else return(null);
     }//LeftMost
BiThrTree RightMost(BiThrTree t)//求结点t最右子孙的右线索
     {BiThrTree p=t;
      while(p->rtag==0) p=p->rchild;    //沿右分支向下
      if(p->rchild!=null) return (p->rchild); else return(null);
     }//RightMost
int IsRightChild(BiThrTree t,father)    //若t是father 的右孩子,返回1,否则返回0
     {father=LeftMost(t);
      if(father &&f ather->rchild==t) return(1); else return(0);
     }//Is RightChild;
void PostOrderInThr(BiThrTree bt) //后序遍历中序线索二叉树bt
     {BiThrTree father, p=bt;
      int flag;
      while(p!=null)
       {while(p->ltag==0 ) p=p->lchild; // 沿左分支向下
        if(p->rtag==0) flag=0;//左孩子为线索,右孩子为链,相当从左返回
        else flag=1;           //p为叶子,相当从右返回
        while(flag==1)
           (visit(*p);//访问结点
            if(IsRightChild(p,father)) {p=father; flag=1;} //修改p指向双亲
            else    //p是左子女,用最右子孙的右线索找双亲
                {p=RightMost(p);
                 if(p&&p->rtag==1) flag=1; else flag=0;
                }
             }// while(flag==1)
         if(flag==0 && p!=null) p=p->rchild; //转向当前结点右分支
} }//结束PostOrderInThr
```

23. 假设在二叉链表的结点中增设两个域：parent 域指示其双亲结点：flag 域（取值为 0．2）区分在遍历过程中到达该结点时应继续向左、向右或访问该结点。试以此存储结构编写不用栈进行后序遍历的递推形式的算法。

答：算法如下：

```
typedef struct node {struct node *lchild,*rchild,*parent;
                     ElemType data;
                     int flag;}BPTree;
void PostOrder(BPTree t)
// 在增加双亲指针parent和标志域flag的二叉树中,不用栈后序遍历二叉树
{BPTree p=t;
 if(p)
```

```
while(p)
      switch (p->flag)
        {case 0: p->flag=1; if(p->lchild) p=p->lchild; //向左
                            break;
         case 1: p->flag=2; if(p->rchild) p=p->rchild; //向右
                            break;
         case 2: p->flag=0; printf(p->data); //访问根结点
                            p=p->parent;      //找被访问结点的双亲
         }//switch
}//结束
```

第 7 章　图

一、选择题

1．图中有关路径的定义正确的是(　　)。

A．由顶点和相邻顶点构成的边所形成的序列

B．由不同顶点所形成的序列

C．由不同边所形成的序列

D．上述定义都不是

【答案】A 查看答案

【解析】顶点 V_p到顶点 V_q之间的一条路径是指顶点序列 V_p，V_{i1}，，…，V_{im}，V_q。路径上边的数目称为路径的长度。

2．设无向图的顶点个数为 n，则该图最多有(　　)条边。

A．n-1　　B．n(n-1) / 2　　C．n(n+1) / 2　　D．0　　E．n^2

【答案】B 查看答案

【解析】在数据结构中仅讨论简单图，在计算无向图的最多边时，不考虑顶点与顶点的边。因此边数最多时，构成的是无向完全图。此时的边数为 n(n-1)/2。

3．要连通具有 n 个顶点的有向图，至少需要(　　)条边。

A．n-1　　B．n　　C．n+l　　D．2n

【答案】B 查看答案

【解析】对于有向图来说，两个顶点之间的边是具有方向的。如果是构成连通的无向图，需要 n-1 条边，而对于有向图来说，只需要再加上第一个顶点和最后一个顶点加上一条边，让其构成环状的图即可，因此最少需要 n 条边。

4.n 个结点的完全有向图含有边的数目(　　)。

A．n*n　　B．n(n+1)　　C．n / 2　　D．n*(n-1)

【答案】D 查看答案

【解析】在有向图中，如果任意两个顶点之间都存在边，则称为有向完全图。顶点个数为 n 的无向图，最多有 C_n^2=n(n-1) / 2 条边。如是有向图，需要在无向图的最多边的基础上乘以 2，则为 n(n-1)。

5．用有向无环图描述表达式(A+B)*((A+B) / A)，至少需要顶点的数目为(　　)。

A．5　　B．6　　C．8　　D．9

【答案】A 查看答案

【解析】一共 5 个结点{*,/,+,A,B}，6 条边{<*,/>,<*,+>,</,+>,<＋，A>,<＋，B>,</,A>}。

6．用邻接表存储图所用的空间大小____。

A．与图的顶点数和边数都有关　　B．只与图的边数有关

C．只与图的顶点数有关　　D．与边数的平方有关

【答案】A 查看答案

【解析】邻接表就是对图 G 中的每个顶点 v_i建立一个单链表，第 i 个单链表中的结点表示依附于顶点 v_i的边，这个单链表就称为顶点 v_i的边表。因此邻接表既存储图的所有顶点，也存储顶点之间的边的信息。

7．图 G 是 n 个顶点的无向完全图，则下列说法不正确的是（）

A．G 的邻接多重表需要 n(n-1)个边结点和 n 个顶点结点

B．G 的连通分量个数最少

C．G 为连通图

D．G 所有顶点的度的总和为 n(n—1)

【答案】A 查看答案

【解析】A 项中 G 的邻接多重表中需要 n(n-1)/2 个边结点和 n 个顶点结点。此时连通分量最少为 1。无向完全图中任意两个顶点之间都存在路径，则 G 必为连通图。每个顶点的度为 n-1，则 n 个结点的度的总和为 n(n-1)。

8．下列哪一种图的邻接矩阵是对称矩阵? (　　)

A．有向图　　B．无向图　　C．AOV 网　　D．AOE 网

【答案】B 查看答案

【解析】邻接矩阵存储，就是用一个一维数组存储图中顶点的信息，用一个二维数组存储图中边的信息，存储顶点之间关系的二维数组称为邻接矩阵。因为无向图中边是没有方向的，所以 A[i][j]=A[j][i],所以无向图的邻接矩阵是对称矩阵。

9．图的 BFS 生成树的树高比 DFS 生成树的树高(　　)。

A．小或相等　　B．小　　C．大或相等　　D．大

【答案】A 查看答案

【解析】BFS 称作广度优先搜索，DFS 称作深度优先搜索。广度优先搜索类似与二叉树的层序遍历算法，深度优先搜索类似于树的先序遍历。因为深度优先搜索算法遵循的策略是尽可能的“深”地搜索一个图。所以图的 BFS 生成树的树高比 DFS 生成树的树高小或者相等。

10．若一个有向图具有拓扑排序序列，那么它的邻接矩阵必定为(　　)。

A．对称矩阵　　B．稀疏矩阵　　C．三角矩阵　　D．一般矩阵

【答案】C 查看答案

【解析】在图论中，由一个有向无环图的顶点组成的序列，当且仅当满足下列条件时，称为改图的一个拓扑排序：①每个顶点出现且出现一次；②若顶点在序列中排在顶点 B 的前面，则在图中不存在从顶点 B 到顶点 A 的路径。由拓扑排序的性质知，有向图的邻接矩阵必定为三角矩阵。

11．下列说法不正确的是(　　)。

A．图的遍历是从给定的源点出发每个顶点仅被访问一次

B．遍历的基本方法有两种：深度遍历和广度遍历

C．图的深度遍历不适用于有向图

D．图的深度遍历是一个递归过程

【答案】C 查看答案

【解析】图的遍历是指从图中的某一个顶点出发，按照某种搜索算法沿着图中的边对图中的所有顶点访问一次且仅访问一次。图的深度遍历类似于树的先序遍历，不仅适合无向图，也适合于有向图。

12．无向图 G=(V，E)，其中：V={a，b，c，d，e，f)}，E={(a，b)，(a，e)，(a，c)，(b，e)，(c，f)，(f,d),(e，d)，对该图进行深度优先遍历，得到的顶点序列正确的是(　　)。

A．a，b，e，c，d，f　B．a，c，f,e，b，d
C．a，e，b，c，f, d D．a，e，d，f，c，b
【答案】D 查看答案
【解析】图的深度优先遍历过程是：从图中某个初始顶点 v 出发，首先访问初始顶点 v，然后选择一个与顶点 v 相邻且没被访问过的顶点 u 为初始顶点。再从 u 出发进行深度优先搜索，直到图中与当前顶点 v 邻接的所有顶点都被访问过为止。
根据 E={（a，b），（a，e），（a，c），（b，e），（c，f），（f，d），（e，d）}可知各顶点之间的邻接关系。依据上面的原则遍历，得出遍历顺序 a，e，d，f，c，b。
13．在有向图的邻接表存储结构中，顶点 v 在链表中出现的次数是(　)。
A．顶点 v 的度　B．顶点 v 的出度　C．顶点 v 的入度　D．依附于顶点 v 的边数
【答案】B 查看答案
【解析】在有向图中，第 j 个链表中的结点个数只是顶点 v_i的出度，为求入度，必须遍历整个邻接表。因此顶点 v 在链表中出现的次数是顶点 v 的出度。
14．已知有向图 G=(V，E)，其中 V={V_1，V_2，V_3，V_4，V_5，V_6，V_7}，E={<V_1，V_2>，<V_1，V_3>，<V_1，V_4>，<V_2，V_5>，<V_3，V_5>，<V_3，V_6>，<V_4，V_6>，<V_5，V_7>，<V_6，V_7>}，G 的拓扑序列是(　)。
A．V_1，V_3，V_4，V_6，V_2，V_5，V_7　B．V_1，V_3，V_2，V_6，V_4，V_5，V_7
C．V_1，V_3，V_5，V_2，V_6，V_7　D．V_1、V_2，V_5，V_3，V_4，V_6，V_7
【答案】A 查看答案
【解析】设 G=（V，E）是一个具有 n 个顶点的有向图，V 中顶点序列 v_1，v_2，…，v_n，能被称为拓扑序列的条件：若<v_i，v_j>是图中的边（即从顶点 v_i到 v_j有一条路径），则在序列中顶点 v_i必须排在顶点 v_j之前。根据上面拓扑序列的定义，就可以得出 G 的拓扑序列是 V_1，V_3，V_4，V_5，V_2，V_5．V_7。
15. 在有向图 G 的拓扑序列中，若顶点 V_i在顶点 V_j之前，则下列情形不可能出现的是(　)。
A．G 中有弧<V_i，V_j>　B．G 中有一条从 V_i到 V_j的路径
C．G 中没有弧<V_i，V_j)　D．G 中有一条从 V_j到 V_i的路径
【答案】D 查看答案
【解析】若想实现图的一个拓扑排序，需要满足的一个条件为：若顶点 A 在序列中排在顶点 B 的前面，则在图中不存在从顶点 B 到顶点 A 的路径。又因为若 G 中有一条从 V_j到 V_i的路径，则在拓扑序列中 V_i不可能在 V_j前。
16．在用邻接表表示图时，拓扑排序算法时间复杂度为(　)。
A．0(n)　B．0(n+e)　C．0(n*n)　D．O(n*n*n)
【答案】B 查看答案
【解析】由于输出每个顶点的同时还要删除以它为起点的边，故拓扑排序的时间复杂度为 0(n+e)
17．关键路径是 AOE 网中(　)。
A．从始点到终点的最短路径
B．从始点到终点的最长路径
C．从始点到终点的边数最多的路径
D．从始点到终点的边数最少的路径
【答案】B 查看答案
【解析】在 AOE-网中有些活动可以并行地进行，所以完成工程的最短时间是从开始点到完成点的最长路径的长度(这里所说的路径长度是指路径上各活动持续时间之和，不是路径上弧的数目)。路径长度最长的路径称作关键路径(critical path)。

18．下面关于求关键路径的说法不正确的是(　　)。
A．求关键路径是以拓扑排序为基础的
B．一个事件的最早开始时间同以该事件为尾的弧的活动最早开始时间相同
C．一个事件的最迟开始时间为以该事件为尾的弧的活动最迟开始时间与该活动的持续时间的差
D．关键活动一定位于关键路径上
【答案】C 查看答案
【解析】一个事件的最迟开始事件是这个事件能够拖到的最晚时间，从这个时刻开始做完这个事件不影响其后续事件的开始时间。
19．下列关于 AOE 网的叙述中，不正确的是(　　)。
A．关键活动不按期完成就会影响整个工程的完成时间
B．任何一个关键活动提前完成，那么整个工程将会提前完成
C．所有的关键活动提前完成，那么整个工程将会提前完成
D．某些关键活动若提前完成，那么整个工程将会提前完成
【答案】B 查看答案
【解析】关键路径是指从有向图的源点到汇点的最长路径。某些关键活动提前完成，那么整个工程将会提前完成，但不是任何一个关键活动提前完成，就能保证整个工程将会提前完成。
20.n 个顶点的无向图的邻接表最多有(　　)个表结点。
A．n^2　　B．n(n-1)　　C．n(n+1)　　D．n(n-1)／2
【答案】B 查看答案
【解析】当 n 个顶点构成的无向图是无向完全图时，则每一个结点都会和其余的 n-1 个结点连接，从而会产生 n(n-1)个表结点。
二、判断题
1．树中的结点和图中的顶点就是指数据结构中的数据元素。(　　)
【答案】√查看答案
【解析】树中的结点和图中的顶点就是指数据结构中的数据元素,而它们的边指的是元素之间的关系。
2．有向图中顶点 V 度等于其邻接矩阵中第 V 行中的 1 的个数。(　　)
【答案】×查看答案
【解析】有向图顶点 V 的出度等于其邻接矩阵第 V 行中的 1 的个数，而有向图中 V 的度等于其邻接矩阵中边表出现的 V 的个数。
3．强连通分量是无向图的极大强连通子图。(　　)
【答案】×查看答案
【解析】强连通分量是对有向图而言的，一般在无向图中讨论连通性。
4．十字链表是无向图的一种存储结构。(　　)
【答案】×查看答案
【解析】十字链表是有向图的另一种链式存储结构。在十字链表中，对应于有向图的每条弧有一个结点，对应于每个顶点也有一个结点。类似于邻接表是无向图的一种链式存储结构。
5．一个有向图的邻接表和逆邻接表中的结点个数一定相等。（　　）
【答案】√查看答案
【解析】图的邻接表表示法类似于树的孩子链表示法。对于图 G 中的每个顶点 v_i，该方法把所以邻接于顶点 v_i的 v_j链接成一个带头。在有向图中，为图中每个顶点 v_i建立一个入边表的方法称为逆邻接表示法。
6．若一个有向图无环，则它一定有唯一的拓扑序列。(　　)

【答案】×查看答案
【解析】有向图无环说明它一定有拓扑序列，但这个拓扑序列不唯一。如果在一个线性有序的序列中，每个顶点有唯一的前驱后继关系，在做拓扑排序时，则排序的结果是唯一的，即它有唯一的拓扑序列。
7．不同的求最小生成树的方法最后得到的生成树是相同的。(　　)
【答案】×查看答案
【解析】对于一个带权连通无向图 G=(V,E)，生成树不同，每棵树的权也可能不同。若生成树 T 上的所有边的权值之和最小，则 T 称为 G 的最小生成树。因此可以看出，最小生成树不是唯一的，最小生成树的权值之和总是唯一的。
8．连通图上各边权值均不相同，则该图的最小生成树是唯一的。(　　)
【答案】√查看答案
【解析】由最小生成树的构建过程知，在确定根结点之后，因为连通图上各边权值均不相同，下一个结点的选择是唯一的，所以该图的最小生成树是唯一的。
9．最小生成树的 Krusakl 算法是一种贪心法。(　　)
【答案】√查看答案
【解析】在构建最小生成树常见的有三种贪心算法：kruskal，prim，soilion。
10．采用深度优先搜索或拓扑排序算法可以判断出一个有向图中是否有环(回路)。
【答案】√查看答案
【解析】采用深度优先搜索算法主要是通过设置标志位可以判断出一个有向图中是否有环。采用拓扑排序算法，如果能够构成一个拓扑排序，则有向图中没有环，否则，有向图中有环。
11．无环有向图才能进行拓扑排序。(　　)
【答案】√查看答案
【解析】在图论中，由一个有向无环图的顶点组成的序列，才能进行拓扑排序。
12．拓扑排序的有向图中，最多存在一条环路。(　　)
【答案】×查看答案
【解析】要进行拓扑排序，需要满足一个条件为：若顶点 A 在序列中排在顶点 B 的前面，则在图中不存在顶点 B 到顶点 A 的路径。如果是一个有环图，则不能满足这个条件,所以拓扑排序的有向图中不能存在环路。
13．即使有向无环图的拓扑序列唯一，也不能唯一确定该图。(　　)
【答案】×查看答案
【解析】如果有向无环图的拓扑序列唯一，则能够确定每个结点的唯一前驱和后继，因此能够确定该图。
14．若一个有向图的邻接矩阵对角线以下元素均为零，则该图的拓扑有序序列必定存在。(　　)
【答案】√查看答案
【解析】因为一个有向图的邻接矩阵对角线以下元素均为零，则该图是一个有向无环图，所以该图的拓扑有序序列必定存在。
15.AOE 网一定是有向无环图。(　　)
【答案】×查看答案
【解析】在带权的有向图中，以顶点表示事件，有向边表示活动，边上的权值表示完成该活动的开销，则称这种有向图表示活动的网络，简称为 AOE 网。因此对 AOE 网是否是有向无环图没有要求。
16．当改变网上某一关键路径上任一关键活动后，必将产生不同的关键路径。(　　)
【答案】×查看答案

【解析】当改变任意关键路径上的关键活动之后，这个活动可能还是关键活动，因此不会产生不同的关键路径。

三、填空题

1．在一个无向图的的邻接表中，若表结点的个数是 m，则图中边的条数是____条。

【答案】m / 2 查看答案

【解析】对于无向图，在邻接表中，如果存在 n 条边，则会有 2n 个表结点。

2．若用 n 表示图中顶点数目，则有____条边的无向图成为完全图。

【答案】n(n-1) / 2 查看答案

【解析】无向完全图中任意一个顶点都和其他 n-1 个顶点都有一条边，即为 n(n-1)。又因为每条边重复出现两次，所有无向完全图的边数为 n(n-1)/2。

3.G 是一个非连通无向图，共有 28 条边，则该图至少有____个顶点。

【答案】9 查看答案

【解析】求该非连通无向图的最少顶点数，则该图为一个孤立的顶点和一个完全连通图。

4．在有 n 个顶点的有向图中，每个顶点的度最大可达____。

【答案】2(n-1)查看答案

【解析】当有向图为完全连通图时每个顶点的度达到最大，出度入度均为 n-1。

5.N 个顶点的连通图用邻接矩阵表示时，该矩阵至少有____个非零元素。

【答案】2(N-1)查看答案

【解析】所谓连通图一定指的是无向图，有向图会称作强连通图。连接 N 个顶点，至少需要 N-1 条边就可以了。由于无向图的每一条边同时关联了两个顶点。因此用邻接矩阵表示时，该矩阵至少有 2(N-1)个非零元素。

6．在图 G 的邻接表表示中，每个顶点邻接表中所含的结点数，对于无向图来说等于该顶点的____；对于有向图来说等于该顶点的____。

【答案】度；出度查看答案

7．遍历图的过程实质上是____，广度优先遍历图的时间复杂度____；深度优先遍历图的时间复杂度____，两者不同之处在于____，反映在数据结构上的差别是____。

【答案】查找顶点的邻接点的过程；0(n+e)；0(n+e)；访问顶点的顺序不同；队列和栈查看答案

【解析】广度优先遍历图使用队列这种数据结构，深度优先遍历图使用栈这种数据结构。

8．在 n 个顶点的非空无向图中，最多有____个连通分量。

【答案】n 查看答案

【解析】当 n 个顶点之间没有边，都是孤立的顶点时，有 n 个连通分量。

9．求图的最小生成树有两种算法，____算法适合于求稀疏图的最小生成树 e

【答案】克鲁斯卡尔查看答案

【解析】克鲁斯卡尔算法是一种按权值的递增次序选择合适的边来构造最小生成树的方法，这种算法中，采用堆来存放边的集合，适合于边稀疏而顶点较多的图。

10．克鲁斯卡尔算法的时间复杂度为____，它对____图较为适合。

【答案】O(eloge)；边稀疏查看答案

11．下面描述的是一种构造最小生成树算法的基本思想。设要处理的无向图包括 n 个顶点 Vl，V_2，…，V_n，用相邻矩阵 A 表示，边的权全是正数。请在下列划线处填上正确叙述。

(1)若(V_i，V_j)是边，则 A(i，j)的值等于____，若(V_i，V_j)不是边，则 A(i，j)的值是一个比任何边的权____，矩阵的对角线元素全为 0。

(2)构造最小生成树过程中，若顶点 V_i已包括进生成树，就把相邻矩阵的对角线元素 A(i，i)置成____，若(V_i，V_j)已包括进生成树，就把矩阵元素 A(i，j)置成____。

(3)算法结束时，相邻矩阵中____的元素指出最小生成树的____。

【答案】(1)(V_i，V_j)边上的权值；都大的数；(2)1；负值；(3)为负；边查看答案

12．求最短路径的 Dijkstra 算法的时间复杂度为____。

【答案】$O(n^2)$查看答案

13．有向图 G=(V, E)，其中 V(G)=[0，1，2，3，4，5｝，用<a，b，d>三元组表示弧<a，b>及弧上的权 d。E(G)为 E(G=｛<0，5，100>，<0，2，10>，<1，2，5>，<0，4，30>，<4，5，60>，<3，5，10>，<2．3，50>，<4，3，20>｝，则从源点 0 到顶点 3 的最短路径长度是____，经过的中间顶点是____。

【答案】50；4 查看答案

14．在拓扑分类中，拓扑序列的最后一个顶点必定是____的顶点。

【答案】出度为 0 查看答案

【解析】如果最后一个顶点的出度不为 0，则必定还有顶点存在，与题目所说的最后一个顶点矛盾，所有最后一个顶点的出度必定为零。

15．应用 prim 算法求解连通网络的最小生成树问题。

(1)针对如图所示的连通网络，试按如下格式给出在构造最小生成树过程中顺序选出的各条边。

（始顶点号，终顶点号，权值）

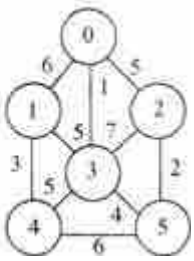
(　　，　　，　　)

(　　，　　，　　)

(　　，　　，　　)

(　　，　　，　　)

(　　，　　，　　)



(2)下面是 Prim 算法的实现，中间有 5 个地方缺失，请阅读程序后将它们补上。

```
const int MaxInt=INT_MAX;                    //INT_MAX的值在<limits.h>中
const int n=6;                               //图的顶点数,应由用户定义
typedef int AdjMatrix[n][n];                 //用二维数组作为邻接矩阵表示
typedef struct{                              //生成树的边结点
   int fromVex,toVex;                        //边的起点与终点
   int weight;                               //边上的权值
}TreeEdgeNode;
typedef TreeEdgeNode MST[n-1];               //最小生成树定义
void PrimMST(AdjMatrix G, MST T,int rt){
   //从顶点rt出发构造图G的最小生成树T,rt成为树的根结点
   TreeEdgeNode e; int i ,k=0,min,minpos,v;
   for(i=0;i<n;i++)                          //初始化最小生成树T
     if(i!=rt){T[k].fromVex=rt;  ①  ;T[k++].weight=G[rt][i]; }
   for(k=0;k<n-1;k++){                 //依次求MST的候选边
     ②
   for (i=k;i<n-1;i++)                       //遍历当前候选边集合
     if(T[i].weight<min)                     //选具有最小权值的候选边
       {min=T[i].weight;  ③  ;}
   if(min= =MaxInt)                  //图不连通,出错处理
     {cerr<<"Graph is disconnected!"<<endl;  ④  ;}
   e=T[minpos];T[minpos]=T[k];  T[k]=e;
   v=T[k].toVex;
   for(i=k+1;i<n-1;i++)      //修改候选边集合
     if(G[v][T[i].toVex]<T[i].weight){
         T[i].weight=G[v][T[i].toVex];
          ⑤  ;
} } }
```

【答案】(1)(0，3，1)；(3，5，4)；(5，2，2)；(3，1，5)；(1，4，3)查看答案

(2)①T[k]；tovex=i②min=Maxint③mispos=i④exit(O)⑤T[i]；fromvex=v

【解析】Prim 算法的执行类似于寻找图的最短路径的 Dijkstra 算法。假设 N={V,E}是连通图，E_T是 N 上最小生成树边的集合。算法从 $V_T=\{u_0\}$，E_T开始，重复执行下述操作：在所有 u 属于 V_T，v 属于 V-V_T的边（u，v）属于 E 中找一条代价最小的边（u_0，v_0）加入集合 E_T，同时将 v_0并入 V_T，直到 V_T=V 为止。

16.n 个顶点的有向图用邻接矩阵 array 表示，下面是其拓扑排序算法，试补充完整。

注：(1)图的顶点号从 0 开始计；

(2)indegree 是有 n 个分量的一维数组，放顶点的入度，

(3)函数 crein 用于记算顶点入度；

(4)有三个函数 push(data)，pop()，check()其含义为数据 data 入栈，出栈和测试栈是否空(不空返回 l，否则 0)。

```
crein( array ,indegree,n)
{ for (i=0;i<n;i++)  indegree[i]= (   (1)   )
  for(i=0,i<n;i++)
    for (j=0;j<n; j++)  indegree[i]+=array[   (2)   ][   (3)   ];
}
topsort (array,indegree,n)
  { count= (   (4)   );
    for (i=0; i<n; i++)  if (   (5)   )  push(i) ;
    while (check( ))
    { vex=pop( );  printf(vex);  count++;
     for (i=0;i<n;i++)
        { k=array   (6)
        if (   (7)   )  { indegree[i]--;  if (   (8)   )  push(i); }
        }
  }
if( count<n)   printf("图有回路");
}
```

【答案】0；j；i；0；indegree[i]=O；[vex][i]；k==l；indegree[i]=0 查看答案

【解析】有向图用邻接矩阵表示时，顶点 i 的入度等于第 i 列的所有元素之和。拓扑排序过程：首先将入度为 0 的顶点全部进栈。然后弹出栈顶结点，并将与弹出的顶点相连的其它顶点的入度减一，然后判断这些顶点的入度是否为零，如果为零，继续进栈，重复这些操作，完成拓扑排序。

四、应用题

1．(1)如果 Gl 是一个具有 n 个顶点的连通无向图，那么 Gl 最多有多少条边?G1 最少有多少条边?

(2)如果 G2 是一个具有 n 个顶点的强连通有向图，那么 G2 最多有多少条边?G2 最少有多少条边?

(3)如果 G3 是一个具有 n 个顶点的弱连通有向图，那么 G3 最多有多少条边?G3 最少有多少条边?

答：(1)G1 最多 n(n-1) / 2 条边，最少 n-1 条边。

(2)G2 最多 n(n-1)条边，最少 n 条边。

(3)G3 最多 n(n-1)条边，最少 n-1 条边。

2．证明：具有 n 个顶点和多于 n-1 条边的无向连通图 G 一定不是树。

答：证明：具有 n 个顶点 n-1 条边的无向连通图是自由树，即没有确定根结点的树，每个结点均可当根。若边数多于 n-1 条，因一条边要连接两个结点，则必因加上这一条边而使两个结点多了一条通路，即形成回路。形成回路的连通图不再是树。

3．请回答下列关于图(Graph)的一些问题：

(1)有 n 个顶点的有向强连通图最多有多少条边?最少有多少条边?

(2)表示一个有 1000 个顶点、1000 条边的有向图的邻接矩阵有多少个矩阵元素?是否稀疏矩阵?

(3)对于一个有向图，不用拓扑排序，如何判断图中是否存在环?

答：(1)最多有 n(n-1)条边；最少有 n 条边。

(2)有有 10^6个矩阵元素；不一定是稀疏矩阵(稀疏矩阵的定义是非零元素个数远小于该矩阵元素个数，且分布无规律)。

(3)使用深度优先遍历，按退出 DFS 过程的先后顺序记录下的顶点是逆向拓扑有序序列。如果在执行 DFS(v)未退出前，出现顶点 u 到 v 的回边，则说明存在包含顶点 v 和顶点 u 的环。

4．解答问题。设有数据逻辑结构为：

B=(K，R)，K={k_1，k_2，…，k_9}

R={<k_1，k_3>，<k_1，k_8>，<k_2，k_3>，<k_2，k_4>，<k_2，k_5>，<k_3，k_9>，<k_5，k_6>，<k_8，k_9>，<k_9，k_7>，<k_4，k_7>，<k_4，k_6>)

(1)画出这个逻辑结构的图示。

(2)相对于关系 R，指出所有的开始结点和终端结点。

(3)分别对关系 R 中的开始结点，举出一个拓扑序列的例子。

(4)分别画出该逻辑结构的正向邻接表和逆向邻接表。

答：

(1)如图 7-1 所示：

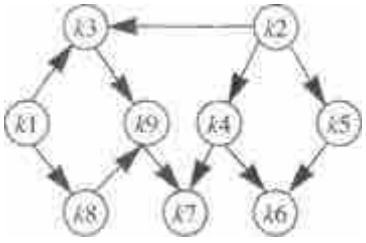


图 7-1

(2)开始结点(入度为 0)：K_1、K_2；终端结点(出度为 0)：K_6、K_7。

(3)拓扑序列：

K_1，K_2，K_3，K_4，K_5，K_6，K_8，K_9，K_7

K_2，K_1，K_3，K_4，K_5，K_6，K_8，K_9，K_7

规则：开始结点为 K_1或 K_2，之后，若遇多个入度为 0 的顶点，按顶点编号顺序选择。

(4)正向邻接表如图 7-2 所示，逆向邻接表如图 7-3 所示：

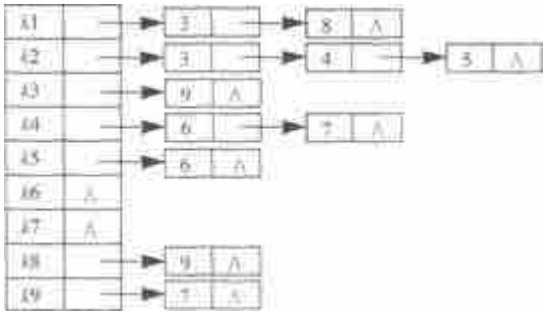


图 7-2 正向邻接表

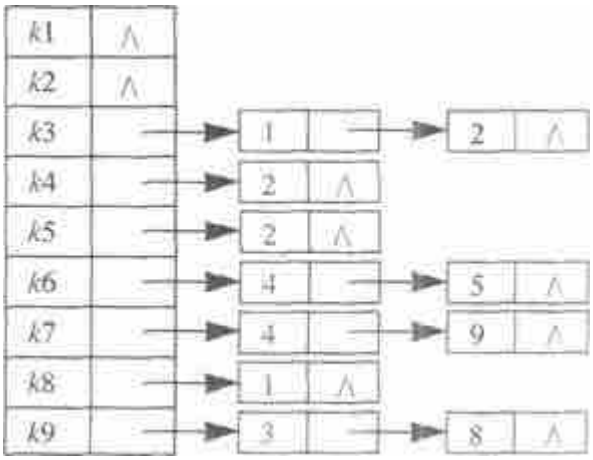
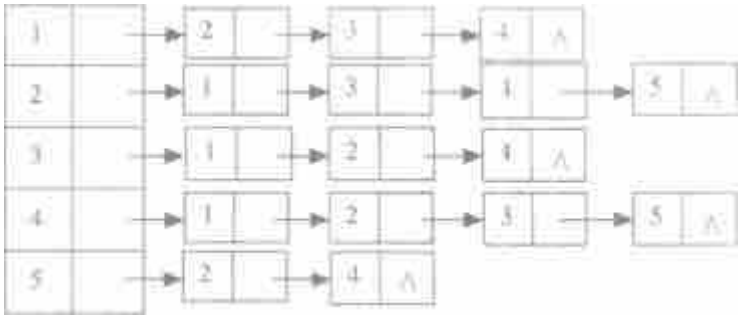


图 7-3 逆邻接表

5．设 G=(V，E)以邻接表存储，如图所示，试画出图的深度优先生成树和广度优先生成树。



答：设从顶点 1 开始遍历，则深度优先生成树如图 7-4 所示，广度优先生成树如图 7-5 所示：



图 7-4

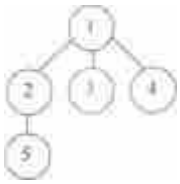


图 7-5

6．已知世界六大城市为：北京(Pe)、纽约(N)、巴黎(Pa)、伦敦(L)、东京(T)、墨西哥(M)，下表给定了这六大城市之间的交通里程：

世界六大城市交通里程表(单位：百公里)

	Pe	N	Pa	L	T	M
Pe		109	82	81	2l	124
N	109		58	55	108	32
Pa	82	58		3	97	92
L	8l	55	3		95	89
T	2l	108	97	95		113
M	124	32	92	89	113	

(1)画出这六大城市的交通网络图；

(2)画出该图的邻接表表示法；

(3)画出该图按权值递增的顺序来构造的最小(代价)生成树。

答：(1)这六大城市的交通网络图如图 7-6 所示：

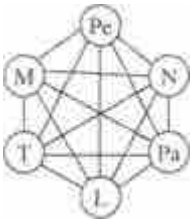


图 7-6

(2)该图的邻接表表示法如图 7-7 所示：

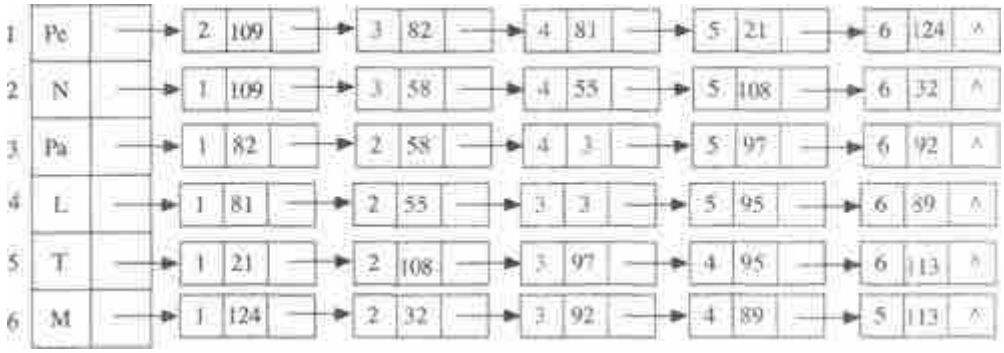


图 7-7

(3)最小生成树有 6 个顶点与条边：V(G)={Pe，N，Pa，L，T，M}

E(G)=({(L，Pa，3)，(Pe，T，21)，(M，N，32)，(L，N，55)，(L，Pe，81)}

7．已知图的邻接矩阵为：

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10
V1	0	1	1	1	0	0	0	0	0	0
V2	0	0	0	1	1	0	0	0	0	0
V3	0	0	0	1	0	1	0	0	0	0
V4	0	0	0	0	0	1	1	0	1	0
V5	0	0	0	0	0	0	1	0	0	0
V6	0	0	0	0	0	0	0	1	1	0
V7	0	0	0	0	0	0	0	0	1	0
V8	0	0	0	0	0	0	0	0	0	1
V9	0	0	0	0	0	0	0	0	0	1
V10	0	0	0	0	0	0	0	0	0	0

当用邻接表作为图的存储结构，且邻接点都按序号从大到小排列时，试写出：

(1)以顶点 Vl 为出发点的唯一的深度优先遍历序列；

(2)以顶点 Vl 为出发点的唯一的广度优先遍历序列；

(3)该图唯一的拓扑有序序列。

答：(1)Vl，V4，V9，V10，V7，V6，V8，V3，V2，V5

(2)Vl，V4，V3，V2，V9，V7，V6，V5，V10，V8

(3)V1，V2，V5，V3，V4，V6，V8，V7，V9，VIO

8．(1)对于有向无环图，叙述求拓扑有序序列的步骤；

(2)对于图 7-8，写出它的四个不同的拓扑有序序列。

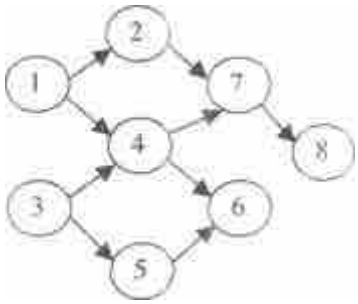


图 7-8

答：(1)对有向图，求拓扑序列步骤为：

1)在有向图中选一个没有前驱(即入度为 0)的顶点并输出。

2)在图中删除该顶点及所有以它为尾的弧。

3)重复 l)和 2)，直至全部顶点输出，这时拓扑排序完成；否则，图中存在环，拓扑排序失败。

(2)拓扑有序序列如图 7-9：

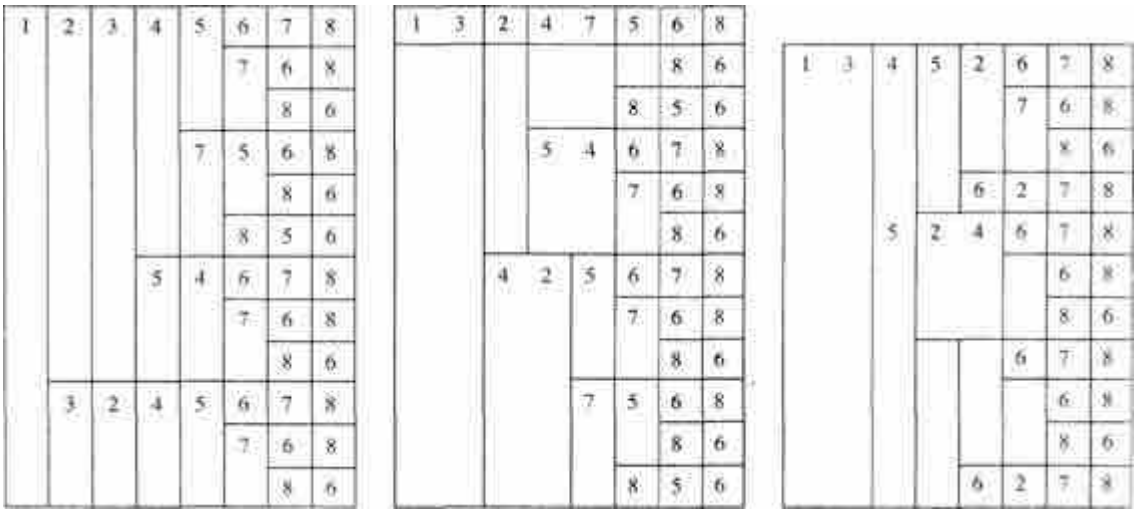


图 7-9 拓扑有序序列

9．请写出应填入下列叙述中(　　)内的正确答案。

某一工程作业的网络图如图 7-10 所示，其中箭头表示作业，箭头边的数字表示完成作业所需的天数。箭头前后的圆圈表示事件，圆圈中的数字表示事件的编号。用事件编号的序列(例如 0-2- 7- 9-11)表示进行作业的路径。

完成此工程的关键路径是(A)完成此工程所需的最少天数为(B)天，此工程中具有最大充裕天数的事件是(C)，充裕天数是(D)。关键路径上的事件的充裕天数是(E)。

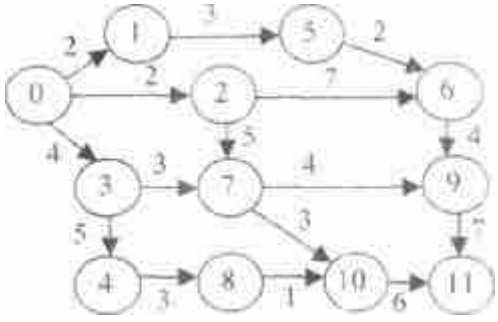


图 7-10

答：A．0-2-6-9-11；B．20；C．事件顶点 1、5 和 7 各充裕两天；D．4 天；E．0

五、算法设计题

1．请用流程图或高级语言表示算法。已知有向图有 n 个顶点，请写算法，根据用户输入的数对建立该有向图的邻接表。即接受用户输入的<vi，vj>(以其中之一为 0 标志结束)，对于每条这样的边，申请一个结点，并插入到的单链表中，如此反复，直到将图中所有边处理完毕。

提示：先产生邻接表的 n 个头结点(其结点数值域从 1 到 n)。

答：算法如下：

```
void CreatAdjList(AdjList g)//建立有向图的邻接表存储结构
 {int n;
  scanf("%d",&n);
  for(i=0;i<n;i++)
     {scanf(&g[i].vertex); g[i].firstarc=null;}//输入顶点信息
  scanf(&v1,&v2);
  while(v1 && v2)//题目要求两顶点之一为0表示结束
     {i=GraphLocateVertex(g,v1);
      p=(ArcNode*)malloc(sizeof(ArcNode));
      p->adjvex=j; p->next=g[i].firstarc; g[i].firstarc=p;
      scanf(&v1&v2);
     }//while  }
```

2．给出以十字链表作存储结构，建立图的算法，输入(i，j，v)，其中 i，j 为顶点号，v 为权值。

答：算法如下：

```
void CreatOrthList(OrthList g)     //建立有向图的十字链表存储结构
 {int i,j,v;                       //假定权值为整型
  scanf("%d",&n);
  for(i=0,i<n;i++)                 //建立顶点向量
     {scanf(&g[i].vertex); g[i].firstin=null; g[i].firstout=null;}
  scanf("%d%d%d",&i,&j,&v);
  while(i && j && v)                                  //当输入i、j、v之一为0时，结束算法运行
     {p=(OrArcNode *)malloc(sizeof(OrArcNode)); //申请结点
     p->headvex=j; p->tailvex=i; p->weight=v;   //弧结点中权值域
     p->headlink=g[j].firstin; g[j].firstin=p;
     p->tailink=g[i].firstout; g[i].firstout=p;
     scanf("%d%d%d",&i,&j,&v);
}//while}//算法结束
```

3．已知无向图采用邻接表存储方式，试写出删除边(i,j)的算法。

答：算法如下：

```
void DeletEdge(AdjList g,int i,j)
//在用邻接表方式存储的无向图g中，删除边(i,j)
  (p=g[i].firstarc;  pre=null; //删顶点i的边结点(i,j),pre是前驱指针
```

```
  while(p)
    if(p->adjvex==j)
      {if(pre==null)g[i].firstarc=p->next; else pre->next=p->next; free(p);}//释放空间
    else {pre=p; p=p->next;}    //沿链表继续查找
p=g[j].firstarc;  pre=null;  //删顶点j 的边结点(j,i)
while(p)
    if(p->adjvex==i)
      {if(pre==null)g[j].firstarc=p->next; else pre->next=p->next; free(p);}//释放空间
    else {pre=p; p=p->next;}    //沿链表继续查找
    }// DeletEdge
```

4. 按图的宽度优先搜索法写一算法判别以邻接矩阵存储的有向图中是否存在由顶点 V_i到顶点 V_j的路径(i≠j)

答：算法如下：

```
int visited[n];   //设有向图有n个顶点
int PathiToj(AdjMatrix g,vertype Vi,vertype Vj)
   // 判断以邻接矩阵方式存储的有向图中是否存在由顶点Vi到顶点Vj的路径
{for(i=0;i<n;i++) visited[i]=0;
 i=GraphLocateVertex(g,Vi);       j=GraphLocateVertex(g,Vj);
 QueueInit(Q);                   //Q是队列,容量足够大,元素是顶点编号(0..n-1)
 visited[i]=1; QueueIn(Q,i); //Vi入队
 while(!QueueEmpty(Q))
    {v=QueueOut(Q);
     for(w=0;i<n;w++)
       {if(g[v][w]==1 && w==j) { printf("Vi到顶点Vj存在路径\n"); return(1); }
        if(g[v][w]==1 && !visited[w]) {visited[w]=1; QueueIn(Q,w);}
       }//for
     }
 printf("Vi到顶点Vj不存在路径\n"); return(0);
 }
```

5. 用邻接多重表存储结构，编写 FIRST-ADJ (G，V)函数，函数返回值为第一个邻接点，若 V 没有邻接点，返回零。

答：算法如下：

```
    int FirstAdj(AdjMuList g , vertype v)
    //在邻接多重表g中,求v的第一邻接点,若存在,返回第一邻接点,否则返回0
     {i=GraphLocateVertex(g,v); //确定顶点v在邻接多重表向量中的下标,不考虑不存在v的情况
      p=g[i].firstedge;          //取第一个边结点
      if(p==null ) return (0);
        else {ifp->(ivex==i) return (p->jvex); else return (p->ivex) }
             //返回第一邻接点,ivex和jvex中必有一个等于i
      }// FirstAdj
```

6. 令 G=(V，E)为一个有向无环图，编写一个给图 G 中每一个顶点赋以一个整数序号的算法，并满足以下条件：若从顶点 i 至顶点 j 有一条弧，则应使 i<j。

答：算法如下：

```
void TopNo(AdjList g)
//对以邻接表存储的DAG图g重新编号,使若有<i,j>,则编号i<j
{int new[MAXSIZE],indegree[n]={0},count=0;
 for(i=0;i<n;i++)
    {p=g[i].firstarc;
     while(p) {indegree[p->adjvex]++; p=p->next;} //求各顶点的入度
    }
 StackInit (S);
 for(i=0; i<n; i++) if(!indegree[i]) Push(S,i);
 while(!StackEmpty(S))
   {Pop(S,i);
    new[i]=n--;  //记录结点的逆序序号
    count++;
    for(p=G.vertices[i].firstarc; p; p=p->nextarc)
       {k=p->adjvex; if(!(--indegree[k])) Push(S,k); }
   }//while
 if(count<G.vexnum) return ERROR;
 for(i=1; i<=n; i++) printf("Old No:%d New No %d\n",i,new[i])
 return OK;
}
```

7. 图 G 有 n 个点，利用从某个源点到其余各点最短路径算法思想，设计一产生 G 的最小生成树的算法。

答：算法如下：

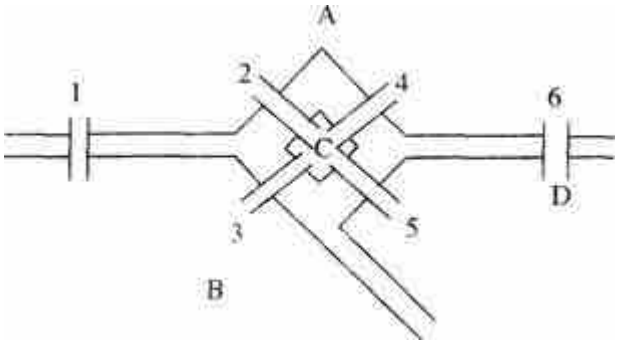
```
void Shortest_PTree ( AdjMatrix G, vertype v0)
   //利用从源点v0到其余各点的最短路径的思想,产生以邻接矩阵表示的图G的最小生成树
  {struct{int u,w,v}dist[],Q[];//dist,Q数组存放生成树
   ints[] ;          //s数组存放顶点是否找到最短路径
   for(i=0;i<n;i++)//初始化,设顶点信息就是编号
     {dist[i].u=v0; dist[i].w=G[v0][i]; dist[i].v=i; s[i]=0;
     }
   s[v0]=1; dist[v0].w=0; dist[v0].v=v0;//从v0开始,求其最小生成树
   QueueIn(Q,dist);
   V=V-{v0};                          //V是尚未到最小生成树的顶点的集合
   for(i=1;i<n;i++)                   //循环n-1次
     {mindis=MAXINT;
      for(j=0;j<n;j++)
       if(s[j]==0 && dist[j].w<mindis && dist[j].v∈V) { u=j; mindis=dist[j].w;}
      s[u]=1; V=V-{u};  //顶点u已找到最短路径下,下面修改相关顶点的最短路径
      for(j=0;j<n;j++)
       if(s[j]==0 && dist[j].w>G[u][j]){dist[j].w=G[u][j];}
     }//for(i=1;i<n;i++)
}//算法结束
```

8．一个有向图 G=(V，E)的平方图 G2=(V，E^2)满足下述性质：(U，w)∈E^2当且仅当存在某个顶点 v∈V，使得(u，v)∈E 且(v，w)∈E。写一个算法从给定的 G 求出 G^2，G 和 G^2可分别用两个邻接表表示。

答：算法如下：

```
void gpingfangtu(AdjList G,AdjList G2)
 {for(i=0,i<n;i++)
    {G2[i].vertex=G[i].vertex; G2[i].firstarc=null; }//G2
 for(i=0;i<n;i++)
    {p=G[i].firstarc;
     while(p)
      {j=p->adjvex; q=G[j].firstarc;
       while(q)
        {k=q->adjvex; s=(ArcNode *)malloc(sizeof(ArcNode));
         s->next=G2[i].firstarc; G2[i].firstarc=s; q=q->next;
        }
        p=p->next;
}//while(p) }//for  }
```

9．设在 4 地(A，B，C，D)之间架设有 6 座桥，如图所示。



要求从某一地出发，经过每座桥恰巧一次，最后仍回到原地。

(1)试就以上图形说明：此问题有解的条件是什么?

(2)设图中的顶点数为 n，试用 C 或 PASCAL 语言描述与求解此问题有关的数据结构并编写一个算法，找出满足要求的一条回路。

答：

(1)只有所有的顶点的度都是偶数，才能有解。

(2)算法如下：

```
#define MAXNODE  …          // 图中顶点的最大个数
typedef struct arc // 弧(边)结点
 {int adjvex; num;//adjvex是邻接点在顶点向量中的下标,num是边的编号
  struct arc  *next;      // 指向下一邻接点的指针
  infotype    *info;      // 与弧(或边)相关的信息指针
 }ArcNode;
typedef struct // 顶点结点
 {vertype vertex; ArcNode *firstarc; }VerNode;// 顶点信息及指向第一邻接点的指针
typedef VerNode AdjList[MAXNODE] // 邻接表
```

修改常规访问标志数组 visited 的含义：当元素值为 l 时表示该边已访问；当元素值为 0 时表示该边尚未访问。

```
visited[1...n]=0; AdjList g;
void dfs(int v0) //用邻接表作为存储结构的深度优先遍历算法
  {p=g[v0].firstarc;  //第一邻接点
   while(p)
       {j=p->num;
        if(visited[j]==0) {visited[j]=1; dfs(p->adjvex); }
        p=p->next;
       }
  }//结束dfs( )
 int degree(AdjList g) //求顶点的度
  {for(i=1;i<=n;i++)
       {count=0; p=g[i].firstarc;
        while(p){count++; p=p->next;}
       if(count==0 || count %2!=0) return 0; // 若顶点度为0,或顶点的度不是偶数,无解
       }
  return 1;
 }
void EulerCycle(AdjList g)
  {int flag=degree(g);
   if(!flag) {printf("无解\n"); exit(0);}
   dfs(1);
  }
```

第 8 章　动态存储管理

一、选择题

动态存储管理系统中，通常可有（　　）种不同的分配策略。

A．1　B．2　C．3　D．4　E．5

【答案】C 查看答案

【解析】动态存储管理系统中有以下三种：首次拟合法、最佳拟合法、最差拟合法。①首次拟合法，从表头指针开始查找可利用空间表，将找到的第一个大小不小于 n 的空闲块的一部分分配给用户。②最佳拟合法，将可利用空间表中一个不小于 n 且最接近 n 的空闲块的一部分分配给用户。则系统在分配前首先要对可利用空间表从头到尾扫视一遍，然后从中找出一块不小于 n 且最接近 n 的空闲块进行分配。③最差拟合法，将可利用空间表中不小于 n 且是链表中最大的空闲块的一部分分配给用户。

二、判断题

1．在伙伴系统中的伙伴是指任意两块大小相同、位置相邻的内存块。（　　）

【答案】×查看答案

【解析】伙伴系统的伙伴不一定是位置相邻的内存块。

起始地址为 P，大小为 2^k的内存块，其伙伴块的起始地址计算公式如下：

$$buddy(p,k)=\begin{cases} p+2^k & 如p\%2^{k+1}=0 \\ p-2^k & 如p\%2^{k+1}=2^k \end{cases}$$

只要符合公式的内存块都是伙伴。

2．在动态存储管理系统中做空间分配时，最佳适配法与最先适配法相比，前者容易增加闲置空间的碎片。（　　）

【答案】√查看答案

【解析】最佳适配法：将可利用空间表中一个不小于 n 且最接近 n 的空闲块的一部分分配给用户；最先适配法：从表头指针开始查找可利用空间表，将找到的第一个大小不小于 n 的空闲块的一部分分配给用户；从适配法选择上可以看出最佳适配法会增加闲置空间。

三、填空题

1．起始地址为 480，大小为 8 的块，其伙伴块的起始地址是_____；若块大小为 32，则其伙伴块的起始地址为_____。

【答案】480+8=488；480-32=448 查看答案

【解析】起始地址为 P，大小为 2^k的内存块，其伙伴块的起始地址计算公式如下：

$$buddy(p,k)=\begin{cases}p+2^k & 如p\%2^{k+1}=0\\ p-2^k & 如p\%2^{k+1}=2^k\end{cases}$$

根据上述公式起始地址就为 488。

2．二进制地址为 011011110000，大小为（4）^{10}和（16）^{10}块的伙伴地址分别为：_____

【答案】011011110100；011011100000 查看答案

【解析】011011110000 是块的起始地址，大小分别为$(4)^{10}$和$(16)^{10}$，其伙伴块的起始地址计算公式如下：

$$buddy(p,k)=\begin{cases}p+2^k & 如p\%2^{k+1}=0\\ p-2^k & 如p\%2^{k+1}=2^k\end{cases}$$

当大小为 4 时，起始地址为 011011110000+0100。当大小为 16 时，起始地址为：011011110000-010000。

3．无用单元是指_____，例_____

【答案】用户不再使用而系统没有回收的结构和变量；p=ma11oc（size）；…，p=nu11 查看答案

四、应用题

1．设内存中可利用空间已连成一个单链表，对用户的存储空间需求，一般有哪三种分配策略?

答：（1）首次适配法

从链表头指针开始查找，找到第一个大于等于所需空间的结点即分配。首次适配法适合事先不知道请求分配和释放信息的情况，分配时需查询，释放时插在表头。

（2）最佳适配法

链表结点大小增序排列，找到第一个大于等于所需空间的结点即分配。最佳适配法适用于请求分配内存大小范围较宽的系统，释放时容易产生存储量很小难以利用的内存碎片，同时保留那些很大的内存块以备将来可能发生的大内存量的需求，分配与回收均需查询。

（3）最差适配法

链表结点大小逆序排列，总从第一个结点开始分配，将分配后结点所剩空间插入到链表适当位置。最差适配法适合请求分配内存大小范围较窄的系统，分配时不查询，回收时查询，以便插入适当位置。

2．试叙述动态存储分配伙伴系统的基本思想，它和边界标识法不同点是什么?

答：（1）动态存储分配伙伴系统的基本思想

在伙伴系统中，无论占用块或空闲块，其大小均为 2^k（k 为大于等于 0 的正整数）。若内存容量为 2^m，则空闲块大小只能是 2^0，2^1，2^2，…，2^m。由同一大块分裂而得的两个小块互称“伙伴空间”，如内存大小为 2^{10}的块分裂成两个大小为 2^9的块。只有两个“伙伴空间”才能合

并成一个大空间。起始地址为 P，大小为 2^k的内存块，其伙伴的起始地址为：buddy（p，k）=p+2^k （若 p%2^{k+1}=0）或 buddy（p，k）=P-2^k （若 p%$^{k+1}$=2^k）。

（2）和边界标识法的不同

边界标识法在每块的首尾均有“占用”/“空闲”标志，空闲块合并方便。伙伴系统算法简单、速度快，但只有互为伙伴的两个空闲块才可合并，因而易产生虽空闲但不能归并的碎片。

3．已知一个大小为 512 个字长的存储，假设先后有 6 个用户申请大小分别为 23，45，52，100，11 和 19 的存储空间，然后再顺序释放大小为 45，52，11 的占用块。假设以伙伴系统实现态存储管理。

（1）画出可利用空间表的初始状态。

（2）画出为 6 个用户分配所需要的存储空间后可利用空间表的状态以及每个用户所得到的存储块的起始地址。

（3）画出在回收 3 个占用块之后可利用空间表的状态。

答：（1）因为 512=2^9，可利用空间表的初始状态图如图 8-1 所示：

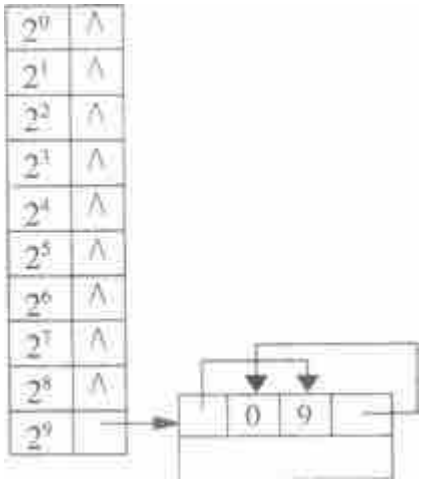


图 8-1 可利用空间表的初始状态

（2）当用户申请大小为 23 的内存块时，因 2^4<23≤2^5，但没有大小为 2^5的块，只有大小为 2^9的块，故将 2^9的块分裂成两个大小为 2^8的块，其中一块挂到可利用空间表上，另一块再分裂成两个大小为 2^7的块。又将其中大小为 2^7的一块挂到可利用空间表上，另一块再分裂成两个大小为 2^6的块。其中一块 2^6的块挂到可利用空间表上，另一块分裂成两个大小为 2^5的块，其中一块挂到可利用空间表上，另一块分给用户（地址 0～31）。如此下去，最后每个用户得到的存储空间的起始地址如图 8-2 所示，为 6 个用户分配所需要的存储空间后可利用空间表的状态如图 8-3 所示。

存储大小	起始地址
23	0
45	64
52	128
100	256
11	32
19	192

图 8-2 每个用户得到的存储空间的起始地址

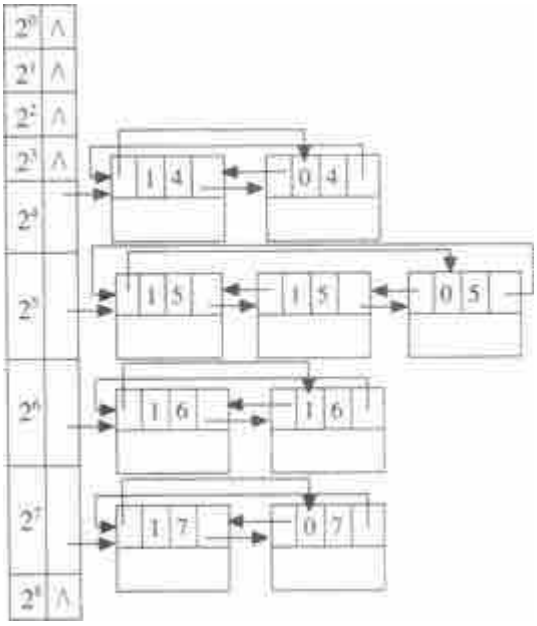


图 8-3 可利用空间表的状态

（3）在回收时，因为给申请 45 的用户分配了大小为 2^6的块，其伙伴地址是 0，在占用中，不能合并，只能挂到可利用空间表上。在回收大小为 52 的占用块时，其伙伴地址是 192，也在占用。回收大小为 11 的占用块时，其伙伴地址是 48，可以合并为大小 2^5的块，挂到可利用空间表上。所以回收 3 个占用块之后可利用空间表的状态如图 8-4 所示：

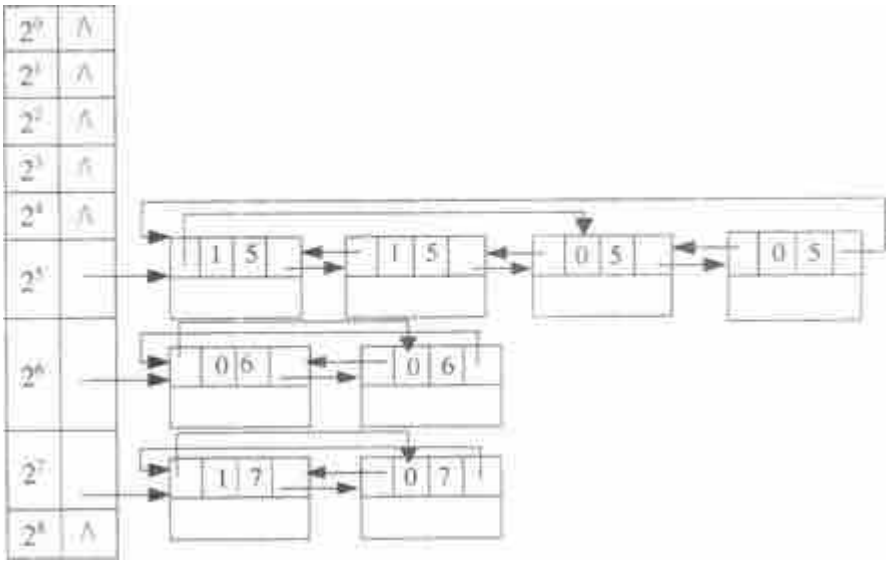


图 8-4

4. 在如图 8-5 所示的伙伴系统中，回收两块首地址分别为 768 及 128、大小为 2^7的存储块，请画出回收后该伙伴系统的状态图。

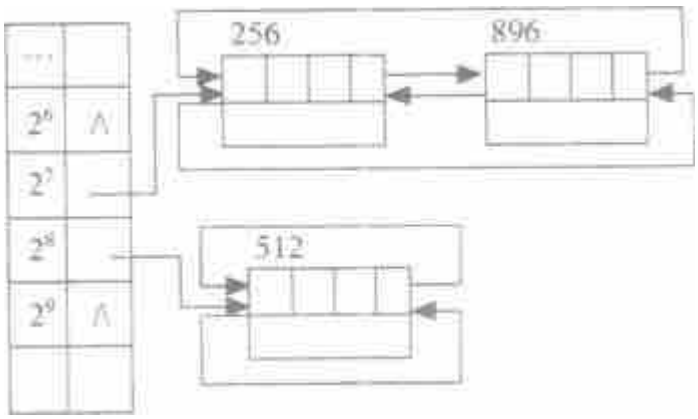


图 8-5

答：因为 $768\%2^{7+1}=0$，所以 768 和 $768+2^7=896$ 互为伙伴，伙伴合并后，首址为 768，块大小为 2^8。因为 $768\%2^{8+1}=2^8$，所以首址 768、；大小为 2^8的块和首址 512、大小为 2^8的块合并，成为首址 512、；大小为 2^9的空闲块。因为 $128\%2^{7+1}=2^7$，其伙伴地址为 $128-2^7=0$，将其插入可利用空间表中。回收后该伙伴系统的状态图如图 8-6 所示：

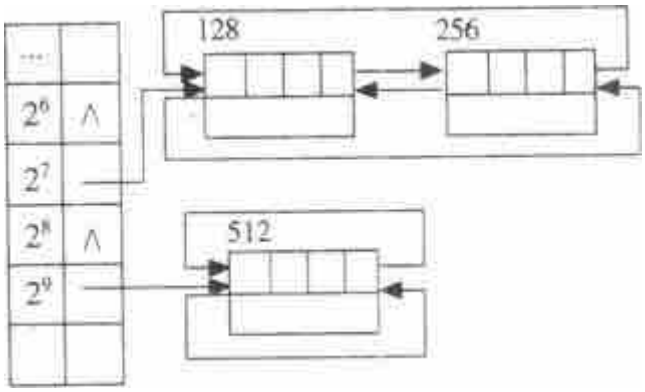
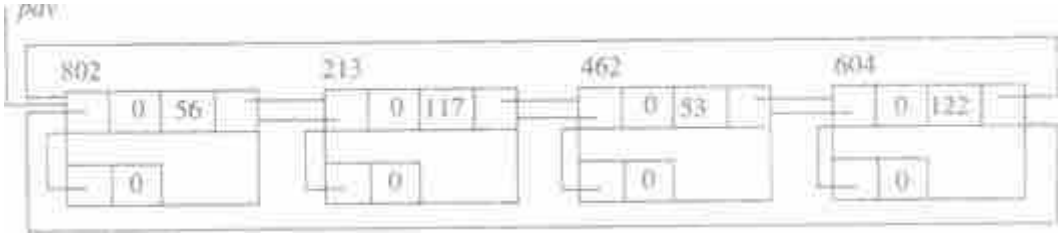


图 8-6 回收后该伙伴系统的状态图

5．假设利用边界标识法，并以首次拟合策略分配，已知在某个时刻可利用空间表的状态如图所示（注：存储块头部 size 域的值和申请分配的存储量均包括头部和尾部的存储空间））请画出：



（1）当系统回收一个起始地址为 559，大小为 45 的空闲块之后的链表状态；

（2）系统继而在接受存储块大小为 100 的请求后，又回收一个起始地址为 515，大小为 44 的空闲块之后的链表状态。

答：（1）系统回收一个起始地址为 559，大小为 45 的空闲块后，因右侧起始地址 604 为空闲块，应与之合并。合并后，成为起始地址为 559，大小为 167 的空闲块。链表状态如图 8-7 所示：

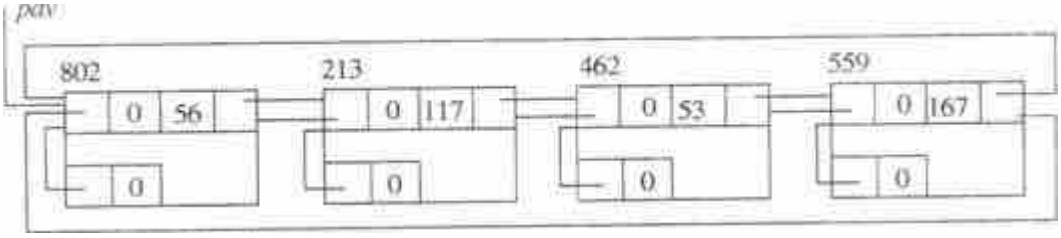


图 8-7

（2）系统在接受存储块大小为 100 的请求后，将大小为 117 的空闲块分出 100 给予用户。在回收一个起始地址为 515，大小为 44 的空闲块之后，因左侧起始地址为 462、大小为 53 和右侧起始地址为 559、大小为 167 均为空闲块，应与之合并。合并后，起始地址为 462、大小为 264 的空闲块。链表状态如图 8-8 所示：

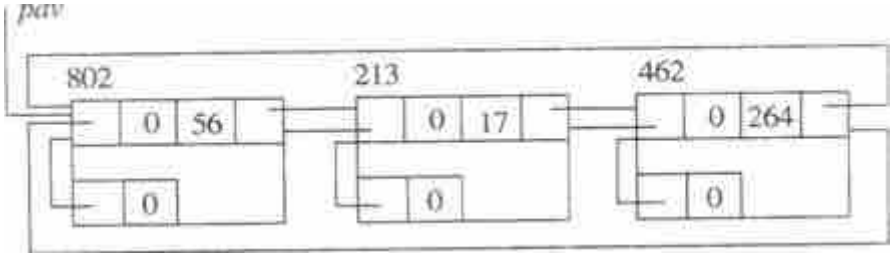


图 8-8

第 9 章　查　找

一、选择题

1．若查找每个记录的概率均等，则在具有 n 个记录的连续顺序文件中采用顺序查找法查找一个记录，其平均查找长度 ASL 为（　　）。

A．（n-1）/ 2　　B．n / 2　　C．（n+1）/ 2　　D．n

【答案】C 查看答案

【解析】最快查找一次成功，最慢查找 n 次成功。平均查找次数为（1+2+3+…+n）/n，那么 ASL=(n+1)/2。

2．在一个有 N 个元素的有序单链表中查找具有给定关键字的结点，平均情况下的时间复杂性为（　　）。

A．O（1）　　B．O（N）　　C．O（N^2）　　D．O（NlogN）

【答案】B 查看答案

【解析】二分查找的时间复杂度为 O(logn)。在一个用 N 个元素的有序单链表中查找具有给定关键字的结点，因为查找是从头结点开始的，需要使用指针顺序往下查找，因此时间复杂度为 0（N）。

3．对线性表进行折半查找时，要求线性表必须（　　）。

A．以顺序方式存储　　B．以顺序方式存储，且数据元素有序

C．以链接方式存储　　D．以链接方式存储，且数据元素有序

【答案】B 查看答案

【解析】二分查找又称折半查找，优点是比较次数少，查找速度快，平均性能好；其缺点是要求待查表为有序表，且插入删除困难。因此，折半查找方法适用于不经常变动而查找频繁的有序列表。折半查找方法适用于对以顺序方式存储的有序表的查找，查找效率较高。

4．下列二叉排序树中查找效率最高的是（　　）。

A．平衡二叉树

B．二叉查找树

C．没有左子树的二叉排序树

D．没有右子树的二叉排序树

【答案】A 查看答案

【解析】平衡二叉树的左子树和右子树的深度之差的绝对值不超过 1。这就保证了二叉树的深度是 $\log_2 n$ 级别的。二叉查找树或者是一颗空数；或者是具有下列性质的二叉树：①若左子树不空，则左子树上所有结点的值均小于它的根结点的值；②若右子树不空，则右子树上所有结点的值均大于它的根结点的值；③左、右子树也分别为二叉排序树。B、C、D 三项均不能保证左子树和右子树的深度之差的绝对值不超过 1，甚至很大，因此查找效率低。

5．当在一个有序的顺序存储表上查找一个数据时，既可用折半查找，也可用顺序查找，但前者比后者的查找速度（　　）。

A．必定快　B．不一定　C．在大部分情况下要快　D．取决于表递增还是递减

【答案】C 查看答案

【解析】对于有序顺序存储表折半查找的效率较高，但是不是所有情况下都是如此，比如要查找的元素就是第一个时，用顺序查找比它就快的多了。这类情况外折半都高于顺序查找。

6．折半查找的时间复杂性为（　　）。

A．O（n^2）　B．O（n）　C．O（nlogn）　D．O（logn）

【答案】D 查看答案

【解析】顺序查找的事件复杂度为 0(n)，因为折半查找是查找效率最高的算法，它的事件复杂度为 0（logn）。

7．如果要求一个线性表既能较快地查找，又能适应动态变化的要求，可以采用下列哪一种查找方法。

A．分块　B．顺序　C．折半　D．哈希

【答案】A 查看答案

【解析】分块查找，把线形表分成若干块，块间是顺序存储的，所以查找速度较快。在每一块中的数据元素的存储顺序是任意的，所以便于线性表的动态变化。

8．分别以下列序列构造二叉排序树，与用其他三个序列所构造的结果不同的是（　　）。

A．（100，80，90，60，120，110，130）　B．（100，120，110，130，80，60，90）

C．（100，60，80，90，20，110，130）　D．（100，80，60，90，120，130，110）

【答案】C 查看答案

【解析】二叉排序树：左右子树都是二叉排序树，且保证右子树都比根结点大，左子树都比根结点小。据以上两点建立二叉排序树。

9．在平衡二叉树中插入一个结点后造成了不平衡，设最低的不平衡结点为 A，并已知 A 的左孩子的平衡因子为 0，右孩子的平衡因子为 l，则应作（　　）型调整以使其平衡

A．LL　B．LR　C．RL D．RR

【答案】C 查看答案

【解析】A 的平衡因子此时为-1，要使插入结点不平衡，必须插在右孩子的左子树上，A 平衡因子变成了-2.则需要进行两次旋转（先右旋后左旋）。

10．下面关于 B 和 B+树的叙述中，不正确的是（　　）

A．B 树和 B+树都是平衡的多叉树　B．B 树和 B+树都可用于文件的索引结构

C．B 树和 B+树都能有效地支持顺序检索　D．B 树和 B+树都能有效地支持随机检索

【答案】C 查看答案

【解析】B 树是一种平衡的多分树，通常我们说 m 阶的 B 树，它必须满足如下条件：①每个结点至多有 m 个子结点；②除根结点和叶结点外，其它每个结点至少有 个子结点；③若根结点不是叶子结点，则至少有两个子结点；④所有的叶结点在同一层；⑤有 k 个子结点的非根结点恰好包含 k-1 个关键码。B+树是 B 树的一种变形树，它与 B 树的差异在于：有 k

个子结点的结点必然有 k 个关键码；非叶结点仅具有索引作用，跟记录有关的信息均存放在叶结点中。其中 B 树适合与随即检索，不适合于顺序检索，所以 C 项错误。

11．一棵 3 阶 B-树中含有 2047 个关键字，包括叶结点层，该树的最大深度为（　　）。

A．11　　B．12　　C．13　　D．14

【答案】B 查看答案

12．下面关于哈希（Hash，杂凑）查找的说法正确的是（　　）。

A．哈希函数构造的越复杂越好，因为这样随机性好，冲突小

B．除留余数法是所有哈希函数中最好的

C．不存在特别好与坏的哈希函数，要视情况而定

D．若需在哈希表中删去一个元素，不管用何种方法解决冲突都只要简单地将该元素删去即可

【答案】C 查看答案

【解析】若数据结构中存在关键字和 K 值相等的记录，则必定在 f(K)的存储位置上，由此，不需要进行比较便可直接取得所查记录。在此，称这个对应关系 f 为哈希(Hash)函数，哈希函数的选择要视具体情况而定。

13．设哈希表长 M=14，哈希函数 H（KEY）=KEY MOD 11。表中已有 4 个结点：ADDR（15）=4，ADDR（38）=5，ADDR（61）=6，ADDR（84）=7，其余地址为空，如用二次探测再哈希法解决冲突，关键字为 49 的结点的地址是（　　）。

【答案】D 查看答案

【解析】15，38，61，84 用哈希函数 H(key)=key%11 计算后得地址：4，5，6，7。49 计算后为 5,发生冲突，用二次探测再散列法解决冲突：　1：(key+1^2)%11=(49+1)%11=6，仍然发生冲突。2：(key-1^2)%11=(49-1)%11=4，仍然发生冲突。3：(key+2^2)%11=(49+4)%11=9，不再发生冲突。

14．假定有 k 个关键字互为同义词，若用线性探测法把这 k 个关键字存入哈希表中，至少要进行多少次探测?（　　）

A．k-1 次　　B．k 次　　C．k+1 次　　D．k（k+1）/ 2 次

【答案】D 查看答案

【解析】至少探测次数=1+2+3+…+k=k*（k+1）/2。

15．采用开址定址法解决冲突的哈希查找中，发生集聚的原因主要是（　　）。

A．数据元素过多　　B．负载因子过大　　C．哈希函数选择不当　　D．解决冲突的算法选择不好

【答案】D 查看答案

【解析】开放定址法就是从发生冲突的那个单元开始，按照一定的次序，从散列表中查找出一个空闲的存储单元，把发生冲突的待插入元素存入到该单元中的一类处理冲突的方法。

16．哈希函数有一个共同的性质，即函数值应当以（　　）取其值域中的每个值。

A．最大概率　　B．最小概率　　C．平均概率　　D．同等概率

【答案】D 查看答案

二、判断题

1．采用线性探测法处理散列时的冲突，当从哈希表删除一个记录时，不应将这个记录的所在位置置空，因为这会影响以后的查找。（　　）

【答案】√查看答案

【解析】从哈希表删除一个记录，不是将这个记录的位置置空，而是设置一个标记，标记这个元素是无效的了。

2．哈希函数越复杂越好，因为这样随机性好，冲突概率小。（　　）

【答案】×查看答案
【解析】随机性好和冲突概率小跟哈希函数的复杂程度无关，是根据具体情况而定的，跟实际的数据有很大关系。
3．哈希表的平均查找长度与处理冲突的方法无关。（　　）
【答案】×查看答案
【解析】常见的处理冲突的方法：开放地址法；再哈希法；链地址法；建立一个公共的溢出区。选取不同的处理冲突的方法，哈希表的平均查找长度可能不同。
4．负载因子（装填因子）是哈希表的一个重要参数，它反映哈希表的装满程度。（　　）
【答案】√查看答案
【解析】查找过程中需和给定值进行比较的关键字的个数取决于三个因素：哈希函数，处理冲突的方法和哈希表的装填因子。其中装填因子标志哈希表的装满程度。
5．哈希表的结点中只包含数据元素自身的信息，不包含任何指针。（　　）
【答案】×查看答案
【解析】哈希表的结点中可以包括指针，指向其元素。如哈希链表。
6．如果数据元素保持有序，则查找时就可以采用折半查找方法。（　　）
【答案】×查看答案
【解析】采用折半查找的条件是数据元素有序且存储方式为顺序存储，对于常见的链式存储，在进行查找时主要依靠指针来操作。
7．折半查找与二元查找树的时间性能在最坏的情况下是相同的。（　　）
【答案】×查看答案
【解析】不是，当二元查找树是一棵单支树时，时间性能是 O（n）。而折半查找依然是 O（$\log_2 n$）
8．用向量和单链表表示的有序表均可使用折半查找方法来提高查找速度。（　　）
【答案】×查看答案
【解析】单链表不能使用折半查找方法。折半查找主要用于数据元素有序且存储方式为顺序存储的表。
9．在索引顺序表中，实现分块查找，在等概率查找情况下，其平均查找长度不仅与表中元素个数有关，而且与每块中元素个数有关。（　　）
【答案】√查看答案
【解析】在进行分块查找时，首先查找元素在哪一块，然后在确定的块中查找元素，因此，在索引顺序表中，进行分块查找的平均查找长度不仅与表中元素的个数有关，而且与每块中的元素个数有关。
10．就平均查找长度而言，分块查找最小，折半查找次之，顺序查找最大。（　　）
【答案】×查看答案
【解析】折半查找最小，分块查找次之，顺序查找最大。分块查找的速度虽然不如折半查找算法，但比顺序查找算法快得多，同时又不需要对全部节点进行排序。当结点很多且块数很大时，对索引表可以采用折半查找，这样能够进一步提高查找的速度。
11．对大小均为 n 的有序表和无序表分别进行顺序查找，在等概率查找的情况下，对于查找成功，它们的平均查找长度是相同的，而对于查找失败，它们的平均查找长度是不同的。（　　）
【答案】√查看答案
【解析】查找成功的情况下，顺序表和无序表的平均查找长度是相同的，对于查找失败，无序表需要查找到表尾，而顺序表不需要查到表尾就能确定，所以顺序表的查找长度小于无序表的查找长度。

12．平衡二叉树中，若某个结点的左、右孩子的平衡因子为零，则该结点的平衡因子一定是零。（　　）
【答案】×查看答案
【解析】平衡因子定义为该结点的左子树的深度减去右子树的深度，一个平衡二叉树中，某节点的左右孩子的平衡因子为 0，说明左孩子的左子树和右子数的深度相同，而且右子树的左子树和右子数的深度相同，但这不能说明该节点的左子树和右子树的深度相同。
13．在二叉排序树中插入一个新结点，总是插入到叶结点下面。（　　）
【答案】×查看答案
【解析】不一定。插入二叉排序树的原则：将新节点元素值与根结点元素值相比较，如小于根结点元素值则插入到左子树口，否则插入到右子树中。所以就可能插在一个度为 1 的结点下面。
14．完全二叉树肯定是平衡二叉树。（　　）
【答案】×查看答案
【解析】从平衡因子定义看，完全二叉树任一结点的平衡因子的绝对值确实是小于等于 1。但是，平衡二叉树本质上是二叉排序树，完全二叉树不一定是排序树。故不能说完全二叉树是平衡二叉树。
15．二叉树中除叶结点外，任一结点 X，其左子树根结点的值小于该结点（X）的值；其右子树根结点的值大于等于该结点（X）的值，则此二叉树一定是二叉排序树。（　　）
【答案】×查看答案
【解析】二叉排序树按中序遍历的结点序列是从小到大的因为某结点的左子树根结点不一定是它的中序前驱，可能会出现某结点的左子树根结点的右子树上的值大于这个结点；其右子树根结点也不一定是它的中序后继。可能会出现右子树根结点的左子树的值小于这个结点。
16．二元查找树的任何结点的左右子树都是二元查找树。（　　）
【答案】√查看答案
【解析】二元查找树也称作二元排序树。二元查找树的性质：①若左子树不空，则左子树上所有结点的值均小于它的根结点的值；②若右子树不空，则右子树所有结点的值均大于它的根结点的值；③左右子树也分别为二元查找树。
17.B-树中所有结点的平衡因子都为零。（　　）
【答案】√查看答案
【解析】一棵 m 阶的 B-树，如果不为空，则所有的叶子结点都出现在同一层次上，所以 B-树总的所有结点的平衡因子都为零。
18.在平衡二叉树中，向某个平衡因子不为零的结点的树中插入一新结点，必引起平衡旋转。（　　）
【答案】×查看答案
【解析】不一定，比如一个平衡因子为 1 的结点，这时往它的右部插入一个新结点，就不会引起平衡旋转
19．如果完全二叉树从根结点开始按层次遍历的输序列为 l，2,3，4，5，6，7，则该完全二叉树是二叉排序树。（　　）
【答案】×查看答案
【解析】不是，2，3 作为 1 的左右孩子，由于左节点 2 比 1 大，所以不具有二叉排序树的性质。
20．若把堆看成是一棵完全二叉树，则该树一定是一棵二叉排序树。（　　）
【答案】×查看答案

【解析】堆中某个节点的值总是不大于或不小于其父节点的值，这个并不是二叉排序树的性质。

21.m 阶 B 树的任何一个结点的左右子树的高度都相等。（　　）

【答案】√查看答案

【解析】由 B 树的性质得知，叶子结点都处于同一层。因此，m 阶 B 树的任何一个结点的左右子树的高度都相等。

22．3 阶的 B-树是平衡的 3 路搜索树。反之，一棵平衡的 3 路搜索树是 3 阶 B-树。（　　）

【答案】×查看答案

【解析】3 路搜索树并不具有 3 阶 B-树的性质。因此一棵平衡的 3 路搜索树不一定是 3 阶 B-树。

23．对两棵具有相同关键字集合的而形状不同的二叉排序树，按中序遍历它们得到的序列的顺序却是一致的。（　　）

【答案】√查看答案

【解析】形状不同的两个二叉排序树（关键字集合相同），在中序遍历下是输出排好序的序列，所以顺序是一致的。

三、填空题

1．顺序查找 n 个元素的顺序表，若查找成功，则比较关键字的次数最多为_____次；当使用监视哨时，若查找失败，则比较关键字的次数为_____。

【答案】n；n+1 查看答案

【解析】最多的情况就是把整个表遍历了一遍。使用监视哨时，需要多一个存储空间来存监视哨。

2．高度为 4 的 3 阶 B-树中，最多有_____个关键字。

【答案】26 查看答案

【解析】第 4 层是叶结点，1 层至 3 层每个结点两个关键字，每个节点的关键字达到最大时，关键字最多。

3．给定一组数据{6，2，7，10，3，12}以它构造一棵哈夫曼树，则树高为_____，带权路径长度 WPL 的值为_____。

【答案】5；96 查看答案

【解析】每次找两个最小的权值构建哈夫曼树：

40

23　17

11　12 7　10

5　6

2 3

WPL=（2+3）*4+6*3+（12+10+7）*2=96.

4．在一棵 m 阶 B-树中，若在某结点中插入一个新关键字而引起该结点分裂，则此结点中原有的关键字的个数是_____；若在某结点中删除一个关键字而导致结点合并，则该结点中原有的关键字的个数是_____。

【答案】m-1；ém / 2ù-1 查看答案

【解析】m 阶 B-树除根结点和叶子结点外，结点中关键字个数最多是 m-1，最少 ém / 2ù-1

5．已知有序表为（12，18，24，35，47，50，62，83，90，115，134）当用二分法查找 90 时，需_____次查找成功，查找 47 时_____成功，查找 100 时，需_____次才能确定不成功。

【答案】2；4；3 查看答案

【解析】二分法查找元素次数列表

下标	1	2	3	4	5	6	7	8	9	10	11
数值	12	18	24	35	47	50	62	83	90	115	134
次数	3	4	2	3	4	1	3	4	2	3	4

查找 100 是找到 115 就停止了。

6．在哈希函数 H（key）=key％p 中，p 值最好取_____。

【答案】小于等于表长的最大素数或不包含小于 20 的质因子的合数查看答案

【解析】在使用除留余数法时，对除数 P 的选择很重要。若 P 选的不好，容易产生同义词。一般情况下，可以选 P 为质数或不包含小于 20 的质因素的合数。

7．假定有 k 个关键字互为同义词，若用线性探测再哈希法把这 k 个关键字存入哈希表中，至少要进行_____次探测。

【答案】k（k+1）／2 查看答案

【解析】当该关键字发生冲突时，用线性探测不会遇到别的关键字冲突，这个时候需要探测的次数最小。总次数为 1+2+3……+k=k（k+1）／2。

8．执行顺序查找时，存储方式可以是__（1）___，折半查找时，要求线性表__（2）___，分块查找时要求线性表__（3）___，而哈希表的查找，要求线性表的存储方式是__（4）___。

【答案】顺序存储或链式存储；顺序存储且有序；块内顺序存储，块间有序；散列存储查看答案

9．如果按关键码值递增的顺序依次将关键码值插入到二叉排序树中，则对这样的二叉排序树检索时，平均比较次数为_____。

【答案】（n+1）／2 查看答案

【解析】如果关键码是排好序的，构建二叉排序树就会形成一个单支树，它的查找效率和顺序查找效率一样为（n+1）／2。

10. 一棵深度为 k 的平衡二叉树，其每个非终端结点的平衡因子均为 0，则该树共有_____个结点。

【答案】2^k一 l 查看答案

【解析】每个非终端结点都是 0 表示该平衡二叉树没有高度落差。也就是说它是一棵满二叉树。故结点个数为 2^k一 l。

11．假定查找有序表 A[1．.12]中每个元素的概率相等，则进行折半查找时的平均查找长度为_____

【答案】37／12 查看答案

【解析】折半查找时每个的次数如表 9-1 所示：

表 9-1

1	2	3	4	5	6	7	8	9	10	11	12
3	4	2	3	4	1	3	4	2	4	3	4

平均查找次数为（3+4+2+3+4+1+3+4+2+4+3+4）/12=37/12。

12. 已知二叉排序树的左右子树均不为空，则_____上所有结点的值均小于它的根结点值，_____上所有结点的值均大于它的根结点的值。

【答案】左子树；右子树查看答案

【解析】二叉排序树(binary sort tree)或者是一棵空树，或者是具有下列性质的二叉树：①若它的左子树不空，则左子树上所有结点的值均小于它的根结点的值；②若它的右子树不空，则右子树上所有结点的值均大于它的根结点的值；③它的左、右子树也分别为二叉排序树。

13．设 T 是一棵结点值为整数的二叉排序树，A 是一个任意给定的整数。在下面的算法中，free_tree（T）在对二叉排序树丁进行后序遍历时释放二叉排序树 T 的所有结点；Delete_subtree

（T，A），首先在二叉排序树 T 中查找值为 A 的结点，根据查找情况分别进行如下处理：（1）若找不到值为 A 的结点，则返回根结点的地址（2）若找到值为 A 的结点，则删除以此结点为根的子树，并释放此子树中的所有结点，若值为 A 的结点是查找树的根结点，删除后变成空的二叉树，则返 NULL；否则返回根结点的地址。

```
typedef struct node{int data, struct node *lchild, *rchild}node;
void  free_tree(node  *T)
  {if(T! =null) {free_tree(T->lchild);free_tree(T->rchild);   (1)   ; }
  }
node *delete_subtree(node  *T,int A)
  {node  *p=null,  *q=T;
  while(   (2)   )
     {p=q; if(A<q->data) q=q->lchild;else   (3)   ;}
      if(q!=null)
         (free_tree(q);
         if(p==null) T=null;else if(A<p->data)   (4)   ; else   (5)   ;
         }
      return(T);
      }
```

【答案】free（T）；q&&q->data!=A；q=q->rchild；p->lchild=nu11；p->rchild=nu11 查看答案

14．阅读下列程序，指出其功能，并写出空格处应填上的语句。

```
void testl(element item,list_pointer ht[])
    {int hash_value=hash(item.key);
    list_pointer ptr, trail=NULL, lead=ht[hash_value];
    for(;lead;trail=lead,lead=lead->link)
   {if(!strcmp(lead->item.key,item.key))
      {fprintf(stderr,"The key is in the table\n"); exit(1);}
   }
ptr=(list_pointer)malloc(sizeof(list));
if(IS_FULL(ptr)) {fprintf(stderr,"The memoty is full\n"); exit(1); }
ptr->item=item; ptr->link=NULL;
if(trail)   (1)   ;else   (2)   ;
}
```

【答案】trai->link=ptr；ht[hash_value]=ptr 查看答案

【解析】本题是在哈希表 ht[]中插入值为 item 的元素，如该元素已在哈希表中，报告出错。

四、应用题

1．证明若二叉排序树中的一个结点存在两个孩子，则它的中序后继结点没有左孩子，则它的中序前驱结点没有右孩子。

答：证明：根据中序遍历的定义，该结点的中序后继是其右子树上按中序遍历的第一个结点：叶结点或仅有右子树的结点；而其中序前驱是其左子树上按中序遍历的最后个结点：叶结点或仅有左子树的结点。命题得证。

2．在采用线性探测法处理冲突的哈希表中，所有同义词在表中是否一定相邻?

答：不一定相邻。哈希地址为 i（0≤i≤m-1）的关键字，和为解决冲突形成的探测序列 f 的同义词，都争夺哈希地址 i。

3．对下面的关键字集{30，15，21，40，25，26，36，37}若查找表的装填因子为 0.8，采用线性探测再哈希方法解决冲突，做：（1）设计哈希函数；（2）画出哈希表；（3）计算查找成功和查找失败的平均查找长度；（4）写出将哈希表中某个数据元素删除的算法。

答：（1）由于装填因子为 0.8，关键字有 8 个，所以表长为 8 / 0.8=10。用除留余数法设计哈希函数，哈希函数为 14（key）=key％7。

（2）哈希表如表 9-2 所示：

表 9-3 哈希表

哈希地址	O	1	2	3	4	5	6	7	8	9
关键字	21	15	30	36	25	40	26	37		
比较次数	1	1	1	3	1	1	2	6		

（3）计算查找失败时的平均查找长度，必须计算不在表中的关键字，当其哈希地址为 i（0≤i≤m-1）时的查找次数。本例中 m=10。故查找失败和查找成功时的平均查找长度分别为：

ASL_{unsucc}=(9+8+7+6+5+4+3+2+1+1)/10=4.6　ASL_{succ}=16/8=2

（4）算法如下：

```
int  Delete(int  h[n], int  k)
// 从哈希表h[n]中删除元素k,若删除成功返回1,否则返回0
{i=k%7;// 哈希函数用H(key)=key % 7
if(h[i]== maxint)//maxint解释成空地址
   printf("无关键字%d\n",k);return  (0);}
if(h[i]==k){h[i]=-maxint  ;return (1);} //被删元素换成最大机器数的负数
else  // 采用线性探测再散列解决冲突
  {j=i;
   for(d=1;d<=n-1;d++)
      {i=(j+d)%n;                    // n为表长,此处为10
       if(h[i]== maxint)return (0); //maxint解释成空地址
       if(h[i]==k){ h[i]=-maxint  ;return  (1);}
       }//for  }
printf("无关键字%d\n",k);return (0)
}
```

5．设哈希（Hash）表的地址范围为 0～17，哈希函数为：H（K）=K MOD 16，K 为关键字，用线性探测再散列法处理冲突，输入关键字序列：（10，24，32，17，31，30，46，47，40，63，49）造出哈希表，试回答下列问题：

（1）画出哈希表示意图；

（2）若查找关键字 63，需要依次与哪些关键字比较?

（3）若查找关键字 60，需要依次与哪些关键字比较?

（4）假定每个关键字的查找概率相等，求查找成功时的平均查找长度。

答：（1）哈希表示意图如表 9-3 所示：

表 9-3 哈希示意图

哈希地址	O	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
关键字	32	17	63	49					24	40	10				30	31	46	47

字

比较次数	1	1	6	3		1	2	1		1	1	3	3

（2）查找关键字 63 时，由于 H（k）=63 MOD 16=15，所以需要依次与 31，46，47，32，17，63 比较。

（3）查找关键字 60 时，由于 H（k）=60 MOD 16=12，哈希地址 12 内为空，查找失败。

（4）$ASL_{成功}=23/11$

6．设 a，b，c，d，e 五个字符的编码分别为 l，2，3，4，5，并设标识符依以下次序出现：ac，Bd，aa，be，ab，ad，cd，bc，ae，ce。要求用哈希（Hash）方法将它们存入具有 10 个位置的表中。

（1）将上述关键字（标识符）构造一个哈希函数，使得发生冲突尽可能地少；

（2）线性探测再散列法解决冲突。写出上述各关键字在表中位置。

答：（1）构造的哈希函数为：哈希函数 H（key）=（关键字各字符编码之和）MOD 7。

（2）关键字在表中的位置如表 9-4 所示：

表 9-4 关键字在表中的位置

哈希地址	0	1	2	3	4	5	6	7	8	9
关键字	be	cd	aa	ab	ac	ad	bd	bc	ae	ce
比较次数	1	2	1	1	1	1	1	3	3	9

7．设散列表为 HT[0．.12]，即表的大小为 m=13。现采用双散列法解决冲突。散列函数和再哈希函数分别为：H_0（key）=key%13；

注：%是求余数运算（=mod）H_i=（H_{i-1}1+REV（key+1）%11+1）%13；i=1，2，3，…，m-1 其中，函数 REV（x）表示颠倒十进制数 x 的各位，如 REV（37）=73，REV（7）=7 等。若插入的关键码序列为（2，8，31，20，19，18，53，27）。

（1）试画出插入这 8 个关键码后的哈希表；

（2）计算搜索成功的平均搜索长度 ASL。

答：（1）插入这 8 个关键码后的哈希表如表 9-5 所示：

表 9-5 插入关键字后的哈希表

哈希地址	0	1	2	3	4	5	6	7	8	9	10	11	12
关键字	27	53	2			31	19	20	8	18			
比较次数	3	1	1			1	1	1	1	2			

（2）$ASL_{成功}=11/8$

8．在 B-树和 B+树中查找关键字时，有什么不同?

答：在 B-树中查找关键字从根结点开始，从根往下查找结点，然后在结点内查找关键字，得出查找成功与否的结论。B+树的非终端结点是索引部分，其查找从根开始，从根往下查到关键字后，要继续查到最下层结点，得到查找成功与否的结论。另外，B+树还可以在最下层从最小关键字开始，从左往右进行顺序查找，B-树则不能作顺序查找。

9．设依以下次序给出关键字：34，16，19，21，5，49，24，62，3，17，45，8，构造 3 阶 B-树。要求从空树开始，每插入一个关键字，画出一棵树。

答：如图 9-1 所示：

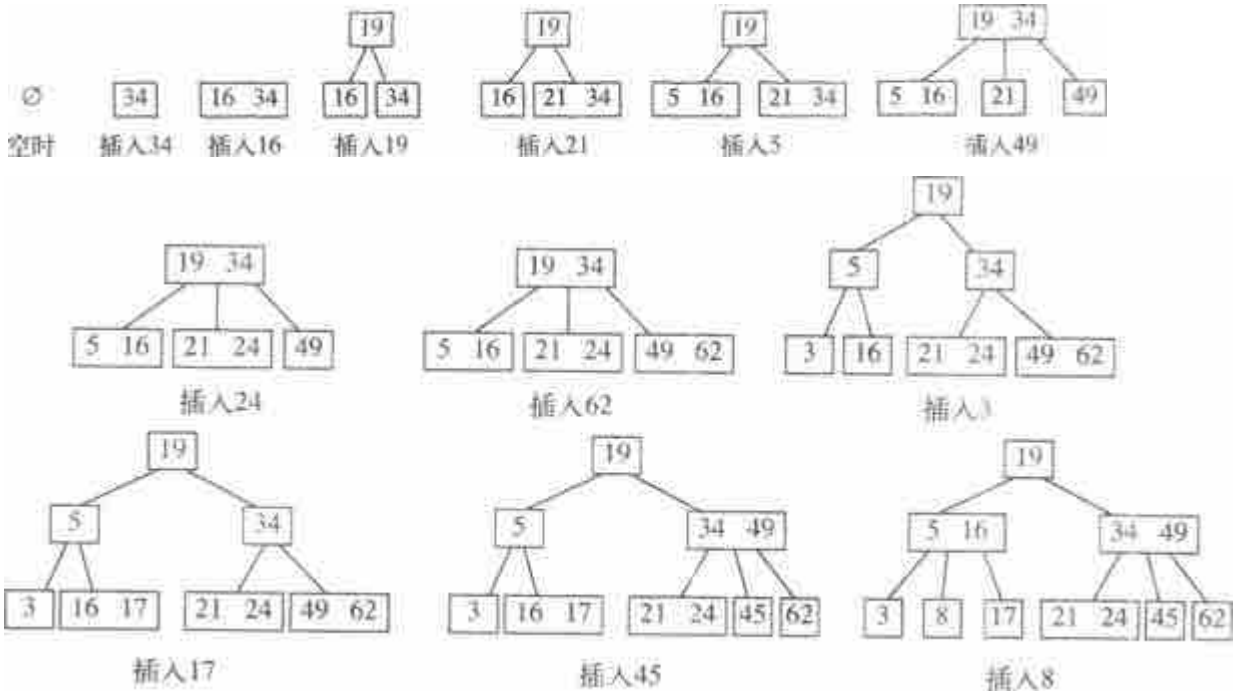
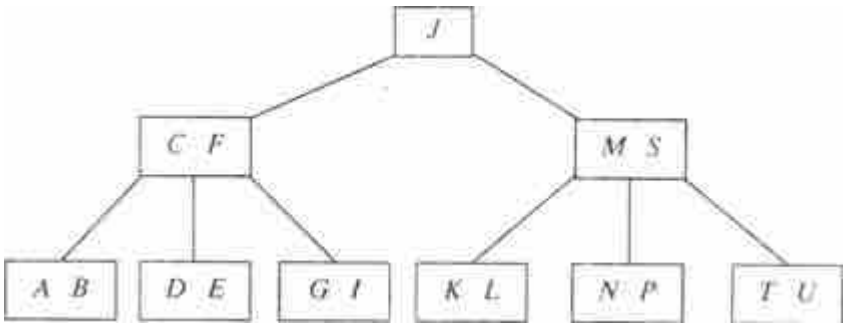
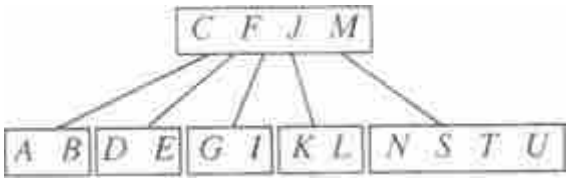


图 9-1

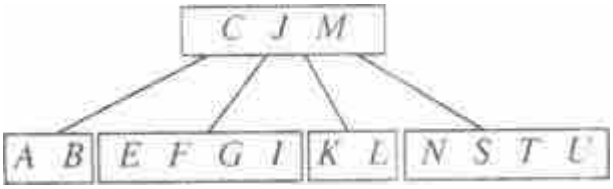
10．下图是 5 阶 B 树，画出删去 P 后的 B 树，再画出删去 D 后的 B 树。



答：删除 P 后



删除 D 后



11．输入一个正整数序列（53，17，12，66，58，70，87，25，56，60），试完成下列各题。

（1）按次序构造一棵二叉排序树 BS。

（2）依此二叉排序树，如何得到一个从大到小的有序序列?

（3）画出在此二叉排序树中删除“66”后的树结构。

答：（1）构造的二叉排序树如图 9-2 所示：

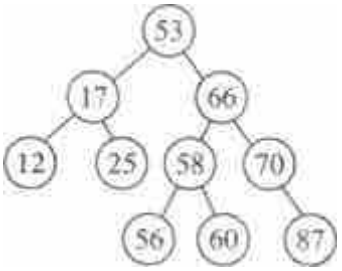


图 9-2 二叉排序树

（2）若二叉树非空，要得到一个从大到小的有序序列可以先中序遍历右子树；再访问根结点；最后中序遍历左子树。

（3）如图 9-3 所示：

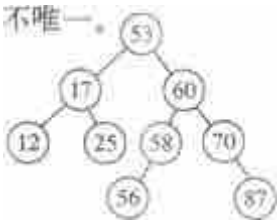


图 9-3

12．假定对有序表：（3，4，5，7，24，30，42，54，63，72，87，95）进行折半查找，试回答下列问题：

（1）画出描述折半查找过程的判定树；

（2）若查找元素 54，需依次与哪些元素比较?

（3）若查找元素 90，需依次与哪些元素比较?

（4）假定每个元素的查找概率相等，求查找成功时的平均查找长度。

答：（1）判定树如图 9-4 所示：

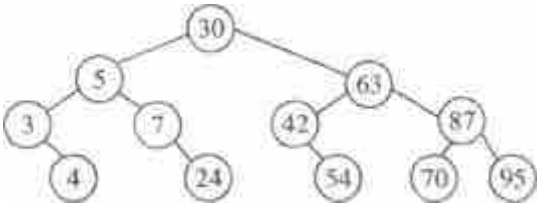


图 9-4 判定树

（2）若查找元素 54，需依次和元素 30、63、42、54 比较，查找成功。

（3）若查找元素 90，需依次和元素 30、63、87、95 比较，查找失败。

（4）$ASL_{succ}=(1*1+2*2+4*3+5*4)/12=37/12$

五、算法设计题

1．请编写一个判别给定二叉树是否为二叉排序树的算法，设二叉树用 llink—rlink 法存储。

答：算法如下：

```
#define true 1
#define false 0
typedef struct node
  {datatype data; struct node *llink,*rlink;} *BTree;
 void  JudgeBST(BTree t,int  flag)
// 判断二叉树是否是二叉排序树,本算法结束后,在调用程序中由flag得出结论
{ if (t!=null && flag)
      { Judgebst(t->llink,flag); // 中序遍历左子树
        if(pre==null)pre=t; // 中序遍历的第一个结点不必判断
        else  if(pre->data<t->data)pre=t; //前驱指针指向当前结点
                else{flag=flase; }    //不是完全二叉树
        Judgebst (t->rlink,flag); // 中序遍历右子树
}//JudgeBST算法结束

int   JudgeBST(BTree t)
 //判断二叉树t是否是二叉排序树,若是,返回true,否则,返回false
 {if(t==null)return   true;
   if (Judgebst(t->llink)&& Judgebst(t->rlink))//若左右子树均为二叉排序树
     {m=max(t->llink); n=min(t->rlink); //左子树中的最大值和右子树中的最小值
       return  (t->data>m && t->data<n); }
   else  return  false; //不是二叉排序树
  }//结束JudgeBST
int   max(BTree p)//求二叉树左子树的最大值
  {if(p==null) return   -maxint; //返回机器最小整数
    else{while(p->rlink! =null)p=p->rlink;
         return  p->data; }
   }
int   min(BTree p)//求二叉树右子树的最小值
  {if(p==null) return    maxint; //返回机器最大整数
    else{while(p->llink! =null)p=p->llink;
         return    p->data; }
   }
```

2．假设 K1，…，Kn 是 n 个关键词，试解答：

（1）试用二叉查找树的插入算法建立一棵二叉查找树，即当关键词的插入次序为 K1，K2，…，Kn 时，用算法建立一棵以 llink—rlink 链接表示的二叉查找树。

答: 算法如下：

```
BSTree BST_Search(BSTree bst,f,Element K,int *tag)
    //在二叉排序树bst上查找值为K的结点,返回其双亲结点指针f
     {if(!bst){*tag=0; return(f);}
      else if(bst->key==K){*tag=1; return(bst);}
             else if(bst->key>K)
                    {f=bst;return BST_Search(bst->llink,f,K, tag);}
                  else{f=bst;return BST_Search(bst->rlink,f,K, tag);}
      }// BST_Search
       BSTree Creat_BST(ElemType R[],int n)
        //以存储在数组R中的n个关键字,建立一棵初始为空的二叉排序树
         {int *flag=0; BSTree t;
          for(i=1;i<=n;i++)
          {f=null;
           f=BST_Search(t,f,K[i], flag);
           if(*tag==1) tag=0; //不再插入相同值结点
           else{s=(BSNode *)malloc(sizeof(BiNode));// 申请结点空间
                s->key=K[i];s->llink=null;s->rlink=null;
                if(f==null)t=s                          //根结点
                else if(s->key<f->data)f->llink=s;//左子女


                else  f->rchild=s;                  //右子女
           }//else
     }//for  ;return t;
}//结束算法
```

（2）设计一个算法，打印出该二叉查找树的嵌套括号表示结构。例如，K1=B，K2=A，K3=D，K4=C，K5=E，则用二叉查找树的插入算法建立如图 9-5 所示的二叉查找树。

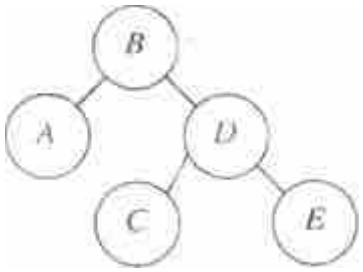


图 9-5

该二叉查找树的嵌套括号表示结构为：B（A，D（C，E））。

答：算法如下：

```
void Print(BSTree  t)
//以嵌套括号表示结构打印二叉排序树
   {if(t!=null)
    {printf(t->data);                  //打印根结点值
     if(t->lchild || t->rchild);       //左子女和右子女中至少有一个不空
      {printf("(");                    //输出左括号
       Print(t->lchild);               //输出左子树的嵌套括号表示
       if(t->rchild)printf(",");       //若右子树不空,输出逗号
       Print(t->rchild);               //输出右子树的嵌套括号表示
       printf(")");                    //输出右括号
     }//if}//if }//Print
```

3．元素集合已存入整型数组 A[1．．n]中，试写出依次取 A 中各值 A[i]（1≤i≤n）构造一棵二叉排序树 T 的非递归算法：CSBT（r，A）

答：算法如下：

```
void  CSBT(BSTree T,ElemType A[],int n)
      //以存储在数组K中的n个关键字,建立一棵初始为空的二叉排序树
      {for(i=1;i<=n;i++)
         {p=T;f=null;//在调用Creat_BST时,T=null
          while(p!=null)
               if(p->data<A[i]){f=p;p=p->rchild; }       // f是p的双亲
               else if(p->data>A[i]){f=p;p=p->lchild;}
          s=(BSTree)malloc(sizeof (BiNode));             // 申请结点空间
          s->data=A[i];s->lchild=null;s->rchild=null;
          if(f==null)T=s;                                //根结点
          else if(s->data<f->data)f->lchild=s;           //左子女
               else  f->rchild=s;                  //右子树根结点的值大于等于根结点的值
          }//算法结束
```

4．设排序二叉树中结点的结构为下述三个域构成：

Data：给出结点数据的值；left：给出本结点的左儿子结点的地址；right：给出本结点的右儿子结点的地址。设 data 域为正整数，该二叉树根结点地址为 T。现给出一个正整数 x。请编写非递归程序，实现将 data 域之值小于等于 x 的结点全部删除掉。

答：算法如下：

```
typedef struct node
  {int data; struct node *left,*right;}BiTNode,*BSTree;
void DelTree(BSTree r)//非递归删除以r为根的二叉排序树
  {BSTree S[];        //栈,容量足够大,栈中元素是二叉排序树结点的指针
   top=0;
   while(r!=null || top>0)
    {while(r!=null) {S[++top]=r;r=r->left;}// 沿左分枝向下
     if(top>0) //出栈,沿栈顶结点的右子树向下删除,释放被删除结点空间
       {p=S[top--];r=p->right;free(p);}
    }//while }// DelTree
    void DeleteAllx(BSTree T,int x)//在二叉排序树T中删除所有小于等于x的结点
    {BSTree p=T,q;
    while(T && T->data<=x) //根结点的值小于等于x
     {p=T;T=T->right;p->right=null;
       DelTree(p); //删除二叉树p,删除持续到"根"结点值大于x或T为空树为止
     }
   if(T)
    {q=T;p=T->left;
     while(p && p->data>x) //沿根结点左分支向下,查小于等于x的结点
       {while(p && p->data>x){q=p;p=p->left;}//q记p的双亲
        if(p) //p结点的值小于等于x
          {q->left=p->right;p->right=null;DelTree(p); }
        p=q->left;// 再查原p的右子树中小于等于x的结点
       }//while
 }//if }// DeleteAllx
```

5．已知二叉树丁的结点形式为（llink，data，count，rlink），在树中查找值为 X 的结点，若找到，则记数（count）加 1；否则，作为一个新结点插入树中，插入后仍为二叉排序树，写出其非递归算法。

答：算法如下：

```
typedef struct node
   {ElemType data;int count;struct  node *llink,*rlink;
   }BiTNode,*BSTree;
 void Search_InsertX(BSTree t,ElemType X)
      //在二叉排序树t中查找值为X的结点,若查到,则其结点的count域值增1,否则,将其
      //插入到二叉排序树中
       {p=t;
        while(p!=null && p->data!=X)  //查找值为X的结点,f指向当前结点的双亲
           {f=p;
            if(p->data<X) p=p->rlink; else p=p->llink;
```

```
    }
  if(!p) // 无值为x的结点,插入之
    {p=(BiTNode *)malloc(sizeof (BiTNode));
     p->data=X;p->llink=null;p->rlink=null;
     if(f->data>X)f->llink=p;else f->rlink=p;
    }
  else  p->count++;// 查询成功,值域为X的结点的count增1
}// Search_InsertX
```

6．设二叉排序树的各元素值均不相同，采用二叉链表作为存储结构，试分别设计递归和非递归算法按递减序打印所有左子树为空，右子树非空的结点的数据域的值。

答：（1）递归算法如下：

```
void  DecPrint(BSTree t)
  //递减序输出二叉排序树t中所有左子树为空右子树非空的结点数据域的值
   {if(t)
      {DecPrint(t->rchild);
       if(!t->lchild && t->rchild)printf(t->data: 4)
       DecPrint(t->lchild);

                    }//if  }//DecPrint
```

（2）非递归算法如下：

```
void  DecPrint(BSTree t)
    //递减序输出二叉排序树t中所有左子树为空、右子树非空的结点的数据域的值
 {BSTree S[];//S是二叉排序树结点指针的栈,容量足够大
  int  top=0;
  while(t || top>0)
    {while(t){S[++top]=t;t=t->rchild ;} //沿右分支向下
     if(top>0)
       {t=S[top--];
        if(!t->lchild && t->rchild)printf(t->data: 4);
        t=t->lchild;  // 去左分支
     }//if }//while }//算法结束
```

7．编写对有序表进行顺序查找的算法，并画出对有序表进行顺序查找的判定树。假设每次查找时的给定值为随机值，又查找成功和不成功的概率也相等，试求进行每一次查找时和给定值进行比较的关键字个数的期望值。

答：算法如下：

```
int  Search(rectype  R[],int  n,K)
//在具有n个元素的有序表R中，顺序查找值为K的结点，查找成功返回其位置
//否则返回-1表示失败
  {i=0;
   while(i<n)
     {if(R[i]==K) return (i);//元素序号
      else if(R[i]>K) return (-1);
      i++;
     }//while
    return  (-1);
   }//结束search
```

期望值分析：在等概率情况下，则查找成功的平均查找长度为（n+1）/ 2，查找失败的平均查找长度为（n+2）/ 2（失败位置除小于每一个，还存在大于最后一个）。若查找成功和不成功的概率也相等，则查找成功时和关键字比较的个数的期望值约为（n+1）/ 4。

8．在二叉排序树的结构中，有些数据元素值可能是相同的，设计一个算法实现按递增有序打印结点的数据域，要求相同的数据元素仅输出一个，算法还应能报出最后被滤掉而未输出的数据元素个数，对如图 9-6 所示的二叉排序树，输出为：10，12，13，15，18，21，27，35，42．滤掉 3 个元素。

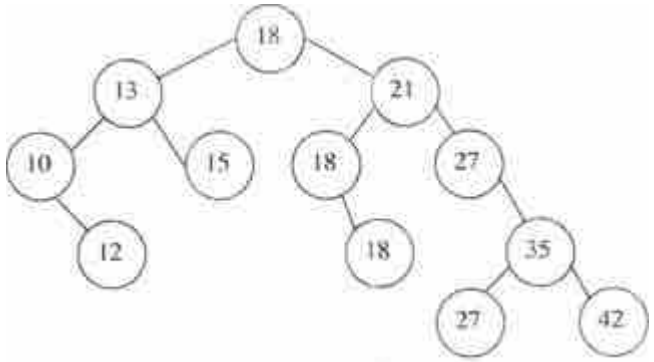


图 9-6

答：算法如下：

```
            void  BSTPrint(BSTree t,int *count)
//递增序输出二叉排序树中结点的值，滤去重复元素
    {if(t)
        {BSTPrint(t->lchild);//中序遍历左子树
         if(pre==null)pre=t;  //pre是当前访问结点的前驱，调用本算法时初值为null
         else if(pre->key==t->key) *count++;      //*count记重复元素，调用本算法时初
         值为0
            else{printf("%4d",t->key);pre=t;} // 前驱后移
         BSTPrint(t->rchild); //中序遍历右子树
       }//if  }//结束BSTPrint算法
```

9．已知某哈希表 HT 的装填因子小于 1，哈希函数 H（key）为关键字的第一个字母在字母表中的序号。

（1）处理冲突的方法为线性探测开放地址法。编写一个按第一个字母的顺序输出哈希表中所有关键字的程序。

（2）处理冲突的方法为链地址法。编写一个计算在等概率情况下查找不成功的平均查找长度的算法。注意，此算法中规定不能用公式直接求解计算。

答：（1）算法如下：

```
void Print(rectype h[])//按关键字第一个字母在字母表中的顺序输出各关键字
 {int   i,j;
   for(i=1;i<=26;i++)     // 哈希地址1-26
      {j=1;printf("\n");
       while(h[j]!=null) // 设哈希表初始值为null
        {if(ord(h[j])==i)// ord()取关键字第一字母在字母表中的序号
         printf("%s",h[j]);
        j=(j+1)% n;
        }//while
    }//for }//end
```

（2）算法如下：

```
int ASLHash(rectype h[])//求链地址解决冲突的哈希表查找不成功时平均查找长度
  {int i,j;count=0;//记查找不成功的总的次数
    LinkedList  p;
    for(i=1;i<=26;i++)
      {p=h[i];j=1;//按我们约定,查找不成功指到空指针为止
       while(p!=null){j++;p=p->next;}
       count+=j;
      }
  return  (count/26.0);
}
```

10．设给定关键字输入序列为（100，90，120，60，78，35，42，31，15）用哈希法散列0-10 的地址区间。要求设计一合理的哈希函数；冲突时用链表法解决，写出哈希算法，并构造出哈希表，在等概率查找情况下查找成功的平均查找长度是多少?

答：算法如下：

```
#define m 11
typedef struct node
    {ElemType key;      //关键字
     struct node *next;//链指针
    }Lnode,*LinkedList;
typedef ElemType int;
typedef struct node *HLK;
void  Creat(HLK HT[m])
   //用链地址法解决冲突,构造哈希表,哈希函数用H(key)=key % 11
    {for(i=0;i<m;i++) H[i]=null;          //初始化
     for(i=0;i<n;i++)                     //输入n(本例中n=9)个关键字
      {scanf("%d",&x);                    //按题意x互不相同
       j=x % m;
       p=(LNode*)malloc(sizeof (LNode));
       p->key=x;p->next=HT[j];HT[j]=p;//将插入结点链入同义词表
      }
     }//Creat
```

构造的哈希表如图 9-7 所示：

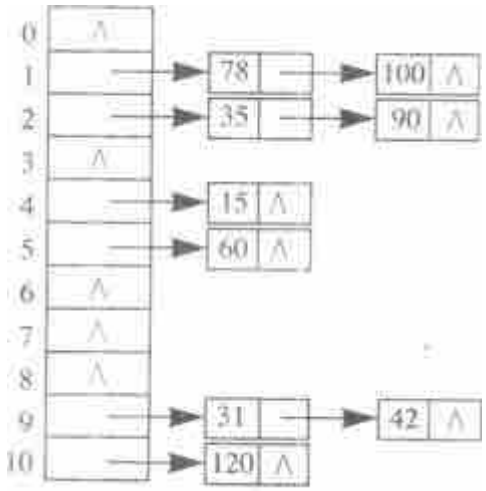


图 9-7 构造的哈希表

查找成功时的平均查找长度 ASL=12 / 9。

11．已知顺序表中有 m 个记录，表中记录不依关键字有序排列，编写算法为该顺序表建立一个有序的索引表，索引表中的每一项含记录的关键字和该记录在顺序表中的序号，要求算法的时间复杂度在最好的情况下能达到 O（m）。

答：算法如下：

```
#define m   //顺序表中记录个数
typedef struct node
  {keytype key;//关键字
   int  adr;    //该关键字在顺序表中的下标
  }idxnode;     //索引表的一项
```

```
typedef struct node
    {keytype key;  //关键字
     anytype other;//记录中的其他数据
    }ElemType;
void IndexT(idxnode index[m+1],ElemType seq[m+1])
    //给有m个记录的顺序表seq建立索引表index
      {index[1].key=seq[1].key; index[1].adr=1;
       for(i=2,i<=m,i++)
        {j=i-1;
         index[0].key=seq[i].key;          //监视哨
         while(index[j].key<index[0].key){index[j+1]=index[j];j--;}
         index[j+1].key=index[0].key;      //关键字放入正确位置
         index[j+1].adr=i;                 //第i个记录的下标
         }//for }//IndexT
```

12．有二叉排序树采用二叉链表方式存放，树中结点值各不相同，欲得到一个由大到小的结点值递减序列，简述处理方法思路，用非递归形式写出算法。

答：算法如下：

```
void InOrder(BSTree bt)//按递减次序输出二叉排序树结点的值
{BiTree s[],p=bt;       //s是元素为二叉树结点指针的栈,容量足够大
 int top=0;
  while(p || top>0)
       {while(p) {s[++top]=p; bt=p->rchild;} //沿右子树向下
        if(top>0){p=s[top--]; printf(p->data); p=p->lchild;}
}    }//结束
```

13．从键盘上输入一串正整数，最后输入-l 作为结束标志。如：8，7，l，22，98，46，…，75，-l。请设计一个非递归程序，创建一棵二叉排序树，并且该二叉排序树也必须是中序线索二叉树。设该二叉排序树上的结点结构为（teft，ltag，data，rta9，right）。其中：data 域为结点的数据场。Itag=0，那么 left 域中存在的是该结点的左儿子结点的地址。ltag=1，那么 left 域中存放的是该结点的按中序遍历次序的前驱结点的地址。rtag=0，那么 right 域中存放的是该结点的右儿子结点的地址。rtag=1，那么 right 域中存放的是该结点的按中序遍历次序的后继结点地址。

答：算法如下：

```
typedef struct node
   {int ltag,rtag,data;struct node *left,*right;}TNode,*BSTree;
BSTree Create()
//从键盘上输入一串正整数,建立一棵初始为空的二叉排序树,同时也是线索二叉树
{int x; BSTree bst=null;
 scanf("%d",&x);
 while(x!=-1)
  {s=(BSTree)malloc(sizeof(TNode));                    //申请结点空间
   s->data=x;s->left=s->right=null;s->ltag=s->rtag=1 //结点赋值,其线索初始化
   p=bst;f=null;
   while(p!=null) //查找结点的插入位置
         if(p->data>x){f=p;p=p->left; } //f是p的双亲
         else {f=p;p=p->right;}
   if(f==null) bst=s;                          //根结点
   else if(s->data<f->data)                    //左子女
         {s->left=f->left; s->right=f;  //修改线索
          f->ltag=0; f->left=s;
         }
         else   //右子树根结点的值大于等于根结点的值
          {s->left=f; s->right=f->right;//修改线索
           f->rtag=0; f->right=s;
          }
         scanf("%d",&x);   //读入下个数
      }//while
  return bst;
}//算法结束
```

14. 编写递归算法，从大到小输出给定二叉排序树中所有关键字不小于 X 的数据元素。要求你的算法的时间复杂度为 O（$\log_2$（x+m）），其中，2 为排序树中所含结点数，m 为输出的关键字个数。

答：算法如下：

```
void output(BSTree bst,int x)
  //从大到小输出二叉排序树bst中所有关键字不小于x 的数据元素
  {if(bst)
   {if(bst->data>=x) output(bst->rchild,x);
    if(bst->data>=x) printf("%4d",bst->data);
    if(bst->data>=x) output(bst->lchild,x);
  }//if  }//output
```

15. 写一算法找出 n 个数的最大值和最小值，要求最坏条件下的元素比较次数为 é3n / 2ù-2。

答：算法如下：

```
void MaxMinSele(SeqList r)
   //用最多 3n/2 -2次比较在n个元素r中选出最大值和最小值
   {for(i=0,j=n-1-i;i<j;i++,j--)
      if(r[i].key>r[j].key) r[i]<-->r[j];
   if(i>j) {i--; j++;}//n为偶数时
   k=0;
   for(m=k+1;m<=i;m++)
     if(r[m].key<r[k].key) k=m;
   printf("最小值=%d\n",r[k].key);
   k=j;
   for(m=j+1;m<n;m++)
    if(r[m].key>r[k].key) k=m;
   printf("最大值=%d\n",r[k].key);
}
```

第 10 章　内部排序

一、选择题

1．排序算法的稳定性是指（　　）。

A．经过排序之后，能使值相同的数据保持原顺序中的相对位置不变

B．经过排序之后，能使值相同的数据保持原顺序中的绝对位置不变

C．算法的排序性能与被排序元素的数量关系不大

D．算法的排序性能与被排序元素的数量关系密切

【答案】A 查看答案

【解析】假定在待排序的记录序列中，存在多个具有相同的关键字的记录，若经过排序，这些记录的相对次序保持不变，即在原序列中，ri=rj，且 ri 在 rj 之前，而在排序后的序列中，ri 仍在 rj 之前，则称这种排序算法是稳定的；否则称为不稳定的。

2．下面给出的四种排序方法中，排序过程中的比较次数与排序方法无关的是（　　）。

A．选择排序法　　B．插入排序法　　C．快速排序法　　D．堆排序法

【答案】A 查看答案

【解析】选择排序的基本思想是：

第 i 趟排序开始时，当前有序区和无序区分别为 R[0．．i-1]和 R[i．．n-1]（_0≤i<n-1），该趟排序则是从当前无序区中选出关键字最小的记录 R[k]，将它与无序区的第 1 个记录 R[i]交换，使 R[0．．i]和 R[i+1．．n-1]分别变为新的有序区和新的无序区。

3．设被排序的结点序列共有 N 个结点，在该序列中的结点已十分接近排序的情况下，用直接插入法、归并法和一般的快速排序法对其排序，这些算法的时间复杂性应为（　　）。

A．O（N），O（N），O（N）　　B．O（N），O（$N*\log_2N$），O（$N*\log_2N$）

C．O（N），O（$N*\log_2N$），O（N^2）　　D．O（N^2），O（$N*\log_2N$），O（N^2）

【答案】C 查看答案

【解析】因为该序列中的结点已经十分接近排序的情况，对于直接插入法，大部分结点只需要直接插入后面即可，因此时间复杂度为 O（N）。对于采用归并法，它是一种稳定的排序方法，它的时间复杂度为 O（$N\log_2N$）。对于一般的快速排序法，序列越接近有序，所需要的比较次数越多，此时的时间复杂度为 O（N^2）。

4．数据序列（8，9，10，4，5，6，20，1，2）只能是下列排序算法中的（　　）的两趟排序后的结果。

A．选择排序　B．起泡排序　C．插入排序　D．堆排序

【答案】C 查看答案

【解析】选择排序、起泡排序和堆排序两趟排序后，在序列的某一端应该有序列的两个最大值或者最小值。

5．下列序列中，（　　）是执行第一趟快速排序后所得的序列。

A．[68，11，18，69][23，93，73]　B．[68，11，69，23][18，93，73]

C．[93，73][68，11，69，23，18]　D．[68，11，69，23，18][93，73]

【答案】C 查看答案

【解析】快速排序将数据划分成两部分，其中一部分关键字比另一部分关键字小。

6．一组记录的关键码为（46，79，56，38，40，84），则利用快速排序的方法，以第一个记录为基准得到的一次划分结果为（　　）。

A．（38，40，46，56，79，84）　B．（40，38，46，79，56，84）

C．（40，38，46，56，79，84）　D．（40，38，46，84，56，79）

【答案】C 查看答案

【解析】快速排序是将待排记录分割成独立的两部分，其中一部分的关键字均比另一部分记录的关键字小。

第一次比较：46 比 84 小，不交换；

第二次比较：40 比 46 小，交换，此时为（40，79，56，38，46，84）；

第三次比较：46 比 79 小，交换，此时为（40，46，56，38，79，84）；

第四次比较：38 比 46 小，交换，此时为（40，38，56，46，79，84）；

第五次比较：56 比 46 大，交换，此时为（40，38，46，56，79，84）；

一次划分结束。

7．就平均性能而言，目前最好的内排序方法是（　　）排序法。

A．起泡　B．希尔插入　C．交换　D．快速

【答案】D 查看答案

【解析】快速排序的平均时间复杂度是 nlogn，所需要的辅助存储为 O(logn)，虽然堆排序的时间复杂度也是 nlog（n），所需要的辅助存储为 O（1），看似堆排序比快速排序的性能好，但是需要注意 nlog（n）仅仅表示的是一个量级，比如 n（logn）和 100*n（logn）的量级都为 nlog（n）。之所以说快排最好，是在综合考虑的情况下。

8．在文件“局部有序”或文件长度较小的情况下，最佳内部排序的方法是（　　）。

A．直接插入排序　B．起泡排序　C．简单选择排序　D．快速排序

【答案】A 查看答案

【解析】当待排序列基本有序时，对冒泡排序来说，若最大关键字位于序列首部，则每趟排序仅能使其“下沉”一个位置，要使其下沉到底部仍需 n-1 趟排序，也即时间复杂度仍为 O(n2)。而对简单选择排序来说，其比较次数与待排序列的初始状态无关；归并排序要求待排序列已经部分有序，而部分有序的含义是待排序列由若干有序的子序列组成，即每个子序列必须有序，并且其时间复杂度为 $O(n \log_2 n)$；直接插入排序在待排序列基本有序时，每趟的比较次数大为降低，也即 n-1 趟，比较的时间复杂度由 $O(n^2)$降至 O(n)。

9.用直接插入排序方法对下面 4 个序列进行排序（由小到大），元素比较次数最少的是（　　）。

A．94，32，40，90，80，46，21，69　B．32，40，21，46，69，94，90，80

C．21，32，46，40，80，69，90，94　D．90，69，80，46，21，32，94，40

【答案】C 查看答案

10．直接插入排序在最好情况下的时间复杂度为（　　）。
A．O（logn）　B．O（n）　C．O（n*logn）D．O（n^2）
【答案】B 查看答案
【解析】当序列是按照直接插入排序的顺序有序时，此时进行插入时，每次都只需要和末尾的一个元素进行比较，此时的时间复杂度最好，为 O（n）。
11．采用简单选择排序，比较次数与移动次数分别为（　　）。
A．O（n），O（logn）　B．O（logn），O（n*n）
C．O（n*n），O（n）　D．O（nlogn），O（n）
【答案】C 查看答案
【解析】简单选择排序只在要交换的时候交换位置，及移动位置，共需移动 n 次。而需要比较的次数为 n-1+n-2+...+1=n(n-1)/2。
12．对序列{15，9，7，8，20，-1，4，}用希尔排序方法排序，经一趟后序列变为{15，-1，4，8，20，9，7}则该次采用的增量是（　　）。
A．1　B．4　C．3　D．2
【答案】B 查看答案
【解析】由所给的序列知，本序列要进行递增排序，经过一趟后 15 的位置没有变化，而给的序列中只有 20 比 15 大，20 的位置和 15 的位置相差 4。所以该次采用的增量是 4。
13. 对关键码序列 28，16，32，12，60，2，5，72 快速排序，从小到大一次划分结果为（　　）。
A．（2，5，12，16）26（60，32，72）　B．（5，16，2，12）28（60，32，72）
C．（2，16，12，5）28（60，32，72）　D．（5，16，2，12）28（32，60，72）
【答案】B 查看答案
【解析】快速排序是将待排记录分割成独立的两部分，其中一部分的关键字均比另一部分记录的关键字小。
第一次比较：28 比 72 小，不交换；
第二次比较：28 比 5 大，交换，此时为（5，16，32，12，60，2，28，72）；
第三次比较：16 比 28 小，不交换；
第四次比较：32 比 28 大，交换，此时为（5，16，28，12，60，2，32，72）；
第五次比较：28 比 2 大，交换，此时为（5，16，2，12，60，28，32，72）；
第六次比较：28 比 12 大，不交换；
第七次比较：28 比 60 小，交换，此时为（5，16，2，12，28，60，32，72）；
一次划分结束。
14. 对 n 个记录的线性表进行快速排序为减少算法的递归深度，以下叙述正确的是（　　）。
A．每次分区后，先处理较短的部分　　B．每次分区后，先处理较长的部分
C．与算法每次分区后的处理顺序无关　D．以上三者都不对
【答案】A 查看答案
【解析】令递归函数为 f，第一次进行递归函数认为递归深度为 1，以后从深度为 n 的递归函数 f 中再调用递归函数 f，此时深度为 n+1。整个 f 的最大深度为递归深度。
15. 在含有 n 个关键字的小根堆（堆顶元素最小）中，关键字最大的记录有可能存储在（　　）位置上。
A．ën / 2」　B．ën / 2」-1　C．1 D．ën / 2」+2
【答案】D 查看答案
【解析】小根堆中，关键字最大的记录只能在叶结点上，故不可能在小于等于 ën / 2」的结点上。
16．下列四个序列中，哪一个是堆（　　）？

A．75，65，30，15，25，45，20，10
B．75，65，45，10，30，25，20，15
C．75，45，65，30，15，25，20，10
D．75，45，65，10，25，30，20，15
【答案】C 查看答案
【解析】堆的定义：
n 个关键字序列 K_1，K_2，…，K_n称为堆，当且仅当该序列满足如下性质（简称为堆性质）：
①$K_i \le K_{2i}$且 $K_i \le K_{2i+1}$
②$K_i \ge K_{2i}$且 $K_i \ge K_{2i+2}$（$1 \le i \le [n/2]$）
小根堆：满足第①种情况的堆；
大根堆：满足第②种情况的堆。
根据堆定义即可得出答案。
17．从堆中删除一个元素的时间复杂度为（　　）。
A．O（1）　B．O（$\log_2 n$）　C．O（n）　D．O（$n\log_2 n$）
【答案】B 查看答案
【解析】堆中删除一个元素，需要重新调整堆，其时间复杂度为 O（$\log_2 n$）。
18．在对 n 个元素的序列进行排序时，堆排序所需要的附加存储空间是（　　）。
A．O（$\log_2 n$）　B．O（1）　C．O（n）　D．O（$n\log_2 n$）
【答案】B 查看答案
【解析】堆排序需要一个空间用于交换，因此堆排序所需要的附加存储空间为 O（1）。
19．归并排序中，归并的趟数是（　　）。
A．O（n）　B．O（logn）　C．O（nlogn）　D．O（n*n）
【答案】B 查看答案
【解析】不妨设归并的趟数为 m，第一次归并每组有两个元素，最后一次归并只剩下一组，这组的元素个数为 n。因此每次归并元素的个数增加一倍。所以 $2^{m-1}<n<=2^m$。所以归并的趟数为 O(logn)。
20．基于比较方法的 n 个数据的内部排序。最坏情况下的时间复杂度能达到的最好下界是（　　）。
A．O（nlogn）　B．O（logn）　C．O（n）　D．O（n*n）
【答案】A 查看答案
【解析】在内部排序中，最坏情况下的时间复杂度为 O（nlogn）。
21．已知待排序的 n 个元素可分为 n / k 个组，每个组包含 k 个元素，且任一组内的各元素均分别大干前一组内的所有元素和小于后一组内的所有元素，若采用基于比较的排序，其时间下界应为（　　）。
A．O（$n\log_2 n$）　B．O（$n\log_2 k$）　C．O（$k\log_2 n$）　D．O（$k\log_2 k$）
【答案】B 查看答案
【解析】因组与组之间已有序，故将 n / k 个组分别排序即可，基于比较的排序方法每组的时间下界为 O（$k\log_2 k$），全部时间下界为 O（$n\log_2 k$）。
22．对{05，46，13，55，94，17，42}进行基数排序，一趟排序的结果是：
A．05，46，13，55，94，17，42　B．05，13，17，42，46，55.94
C．42，13，94，05，55，46，17　D．05，13，46，55，17，42，94
【答案】C 查看答案
【解析】基数排序有两种：最低位优先和最高位优先。
①最低位优先的过程

先按最低位的值对记录进行排序，在此基础上，再按次低位进行排序，依此类推。由低位向高位，每趟都是根据关键字的一位并在前一趟的基础上对所有记录进行排序，直至最高位，则完成了基数排序的整个过程。

②以 r 为基数的最低位优先排序的过程

假设线性表由结点序列 a_0，a_1，…，a_{n-1}构成，每个结点 a_j的关键字由 d 元组（k，k…，k，k）组成，其中 0≤k≤r-1（0≤j<n，0≤i≤d-1）。在排序过程中，使用 r 个队列 Q_0，Q_1，…，Q_{r-1}。排序过程就是对 i=0，1，…，d-1，依次做一次“分配”和“收集”。

分配：开始时，把 Q_0，Q_1，…，Q_{r-1}各个队列置成空队列，然后依次考察线性：表中的每一个结点 a_j（==0.1，…，n-1）。如果 a_j的关键字 k=k，就把 a_j放进 Q_k队列中。

收集：把 Q_0，Q_1，…，Q_{r-1}各个队列中的结点依次首尾相接，得到新的结点序列，从而组成新的线性表。

二、判断题

1．内排序要求数据一定要以顺序方式存储。（　　）

【答案】×查看答案

【解析】由于待排序的记录数量不同，使得排序过程中涉及的存储器不同，可将排序方法分为两大类：一类是内部排序；另一类是外部排序。因此，内部排序没有要求数据一定是以顺序方式存储。

2．在执行某个排序算法过程中，出现了排序码朝着最终排序序列位置相反方向移动，则该算法是不稳定的。（　　）

【答案】×查看答案

【解析】例如起泡排序是稳定排序，将 4，3，2，1 按起泡排序排成升序序列，第一趟变成 3，2，1，4，此时 3 就朝向最终位置的相反方向移动。

3．两分法插入排序所需比较次数与待排序记录的初始排列状态相关。（　　）

【答案】×查看答案

【解析】折半插入排序所需的附加存储空间和直接插入排序相同，从时间上比较，折半插入排序仅减少了关键字间的比较次数，而记录的移动次数不变。因此，折半插入排序的时间复杂度仍为 0（n^2），与待排序记录的初始排列状态无关。

4．在初始数据表已经有序时，快速排序算法的时间复杂度为 0（$n\log_2 n$）。（　　）

【答案】×查看答案

【解析】当初始数据表有序，此时快速排序所需要比较的次数最多，快速排序算法的时间复杂度为 $O(n^2)$。

5．在待排数据基本有序的情况下，快速排序效果最好。（　　）

【答案】×查看答案

【解析】在待排数据基本有序的情况下，插入排序效果最好。

6．堆肯定是一棵平衡二叉树。（　　）

【答案】×查看答案

【解析】堆是 n 个元素的序列，可以看成是完全二叉树，但相对于根并无左小右大的要求，故其既不是二叉排序树，更不会是平衡二叉树。

7．堆是满二叉树。（　　）

【答案】×查看答案

【解析】堆的定义：

n 个关键字序列 K_1，K_2，…，K_n称为堆，当且仅当该序列满足如下性质（简称为堆性质）：

①$K_i \le K_{2i}$且 $K_i \le K_{2i+1}$

②$K_i \ge K_{2i}$且 $K_i \ge K_{2i+2}$（$1 \le i \le [n/2]$）

小根堆：满足第①种情况的堆；
大根堆：满足第②种情况的堆。
因此并不能保证堆是满二叉树。
8．在用堆排序算法排序时，如果要进行增序排序，则需要采用“大根堆”。（　　）
【答案】√查看答案
【解析】堆排序：基本思想先将原始序列构造成一个堆(初始堆)，使其 n 个元素的最大(小)值处于序列的第一个位置；然后交换序列第一个元素与最后一个元素的位置。
9．由于希尔排序的最后一趟与直接插入排序过程相同，因此前者一定比后者花费的时间更多。
【答案】×查看答案
【解析】希尔排序把序列分成很多小序列，对于直接插入排序，记录少时的效率会大大提高。并且序列在基本有序的情况下，直接插入排序也会提高。
10．若中序遍历平衡的二叉排序树，可得到排好序的关键码序列。（　　）
【答案】√查看答案
【解析】二叉排序树对于每一个结点，它的左子树的所有结点的值都小于这个结点的值，而它的右子树的所有结点的值都大于这个结点的值。而采用中序遍历，遍历顺序为左根右，因此可以得到按增序排列的关键码序列。
11．用希尔（She11）方法排序时，若关键字的初始排序杂乱无序，则排序效率就低。（　　）
【答案】×查看答案
【解析】希尔排序的基本思想是：先将整个待排记录序列分割成为若干子序列分别进行直接插入排序，待整个序列中的记录“基本有序”时，再对全体记录进行一次直接插入排序。
12．基数分类只适用于以数字为关键字的情况，不适用于以字符串为关键字的情况。（　　）
【答案】×查看答案
【解析】如果用字符串为关键字，可以将其中的字符串的每一位用 ASCII 码进行比较。
13．一个排序算法是否稳定，是指该算法在各种情况下的时间效率是否相差不大。（　　）
【答案】×查看答案
【解析】排序的稳定性指：假定在待排序的记录序列中，存在多个具有相同的关键字的记录，若经过排序，这些记录的相对次序保持不变，即在原序列中，ri=rj，且 ri 在 rj 之前，而在排序后的序列中，ri 仍在 rj 之前，则称这种排序算法是稳定的；否则称为不稳定的。

三、填空题

1．若不考虑基数排序，则在排序过程中，主要进行的两种基本操作是关键字的_____和记录的_____。
【答案】比较；移动查看答案
2．属于不稳定排序的有_____。
【答案】希尔排序、简单选择排序、快速排序、堆排序等查看答案
3．分别采用堆排序，快速排序，起泡排序和归并排序，对初态为有序的表，则最省时间的是_____算法，最费时间的是_____算法。
【答案】起泡；快速查看答案
【解析】当初态为有序表时，冒泡排序只需要进行一趟比较即可，此时时间复杂度为 O（n），而快速排序算法需要比较的次数达到最大，时间复杂度为 O（n^2）。
4．对 n 个记录的表 r[1．．n]进行简单选择排序，所需进行的关键字间的比较次数为_____。
【答案】n（n-1）／2 查看答案
【解析】第一次需要 n-1 次比较，第 i 此需要 n-i 此比较，所以共需要 n-1+n-2+...+1=n(n-1)/2。

5．设用希尔排序对数组{98，36，-9，0，47，23，1，8，10，7}进行排序，给出的步长（也称增量序列）依次是 4，2，1 则排序需_____趟，写出第一趟结束后，数组中数据的排列次序_____。

【答案】3；（10，7，-9，0，47，23，1，8，98，36）查看答案

6．从平均时间性能而言，_____排序最佳。

【答案】快速查看答案

【解析】快速算法的平均时间复杂度为 nlogn。

7．高度为 h 的堆中，最多有_____元素，最少有_____个元素。

【答案】2^h-1；2^{h-1}查看答案

【解析】当这个堆构成的是满二叉树时，元素的个数最多，元素个数为 2^h-1。当最后一层只有一个元素时，此时堆的元素个数最少，元素个数为 2^{h-1}。

8．堆是一种有用的数据结构。堆排序是一种___（1）___排序，堆实质上是一棵___（2）___结点的层次序列。对含有 N 个元素的序列进行排序时，堆排序的时间复杂度是___（3）___，所需的附加存储结点是___（4）___。关键码序列 05，23，16，68，94，72，71，73 是否满足堆的性质___（5）___。

【答案】选择；完全二叉树；O（$Nlog_2N$）；O（1）；满足堆的性质查看答案

9．对 n 个元素的序列进行起泡排序时，最少的比较次数是_____。

【答案】n-1 查看答案

【解析】如果序列是正序，冒泡排序第一次只要进行 n-1 次比较，发现没有移动元素，说明序列有序。

10．下列程序是快速排序的非递归算法，请填写适当的语句，完成该功能。

```
#define n 10
int split(int a[n],int low,int high)
    {int j,k,x; k=low; j=high; x=a[k];
     while(k<j)
        {while(a[j]>=x && k<j) j--;
            if(k<j) {a[k]=a[j]; k++;}
         while(a[k]<=x && k<j) k++;
            if(k<j) {   (1)   ; j--; }
          }
     a[k]=x; return k;
      }
void qsort(int a[n])
   {int k, low=0, high=n-1; int stack[20][2], top=-1;
    top++; stack[top][0]=low; stack[top][1]=high;
    while(top>=0)
      {   (2)   ; high=stack[top][1]; top--;
       k=split(a,low,high);
       if(low<high){top++; stack[top][0]=low; stack[top][1]=k-1;
                    top++;    (3)    ;          stack[top][1]=high;
                    }
      }    }
main()
   {int k,a[n]={2,8,3,6,9,5,1,4,0,7}; //a中存放待排序的关键字
    qsort(a);
    for(k=0; k<n; k++) printf(" %d",a[k]);
}
```

【答案】a[j]=a[k]；low=stack[top][0]；stack[top][0]=k+1 查看答案

【解析】快速排序(quick sort)的基本思想是，通过一趟排序将待排记录分割成独立的两部分，其中一部分记录的关键字均比另一部分记录的关键字小，则可分别对这两部分记录继续进行排序，以达到整个序列有序。

四、应用题

1．设有 n 个元素采用起泡排序法进行排序，通常需要进行多少趟排序?对于第 J 趟起泡通常需要进行多少次关键字比较?在程序设计中如何设置判断条件，有可能使起泡趟数可以减少并且能完成排序。

答：n 个元素采用起泡排序法进行排序，通常需要进行 n-1 趟排序。第 j 趟起泡排序要进行 n-j 次比较。在一趟排序中，若没有记录交换，则表示排序完成。因而，可通过设标记来控制排序结束，下面语句段说明了标记 flag 的使用。

```
int j=1,flag=1;        //flag用作控制标记
 while(j<n && flag) //j是起泡排序趟数,最多n-1趟
  {flag=0;          //设标记,若本趟不发生交换,本趟起泡排序后,算法结束
   for(i=1;i<=n-j;i++)
     if(r[i]>r[i+1]) {flag=1; r[i]<-->r[i+1];}//发生了交换,还要进行下趟
   }
```

2．设 LS 是一个线性表，LS=（a_1，a_2，…，a_n），若采用顺序存储结构，则在等概率的前提下，插入一个元素需要平均移动的元素个数是多少?若元素插在 a_i 与 a_{i+1} 之间（0≤i≤n-1）的概率为（n-i）/（n*（n+1）/ 2），则插入一个元素需要平均移动的元素个数又是多少?

答：需要分两种情况讨论：

（1）等概率（后插），插入位置 0．．n，则平均移动个数为 n / 2。

（2）若不等概率，则平均移动的元素个数为

$$\sum_{i=0}^{n-1}(n-i)/(n*(n+1)/2)*(n-i)=\frac{2n+1}{3}$$

3．在堆排序、快速排序和合并排序中：

（1）若只从存储空间考虑，则应首先选取哪种排序方法，其次选取哪种排序方法，最后选取哪种排序方法?

（2）若只从排序结果的稳定性考虑，则应选取哪种排序方法?

（3）若只从平均情况下排序最快考虑，则应选取哪种排序方法?

（4）若只从最坏情况下排序最快并且要节省内存考虑，则应选取哪种排序方法?

答：（1）应首先选取堆排序方法，其次选取快速排序方法，最后选取归并排序方法。

（2）应选取归并排序方法

（3）应选取快速排序方法

（4）应选取堆排序方法

4．请阅读下列算法，回答问题。

```
PROCEDURE  sort(r,n);
BEGIN
  FOR  i:=2  TO  n DO
    BEGIN
      x:=r[i];r[0]:=x;j:=i-1;
      WHILE  x.key<r[j].key  DO

        BEGIN
            r[j+1]:=r[j]; j:=j-1
        END;
    r[j+1]:=x
  END
END;
```

问题一：这是什么类型的排序算法，该排序算法稳定吗?

问题二：设置 r[0]的作用是什么?若将 WHILE--DO 语句中判断条件改为 X. key<=r[j]. KEY，该算法将会有什么变化，是否还能正确工作?

答：问题一：此为直接插入排序算法，该算法稳定。
问题二：r[0]的作用是监视哨，免去每次检测文件是否到尾，提高了排序效率。
采用 x．key<=r[j]．key 描述算法后，算法变为不稳定排序，但能正常工作。
5．写出下列排序算法的基本思想，并写出对序列（23，12，35，47，16，25，36，19，21，16）进行排序时每一趟的结果。

```
PROC   bbsort(VAR  r: sequence;  n: integer);
(r是一个数组)
  d:=1;  pos[-1]:=1; pos[1]:=n;  i:=1;  exchanged:= true;
  WHILE  exchanged  DO
    [ exchanged:= false;
     WHILE  i<>pos[d]  DO
        [IF (r[i]-r[i+d])*d>0 THEN [r[i]与r[i+d]交换; exchanged:=true;];
         i:=i+d;
        ]
     pos[d]:=pos[d]-d; i:=pos[d]; d:=-d;
  ]
ENDP;
```

答：此排序为双向起泡排序：从前向后一趟排序下来得到一个最大值，若其中发生交换，则再从后向前一趟排序，得到一个最小值。
第一趟：12，23，35，16，25，36，19，21，16，47
第二趟：12，16，23，35，16，25，36，19，21，47
第三趟：12，16，23，16，25，35，19，21，36，47
第四趟：12，16，16，23，19，25，35，21，36，47
第五趟：12，16，16，19，23，25，21，35，36，47
第六趟：12，16，16，19，21，23，25，35，36，47
第七趟：12，16，16，19，21，23，25，35，36，47
6．设与记录 R_1，R_2，…，R_n对应的关键字分别是 K_1，K_2，…，K_n。如果存在 R_i和 R_j使得 j<i 且 $K_i<K_j$成立，试证明经过一趟起泡后，一定有记录与 R_i进行交换。
答：证明：起泡排序思想是相邻两个记录的关键字比较，若反序则交换，一趟排序完成得到极值。由题假设知 R_j在 R_i之前且 $K_j>K_i$，即说明 R_j和 R_i是反序；设对于 R_i之前全部记录 R_1～$R_{(i-1)}$（其中包括 K_j）中关键字最大为 K_{max}，则 $K_{max}\geq K_j$，故经过起泡排序前 i-2 次比较后，R_{i-1}的关键字一定为 K_{max}，又因 $K_{max}\geq K_j>R_i$，故 R_{j-1}和 Rf 为反序，由此可知 R_{i-1}和 R_i必定交换，证毕。
7．快速排序的最大递归深度是多少?最小递归深度是多少?
答：设待排序记录的个数为 n，则快速排序的最小递归深度为 $\ddot{e}\log_2 n\rfloor+1$，最大递归深度 n。
8．我们知道，对于 n 个元素组成的线性表进行快速排序时，所需进行的比较次数与这 n 个元素的初始排序有关。问：
（1）当 n=7 时，在最好情况下需进行多少次比较?请说明理由。
（2）当 n=7 时，给出一个最好情况的初始排序的实例。
（3）当 n=7 时，在最坏情况下需进行多少次比较?请说明理由。
（4）当 n=7 时，给出一个最坏情况的初始排序的实例。
答：（1）在最好情况下，每次划分能得到两个长度相等的子文件。假设文件的长度 $n=2^k-1$，那么第一趟划分得到两个长度均为 $\ddot{e}n/2\rfloor$ 的子文件，第二趟划分得到 4 个长度均为 $\ddot{e}n/4\rfloor$ 的子文件，以此类推，总共进行 $k=\log_2(n+1)$ 趟划分，各子文件的长度均为 1，排序完毕。

当 n=7 时，k=3，在最好情况下，第一趟需比较 6 次，第二趟分别对两个子文件（长度均为 3，k=2）进行排序，各需 2 次，共 10 次即可。

（2）在最好情况下快速排序的原始序列实例：4，1，3，2，6，5，7。

（3）在最坏情况下，若每次用来划分的记录的关键字具有最大（或最小）值，那么只能得到左（或右）子文件，其长度比原长度少 1。因此，若原文件中的记录按关键字递减次序排列，而要求排序后按递增次序排列时，快速排序的效率与起泡排序相同，其时间复杂度为 O（n^2）。所以当 n=7 时，最坏情况下的比较次数为 21 次。

（4）在最坏情况下快速排序的初始序列实例：7，6，5，4，3，2，1，要求按递增排序。

9．对给定文件（28，07，39，10，65，14，61，17，50，21）选择第一个元素 28 进行划分，写出其快速排序第一遍的排序过程。

答：快速排序的思想如下：首先将待排序记录序列中的所有记录作为当前待排序区域，以第一个记录的关键字作为枢轴（或支点），凡其关键字不大于枢轴的记录均移动至该记录之前，凡关键字不小于枢轴的记录均移动至该记录之后。致使一趟排序之后，记录的无序序列 R[s..t]将分割成两部分：R[s..i-1]和 R[i+1．．t]，且 R[j]．key<<．R[i]．key≤R[k].key（s≤j≤i，i≤k≤t），然后再递归地将 R[s..i-1]和 R[i+1．．t]进行快速排序。快速排序在记录有序时蜕变为起泡排序，可用“三者取中”法改善其性能，避免最坏情况的出现。本题解答如下：

初始序列：[28]，07，39，10，65，14，61,17，50，21

21 移动：21，O7，39，10，65，14，61，17，50，[]

39 移动：21，O7，[]，10，65，14，61，17，50，39

17 移动：21，07，17，10，65，14，61，[]，50，39

65 移动：21，O7，17，10，[]，14，61，65，50，39

14 移动：21，07，17，10，14，[28]，61，65，50，39

10．如果只要找出一个具有 n 个元素的集合的第 k（1≤k≤n）个最小元素，你所学过的排序方法中哪种最适合?给出实现的思想。

答：在具有 n 个元素的集合中找第 k（1≤k≤n）个最小元素，应使用快速排序方法。其基本思想如下：设 n 个元素的集合用一维数组表示，其第一个元素的下标为 1，最后一个元素下标为 n。以第一个元素为“枢轴”，经过快速排序的一次划分，找到“枢轴”的位置 i，若 i=k，则该位置的元素即为所求；若 i>k，则在 1 至 i-1 间继续进行快速排序的划分；若 i<k，则在 i+1 至 n 间继续进行快速排序的划分。这种划分一直进行到 i=k 为止，第 i 位置上的元素就是第 k（1≤k≤n）个最小元素。

11．请回答下列关于堆（Heap）的一些问题：

（1）堆的存储表示是顺序的还是链接的?

（2）设有一个最小堆，即堆中任意结点的关键码均不大于它的左子女和右子女的关键码。其具有最大值的元素可能在什么地方?

（3）对 n 个元素进行初始建堆的过程中，最多做多少次数据比较（不用大 O 表示法）?

答：（1）堆的存储是顺序的。

（2）最大值元素一定是叶结点，在最下两层上。

（3）在建含有 n 个元素、深度为 h 的堆时，其比较次数不超过 4n，推导如下：

由于第 i 层上的结点数至多是 2^{i-1}，以它为根的二叉树的深度为 h-i+1，则调用 ën / 2」次筛选算法时总共进行的关键字比较次数不超过下式之值：

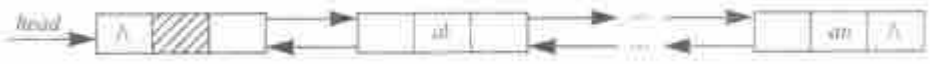
$$\sum_{i=h-1}^{1} 2^{i-1}\cdot 2(h-i)=\sum_{i=h-1}^{1} 2^{i}\cdot(h-i)=\sum_{j=1}^{h-1} 2^{h-j}\cdot j\leqslant(2n)\sum_{j=1}^{h-1} j/2^{j}\leqslant 4n$$

12．（1）判定起泡排序的结束条件是什么?

（2）请简单叙述希尔排序的基本思想。
（3）将下列序列调整成堆（堆顶为最小值）。
112　70　33　65　24　56　48　92　80　13
（4）在 16 个关键字中选出最小的关键字至少要进行多少次比较?再选出次小的关键字至少要进行多少次比较?请简要说明选择的方法和过程。
答：（1）至多进行 n-1 趟起泡排序，或一趟起泡排序中未发生交换（即已有序）时，结束排序。
（2）希尔排序是对直接插入排序算法的改进，它从“记录个数少”和“基本有序”出发，将待排序的记录划分成几组（缩小增量分组），从而减少参与直接插入排序的数据量，当经过几次分组排序后，记录的排列已经基本有序，这个时候再对所有的记录实施直接插入排序。
（3）13，24，33，65，70，56，48，92，80，112
（4）采用树形锦标赛排序选出最小关键字至少要 15 次比较。再选出次小关键字比较 4 次（两两比较 8 次选出 8 个优胜，再两两比较 4 次选出 4 个优胜，半决赛两场，决赛一场，共比较了 15 次。将冠军的叶结点改为最大值，均与兄弟比较，共 4 次选出亚军）。
13．请写出应填入下列叙述中（　　）内的正确答案。
排序有各种方法，如插入排序、快速排序、堆排序等。
设一数组中原有数据如下：15，13，20，18，12，60。下面是一组用不同排序方法进行一遍排序后的结果。
（　　）排序的结果为：12，13，15，18，20，60
（　　）排序的结果为：13，15，18，12，20，60
（　　）排序的结果为：13，15，20，18，12，60
（　　）排序的结果为：12，13，20，18，15，60
答：①快速排序 ②起泡排序 ③直接插入排序 ④堆排序
五、算法设计题
1．起泡排序算法是把大的元素向上移（气泡的上浮），也可以把小的元素向下移（气泡的下沉；请给出上浮和下沉过程交替的起泡排序算法。
答：算法如下：

```
void BubbleSort2(int a[],int n) //相邻两趟向相反方向起泡的起泡排序算法
  {change=1;low=0;high=n-1;     //起泡的上下界
   while(low<high && change)
       {change=0;                //设不发生交换
        for(i=low;i<high;i++)    //从上向下起泡
           if(a[i]>a[i+1]){a[i]<-->a[i+1];change=1;} //有交换,修改标志change
        high--; //修改上界
          for(i=high;i>low;i--)  //气泡下沉,小元素上浮(向左)
            if(a[i]<a[i-1]){a[i]<-->a[i-1];change=1;}
          low++; //修改下界
       }//while   }//BubbleSort2
```

2.有 n 个记录存储在带头结点的双向链表中，现用双向起泡排序法对其按上升序进行排序，请写出这种排序的算法（注：双向起泡排序即相邻两趟排序向相反方向起泡）。



答：算法如下：

```
void  TwoWayBubbleSort(DLinkedList head)
   //对存储在带头结点的双向链表head中的元素进行双向起泡排序
{int exchange=1;  // 设标记
 DLinkedList p,temp,tail;
 tail=null;       //双向链表尾,算法过程中是向上起泡的开始结点
 while(exchange)
   {p=head->next;                  //p是工作指针,指向当前结点
    exchange=0;                    //假定本趟无交换
    while(p->next!=tail)           //向下(右)起泡,一趟有一个最大元素沉底
     if(p->data>p->next->data)     //交换两结点指针,涉及6条链
      {temp=p->next; exchange=1;//有交换

       p->next=temp->next;temp->next->prior=p //先将结点从链表上摘下
       temp->next=p; p->prior->next=temp;      //将temp插到p结点前
       temp->prior=p->prior; p->prior=temp;
      }
      else p=p->next;//无交换,指针后移
      tail=p;          //准备向上起泡
      p=tail->prior;
     while(exchange && p->prior!=head)  //向上(左)起泡,一趟有一个最小元素冒出
      if(p->data<p->prior->data)        //交换两结点指针,涉及6条链
       {temp=p->prior; exchange=1;      //有交换
        p->prior=temp->prior;temp->prior->next=p; //先将temp结点从链表上摘下
        temp->prior=p; p->next->prior=temp;       //将temp插到p结点后(右)
        temp->next=p->next; p->next=temp;
       }
     else p=p->prior;  //无交换,指针前移
   head=p;             //准备向下起泡
  }// while(exchange)
} //算法结束
```

3. 写出一趟快速排序算法。

答：算法如下：

```
int  Partition(RecType R[],int l,int h)
//一趟快速排序算法,枢轴记录到位,并返回其所在位置
  {int i=l; j=h; R[0]=R[i]; x=R[i].key;
     while(i<j)
```

```
    {while(i<j && R[j].key>=x)   j--;
     if(i<j) R[i]=R[j];
     while(i<j && R[i].key<=x)   i++;
     if(i<j) R[j]=R[i];
     }//while
   R[i]=R[0];
   return  i;
}//Partition
```

4．设有一个数组中存放了一个无序的关键序列 K_1、K_2、…、K_n。现要求将 K_n放在将元素排序后的正确位置上，试编写实现该功能的算法，要求比较关键字的次数不超过 n（注：用程序实现）。

答：算法如下：

```
int Partition(RecType K[],int l,int n)
  { int i=l; j=n; K[0]=K[j]; x=K[j].key;
    while(i<j)
     { while(i<j && K[i].key<=x)  i++;
       if(i<j) K[j]=K[i];
       while(i<j && K[j].key>=x)  j--;
       if(i<j) K[i]=K[j];
      }//while
    K[i]=K[0]; return  i;
}//Partition
```

5．借助于快速排序的算法思想，在一组无序的记录中查找给定关键字值等于 key 的记录。设此组记录存放于数组 r[1．．h]中。若查找成功，则输出该记录在 r 数组中的位置及其值，否则显示“not find”信息。请编写出算法并简要说明算法思想。

答：算法如下：

```
int index (RecType R[],int l,h,datatype key)
 {int i=l,j=h;
  while(i<j)
   { while(i<=j && R[j].key>key) j--;
     if(R[j].key==key) return  j;
     while(i<=j && R[i].key<key) i++;
     if(R[i].key==key) return  i;
     if(i<j) R[i]<-->R[j]; //本句的有无不影响查找结果
   }
  printf("not find") ; return  0;
}//index
```

6．辅助地址表的排序是不改变结点物理位置的排序。辅助地址表实际上是一组指针，用它来指出结点排序后的逻辑顺序地址。设用 K[1]，K[2]，…，K[n]表示 n 个结点的值，用 T[1]，T[2]，…，T[n]表示辅助地址表。初始时 T[i]：=i，在排序中，凡需对结点交换就用它的地

址来进行。例如当 n=3 时，对 K（31，11，19）则有 T（2，3，1）。试编写实现辅助地址表排序（按非递减序）算法的语句序列。

答：算法如下：

```
void sort (RecType K[],int n)
{ for(i=1;i<=n;i++) T[i]=i;
  for(i=1;i<n;i++)
     {exchange=1;
      for(j=1;j<=n-i;j++)
            if(K[T[j]]>K[T[j+1]]) {t=T[j];T[j]=T[j+1];T[j+1]=t;exchange=0;}
      if(exchange) break; //一趟排序无交换发生,结束
      }
}//sort
```

7．二路插入排序是将待排关键字序列 r[1．．n]中关键字分二路分别按序插入到辅助向量 d[1．．n]前半部和后半部（注：向量 d 可视为循环表），其原则为，先将 r[1]赋给 d[1]，再从 r[2]记录开始分二路插入。编写实现 2-路插入排序算法。

答：算法如下：

```
void BiInsertSort(RecType  R[],int n) //二路插入排序的算法
{int d[n+1];                  //辅助存储
 d[1]=R[1];first=1;final=1;
  for(i=2;i<=n;i++)
    {if(R[i].key>=d[1].key) //插入后部
     {low=1;high=final;
      while(low<=high)       //折半查找插入位置
          {m=(low+high)/2;
           if(R[i].key<d[m].key) high=m-1; else low=m+1;
          }//while
          for(j=final;j>=high+1;j--) d[j+1]=d[j];//移动元素
          d[high+1]=R[i]; final++; //插入有序位置
       }//if
     else //插入前部
      {if(first==1){ first=n; d[n]=R[i];}
       else{low=first;high=n;
           while(low<=high)
               {m=(low+high)/2;
                if(R[i].key<d[m].key) high=m-1; else low=m+1;
               }//while
            for(j=first;j<=high;j++) d[j-1]=d[j]; //移动元素
              d[high]=R[i]; first--;
            }//if
       }//if
    }//for
  R[1] =d[first];
  for(i=first%n+1,j=2;i!=fisrt;i=i%n+1,j++) R[j]=d[i]; //将序列复制回去
 }//BiInsertSort
```

8．叙述基数排序算法，并对下列整数序列图示其基数排序的全过程。（179，208，93，306，55，859，984，9，271，33）

答：算法如下：

```
                    #define   n   待排序记录的个数

typedef  struct
  {int  key[d];       //关键字由d个分量组成
   int  next;         //静态链域
   AnyType  other;    //记录的其他数据域
  } SLRecType;
SLRecType  R[n+1];    //R[1..n]存放n个记录
typedef  struct
{ int  f,e;           //队列的头、尾指针
} SLQueue;
SLQueue B[m]          //用队列表示桶，共m个
int   RadixSort(SLRecType  R[], int n)
{ //对R[1..n]进行基数排序，返回收集用的链头指针
  for(i=1;i<n;i++)  R[i].next=i+1;//将R[1..n]链成一个静态链表
  R[n].next=-1; p=1;    //将初始链表的终端结点指针置空，p指向链表的第一个结点
  for(j=d-1;j>=0;j--)   //进行d趟排序
    {for(i=0;i<m;i++) { B[i].f=-1;B[i].e=-1;}//初始化桶
       while(p!=-1)           //按关键字的第j个分量进行分配
         {k=R[p].key[j];      //k为桶的序号
          if(B[k].f==-1)  B[k].f=p;      //将R[p]链到桶头
          else  R[B[k].e].next=p;        //将R[p]链到桶尾
          B[k].e=p;                      //修改桶的尾指针
          p=R[p].next;                   //扫描下一个记录
         }//while
         i=0;
         while(B[i].f==-1)  i++;    //找第一个非空的桶
         t=B[i].e;  p=B[i].f        //p为收集链表的头指针，t为尾指针
         while(i<m-1)
           {i++;
            if(B[i].f!=-1){ R[t].next=B[i].f; t=B[i].e; } //连接非空桶
           }//while
         R[t].next=-1;          //本趟收集完毕，将链表的终端结点指针置空
       }//for
     return  p;
}//RadixSort
```

9．设有顺序放置的 n 个桶，每个桶中装有一粒砾石，每粒砾石的颜色是红、白、蓝之一。要求重新安排这些砾石，使得所有红色砾石在前，所有白色砾石居中，所有蓝色砾石居后。重新安排时，对每粒砾石的颜色只能察看一次，并且只允许交换操作来调整砾石的位置。

答：算法如下：

```
void QkSort(rectype r[],int n)
// r为含有n个元素的线性表,元素是具有红、白和蓝色的砾石,用顺序存储结构存储
//本算法对其排序,使所有红色砾石在前,白色居中,蓝色在最后
  {int i=1,j=1,k=n,temp;
   while(j<=k)
    if(r[j]==1)                  //当前元素是红色
     {temp=r[i]; r[i]=r[j]; r[j]=temp; i++;j++; }
    else if(r[j]==2) j++;        //当前元素是白色
         else                    //(r[j]==3  当前元素是蓝色
           {temp=r[j]; r[j]=r[k]; r[k]=temp; k--; }
```

10．对给定关键字序号 j（1<j<n），要求在无序记录 A[1..n]中找到关键字从小到大排在第 j 位上的记录，写一个算法利用快速排序的划分思想实现上述查找（要求用最少的时间和最少的空间）。

例如：给定无序关键字{7，5，1，6，2，8，9，3}，当 j=4 时，找到的关键字应是 5。

答：算法如下：

```
int partition(RecType A[],int l,n)
    {int i=l,j=n;x=A[i].key;
     i=l;
     while(i<j)
       {while(i<j && A[j].key>=x) j--;
        if(i<j) A[i++]=A[j];
        while(i<j && A[i].key<=x) i++;
        if(i<j) A[j--]=A[i];
        }
      return  i;
      }
    void Find_j(RecType A[],int n,int j);
      {i=partition (A,1,n);
       while(i!=j)
         if(i<j) i=quicksart(A,i+1,n);    //在后半部分继续进行划分
         else i=quicksart(R,1,i-1);       //在前半部分继续进行划分
       }
```

11．请运用快速排序思想，设计递归算法实现求 n（n>1）个不同元素集合中的第 f（1≤i≤n）小元素。

答：算法如下：

```
int partition(RecType A[],int l,n)
    {int i=l,j=n;x=A[i].key;
     i=l;
     while(i<j)
       {while(i<j && A[j].key>=x) j--;
        if(i<j) A[i++]=A[j];
        while(i<j && A[i].key<=x) i++;
        if(i<j) A[j--]=A[i];
        }
      return  i;
      }
    void Find_j(RecType A[],int n,int j);
      {i=partition (A,l,n);
       while(i!=j)
         if(i<j) i=quicksart(A,i+1,n);     //在后半部分继续进行划分
         else i=quicksart(R,l,i-1);        //在前半部分继续进行划分
      }
```

12．结点类型和存储结构如下：

```
typedef struct {int key; datatype data; int count; }node;
node R[n];
```

试设计一个排序算法，要求不移动结点的存储位置，只在结点的 count 字段记录结点在排序中的序号，并将排序结果按升序输出。

答：算法如下：

```
void StaticList(node R[],int n)
   //对存储在数组R中的记录进行排序
   {for(i=0;i<=n;i++) R[i].count=i+1; //初始化
    R[0].key=Max; R[0].count=1;       //设R[0]作监视哨,Max是该类型最大元素
    R[1].count=0;                     //初始假定第一个元素有序
    for(i=2;i<=n;i++)
      {pre=0;          //前驱
       p=R[0].count;   //第一个元素
       while(p!=0 && R[p].key<R[i].key) //查找第i个元素的序号
         {pre=p; p=R[p]count;}
       R[pre].count=i; R[i].count=p;     //将第i个元素链入静态链表
       }
   }
```

第 11 章　外部排序

一、选择题

1．下列排序算法中，其中（　　）是稳定的。

A．堆排序，起泡排序　　B．快速排序，堆排序

C．直接选择排序，归并排序　　D．归并排序，起泡排序

【答案】D 查看答案

2．若需在O（$nlog_2n$）的时间内完成对数组的排序，且要求排序是稳定的，则可选择的排序方法是（　　）。

A．快速排序　　B．堆排序　　C．归并排序　　D．直接插入排序

【答案】C 查看答案

【解析】稳定排序有：插入排序、起泡排序、归并排序、基数排序。不稳定排序有：快速排序、堆排序、shell 排序。时间复杂度平均为O（$nlog_2n$）的有：归并排序、堆排序、shell 排序、快速排序。

3．在下面的排序方法中，辅助空间为O（n）的是（　　）。

A．希尔排序　　B．堆排序　　C．选择排序　　D．归并排序

【答案】D 查看答案

4．下列排序算法中，占用辅助空间最多的是（　　）。

A．归并排序　　B．快速排序　　C．希尔排序　　D．堆排序

【答案】A 查看答案

【解析】归并排序的辅助空间为O（n），快速排序所占用的辅助空间为O（logn），堆排序所占用的辅助空间为O（1）。

5．将两个各有N个元素的有序表归并成一个有序表，其最少的比较次数是（　　）。

A．N　　B．2N-1　　C．2N　　D．N-1

【答案】A 查看答案

【解析】归并排序基本思想：归并排序是多次将两个或两个以上的有序表合并成一个新的有序表。最简单的归并是直接将两个有序的子表合并成一个有序的表。归并排序最好情况下的复杂度为O（n）。

6．从未排序序列中依次取出一个元素与已排序序列中的元素依次进行比较，然后将其放在已排序序列的合适位置，该排序方法称为（　　）排序法。

A．插入　　B．选择　　C．希尔　　D．二路归并

【答案】A 查看答案

【解析】解此题需要熟知各种排序方法的基本思想。插入排序的基本思想是：假设待排序的记录存放在数组R[0．．n-1]中，排序过程的某一中间时刻，R被划分成两个子区间R[0．．i-1]和R[i．．n-1]，其中：前一个子区间是已排好序的有序区，后一个子区间则是当前未排序的部分，不妨称其为无序区。将当前无序区的第1个记录R[i]插入到有序区R[0．．i-1]中适当的位置上。使R[0．．i]变为新的有序区。这种方法通常称为增量法，因为它每次使有序区增加1个记录。

二、判断题

1．排序的稳定性是指排序算法中的比较次数保持不变，且算法能够终止。（　　）

【答案】×查看答案

【解析】排序的稳定性指：假定在待排序的记录序列中，存在多个具有相同的关键字的记录，若经过排序，这些记录的相对次序保持不变，即在原序列中，ri=rj，且ri在rj之前，而在排序后的序列中，ri仍在rj之前，则称这种排序算法是稳定的；否则称为不稳定的。

2．排序算法中的比较次数与初始元素序列的排列无关。（　　）

【答案】×查看答案

【解析】这个要看是哪个排序算法，比如快速排序，初始序列为有序的情况比较的次数就相对于无序的多。

3．归并排序辅助存储为O（1）。（　　）

【答案】×查看答案

【解析】归并排序的辅助存储是O（n）。

4．快速排序和归并排序在最坏情况下的比较次数都是 O（$nlog_2n$）。（　　）
【答案】×查看答案
【解析】快速排序最坏的情况下比较次数是 O（n^2）。
5．在任何情况下，归并排序都比简单插入排序快。（　　）
【答案】×查看答案
【解析】错误。待排序序列为正序时，简单插入排序比归并排序快。
6．外部排序是把外存文件调入内存，可利用内部排序的方法进行排序，因此排序所花的时间取决于内部排序的时间。（　　）
【答案】×查看答案
【解析】外部排序方法：按可用内存大小，将外存上含 n 个记录的文件分成若干长度为 z 的子文件或段，依次读入内存并利用有效的内部排序方法对它们进行排序，并将排序后得到的有序子文件重新写入外存，通常称这些有序子文件为归并段或顺串(run)。对这些归并段进行逐趟归并，使归并段(有序的子文件)逐渐由小至大，直至得到整个有序文件为止。一般情况下，外部排序所需总的时间=内部排序(产生初始归并段)所需的时间 $m*t_{IS}$+外存信息读写的时间 $d*t_{IO}$+内部归并所需的时间 $S*ut_{mg}$。
7．在外部排序过程中，对长度为 n 的初始序列进行“置换-选择”排序时，可以得到的最大初始有序段的长度不超过 n / 2。（　　）
【答案】×查看答案
【解析】当输入文件以关键字的升序排序时，只能得到一个长度为文件长度的初始归并段
8．为提高排序速度，进行外排序时，必须选用最快的内排序算法。（　　）
【答案】×查看答案
【解析】外部排序算法最常用的是多路归并，即将原文件分解成多个能够一次性装人内存的部分，分别把每一部分调入内存完成排序。然后，对已经排序的子文件进行归并排序。
三、填空题
1．外排序的基本操作过程是_____和_____。
【答案】生成有序归并段（顺串）；归并查看答案
2．关键码序列（Q，H，C，Y，Q，A，M，S，R，D，F，X），要按照关键码值递增的次序进行排序，若采用初始步长为 4 的希尔排序法，则一趟扫描的结果是_____；若采用以第一个元素为分界元素的快速排序法，则扫描一趟的结果是_____。
【答案】（Q，A，C，S，Q，D，F，X，R，H，M ,Y）；（F，H，C，D，a，A，M, Q，R，S，Y，X）查看答案
【解析】希尔排序的基本思想是：先将整个待排记录序列分割成为若干子序列分别进行直接插入排序，待整个序列中的记录“基本有序”时，再对全体记录进行一次直接插入排序。
快速排序(quick sort)的基本思想是，通过一趟排序将待排记录分割成独立的两部分，其中一部分记录的关键字均比另一部分记录的关键字小，则可分别对这两部分记录继续进行排序，以达到整个序列有序。
3. 在基于关键字比较且时间为 O（$nlog_2n$）的排序中，若要求排序是稳定的，则可选用_____排序；若要求就地排序（及辅助空间为 O（1）），则可选用_____排序。
【答案】归并；堆查看答案
4. 按 LSD 进行关键字排序，除最次位关键字之外，对每个关键字进行排序时，只能用_____的排序方法。
【答案】稳定查看答案
四、应用题

1．在各种排序方法中，哪些是稳定的?哪些是不稳定的?并为每一种不稳定的排序方法举出一个不稳定的实例。

答：各种排序算法稳定性的归纳如图 11-1 所示：

排序方法	平均时间	最坏情况	辅助空间	稳定性	不稳定排序举例
直接插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	
折半插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	
二路插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	稳定	
表插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	
起泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	
直接选择排序	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定	2, 2', 1
希尔排序	$O(n^{1.3})$	$O(n^{1.3})$	$O(1)$	不稳定	3, 2, 2', 1(d=2, d=1)
快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n)$	不稳定	2, 2', 1
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定	2, 1, 1'（极大堆）
2-路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定	
基数排序	$O(d*(rd+n))$	$O(d*(rd+n))$	$O(rd)$	稳定	

图 11-1 各种排序算法稳定性归纳

2．在多关键字排序时，LSD 和 MSD 两种方法的特点是什么?

答：（1）最高位优先（MSD）法

先对最高位关键字 K^0进行排序，将序列分成若干子序列，每个子序列中的记录都具有相同的 K^0值，然后，分别就每个子序列对关键字 K^1进行排序，按 K^1值不同再分成若干更小的子序列，依次重复，直至最后对最低位关键字排序完成，将所有子序列依次连接在一起，成为一个有序子序列。

（2）最低位优先（LSD）法

先对最低位关键字 K^{d-1}进行排序，然后对高一级关键字 K^{d-2}进行排序，依次重复，直至对最高位关键字 K^0排序后便成为一个有序序列。进行排序时，不必分成子序列，对每个关键字都是整个序列参加排序，但对 K^i（0<=i<d-1）排序时，只能用稳定的排序方法。另一方面，按 LSD 进行排序时，可以不通过关键字比较实现排序，而是通过若干次“分配”和“收集”来实现排序。

3．给出一组关键字：29，18，25，47，58，12，51，10，分别写出按下列各种排序方法进行排序时的变化过程：

（1）归并排序，每归并一次书写一个次序。

（2）快速排序，每划分一次书写一个次序。

（3）堆排序，先建成一个堆，然后每从堆顶取下一个元素后，将堆调整一次。

答：（1）2 一路归并第一趟：18，29，25，47，12，58，10，51；

第二趟：18，25，29，47，10，12，51，58；

第三趟：10，12，18，25，29，47，51，58

（2）快速排序第一趟：10，18，25，12，29，58，51，47；

第二趟：10，18，25，12，29，47，51，88；

第三趟：10，12，18，25，29，47，51，88

（3）堆排序

建大堆：58，47，51，29，18，12，25，10；

①51，47，25，29，18，12，10，58；

②47，29，25，10，18，12，51，58；

③29，18，25，10，12，47，51，58；

④25，18，12，10，29，47，51，58；
⑤18，10，12，25，29，47，51，58；
⑥12，10，18，25，29，47，51，58；
⑦10，12，18，25，29，47，51，58

4．设某文件经内排序后得到 100 个初始归并段（初始顺串），若使用多路归并排序算法，并要求三趟归并完成排序，问归并路数最少为多少?

答：设归并路数为 k，归并趟数为 s，则 s=|$\log_k 100$|，因|$\log_k 100$|=3，且 k 为整数，故 k=5，即最少 5-路归并可以完成排序。

5．证明：置换选择排序法产生的初始归并段的长度至少为 m（m 是所用缓冲区的长度）。

答：证明：由置换选择排序思想，第一个归并段中第一个元素是缓冲区中最小的元素，以后每选一个元素都不应小于前一个选出的元素，故当产生第一个归并段时（即初始归并段），缓冲区中 m 个元素中除最小元素之外，其他 m-1 个元素均大于第一个选出的元素，即当以后读入元素均小于输出元素时，初始归并段中也至少能有原有的 m 个元素。证毕。

6．以归并算法为例，比较内部排序和外部排序的不同，说明外部排序如何提高操作效率。

答：（1）内部排序中的归并排序是在内存中进行的归并排序，辅助空间为 O（n）。外部归并排序是将外存中的多个有序子文件合并成一个有序子文件，将每个子文件中记录读入内存后的排序方法可采用多种内排序方法。外部排序的效率主要取决于读写外存的次数，即归并的趟数。

（2）因为归并的趟数 s=「$\log_k$mù，其中，m 是归并段个数，k 是归并路数。增大 k 和减少 m 都可减少归并趟数。应用中通过败者树进行多（k）路平衡归并和置换一选择排序减少 m，来提高外部排序的效率。

7．外排序中为何采用 k 一路（k>2）合并而不用 2 一路合并?这种技术用于内排序有意义吗?为什么?

答：外排序用 k-路归并（k>2）是因为 k 越小，归并趟数越多，读写外存次数越多，时间效率越低，故一般应大于最少的 2-路归并。若将 k-路归并的败者树思想单纯用于内排序，因其由胜者树改进而来，且辅助空间大，完全可由堆排序取代，故将其用于内排序效率并不高。

五、算法设计题

1．已知关键字序列（K_1，K_2，K_3，…，K_{n-1}）是大根堆。

（1）试写出一算法将（K_1，K_2，K_3，…，K_{n-1}，K_n）调整为大根堆；

（2）利用（1）的算法写一个建大根堆的算法。

答：（1）算法如下：

```
void sift(RecType R[],int n)
{ //假设 R[1..n-1]是大堆 ,本算法把R[1..n]调成大堆
  j=n;  R[0]=R[j];
  for(i=n/2;i>=1;i=i/2)
     if(R[0].key>R[i].key) {R[j]=R[i];j=i;} else break;
  R[j]=R[0];
}//sift
```

(2)

```
void HeapBuilder(RecType R[],int n)
  {for(i=2;i<=n;i++) sift (R,i); }
```

2．设记录 R[i]的关键字为 R[i]。KEY（1≤i≤k），树结点 T[i]（1≤i≤K-1）指向败者记录，T[0]为全胜记录下标。写一算法产生对应上述 R[f]（1≤f≤k）的败者树，要求除 R[1..k]和 Tr[0..k-1]以外，只用 O（1）辅助空间。

答：算法如下：

```
void  Adjust(int T[],int s)
    {//选得最小关键字记录后,沿从叶结点R[s]到根结点T[0]的路径调整败者树
     t=(s+k)/2;       //T[t]是R[s]的双亲结点
     while(t>0)
       {if(R[s].key>R[T[t]].key)  s<-->T[t];   //s指示新的胜者
        t=t/2;
       }//while
          T[0]=s;
      }//Adjust
void  CreateLoserTree(int T[])
  { //R[0]到R[k-1]为完全二叉树T的叶结点,本算法建立败者树
    R[k].key=MINKEY;                  //MINKEY是与题中要求的关键字类型相同的机器最小数
    for(i=0;i<k;i++) T[i]=k;          //设置T中"败者"的初值
    //依次从R[k-1],R[k-2],…,R[0]出发调整败者
    for(i=k-1;k>=0;i--)  Adjust(T,i);
}//CreateLoserTree
```

第 12 章　文　件

一、选择题

1．哈希文件使用哈希函数将记录的关键字值计算转化为记录的存放地址，因为哈希函数是一对一的关系，则选择好的（　　）方法是哈希文件的关键。

A．哈希函数　　B．除余法中的质数　　C．冲突处理　　D．哈希函数和冲突处理

【答案】D 查看答案

【解析】哈希表是根据文件中关键字的特点设计一种哈希函数和处理冲突的方法将记录散列到存储设备上。

2．下述文件中适合于磁带存储的是（　　）。

A．顺序文件　　B．索引文件　　C．哈希文件　　D．多关键字文件

【答案】A 查看答案

【解析】磁带存储是一种顺序存储，顺序文件(sequential file)是记录按其在文件中的逻辑顺序依次进入存储介质而建立的，即顺序文件中物理记录的顺序和逻辑记录的顺序是一致的。因此顺序文件适合磁带存储。

二、判断题

1．倒排文件是对次关键字建立索引。（　　）

【答案】√查看答案

【解析】倒排文件是对每一个次关键字项建立次关键字索引(称为倒排表)，将所有具有相同次关键字的记录的物理记录号都填入倒排表为此次关键字的表中。

2．倒排序文件的优点是维护简单。（　　）

【答案】×查看答案

【解析】倒排文件的优点是检索记录较快。特别是对某些询问，不用读取记录，就可得到解答。

3．哈希表与哈希文件的唯一区别是哈希文件引入了“桶”的概念。（　　）

【答案】×查看答案

【解析】哈希文件是使用一个函数（算法）来完成一种将关键字映射到存储器地址的映射，根据用户给出的关键字，经函数计算得到目标地址，再进行目标的检索。哈希表是根据关键码值而直接进行访问的数据结构。

4．文件系统采用索引结构是为了节省存储空间。（　　）
【答案】×查看答案
【解析】是为了缩短查找的时间，牺牲了一部分存储空间。
5. 对处理大量数据的外存介质而言，索引顺序存取方法是一种方便的文件组织方法。（　　）
【答案】×查看答案
【解析】索引顺序存取方法插入操作比较麻烦，对于处理大量数据，会有大量的记录进入溢出区，而基本区中又浪费很多空间。
6．对磁带机而言，ISAM 是一种方便的文件组织方法。（　　）
【答案】×查看答案
【解析】ISAM 是一种专为磁盘存取设计的文件组织方式。
7．直接访问文件也能顺序访问，只是一般效率不高。（　　）
【答案】×查看答案
【解析】直接访问文件不能进行顺序访问，只能按关键字随机存取。在 ISAM 文件上检索记录时，先从主索引出发找到相应的柱面索引，再从柱面索引找到记录所在柱面的磁道索引，最后从磁道索引找到记录所在磁道的第一个记录的位置，由此出发在该磁道上进行顺序查找直至找到为止。
8．倒排文件的目的是为了多关键字查找。（　　）
【答案】√查看答案
【解析】多关键字文件的特点是，在对文件进行检索操作时，不仅对主关键字进行简单询问，还经常需要对次关键字进行其它类型的询问检索。常见的多关键字文件为：多重表文件和倒排文件。

三、填空题

1．文件可按其记录的类型不同而分成两类，即_____和_____文件。
【答案】操作系统文件；数据库查看答案
2．从用户的观点看，文件的逻辑结构通常可以区分为两类：一类是如 NdBASE 中数据库文件那样的文件组织结构，称为___（1）___文件；另一种是诸如用各种文字处理软件编辑成的文本文件，称为___（2）___文件。从文件在存储器上的存放方式来看，文件的物理结构往往可区分为三类，即___（3）___，___（4）___和___（5）___。B+树适用于组织___（6）___的索引结构，m 阶 B+树每个结点至多有___（7）___个儿子，除根结点外每个结点至少有___（8）___个儿子，根结点至少有___（9）___个儿子，有 k 个儿子的结点必有___（10）___个关键码。
【答案】数据库；文本；顺序组织；随机组织；链组织；随机组织；m；ém/2ù；2；k 查看答案
3．文件由_____组成；记录由_____组成。
【答案】记录；数据项查看答案
4．索引顺序文件既可以顺序存取，也可以_____存取。
【答案】随机查看答案
5．建立索引文件的目的是_____。
【答案】提高查找速度查看答案
6．检索是为了在文件中寻找满足一定条件的记录而设置的操作。检索可以按_____检索。也可以按_____检索；按_____检索又可以有_____检索和_____检索。
【答案】关键字；记录号；记录号；顺序；直接查看答案
7.VSAM 系统是由_____、_____、_____构成的。
【答案】索引集；顺序集；数据集查看答案

8.VSAM（虚拟存储存取方法）文件的优点是：动态地_____，不需要文件进行_____，并能较快地_____进行查找。

【答案】分配和释放存储空间；重组；对插入的记录查看答案

四、应用题

1．文件存储结构的基本形式有哪些?一个文件采用何种存储结构应考虑哪些因素?

答：文件的基本组织方式有顺序组织、索引组织、散列组织和链组织，常用的结构有顺序结构、索引结构、散列结构。

（1）顺序结构，相应文件为顺序文件，其记录按存入文件的先后次序顺序存放。顺序文传本质上就是顺序表。若逻辑上相邻的两个记录在存储位置上相邻，则为连续文件；若记录之间以指针相链接，则称为串联文件。顺序文件只能顺序存取，要更新某个记录，必须复制整个文件。顺序文件连续存取的速度快，主要适用于顺序存取，批量修改的情况。

（2）带索引的结构，相应文件为索引文件。索引文件包括索引表和数据表，索引表中的索引项包括数据表中数据的关键字和相应地址，索引表有序，其物理顺序体现了文件的逻辑次序，实现了文件的线性结构。索引文件只能是磁盘文件，既能顺序存取，又能随机存取。

（3）散列结构，也称计算寻址结构，相应文件称为散列文件，其记录是根据关键字值经哈希函数计算确定其地址，存取速度快，不需索引，节省存储空间。不能顺序存取，只能随机存取。

文件采用何种存储结构应综合考虑各种因素，如：存储介质类型、记录的类型、大小和关键字的数目以及对文件作何种操作。

2．索引顺序存取方法（ISAM）中，主文件已按关键字排序，为何还需要主关键字索引?

答：ISAM 是专为磁盘存取设计的文件组织方式。即使主文件关键字有序，但因磁盘是以盘组、柱面和磁道（盘面）三级地址存取的设备，因此通常对磁盘上的数据文件建立盘组、柱面和磁道（盘面）三级索引。在 ISAM 文件上检索记录时，先从主索引（柱面索引的索引）找到相应柱面索引。再从柱面索引找到记录所在柱面的磁道索引，最后从磁道索引找到记录所在磁道的第一个记录的位置，由此出发在该磁道上进行顺序查找直到查到为止；反之，若找遍该磁道而未找到所查记录，则文件中无此记录。

3. 分析 ISAM 文件（Indexed Sequential Access Methord）和 VSAM 文件（Virtual storageAccess Methord）的应用场合、优缺点等。

答：ISAM 是一种专为磁盘存取设计的文件组织形式，采用静态索引结构，对磁盘上的数据文件建立盘组、柱面、磁道三级索引。ISAM 文件中记录按关键字顺序存放，插入记录时需移动记录并将同一磁道上最后的一个记录移至溢出区，同时修改磁道索引项，删除记录只需在存储位置作标记，不需移动记录和修改指针。经过多次插入和删除记录后，文件结构变得不合理，需周期整理 ISAM 文件。

VSAM 文件采用 B+树动态索引结构，文件只有控制区间和控制区域等逻辑存储单位，与外存储器中柱面、磁道等具体存储单位没有必然联系。VSAM 文件结构包括索引集、顺序集和数据集三部分，记录存于数据集中，顺序集和索引集构成 B+树，作为文件的索引部分可实现顺链查找和从根结点开始的随机查找。

优缺点：与 ISAM 文件相比，VSAM 文件有如下优点：动态分配和释放存储空间，不需对文件进行重组；能保持较高的查找效率，且查找先后插入记录所需时间相同。因此，基于 B+树的 VSAM 文件通常作为大型索引顺序文件的标准组织。

4．一个 ISAM 文件除了主索引外，还包括哪两级索引?

答：ISAM 文件有三级索引：磁盘组、柱面和磁盘，柱面索引存放在某个柱面上，若柱面索引较大，占多个磁道时，可建立柱面索引的索引——主索引。故还包括的两级索引是盘组和磁道。

5．为什么在倒排文件组织中，实际记录中的关键字域可删除以节约空间?而在多重表结构中这样做为什么要牺牲性能?

答：因倒排文件组织中，倒排表有关键字值及同一关键字值的记录的所有物理记录号，可方便地查询具有同一关键字值的所有记录；而多重表文件中次关键字索引结构不同，删除关键字域后查询性能受到影响。

6．组织待检索文件的倒排表的优点是什么?

答：倒排表作为索引的优点是索引记录快，因为从次关键字值直接找到各相关记录的物理记录号，倒排因此而得名（因通常的查询是从关键字查到记录）。在插入和删除记录时，倒排表随之修改，倒排表中具有相同次关键字的记录号是有序的。

7．为什么文件的倒排表比多重表组织方式节省空间?

答：倒排表有两项，一是次关键字值，二是具有相同次关键字值的物理记录号，这些记录号有序且顺序存储，不使用多重表中的指针链接，因而节省了空间。

8．试比较顺序文件、索引非顺序文件、索引顺序文件、哈希文件的存储代价，检索、插入、删除记录时的优点和缺点。

答：（1）顺序文件只能顺序查找，优点是批量检索速度快，不适于单个记录的检索。顺序文件不能像顺序表那样插入、删除和修改，因文件中的记录不能象向量空间中的元素那样“移动”，只能通过复制整个文件实现上述操作。

（2）索引非顺序文件适合随机存取，不适合顺序存取，因主关键字未排序，若顺序存取会引起磁头频繁移动。索引顺序文件是最常用的文件组织，因主文件有序，既可顺序存取也可随机存取。索引非顺序文件是稠密索引，可以“预查找”，索引顺序文件是稀疏索引，不能“预查找”，但由于索引占空间较少，管理要求低，提高了索引的查找速度。

（3）散列文件也称直接存取文件，根据关键字的哈希函数值和处理冲突的方法，将记录散列到外存上。这种文件组织只适用于像磁盘那样的直接存取设备，其优点是文件随机存放，记录不必排序，插入、删除方便，存取速度快，无需索引区，节省存储空间。缺点是散列文件不能顺序存取，且只限于简单查询。经多次插入、删除后，文件结构不合理，需重组文件，这很费时。

9．已知两个各包含Ⅳ和 M 个记录的排好序的文件能在 O（N+M）时间内合并为一个包含 N+M 个记录的排好序的文件。当有多于两个排好序的文件要被合并在一起时，只需重复成对地合并便可完成。合并的步骤不同，所需花费的记录移动次数也不同。现有文件，F1，F2，F3，F4，F5，各有记录数为 20，30，10，5 和 30，试找出记录移动次数最少的合并步骤。

答：类似最优二叉树（哈夫曼树），可先合并含较少记录的文件，后合并较多记录的文件，使移动次数减少,如图 12-1 所示的哈夫曼树。

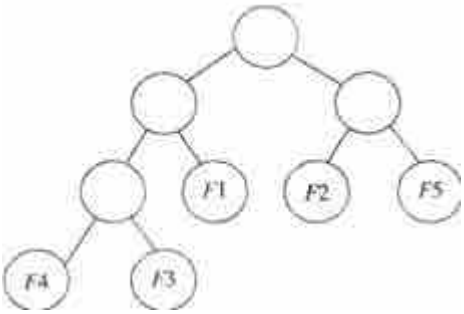


图 12-1 哈夫曼树

第三部分　模拟试题

严蔚敏《数据结构》（C 语言版）配套模拟试题及详解（一）

一、单项选择题（每小题 2 分，共 20 分）

1．以下数据结构中哪一个是线性结构？（　　）

A．有向图　B．栈　C．二叉树　D．B 树

【答案】B 查看答案

【解析】线性结构指结构中的数据元素之间存在一个对一个的关系。

2．若某链表最常用的操作是在最后一个节点之后插入一个节点和删除最后一个节点，则采用（　　）存储方式最节省时间。

A．单链表

B．双链表

C．带头节点的双循环链表

D．单循环链表

【答案】C 查看答案

【解析】因为需要经常在最后一个节点插入元素，就需要得到迅速得到最后一个节点。而删除最后一个节点，需要得到最后一个节点的前继。

3．（　　）不是队列的基本运算。

A．在队列第 i 个元素之后插入一个元素

B．从队头删除一个元素

C．判断一个队列是否为空

D．读取队头元素的值

【答案】A 查看答案

【解析】作为队列必须遵循先进先出的特性，插入的元素都在队尾。

4．字符 A、B、C、D 依次进入一个栈，按出栈的先后顺序组成不同的字符串，至多可以组成（　　）个不同的字符串。

A.15　B．14 C．16 D．21

【答案】B 查看答案

【解析】n 个不同字符入栈，先后出栈组成不同字符串，可以根据卡塔南数列计算公式

$C_n = \frac{1}{n+1} C_{2n}^{n}$ 算出，把 4 代入得到 14。

5．由权值分别为 4，7，6，2 的叶子生成一棵哈夫曼树，它的带权路径长度为（　　）。

A．11　B．37　C．19　D．53

【答案】B 查看答案

【解析】构造哈夫曼树　19

　　7　12

　6　6

　2　4

图 4-1

根据图 4-1，计算权值 W = （2+4）*3+6*2+7*1 = 37。

以下（6）～（8）题基于下面的叙述：

若某二叉树节点的中序遍历的序列为 A、B、C、D、E、F、G，后序遍历的序列为 B、D、C、A、F、G、E。

6．则该二叉树节点的前序遍历的序列为（　　）。

A. E、G、F、A、C、D、B

B．E、A、G、C、F、B、D

C．E、A、C、B、D、G、F
D．E、G、A、C、D、F、B
【答案】C 查看答案
【解析】根据题干已知条件可以画出二叉树

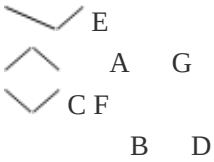
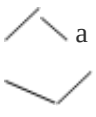


图 4-2
根据图 4-2 可知其前序遍历的序列为 E A C B D G F
7．该二叉树有（　　）个叶子。
A．3 B．2　C．5　D．4
【答案】A 查看答案
【解析】根据图 2，易见叶子节点的个数为 B、D、 F。
8．该二叉树的按层遍历的序列为（　　）。
A．E、G、F、A、C、D、B B．E、A、C、B、D、G、F
C．E、A、G、C、F、B、D　D．E、G、A、C、D、F、B
【答案】C 查看答案
【解析】根据图 2，二叉树可知层次遍历为 E、A、G、C、F、B、D
9．下面的二叉树中，（　　）不是完全二叉树。



A.　B C　D.
【答案】C 查看答案
【解析】一颗深度为 h 完全二叉树，指除了 h 层外，其余（1-h-1）层结点个数都达到了最大个数。且 h 层的结点都集中在最左边。
10．设有关键码序列（q，g，m，z，a），（　　）序列是从上述序列出发建的小根堆的结果。
A. a,g ,m,q,z
B. a,g,m,z,q
C. g,m,q,a,z
D. g, m, a,q,z
【答案】B 查看答案
【解析】根据该序列建立小根堆的结果如下



小根堆建立原则是从树叶开始确保所有的双亲比其左右孩子都小。

二、（本题 10 分）

试述顺序查找法、折半查找法和分块查找法对被查找的表中元素的要求，对长度为 n 的查找表来说，三种查找法在查找成功时的查找长度各是多少？

答：三种方法对查找的要求分别如下：

（1）顺序查找法：表中元素可以任意存放；

（2）折半查找法：表中元素必须以关键字的大小递增或递减的次序存放；

（3）分块查找法：表中元素每块内的元素可任意存放，但块与块之间必须以关键字的大小递增（或递减）存放，即前一块内所有元素的关键字都不能大于（或小于）后一块内任何元素的关键字。

三种方法的平均查找长度分别如下：

（1）顺序查找法：查找成功的平均查找长度为 $\frac{n+1}{2}$；

（2）折半查找法：查找成功的平均查找长度为 $\log_2(n+1)-1$；

（3）分块查找法：若用顺序查找确定所在的块，平均查找长度为 $\frac{1}{2}(\frac{n}{s}+s)+1$；

若用折半确定所在块，平均查找长度为 $\log_2(\frac{n}{s}+1)+\frac{s}{2}$。

三、（本题 10 分）

设有一个输入数据的序列是{46，25，78，62，12，80}，试画出从空树起，逐个输入各个数据而生成的二叉排序树。

答：建立二叉排序树需要注意：①左子树上所有节点的值均小于它的根节点的值；②右子树上所有节点的值均大于或等于它的根节点的值；③它的左、右子树也分别为二叉排序树；如图 4-3 所示。

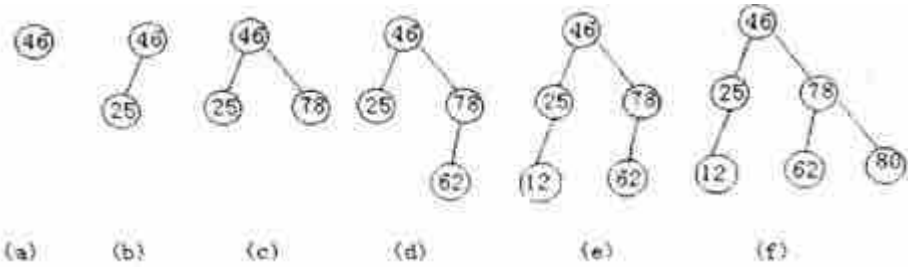


图 4-3 构造二叉排序树示意图

四、（本题 10 分）

给定一个关键字序列{24，19，32，43，38，6，13，22}，请写出快速排序第一趟的结果：堆排序时所建的初始堆；然后回答上述两种排序方法中哪一种方法使用的辅助空间最小。在最坏情况下哪种方法的时间复杂度最差？

答：快速排序：找到第一个元素从序列尾部开始找该元素小的，找到后互换。再从该元素上一次位置后开始找比该元素大的，找到后互换。依次类推就完成了一趟快速排序。

快速排序的第一趟结果为（22，19，13，6，24，38，43，32）；

堆排序：如将上面的序列对应一维数组看成为一个完全二叉树，则堆的含义表明完全二叉树中所有非终端节点的值都不大于（或不小于）其左右孩子节点的值

堆排序时所建立的初始小顶堆如图 4-4 所示。

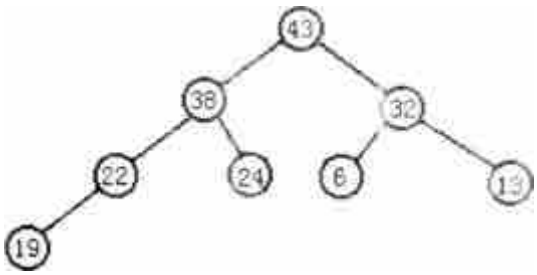


图 4-4 初始小顶堆示意图

两种排序方法所需辅助空间：堆是 O（1），快速排序是 O（1ogn），可见堆排序所需辅助空间较少；在最坏情况下两种排序方法所需时间：堆是 O（nlogn），快这排序是 O（n^2），所以，可见快速排序时间复杂度最差。

注意：快速排序的平均时排序速度最快，但在最坏情况下不一定比其他排序方法快。

五、（本题 10 分）

发二维数组 A[0：10，-5：0]，按行优先顺序存储，每个元素占 4 个单元，A[0][-5]的存储地址为 1000，则 A[9][-2]的存储地址为多少？

答：二维数组存储位置计算公式

LOC（i，j）=LOC（c_1，c_2）+[（d_2-c_2+1）（i-c_1）+（i-c_2）]l

依题意 A 的起始地址为 1000，则有：

Loc(9,-2)=1000+[(9-0)*(0-(-5)+1)+(-2-(-5))]*4=1228。

六、（本题 10 分）

用一维数组存放的一棵完全二叉树：ABCDEFGHIJKL。请写出后序遍历该二叉树的访问节点序列。

答：深度为 K 的完全二叉树，指除了第 K 层外，1～k-1 层的结点个数达到最大。

第 K 层的所有结点都在最左部。

根据上面的原则先画出该二叉树的树形结构。

A

B C

D E F G

H I J K L

对其进行后序遍历得到后序序列为：HIDJKEBLFGCA。

七、（本题 10 分）

请说明对一棵二叉树进行前序、中序和后序遍历，其叶节点的相对次序是否会发生改变？为什么？

答：二叉树任意两个中叶节点必在某节点的左/右子树中，三种遍历方法对左右子树的遍历都是按左子树在前、右子树在后的顺序进行遍历的。所以在三种遍历序列中时节点的相对次序是不会发生改变的。

八、（本题 10 分）

对于如图 4-5 所示的 G，用 Kruskal 算法构造最小生成树，要求图示出每一步的变化情况。

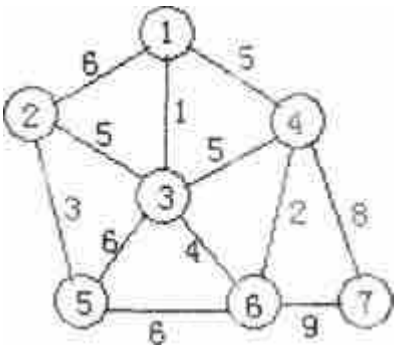


图 4-5 无向图示意图

答：克鲁斯卡尔算法：每次都选用代价最小的边（最短的），且该边的两个顶点分别在两个不同的连通分量上，符合这两个条件就把该边加入最小生成树。

用 Kruskal 算法构造最小生成树的过程如图 4-6 所示：

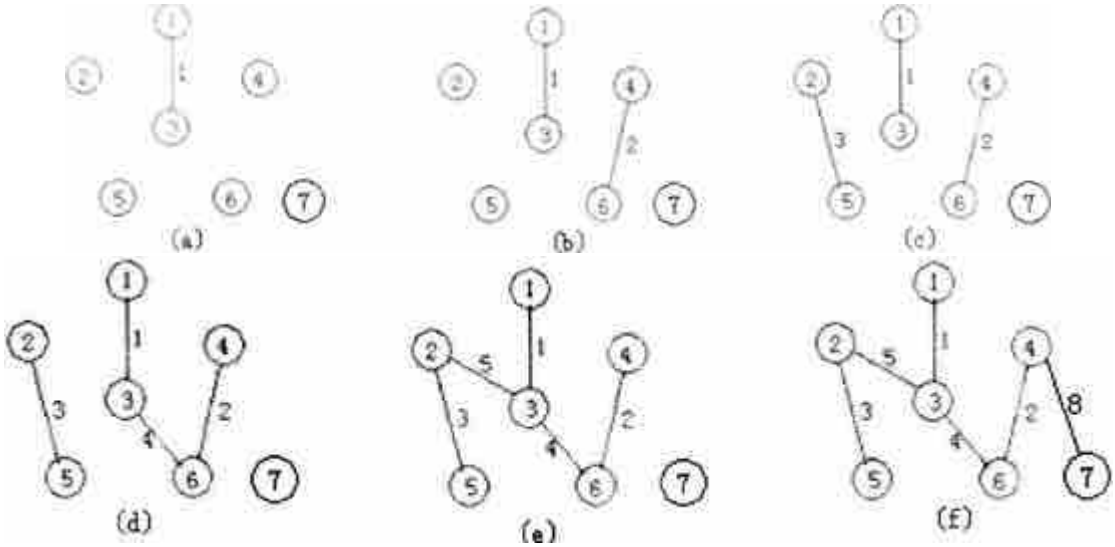


图 4-6 用 Kruskal 算法构造最小生成树的过程示意图

九、（本题 10 分）

在二叉树中显示值为 e 的节点的所有祖先，假定值为 e 的节点不多于一个。

答：用栈 S 存储值为 e 的节点的所有祖先，采用先序遍历方式搜索值为 e 的节点，如搜索成功，则用栈 S 中的节点元素值。

具体算法如下：

```
Satus DispAncestor（BiTree T，TElemType e，SqStack &S）
// 显示二叉树 T 中元素 e 的所有祖先
```

```
{
        BiTNode *p;
        if(T)
        {
            if(T->data==e)
            {   //显示e的祖先，也就是S中节点的元素值
                while(StackEmpty(S)==FALSE)
                {
                    Pop(S,p); cout<<p->data;
                }
                return TRUE;
}
    Push(S,T);
    if(DispAncestor(T->lchild,e,S)==TRUE)return TRUE;//对左子树进行搜索e
    if(DispAncestor(T->rchild,e,S)==TRUE)return TRUE;//对右子树进行搜索e
    Pop(S,p);
                }
            return FALSE;
        }
        Status DispAncestor(BiTree T,TElemType e)
        //    去掉调用时不必要的参数S
        {
            SqStack S;InitStack(S);
            return DispAncestor(T,e,S);
        }
```

严蔚敏《数据结构》（C 语言版）配套模拟试题及详解（二）

一、单项选择题（每小题 2 分，共 20 分）

1．对一个算法的评价，不包括如下（　　）方面的内容。

A．健壮性和可读性

B．并行性

C．正确性

D．时空复杂度

【答案】B 查看答案

【解析】并行性是算法的重要特征，但不是作为评价算法的准则。

2. 在带有头结点的单链表 HL 中，要向表头插入一个由指针 p 指向的结点，则执行（　　）。

A. p->next=HL->next; HL->next=p

B. p->next=HL; HL=p

C. p->next=HL; p=HL

D. HL=p; p->next=HL

【答案】A 查看答案

【解析】本题要在 HL 后面插入一个指针 p，那么就把 p 的后继指向 HL 的后继。即 p->next=HL->next，而 HL 的新后继为 p，即 HL->next=p。

3．对线性表，在下列哪种情况下应当采用链表表示（　　）

A．经常需要随机地存取元素

B．经常需要进行插入和删除操作

C．表中元素需要占据一片连续的存储空间

D．表中元素的个数不变

【答案】B 查看答案

【解析】ACD 项适合顺序表，顺序表存储结构是用一组连续地址的固定大小存储单元，所以可以随机存取元素，但是插入和删除操作是需要挪动大量的元素，效率不高。而用链表相对较好。

4．一个栈的输入序列为 123，则下列序列中不可能是栈的输出序列的是（　　）。

A．231　B．321 C．312　D．123

【答案】C 查看答案

【解析】根据栈的先进后出的特性，那么 123 的入栈顺序可能有以下几种情况：123、321、132、231、213。

5．如将一个由置换一选择排序得到的输出文件 F0 作为输入再次进行置换一选择排序，得到输出文件 F1，问 F0 与 F1 有何差异？（　　）

A．F1 的归并段个数减少

B．F1 中归并段的最大长度增大

C．归并段个数及各归并段长度均不变，但 F1 中可能存在与 F0 不同的归并段

D．F1 与 F0 无差异

【答案】D 查看答案

【解析】F0 为已经排好序的初始归并段。根据置换选择排序的特性，当输入的文件 F0 已经是排好序的升序序列，那么 F1 长度为 F0 的初始归并段。

6．采用开放定址法处理散列表的冲突时，其平均查找长度（　　）。

A．低于链接法处理冲突

B．高于链接法处理冲突

C．与链接法处理冲突相同

D．高于二分查找

【答案】B 查看答案

【解析】利用开放定址法来处理冲突查找成功的平均查找长度公式为 $S_{nr} \approx -\frac{1}{\alpha}\ln(1-\alpha)$，

$S_{nc} \approx 1+\frac{\alpha}{2}$

而利用链接法来处理冲突的平均查找长度为 ，（a 是指表中填入的元素记录数比上哈希表的长度）当 a=1 前者比后者高的多。

7．若需要利用形参直接访问实参时，应将形参变量说明为（　　）参数。

A．值　B．函数　C．指针 D．引用

【答案】D 查看答案

【解析】引用类似与指针，相当于让形参直接指向了实参，这样对形参的所有操作都会引起实参的变化。

8．在稀疏矩阵的带行指针向量的链接存储中，每个单链表中的结点都具有相同的（　　）。

A．行号 B．列号　C．元素值 D．非零元素个数

【答案】A 查看答案

【解析】链接存储稀疏矩阵，那么每一个链表表示一行，所以行号相同。

9．快速排序在最坏情况下的时间复杂度为（　　）。

A．O（$1og_2n$）　B．O（$nlog_2n$）

C．O（n）　D．O（n^2）

【答案】D 查看答案

【解析】当序列本身就是有序的情况下，那么快速排序每次都会选择序列中最大和最小的那个元素，以这个元素作为枢轴的话，那么没有排序的元素都在一边。根据快速排序的特点，这样时间复杂度就为 O（n^2）。

10．从二叉搜索树中查找一个元素时，其时间复杂度大致为（　　）。

A．O（n）　B．O（1）　C．O（$1og_2n$）　D．O（n^2）

【答案】C 查看答案

【解析】在一个二叉搜索树中元素都是排好序的，就是说左子树的元素都比根结点小，而右子树的元素都比根结点大。例如二叉搜索树的结点个数为 n 那么深度大致为 $1og_2n$，那么当查找一个元素时最多只需要查找 $1og_2n$ 次。

二、（本题 10 分）

已知一个图的顶点集 V 和边集 E 分别为：

V={1,2,3,4,5,6,7};

E={(1,2)3,(1,3)5,(1,4)8,(2,5)10,(2,3)6,(3,4)15,(3,5)12,(3,6)9,(4,6)4,(4,7)20,(5,6)18,(6,7)25};

用克鲁斯卡尔算法得到最小生成树，试写出在最小生成树中依次得到的各条边。

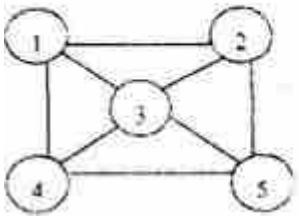
答：克鲁斯卡尔算法：每次都选用代价最小的边（最短的），且该边的两个顶点分别在两个不同的连通分量上，符合这两个条件就把该边加入最小生成树。

用克鲁斯卡尔算法得到的最小生成树为：

（1，2）3，　（4，6）4，　（1，3）5，　（1，4）8，　（2，5）10，　（4．7）20

三、（共 10 分）

设有如图 3-3 所示的 AOE 网（其中 vi（i=1，2，…，6）表示事件，弧上表示活动的天数）。



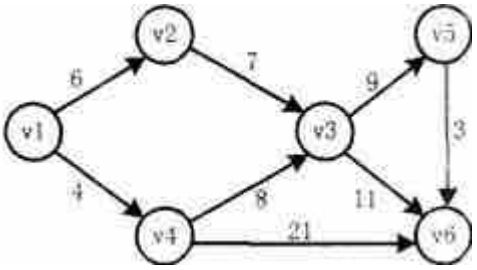


图 3-1 无向图示意图　　图 3-3 AOE 网示意图

（1）找出所有的关键路径。

（2）v3 事件的最早开始时间是多少。

答：（1）找出所有的关键路径有：v1→v2→v3→v5→V6，以及 v1→v4→v6。

（2）v3 事件的最早开始时间是 13。

四、（本题 10 分）

如果在 1000000 个记录中找出两个最小的记录，你认为采用什么样的排序方法所需的关键字比较次数最少？最多比较多少次？

答：树形选择排序。首先对 n 个记录的关键字进行两两比较，然后在 n/2 个较小者之间再进行两两比较，如此重复，直至选出最小的记录为止。

采用树形选择排序方法所需的关键字比较次数最少，最多比较次数
=999999+$\lceil \log_2 1000000 \rceil$=1000019 次。

五、（本题 10 分）

假设把 n 个元素的序列

$(a_1,a_2,\ldots,a_n)$ 满足条件 $a_k<\max\{a_t|1\leqslant t\leqslant k\}$的元素 a_k

称为“逆序元素”。若在一个无序序列中有一对元素$a_i>a_j(i<j)$，试问，当 a_i 与 a_j 相互交换后，该序列中逆序元素的个数一定不会增加，这句话对不对？如果对，请说明为什么？如果不对，请举一例说明。

答：不对，例如序列{3、3、4、2、1}的“逆序元素”个数是 2，2 和 1 是“逆序元素”；但是将第二个 3 和 2 交换后，成为{3、2、4、3、1}，此时“逆序元素”个数是 3，2、3 和 1 是“逆序元素”。然而交换后一定减少的是“逆序对”的个数，例如上例中{3、3、4、2、1}的逆序对的个数是 7，交换第二个 3 和 2 后，（3、2、4、3、1）的逆序对的个数是 6。

六、（共 10 分）

设内存有大小为 6 个记录的缓冲区供内排序使用，文件的关键字序列为(29,50,70,33,38,60,28,31,43,36,25,9,80,100,57,18,65,2,78,30,14,20,17,99)。

试列出：

（1）用内排序求出初始归并段；

（2）用置换-选择排序求初始归并段。

答：（1）用内排序求出初始归并段为：

依次读入六个元素排好序作为归并段，这样就得到了如下四个归并段。

归并段 1：29，33，38，50，60，70；

归并段 2：9，25，28，31，36，43；

归并段 3：2，18，57，65，80，100；

归并段 4：14，17，20，30，78，99.

（2）用置换一选择排序求初始归并段为：

依次读入六个元素，选出最小者 MIN，继续读入元素，选出比上一次大的最小的元素。依次类推。

归并段 1：29，33，38，50，60，70，80，100；

归并段 2：9，18，25，28，31，36，57，65，78，99；

归并段 3：2，14，17，20，30.

七、（本题 10 分）

已知一组关键字为（19，14，23，1，68，20，84，27，55，11，10，79），哈希函数：H（key）=key MOD 13，哈希地址空间为 0～12，请构造用链地址法处理冲突的哈希表，并求平均查找长度。

答：将所有关键字为同义词的记录存储在同一线性链表中，当出现冲突就插入链表尾部。用链地址法处理冲突的哈希表如图 3-4 所示。

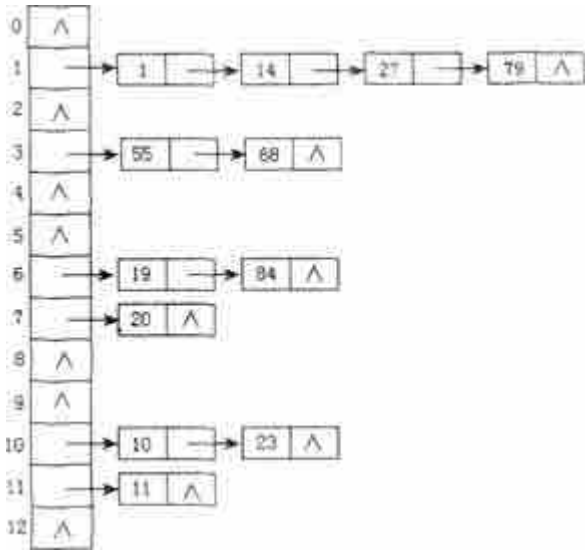


图 3-4 用链地址法处理冲突的哈希表示意图

$$ASL=\frac{1}{12}(1*6+2*4+3*1+4*1)=1.75$$

八、（本题 10 分）

已知深度为 h 的二叉树以一维数组 BT[1:2^h-1]作为存储结构，对于虚结点的数据元素的值为 NULL。请写-算法，采用中序遍历的方法求二叉树中叶结点的个数。

答：根据完全二叉树的性质，顶点 BT[i]如左孩子存在，则左孩子为 BT[2*i]，如右孩子存在，则右孩子为

BT[2*i+1]，

这样与二叉链表表示法类似的可得到递归算法求得叶子结点数。
具体算法如下：

```
int CountLeaf(STElemType BT[],int h,int root=1)
//递归方式求二叉树的顺序存储结点的叶子结点个数
{
    int len=(int)(pow(2,h)-1);      //BT下标范围为[1:len]
    int nl=0,nr=0;          //nl,nr左/右子树所含的叶子节点数
    if(root<=len&&BT[root]!=NULL)
    {
if(2*root<=len&&BT[2*root]!=NULL)
    nl=CountLeaf(BT,h,2*root);//存在左孩子，对左子树进行递归
if(2*root+1<=len&&BT[2*root+1]!=NULL)
    nr=CountLeaf(BT,h,2*root+1);//存在右孩子，对右子树进行递归
if(nl+nr>0)return nl+nr;    //此时根节点存在孩子，非叶节点
else return 1;                  //此时根节点无孩子，为叶节点
```

```
        }
    else
            return 0;                    //空二叉树，无叶节点
}
```

九、（本题 10 分）

设符号表 T 中的标识符（关键字）X 满足：1≤x≤m，n 为对 T 表的最大插入次数，设计符号表 T 的表示结构，允许使用以 O（m+n）空间，并写出 T 的初始化、查找、插入和删除算法，要求它们的时间复杂度都是 O（1）。

答：在理想情况下，初始化一个空的哈希表需要 O（k）（k 为哈希表中位置的个数），查找、插入、删除的操作时间均为 O（1）。本题要求初始化的操作时间也为 O（1），是由于已经知道了符号表中标识符的范围和对此哈希表的最大插入次数，可作理想化处理哈希方法进行修改：将索引与数据在一起的哈希表一分为二：采用两个数组 HT 和 elem，其中 HT 是从 1 到 m 的整数数组（做索引用）；elem 数组的数据类型为哈希表中各元素的类型，其元素个数为 n（即最大的插入次数），两个数组都不需要初始化。使用计数器 LastE 表示上次所用到的 elem 中的位置，初值为-1。HT[k]中的值可能无效也可能是 elem 的索引（用来寻找关键字为 k 的元素），如图 3-5 所示。

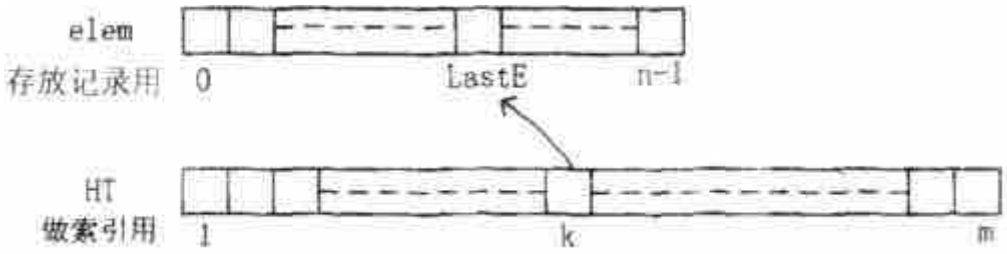


图 3-5 理想化哈希表示意图

具体算法如下：

```
typedef struct{
        KeyType key;
        void *OtherInfo;
}HElemType;

const int n=10;
const int m=200;

typedef struct{
        int HT[m+1];          //用作elem的索引
    HElemType elem[n];        //存放插入的哈希表的元素
        int LastE;            //计数器，表示上次用到的elem位置
}IdealHashTable;
```

```
void InitT(IdealHashTable &T)
//    只为LastE赋初值，时间复杂度为O(1)
{
    T.LastE=-1;
}
Status SerachT(IdealHashTable T,int k,HElemType &e)
//    关键字k的范围在[1:m]，HT[k]应在[0:LastE]范围内
{
    if(k<1||k>m)return FALSE;   //范围错
    if(T.HT[k]<0||T.HT[k]>T.LastE||T.elem[T.HT[k]].key!=k)
        return FALSE;           //表中没有匹配记录
e=T.elem[T.HT[k]];
    return TRUE;                //查找成功
}

Status InsertT(IdealHashTable &T,HElemType e)
//    将记录插入到elem数组LastE位置，同时修改索引值
{
  int k=e.key;
  if(k<1||k>m)return FALSE;   //范围错
HElemType x;
//原哈希表中存在同样的关键字，不允许重复
  if(SerachT(T,k,x)==TRUE)return FALSE;
  if(T.LastE==n-1)return FALSE;//超过最大插入次数
  T.elem[++T.LastE]=e;   //修改计数器，将记录插入到表中
T.HT[k]=T.LastE;   //索引表HT的k位置记录关键字的记录在elem中的位置
  return TRUE;
}

Status   DeleteT(IdealHashTable &T,int k,HElemType &e)
```

/* 通过将 LastE 位置记录移到所删除位置，LastE 指向倒数第二个记录，逻辑上完成删除*
/

```
{
    if(SerachT(T,k,e)==FALSE)return FALSE;//表中不要删除的记录
        //为不中断elem数组,将最后记录移到所删除处,将修改LastE指向新的末记录
        T.elem[T.HT[k]]=T.elem[T.LastE--];
        int key=T.elem[T.HT[k]].key;
    T.HT[key]=T.HT[k];              //由于记录的移动要修改索引值
        return TRUE;
}
```